

CELL 1: Install Ollama + Pull Model (idempotent)

In []:

```
# @title
import os, time, shutil

# Step 1: Install zstd if needed
if not shutil.which("zstd"):
    print("❑ Installing zstd...")
    !sudo apt-get install -y -qq zstd > /dev/null 2>&1
else:
    print("❑ zstd already installed")

# Step 2: Install Ollama if needed
if not shutil.which("ollama"):
    print("❑ Installing Ollama...")
    !curl -fsSL https://ollama.com/install.sh | sh
else:
    print("❑ Ollama already installed")

# Step 3: Stop systemd service (Colab can't use it)
!sudo systemctl stop ollama 2>/dev/null; true

# Step 4: Set GPU environment
os.environ['LD_LIBRARY_PATH'] = '/usr/lib64-nvidia'
os.environ['PATH'] += ':/usr/local/cuda/bin'

# Step 5: Start server if not already running
import requests
server_ok = False
try:
    r = requests.get("http://localhost:11434/api/tags", timeout=3)
    if r.status_code == 200:
        server_ok = True
        print("❑ Ollama server already running")
except:
    pass

if not server_ok:
    print("❑ Starting Ollama server...")
    os.system("pkill -f 'ollama serve' 2>/dev/null; true")
    time.sleep(1)
    os.system("nohup ollama serve > /tmp/ollama.log 2>&1 &")
    time.sleep(5)
    for attempt in range(15):
        try:
            r = requests.get("http://localhost:11434/api/tags", timeout=3)
            if r.status_code == 200:
                print("❑ Ollama server started!")
                server_ok = True
                break
        except:
            pass
        time.sleep(2)
    if not server_ok:
        print("❑ Server failed to start. Logs:")
        !cat /tmp/ollama.log | tail -20

# Step 6: Pull model if not already downloaded
if server_ok:
    r = requests.get("http://localhost:11434/api/tags", timeout=5)
    models = [m["name"] for m in r.json().get("models", [])]
    if any("qwen2.5:7b" in m for m in models):
        print("❑ qwen2.5:7b already downloaded")
    else:
        print("❑ Pulling qwen2.5:7b (3-5 minutes)...")
        !ollama pull qwen2.5:7b
        print("❑ Model ready!")
        !ollama list
```

❑ zstd already installed			
❑ Ollama already installed			
❑ Starting Ollama server...			
❑ Ollama server started!			
❑ qwen2.5:7b already downloaded			
NAME	ID	SIZE	MODIFIED

CELL 2: Project Structure + Aquatic Environment

In []:

```
# @title
# CELL 2: Project Structure + Aquatic Environment
import os

BASE = "/content/AquaLLM"

# Create directories
for d in [f"{BASE}/core", f"{BASE}/agent", f"{BASE}/logs", f"{BASE}/plots"]:
    os.makedirs(d, exist_ok=True)

# Create __init__.py files
for pkg in ["core", "agent"]:
    init_path = f"{BASE}/{pkg}/__init__.py"
    if not os.path.exists(init_path):
        with open(init_path, "w") as f:
            f.write("")

# =====
# FILE 1: entities.py
# =====

ENTITIES_PY = r'''#!/usr/bin/env python3
"""
entities.py -- Simulation entity dataclasses (3D).
PondGenotype is a pure data container -- EA operations live in ea.py.
"""

from dataclasses import dataclass
from typing import Tuple

from constants import (
    FOOD_PRICE, PROBIOTIC_PRICE, OXYGEN_PRICE,
    VELOCITY_HEALTH_THRESHOLD, VELOCITY_FULLNESS_THRESHOLD,
    PARASITE_VELOCITY_MULT, FLOOR_HAZARD_HEIGHT,
)

@dataclass
class PondGenotype:
    fish_count: int = 30
    food_interval: int = 6
    food_quantity: int = 3
    food_location: int = 0
    probiotic_interval: int = 24
    probiotic_quantity: int = 2
    probiotic_location: int = 0
    oxygen_interval: int = 8
    oxygen_duration: int = 2
    oxygen_location: int = 0

    def total_cost(self, runtime: int) -> float:
        fc = (runtime / self.food_interval) * self.food_quantity * FOOD_PRICE
        pc = (runtime / self.probiotic_interval) * self.probiotic_quantity * PROBIOTIC_PRICE
        oc = (runtime / self.oxygen_interval) * self.oxygen_duration * OXYGEN_PRICE
        return fc + pc + oc

    def per_cycle_cost(self) -> float:
        return (self.food_quantity * FOOD_PRICE +
                self.probiotic_quantity * PROBIOTIC_PRICE +
                self.oxygen_duration * OXYGEN_PRICE)

    def to_dict(self) -> dict:
        return {k: getattr(self, k) for k in [
            'fish_count',
            'food_interval', 'food_quantity', 'food_location',
            'probiotic_interval', 'probiotic_quantity', 'probiotic_location',
            'oxygen_interval', 'oxygen_duration', 'oxygen_location'
        ]}

@dataclass
class Obstacle:
    x: float; y: float; z: float; w: float; h: float; d: float
```

```

def contains(self, px, py, pz) -> bool:
    return (self.x <= px <= self.x + self.w and
            self.y <= py <= self.y + self.h and
            self.z <= pz <= self.z + self.d)

def nearest_surface(self, px, py, pz) -> Tuple[float, float, float]:
    cx = max(self.x, min(px, self.x + self.w))
    cy = max(self.y, min(py, self.y + self.h))
    cz = max(self.z, min(pz, self.z + self.d))
    return cx, cy, cz

def center(self) -> Tuple[float, float, float]:
    return self.x + self.w / 2, self.y + self.h / 2, self.z + self.d / 2

def top_z(self) -> float:
    return self.z

def xy_contains(self, px, py) -> bool:
    return self.x <= px <= self.x + self.w and self.y <= py <= self.y + self.h

@dataclass
class DynObj:
    x: float; y: float; z: float; kind: str; value: float = 5.0
    age: int = 0; max_age: int = 30
    vx: float = 0.0; vy: float = 0.0; vz: float = 0.0
    alive: bool = True
    on_floor: bool = False
    float_timer: int = 0
    obj_id: int = -1

@dataclass
class Hazard:
    x: float; y: float; z: float; radius: float; kind: str
    age: int = 0; max_age: int = 50; alive: bool = True
    vx: float = 0.0; vy: float = 0.0; vz: float = 0.0
    is_floor: bool = False
    follow_dead_fish: bool = False
    follow_id: int = -1

    def contains(self, px, py, pz) -> bool:
        if self.is_floor:
            if pz < self.z - FLOOR_HAZARD_HEIGHT:
                return False
            return (px - self.x) ** 2 + (py - self.y) ** 2 <= self.radius ** 2
        return ((px - self.x) ** 2 + (py - self.y) ** 2 +
                (pz - self.z) ** 2) <= self.radius ** 2

@dataclass
class Fish:
    fid: int = 0
    x: float = 0.0; y: float = 0.0; z: float = 0.0
    vx: float = 0.0; vy: float = 0.0; vz: float = 0.0
    mouth_size: float = 5.0; body_size: float = 6.0
    health: float = 100.0; max_health: float = 100.0
    fullness: float = 60.0; max_fullness: float = 60.0
    immunity: float = 80.0; max_immunity: float = 80.0
    oxygen: float = 90.0; max_oxygen: float = 90.0
    base_velocity: float = 2.0
    is_infected: bool = False; has_parasite: bool = False
    alive: bool = True; fecal_timer: int = 0

    def eff_vel(self) -> float:
        v = self.base_velocity
        if self.health <= self.max_health * VELOCITY_HEALTH_THRESHOLD: v *= 0.5
        if self.fullness <= self.max_fullness * VELOCITY_FULLNESS_THRESHOLD: v *= 0.5
        if self.has_parasite: v *= PARASITE_VELOCITY_MULT
        return v

    def norm_stats(self) -> float:
        h_n = max(0, self.health) / self.max_health
        fu_n = max(0, self.fullness) / self.max_fullness
        im_n = max(0, self.immunity) / self.max_immunity
        vl_n = min(1.0, self.eff_vel() / (self.base_velocity + 1e-9))
        return (h_n + fu_n + im_n + vl_n) / 4.0

    def snapshot(self) -> dict:
        return {
            'id': self.fid,

```

```
'x': round(self.x, 1), 'y': round(self.y, 1), 'z': round(self.z, 1),
'health': round(self.health, 1), 'max_health': round(self.max_health, 1),
'fullness': round(self.fullness, 1), 'max_fullness': round(self.max_fullness, 1),
'immunity': round(self.immunity, 1), 'max_immunity': round(self.max_immunity, 1),
'oxygen': round(self.oxygen, 1), 'max_oxygen': round(self.max_oxygen, 1),
'alive': self.alive,
'is_infected': self.is_infected, 'has_parasite': self.has_parasite,
'body_size': round(self.body_size, 1), 'mouth_size': round(self.mouth_size, 1),
'base_velocity': round(self.base_velocity, 2),
'vx': round(self.vx, 2), 'vy': round(self.vy, 2), 'vz': round(self.vz, 2),
    }

'''

# =====
# FILE 2: helpers.py
# =====

HELPERS_PY = r'''#!/usr/bin/env python3
"""
helpers.py -- Pure utility functions and fish factory (3D).
"""

import math
import random

from constants import (
    POND_WIDTH, POND_HEIGHT, POND_DEPTH, DropLocation,
    MOUTH_SIZE_RANGE, BODY_SIZE_RANGE,
    HEALTH_RANGE, FULLNESS_RANGE, IMMUNITY_RANGE, OXYGEN_RANGE,
    VELOCITY_RANGE,
)
from entities import Fish

def _dist(x1, y1, z1, x2, y2, z2):
    return math.sqrt((x1 - x2) ** 2 + (y1 - y2) ** 2 + (z1 - z2) ** 2)

def _clamp(v, lo, hi):
    return max(lo, min(hi, v))

def _norm(vx, vy, vz):
    m = math.sqrt(vx * vx + vy * vy + vz * vz)
    return (vx / m, vy / m, vz / m) if m > 1e-8 else (0.0, 0.0, 0.0)

def _drop_pos(loc: int):
    """Return (x, y) for a drop location. Z is handled by caller."""
    W, H = POND_WIDTH, POND_HEIGHT
    mx, my = W * 0.15, H * 0.15
    jx = random.uniform(-mx * 0.3, mx * 0.3)
    jy = random.uniform(-my * 0.3, my * 0.3)

    if loc == DropLocation.CENTER:
        return W / 2 + jx, H / 2 + jy
    elif loc == DropLocation.TOP_LEFT:
        return mx + jx, my + jy
    elif loc == DropLocation.TOP_RIGHT:
        return W - mx + jx, my + jy
    elif loc == DropLocation.BOT_LEFT:
        return mx + jx, H - my + jy
    elif loc == DropLocation.BOT_RIGHT:
        return W - mx + jx, H - my + jy
    elif loc == DropLocation.TOP_CENTER:
        return W / 2 + jx, my + jy
    elif loc == DropLocation.BOT_CENTER:
        return W / 2 + jx, H - my + jy
    elif loc == DropLocation.LEFT_CENTER:
        return mx + jx, H / 2 + jy
    elif loc == DropLocation.RIGHT_CENTER:
        return W - mx + jx, H / 2 + jy
    else:
        return random.uniform(10, W - 10), random.uniform(5, H - 5)

def _make_fish(fid: int) -> Fish:
    f = Fish(fid=fid)
    f.x = random.uniform(15, POND_WIDTH - 15)
    f.y = random.uniform(10, POND_HEIGHT - 10)
    f.z = random.uniform(5, POND_DEPTH - 5)
```

```
f.mouth_size = random.uniform(*MOUTH_SIZE_RANGE)
f.body_size = random.uniform(*BODY_SIZE_RANGE)
f.max_health = random.uniform(*HEALTH_RANGE); f.health = f.max_health
f.max_fullness = random.uniform(*FULLNESS_RANGE); f.fullness = f.max_fullness * 0.8
f.max_immunity = random.uniform(*IMMUNITY_RANGE); f.immunity = f.max_immunity
f.max_oxygen = random.uniform(*OXYGEN_RANGE); f.oxygen = f.max_oxygen
f.base_velocity = random.uniform(*VELOCITY_RANGE)
f.vx = random.uniform(-1, 1)
f.vy = random.uniform(-1, 1)
f.vz = random.uniform(-0.5, 0.5)
return f
```

```
FISH_DICT_KEYS = [
    'fid', 'x', 'y', 'z', 'vx', 'vy', 'vz', 'mouth_size', 'body_size',
    'health', 'max_health', 'fullness', 'max_fullness',
    'immunity', 'max_immunity', 'oxygen', 'max_oxygen', 'base_velocity',
    'is_infected', 'has_parasite', 'alive', 'fecal_timer']
```

```
def _fish_to_dict(f: Fish) -> dict:
    return {s: getattr(f, s) for s in FISH_DICT_KEYS}
```

```
def _dict_to_fish(d: dict) -> Fish:
    f = Fish()
    for k, v in d.items():
        if hasattr(f, k):
            setattr(f, k, v)
    return f
```

```
class SpatialGrid:
    """3D spatial hash grid for fast neighbor lookups."""

    def __init__(self, cell_size=30.0):
        self.cell_size = cell_size
        self.cells = {}

    def clear(self):
        self.cells.clear()

    def _key(self, x, y, z):
        return (int(x // self.cell_size),
                int(y // self.cell_size),
                int(z // self.cell_size))

    def insert(self, obj):
        k = self._key(obj.x, obj.y, obj.z)
        if k not in self.cells:
            self.cells[k] = []
        self.cells[k].append(obj)

    def insert_all(self, objects):
        self.clear()
        for obj in objects:
            self.insert(obj)

    def get_nearby(self, x, y, z, radius):
        cs = self.cell_size
        r_cells = int(radius // cs) + 1
        cx, cy, cz = int(x // cs), int(y // cs), int(z // cs)
        r2 = radius * radius
        for dx in range(-r_cells, r_cells + 1):
            for dy in range(-r_cells, r_cells + 1):
                for dz in range(-r_cells, r_cells + 1):
                    cell = self.cells.get((cx + dx, cy + dy, cz + dz))
                    if cell:
                        for obj in cell:
                            ddx = obj.x - x
                            ddy = obj.y - y
                            ddz = obj.z - z
                            if ddx * ddx + ddy * ddy + ddz * ddz <= r2:
                                yield obj
```

```
'''
```

```
# =====
# FILE 3: physics.py
# =====
```

```
PHYSICS_PY = r'#!/usr/bin/env python3
```

```
"""
physics.py -- Environment physics: object movement, hazard updates,
            pollutant transformation, floor stacking.
"""

import random, math

from constants import (
    POND_WIDTH, POND_HEIGHT, POND_DEPTH,
    OXYGEN_BUBBLE_SPEED, NH3_BUBBLE_SPEED, NH3_AREA_RADIUS_RANGE,
    NH3_EXPIRE_TIMESTEPS, DISEASE_AREA_DECAY, PARASITE_AREA_DECAY,
    POLLUTANT_TO_HAZARD_TIMESTEPS, DISEASE_AREA_RADIUS_DECAY,
    POLLUTANT_TO_NH3_CHANCE, POLLUTANT_TO_DISEASE_CHANCE,
    POLLUTANT_TO_PARASITE_CHANCE, POLLUTANT_TO_BOTH_CHANCE,
    POLLUTANT_TO_PLANT_CHANCE, POLLUTANT_TO_OBSTACLE_CHANCE,
    POLLUTANT_RADIUS_SCALE, DEAD_FISH_POLLUTANT_MULT,
    POLLUTANT_OBSTACLE_AREA_RANGE,
    OBSTACLE_MAX_WIDTH, OBSTACLE_MAX_HEIGHT, OBSTACLE_MAX_DEPTH,
    SINK_SPEED, SINK_SPEED_HEAVY,
    DEAD_FISH_NH3_RADIUS, DEAD_FISH_FLOAT_SPEED,
    FECAL_SINK_SPEED,
    STACK_RADIUS,
)
from entities import Obstacle, DynObj, Hazard
from helpers import _dist, _clamp

_WALL = 2.0
_FLOOR_Z = POND_DEPTH - _WALL

def upd_objs(sim):
    """Move/age all dynamic objects. Handles sinking, drifting, expiry, pollutant conversion."""
    keep = []
    for o in sim.objs:
        if o.kind == 'oxygen':
            o.age += 1
            o.x += o.vx; o.y += o.vy; o.z += o.vz
            if o.x < _WALL or o.x > POND_WIDTH-_WALL: o.vx *= -1
            if o.y < _WALL or o.y > POND_HEIGHT-_WALL: o.vy *= -1
            if o.z < _WALL or o.z > POND_DEPTH-_WALL: o.vz *= -1
            o.x, o.y, o.z = sim._cp(o.x, o.y, o.z)
            if any(h.contains(o.x, o.y, o.z) and h.kind == 'nh3' and h.alive for h in sim.hazards):
                continue
            keep.append(o); continue

        if o.kind == 'plant':
            keep.append(o); continue

        if o.kind in ('food', 'probiotic'):
            rest_z = sim._resting_z(o.x, o.y)
            if not o.on_floor:
                o.x += o.vx; o.y += o.vy; o.z += o.vz
                if o.x < _WALL or o.x > POND_WIDTH-_WALL: o.vx *= -1
                if o.y < _WALL or o.y > POND_HEIGHT-_WALL: o.vy *= -1
                o.x = _clamp(o.x, _WALL, POND_WIDTH-_WALL)
                o.y = _clamp(o.y, _WALL, POND_HEIGHT-_WALL)
                o.z = max(o.z, _WALL)
                if o.z >= rest_z:
                    o.z = rest_z
                    o.on_floor = True
                    o.vx = 0.0; o.vy = 0.0; o.vz = 0.0
            if o.on_floor: o.age += 1

            if o.age >= o.max_age and o.alive:
                if o.value > 0:
                    keep.append(DynObj(o.x, o.y, o.z, 'pollutant', o.value,
                                       max_age=POLLUTANT_TO_HAZARD_TIMESTEPS, on_floor=True))
                    continue
            if o.alive: keep.append(o)
            continue

        if o.kind == 'dead_fish':
            rest_z = sim._resting_z(o.x, o.y)

            if o.float_timer > 0:
                o.z = max(_WALL, o.z - DEAD_FISH_FLOAT_SPEED)
                if o.z <= _WALL:
                    o.z = _WALL
                o.float_timer -= 1
            elif o.z < rest_z:
                o.z = min(o.z + SINK_SPEED_HEAVY, rest_z)
```

```

# Follower NH3 tracks this specific body via matching ID
for h in sim.hazards:
    if h.follow_dead_fish and h.kind == 'nh3' and h.follow_id == o.obj_id:
        h.x, h.y, h.z = o.x, o.y, o.z

o.on_floor = o.z >= rest_z - 0.1
if o.on_floor: o.age += 1

if o.age >= o.max_age and o.alive:
    # Kill the linked follower NH3
    for h in sim.hazards:
        if h.follow_dead_fish and h.kind == 'nh3' and h.follow_id == o.obj_id:
            h.alive = False
    if o.value > 0:
        pv = o.value * DEAD_FISH_POLLUTANT_MULT
        if not any(h.contains(o.x, o.y, o.z) and h.kind == 'nh3' and h.alive for h in sim.hazards):
            keep.append(DynObj(o.x, o.y, o.z, 'pollutant', pv,
                               max_age=POLLUTANT_TO_HAZARD_TIMESTEPS, on_floor=True))
        continue
    if o.alive: keep.append(o)
    continue

if o.kind == 'fecal':
    rest_z = sim._resting_z(o.x, o.y)
    if not o.on_floor:
        o.z = min(o.z + FECAL_SINK_SPEED, rest_z)
        if o.z >= rest_z:
            o.z = rest_z
            o.on_floor = True

    if o.on_floor: o.age += 1

    if o.age >= o.max_age and o.alive:
        if o.value > 0:
            if not any(h.contains(o.x, o.y, o.z) and h.kind == 'nh3' and h.alive for h in sim.hazards):
                keep.append(DynObj(o.x, o.y, o.z, 'pollutant', o.value,
                                   max_age=POLLUTANT_TO_HAZARD_TIMESTEPS, on_floor=True))
            continue
        if o.alive: keep.append(o)
        continue

# pollutant and other kinds
o.on_floor = o.z >= sim._resting_z(o.x, o.y) - 0.1
if o.on_floor: o.age += 1

if o.age >= o.max_age and o.alive:
    if o.kind == 'pollutant':
        transform_pollutant(sim, o)
        continue
    if o.alive: keep.append(o)
sim.objs = keep

def transform_pollutant(sim, o):
    """Convert an expired pollutant into a hazard, plant, obstacle, or nothing."""
    r = o.value * POLLUTANT_RADIUS_SCALE
    roll = random.random()
    cumul = 0.0
    cumul += POLLUTANT_TO_NH3_CHANCE
    if roll < cumul:
        sim.hazards.append(Hazard(o.x, o.y, o.z, r, 'nh3', max_age=NH3_EXPIRE_TIMESTEPS,
                                  vx=random.uniform(-NH3_BUBBLE_SPEED, NH3_BUBBLE_SPEED),
                                  vy=random.uniform(-NH3_BUBBLE_SPEED, NH3_BUBBLE_SPEED),
                                  vz=random.uniform(-NH3_BUBBLE_SPEED*0.5, NH3_BUBBLE_SPEED*0.5)))
        return
    cumul += POLLUTANT_TO_DISEASE_CHANCE
    if roll < cumul:
        sim.hazards.append(Hazard(o.x, o.y, _FLOOR_Z, r, 'disease', max_age=DISEASE_AREA_DECAY, is_floor=True))
        return
    cumul += POLLUTANT_TO_PARASITE_CHANCE
    if roll < cumul:
        sim.hazards.append(Hazard(o.x, o.y, _FLOOR_Z, r, 'parasite', max_age=PARASITE_AREA_DECAY, is_floor=True))
        return
    cumul += POLLUTANT_TO_BOTH_CHANCE
    if roll < cumul:
        sim.hazards.append(Hazard(o.x, o.y, _FLOOR_Z, r, 'disease', max_age=DISEASE_AREA_DECAY, is_floor=True))

```

```

True))
    sim.hazards.append(Hazard(o.x, o.y, _FLOOR_Z, r*0.8, 'parasite', max_age=PARASITE_AREA_DECAY, is_
floor=True))
    return
    cumul += POLLUTANT_TO_PLANT_CHANCE
    if roll < cumul:
        sim.objs.append(DynObj(o.x, o.y, _FLOOR_Z, 'plant', 0, max_age=999999))
        return
    cumul += POLLUTANT_TO_OBSTACLE_CHANCE
    if roll < cumul:
        area = random.uniform(*POLLUTANT_OBSTACLE_AREA_RANGE)
        aspect = random.uniform(0.5, 2.0)
        w = math.sqrt(area * aspect)
        h = area / w if w > 0 else math.sqrt(area)
        w = min(w, OBSTACLE_MAX_WIDTH); h = min(h, OBSTACLE_MAX_HEIGHT)
        d = random.uniform(2, 5); d = min(d, OBSTACLE_MAX_DEPTH)
        ox = _clamp(o.x - w/2, _WALL, POND_WIDTH - _WALL - w)
        oy = _clamp(o.y - h/2, _WALL, POND_HEIGHT - _WALL - h)
        oz = _FLOOR_Z - d
        sim.obstacles.append(Obstacle(ox, oy, oz, w, h, d))
        return
    # Remaining POLLUTANT_TO_NOTHING_CHANCE: harmless decomposition

def upd_haz(sim):
    """Move/age all hazards. Expired NH3 becomes pollutant."""
    keep = []
    for h in sim.hazards:
        if not h.alive:
            continue
        h.age += 1
        if h.kind == 'nh3' and not h.follow_dead_fish:
            h.x += h.vx; h.y += h.vy; h.z += h.vz
            if h.x < 5 or h.x > POND_WIDTH-5: h.vx *= -1
            if h.y < 3 or h.y > POND_HEIGHT-3: h.vy *= -1
            if h.z < 3 or h.z > POND_DEPTH-3: h.vz *= -1
            h.x = _clamp(h.x, 5, POND_WIDTH-5)
            h.y = _clamp(h.y, 3, POND_HEIGHT-3)
            h.z = _clamp(h.z, 3, POND_DEPTH-3)
        if h.age >= h.max_age:
            if h.kind == 'nh3':
                sim.objs.append(DynObj(h.x, h.y, _FLOOR_Z, 'pollutant', h.radius*0.5,
max_age=POLLUTANT_TO_HAZARD_TIMESTEPS, on_floor=True))
            continue
        if h.kind in ('disease', 'parasite'):
            h.radius = max(0.5, h.radius * DISEASE_AREA_RADIUS_DECAY)
        keep.append(h)
    sim.hazards = keep

def stack_floor_objs(sim):
    """Merge stackable objects (food, probiotic, fecal) on the floor within STACK_RADIUS."""
    floor_objs = [o for o in sim.objs if o.alive and o.on_floor and o.kind in ('food', 'probiotic', 'fecal
')]
    if not floor_objs:
        return
    by_kind = {}
    for o in floor_objs:
        by_kind.setdefault(o.kind, []).append(o)

    for kind, group in by_kind.items():
        for i in range(len(group)):
            a = group[i]
            if not a.alive:
                continue
            for j in range(i + 1, len(group)):
                b = group[j]
                if not b.alive:
                    continue
                if _dist(a.x, a.y, a.z, b.x, b.y, b.z) < STACK_RADIUS:
                    a.value += b.value
                    b.alive = False

'''

# =====
# FILE 4: behavior.py
# =====

BEHAVIOR_PY = r'#!/usr/bin/env python3
'''
behavior.py -- Fish behavior: PSO movement, eating, decay, cannibalism,

```

```

        fecal production, death.
    """

import random, math

from constants import (
    POND_WIDTH, POND_HEIGHT, POND_DEPTH, BODY_SIZE_RANGE,
    OXYGEN_DECAY, OXYGEN_DECAY_NH3_MULT, OXYGEN_PASSIVE_REGEN,
    FULLNESS_DECAY, FULLNESS_COST_MOVE,
    HEALTH_DECAY_NO_FULLNESS, HEALTH_DECAY_INFECTED, HEALTH_DECAY_PARASITE,
    HEALTH_DECAY_IN_NH3, HEALTH_REGEN, DISEASE_SELF_CURE_CHANCE,
    IMMUNITY_DECAY_IN_DISEASE, IMMUNITY_DECAY_IN_NH3, IMMUNITY_REGEN,
    PARASITE_CONTACT_CHANCE,
    PARASITE_FULLNESS_EFFICIENCY, PARASITE_EXTRA_FULLNESS_DRAIN,
    FOOD_FULLNESS_GAIN, PROBIOTIC_IMMUNITY_GAIN, OXYGEN_BUBBLE_GAIN,
    FISH_EAT_RANGE, INFECTED_FISH_DISEASE_RADIUS_MULT,
    SENSITIVE_DISTANCE, SOCIAL_DISTANCE, SELFISH_DISTANCE,
    PSO_INERTIA, PSO_FOOD_WEIGHT, PSO_FOOD_URGENT_MULT,
    PSO_FOOD_URGENT_THRESHOLD,
    PSO_PROBIOTIC_WEIGHT,
    PSO_OXYGEN_WEIGHT, PSO_OXYGEN_CRITICAL_MULT,
    PSO_OXYGEN_THRESHOLD, PSO_OXYGEN_CRITICAL_THRESHOLD,
    PSO_SOCIAL_WEIGHT, PSO_SELFISH_WEIGHT,
    PSO_NH3_WEIGHT, PSO_NH3_HUNGRY_OVERRIDE,
    PSO_DISEASE_WEIGHT, PSO_PARASITE_WEIGHT,
    PSO_RUN_WEIGHT, PSO_OBSTACLE_WEIGHT,
    PSO_SWARM_HUNGRY_WEIGHT, PSO_SWARM_HUNGRY_THRESHOLD,
    CANNIBAL_FULLNESS_THRESHOLD, CANNIBAL_BASE_CHANCE, CANNIBAL_HUNGER_MULT,
    CANNIBAL_FULLNESS_GAIN_MULT, CANNIBAL_COLLISION_RADIUS_MULT,
    FECAL_DROP_INTERVAL, FECAL_BASE_CHANCE, FECAL_VALUE, FECAL_SINK_SPEED,
    FECAL_EXPIRE_TIMESTEPS, MAX_SINKING_FECAL,
    DEAD_FISH_DECAY_TIMESTEPS, DEAD_FISH_NH3_RADIUS,
    DEAD_FISH_FLOAT_CHANCE, DEAD_FISH_FLOAT_DURATION_RANGE,
    DISEASE_AREA_DECAY, PARASITE_AREA_DECAY,
    PARASITE_VELOCITY_MULT,
)

from entities import DynObj, Hazard
from helpers import _dist, _clamp, _norm

_WALL = 2.0
_FLOOR_Z = POND_DEPTH - _WALL

def decay(sim):
    """Apply per-tick stat decay, environmental effects, disease/parasite mechanics."""
    for f in sim.fish:
        if not f.alive: continue

        f.oxygen -= OXYGEN_DECAY
        f.fullness -= FULLNESS_DECAY * (PARASITE_EXTRA_FULLNESS_DRAIN if f.has_parasite else 1.0)

        in_nh3 = any(h.kind == 'nh3' and h.contains(f.x, f.y, f.z) for h in sim.hazards)
        if not in_nh3 and f.oxygen < f.max_oxygen:
            f.oxygen = min(f.max_oxygen, f.oxygen + OXYGEN_PASSIVE_REGEN)

        for h in sim.hazards:
            if h.kind == 'nh3' and h.contains(f.x, f.y, f.z):
                f.oxygen -= OXYGEN_DECAY * OXYGEN_DECAY_NH3_MULT
                f.health -= HEALTH_DECAY_IN_NH3
                f.immunity -= IMMUNITY_DECAY_IN_NH3

        if f.fullness <= 0: f.health -= HEALTH_DECAY_NO_FULLNESS
        if f.is_infected: f.health -= HEALTH_DECAY_INFECTED
        if f.has_parasite and sim.ts % 3 == 0: f.health -= HEALTH_DECAY_PARASITE

        fr = f.fullness / f.max_fullness
        or_ = f.oxygen / f.max_oxygen
        ir = f.immunity / f.max_immunity
        if (not f.is_infected and not f.has_parasite and
            fr >= 0.7 and or_ >= 0.7 and ir > 0.7):
            f.health = min(f.max_health, f.health + HEALTH_REGEN)

        if not f.is_infected and f.immunity < f.max_immunity:
            f.immunity = min(f.max_immunity, f.immunity + IMMUNITY_REGEN)

        if f.is_infected:
            if fr >= 0.8 and or_ >= 0.8 and ir >= 0.9:
                if random.random() < DISEASE_SELF_CURE_CHANCE:
                    f.is_infected = False

        for h in sim.hazards:

```

```

        if h.kind == 'disease' and h.contains(f.x, f.y, f.z):
            f.immunity -= IMMUNITY_DECAY_IN_DISEASE
            if f.immunity <= 0: f.is_infected = True; f.immunity = 0
        if h.kind == 'parasite' and h.contains(f.x, f.y, f.z):
            if random.random() < PARASITE_CONTACT_CHANCE: f.has_parasite = True

        f.health = max(0, f.health)
        f.oxygen = max(0, f.oxygen)
        f.fullness = max(0, f.fullness)
        f.immunity = max(0, f.immunity)

def eat(sim):
    """Fish consume nearby food, probiotic, and oxygen objects."""
    sim._obj_grid.insert_all([o for o in sim.objs if o.alive and o.kind != 'plant'])
    for f in sim.fish:
        if not f.alive: continue
        for o in sim._obj_grid.get_nearby(f.x, f.y, f.z, f.body_size + FISH_EAT_RANGE):
            if not o.alive or o.value <= 0: continue
            if o.kind == 'food' and f.fullness < f.max_fullness:
                o.value -= 1
                g = FOOD_FULLNESS_GAIN * (PARASITE_FULLNESS_EFFICIENCY if f.has_parasite else 1.0)
                f.fullness = min(f.max_fullness, f.fullness + g)
                if o.value <= 0: o.alive = False
            elif o.kind == 'probiotic' and f.immunity < f.max_immunity:
                o.value -= 1
                f.immunity = min(f.max_immunity, f.immunity + PROBIOTIC_IMMUNITY_GAIN)
                if o.value <= 0: o.alive = False
            elif o.kind == 'oxygen':
                f.oxygen = min(f.max_oxygen, f.oxygen + OXYGEN_BUBBLE_GAIN)
                o.alive = False

def _intercept_pos(f, o):
    """Predict where a moving object will be when the fish can reach it."""
    vel = f.eff_vel()
    if vel < 0.01:
        return o.x, o.y, o.z
    tx, ty, tz = o.x, o.y, o.z
    for _ in range(2):
        d = math.sqrt((f.x - tx)**2 + (f.y - ty)**2 + (f.z - tz)**2)
        eta = d / vel
        tx = o.x + o.vx * eta
        ty = o.y + o.vy * eta
        tz = o.z + o.vz * eta
        tx = _clamp(tx, _WALL, POND_WIDTH - _WALL)
        ty = _clamp(ty, _WALL, POND_HEIGHT - _WALL)
        tz = _clamp(tz, _WALL, POND_DEPTH - _WALL)
    return tx, ty, tz

def _vecs(f, sim, alive, swarm_cx, swarm_cy, swarm_cz):
    """Compute all PSO steering vectors for a single fish."""
    vecs = []; fr = max(0, f.fullness) / f.max_fullness

    if fr < 1.0:
        fw = PSO_FOOD_WEIGHT * math.sqrt(1.0 - fr)
        if fr < PSO_FOOD_URGENT_THRESHOLD: fw *= PSO_FOOD_URGENT_MULT
        nd, no = float('inf'), None
        for o in sim._obj_grid.get_nearby(f.x, f.y, f.z, SENSITIVE_DISTANCE):
            if o.alive and o.kind == 'food' and o.value > 0:
                tx, ty, tz = _intercept_pos(f, o)
                d = _dist(f.x, f.y, f.z, tx, ty, tz)
                if d < nd: nd, no = d, o
        if no:
            tx, ty, tz = _intercept_pos(f, no)
            dx, dy, dz = _norm(tx-f.x, ty-f.y, tz-f.z); vecs.append((fw, dx, dy, dz))

    ir = max(0, f.immunity)/f.max_immunity
    if ir < 1.0:
        pw = PSO_PROBIOTIC_WEIGHT*(1.0-ir)
        nd, no = float('inf'), None
        for o in sim._obj_grid.get_nearby(f.x, f.y, f.z, SENSITIVE_DISTANCE):
            if o.alive and o.kind == 'probiotic' and o.value > 0:
                tx, ty, tz = _intercept_pos(f, o)
                d = _dist(f.x, f.y, f.z, tx, ty, tz)
                if d < nd: nd, no = d, o
        if no:
            tx, ty, tz = _intercept_pos(f, no)
            dx, dy, dz = _norm(tx-f.x, ty-f.y, tz-f.z); vecs.append((pw, dx, dy, dz))

```

```

orr = max(0, f.oxygen/f.max_oxygen)
if orr < PSO_OXYGEN_THRESHOLD:
    ow = PSO_OXYGEN_WEIGHT*(1.0-orr)
    if orr < PSO_OXYGEN_CRITICAL_THRESHOLD: ow *= PSO_OXYGEN_CRITICAL_MULT
    nd, no = float('inf'), None
    for o in sim._obj_grid.get_nearby(f.x, f.y, f.z, SENSITIVE_DISTANCE):
        if o.alive and o.kind == 'oxygen':
            tx, ty, tz = _intercept_pos(f, o)
            d = _dist(f.x, f.y, f.z, tx, ty, tz)
            if d < nd: nd, no = d, o
    if no:
        tx, ty, tz = _intercept_pos(f, no)
        dx,dy,dz = _norm(tx-f.x,ty-f.y,tz-f.z); vecs.append((ow,dx,dy,dz))

svx,svy,svz,sc = 0,0,0,0; rvx,rvy,rvz,rc = 0,0,0,0
for o in sim._fish_grid.get_nearby(f.x, f.y, f.z, SOCIAL_DISTANCE):
    if o.fid == f.fid: continue
    d = _dist(f.x,f.y,f.z,o.x,o.y,o.z)
    if d < SELFISH_DISTANCE and d > 0.1:
        dx,dy,dz = _norm(f.x-o.x,f.y-o.y,f.z-o.z)
        rvx += dx/d; rvy += dy/d; rvz += dz/d; rc += 1
    elif d < SOCIAL_DISTANCE:
        svx += o.x-f.x; svy += o.y-f.y; svz += o.z-f.z; sc += 1
if sc > 0:
    dx,dy,dz = _norm(svx/sc,svy/sc,svz/sc); vecs.append((PSO_SOCIAL_WEIGHT,dx,dy,dz))
if rc > 0:
    dx,dy,dz = _norm(rvx,rvy,rvz); vecs.append((PSO_SELFISH_WEIGHT,dx,dy,dz))

for h in sim.hazards:
    if h.kind == 'nh3':
        d = _dist(f.x,f.y,f.z,h.x,h.y,h.z)
        if d < h.radius+SENSITIVE_DISTANCE*0.5:
            w = PSO_NH3_WEIGHT
            if fr < 0.1 and any(o.alive and o.kind=='food' and h.contains(o.x,o.y,o.z) for o in sim.objs):
                w = PSO_NH3_HUNGRY_OVERRIDE
            dx,dy,dz = _norm(f.x-h.x,f.y-h.y,f.z-h.z); vecs.append((w,dx,dy,dz))
for h in sim.hazards:
    if h.kind == 'disease':
        d = _dist(f.x,f.y,f.z,h.x,h.y,h.z)
        if d < h.radius+SENSITIVE_DISTANCE*0.5:
            dx,dy,dz = _norm(f.x-h.x,f.y-h.y,f.z-h.z); vecs.append((PSO_DISEASE_WEIGHT,dx,dy,dz))
for h in sim.hazards:
    if h.kind == 'parasite':
        d = _dist(f.x,f.y,f.z,h.x,h.y,h.z)
        if d < h.radius+SENSITIVE_DISTANCE*0.5:
            dx,dy,dz = _norm(f.x-h.x,f.y-h.y,f.z-h.z); vecs.append((PSO_PARASITE_WEIGHT,dx,dy,dz))

for o in sim._fish_grid.get_nearby(f.x, f.y, f.z, SENSITIVE_DISTANCE):
    if o.fid == f.fid: continue
    if o.mouth_size > f.body_size:
        d = _dist(f.x,f.y,f.z,o.x,o.y,o.z)
        if 0.1 < d < SENSITIVE_DISTANCE:
            w = PSO_RUN_WEIGHT*(SENSITIVE_DISTANCE/(d+1))
            dx,dy,dz = _norm(f.x-o.x,f.y-o.y,f.z-o.z); vecs.append((w,dx,dy,dz))

for obs in sim.obstacles:
    ssx,ssy,ssz = obs.nearest_surface(f.x,f.y,f.z)
    d = _dist(f.x,f.y,f.z,ssx,ssy,ssz)
    if 0 < d < SENSITIVE_DISTANCE:
        w = PSO_OBSTACLE_WEIGHT*(1.0-d/SENSITIVE_DISTANCE)
        dx,dy,dz = _norm(f.x-ssx,f.y-ssy,f.z-ssz); vecs.append((w,dx,dy,dz))
    elif d < 0.01:
        ocx,ocy,ocz = obs.center()
        dx,dy,dz = _norm(f.x-ocx,f.y-ocy,f.z-ocz); vecs.append((PSO_OBSTACLE_WEIGHT,dx,dy,dz))

if sc == 0 and rc == 0:
    if fr < PSO_SWARM_HUNGRY_THRESHOLD:
        urgency = PSO_SWARM_HUNGRY_WEIGHT * math.sqrt(1.0 - fr)
        dx,dy,dz = _norm(swarm_cx - f.x, swarm_cy - f.y, swarm_cz - f.z)
        vecs.append((urgency, dx, dy, dz))
    else:
        cx, cy, cz = POND_WIDTH/2, POND_HEIGHT/2, POND_DEPTH*0.4
        dx,dy,dz = _norm(cx-f.x, cy-f.y, cz-f.z)
        vecs.append((PSO_SOCIAL_WEIGHT * 0.5, dx, dy, dz))
wx = random.uniform(-1, 1)
wy = random.uniform(-1, 1)
wz = random.uniform(-0.5, 0.5)
dx,dy,dz = _norm(wx, wy, wz)
vecs.append((0.3, dx, dy, dz))

```

```

return vecs

def pso(sim):
    """PSO-based fish movement with obstacle/wall collision."""
    alive = [f for f in sim.fish if f.alive]
    if not alive: return
    sim._fish_grid.insert_all(alive)
    sim._obj_grid.insert_all([o for o in sim.objs if o.alive and o.kind != 'plant'])

    n = len(alive)
    swarm_cx = sum(a.x for a in alive) / n
    swarm_cy = sum(a.y for a in alive) / n
    swarm_cz = sum(a.z for a in alive) / n

    for f in alive:
        vel = f.eff_vel()
        nvx, nvy, nvz = PSO_INERTIA*f.vx, PSO_INERTIA*f.vy, PSO_INERTIA*f.vz
        for w, dx, dy, dz in _vecs(f, sim, alive, swarm_cx, swarm_cy, swarm_cz):
            nvx += w*dx; nvy += w*dy; nvz += w*dz
        m = math.sqrt(nvx**2 + nvy**2 + nvz**2)
        if m > 0.01: nvx = nvx/m*vel; nvy = nvy/m*vel; nvz = nvz/m*vel
        f.vx, f.vy, f.vz = nvx, nvy, nvz
        nx, ny, nz = f.x+f.vx, f.y+f.vy, f.z+f.vz

    for obs in sim.obstacles:
        if obs.contains(nx, ny, nz):
            ssx, ssy, ssz = obs.nearest_surface(nx, ny, nz)
            norm_x, norm_y, norm_z = _norm(nx-ssx, ny-ssy, nz-ssz)
            if abs(norm_x)<1e-8 and abs(norm_y)<1e-8 and abs(norm_z)<1e-8:
                ocx, ocy, ocz = obs.center()
                norm_x, norm_y, norm_z = _norm(f.x-ocx, f.y-ocy, f.z-ocz)
                if abs(norm_x)<1e-8 and abs(norm_y)<1e-8 and abs(norm_z)<1e-8:
                    norm_x, norm_y, norm_z = _norm(random.uniform(-1,1), random.uniform(-1,1), random
.uniform(-1,1))
                nx = ssx+norm_x*0.5; ny = ssy+norm_y*0.5; nz = ssz+norm_z*0.5
                dot = f.vx*norm_x + f.vy*norm_y + f.vz*norm_z
                f.vx -= dot*norm_x; f.vy -= dot*norm_y; f.vz -= dot*norm_z

    if nx < _WALL:
        nx = _WALL; f.vx = 0
    elif nx > POND_WIDTH-_WALL:
        nx = POND_WIDTH-_WALL; f.vx = 0
    if ny < _WALL:
        ny = _WALL; f.vy = 0
    elif ny > POND_HEIGHT-_WALL:
        ny = POND_HEIGHT-_WALL; f.vy = 0
    if nz < _WALL:
        nz = _WALL; f.vz = 0
    elif nz > POND_DEPTH-_WALL:
        nz = POND_DEPTH-_WALL; f.vz = 0

    f.x, f.y, f.z = sim._cp(nx, ny, nz)
    mc = FULLNESS_COST_MOVE * vel
    if f.has_parasite: mc *= PARASITE_EXTRA_FULLNESS_DRAIN
    f.fullness -= mc

def cannibal(sim):
    """Hungry fish may eat smaller fish."""
    alive = [f for f in sim.fish if f.alive]
    sim._fish_grid.insert_all(alive)
    for f in alive:
        if not f.alive: continue
        fr = f.fullness / f.max_fullness if f.max_fullness > 0 else 1.0
        if fr >= CANNIBAL_FULLNESS_THRESHOLD: continue
        eat_range = f.body_size + FISH_EAT_RANGE
        has_food = False
        for o in sim._obj_grid.get_nearby(f.x, f.y, f.z, eat_range * 2):
            if o.alive and o.kind == 'food' and o.value > 0:
                has_food = True; break
        if has_food: continue
        chance = CANNIBAL_BASE_CHANCE + (1.0 - fr) * CANNIBAL_HUNGER_MULT
        if random.random() > chance: continue
        for t in sim._fish_grid.get_nearby(f.x, f.y, f.z,
            max(BODY_SIZE_RANGE[1] * CANNIBAL_COLLISION_RADIUS_MULT, 20)):
            if t.fid == f.fid or not t.alive: continue
            if f.mouth_size > t.body_size:
                d = _dist(f.x, f.y, f.z, t.x, t.y, t.z)
                if d <= t.body_size * CANNIBAL_COLLISION_RADIUS_MULT:
                    t.health = 0; t.alive = False

```

```
f.fullness = min(f.max_fullness, f.fullness + t.body_size * CANNIBAL_FULLNESS_GAIN_MU
LT)

sim.cannibal_events.append({'x':t.x,'y':t.y,'z':t.z,'predator':f.fid,'prey':t.fid})
break

def fecal(sim):
    """Fish produce fecal waste, capped by MAX_SINKING_FECAL."""
    sinking_count = sum(1 for o in sim.objs if o.alive and o.kind == 'fecal' and not o.on_floor)

    for f in sim.fish:
        if not f.alive: continue
        f.fecal_timer += 1
        if f.fecal_timer >= FECAL_DROP_INTERVAL and f.fullness > 0:
            f.fecal_timer = 0
            if sinking_count >= MAX_SINKING_FECAL:
                continue
            if random.random() < (f.fullness/f.max_fullness)*FECAL_BASE_CHANCE:
                sim.objs.append(DynObj(
                    f.x + random.uniform(-3, 3),
                    f.y + random.uniform(-2, 2),
                    f.z + random.uniform(-1, 1),
                    'fecal', FECAL_VALUE,
                    max_age=FECAL_EXPIRE_TIMESTEPS,
                    vz=FECAL_SINK_SPEED))
                sinking_count += 1

def death(sim):
    """Detect dead fish, spawn dead body + NH3 + disease/parasite zones."""
    for f in sim.fish:
        if not f.alive: continue
        if f.health <= 0 or f.oxygen <= 0:
            f.alive = False

            will_float = random.random() < DEAD_FISH_FLOAT_CHANCE
            ft = random.randint(*DEAD_FISH_FLOAT_DURATION_RANGE) if will_float else 0

            link_id = sim._new_obj_id()

            sim.objs.append(DynObj(f.x, f.y, f.z, 'dead_fish', f.body_size,
                                   max_age=DEAD_FISH_DECAY_TIMESTEPS,
                                   float_timer=ft,
                                   obj_id=link_id))

            sim.hazards.append(Hazard(f.x, f.y, f.z, DEAD_FISH_NH3_RADIUS, 'nh3',
                                      max_age=999999, follow_dead_fish=True, follow_id=link_id))

            if f.is_infected:
                sim.hazards.append(Hazard(f.x, f.y, f.z,
                                           f.body_size*INFECTED_FISH_DISEASE_RADIUS_MULT, 'disease',
                                           max_age=DISEASE_AREA_DECAY, is_floor=True))

            if f.has_parasite:
                sim.hazards.append(Hazard(f.x, f.y, f.z,
                                           f.body_size*INFECTED_FISH_DISEASE_RADIUS_MULT*0.8, 'parasite',
                                           max_age=PARASITE_AREA_DECAY, is_floor=True))

'''

# =====
# FILE 5: pond.py
# =====

POND_PY = r'''#!/usr/bin/env python3
'''
pond.py -- PondSim: the aquaculture simulation orchestrator (3D).
Delegates physics to physics.py and fish behavior to behavior.py.
'''

import random, math, copy

from constants import (
    POND_WIDTH, POND_HEIGHT, POND_DEPTH,
    W1_YIELD, W2_SAVING, W3_HEALTHINESS, MAX_FISH_COUNT,
    NUM_OBSTACLES, OBSTACLE_AREA_RANGE, OBSTACLE_ASPECT_RANGE,
    OBSTACLE_MAX_WIDTH, OBSTACLE_MAX_HEIGHT, OBSTACLE_MAX_DEPTH,
    OBSTACLE_DEPTH_RANGE,
    OXYGEN_BUBBLE_SPEED, NH3_BUBBLE_SPEED, NH3_AREA_RADIUS_RANGE,
    FOOD_PRICE, PROBIOTIC_PRICE, OXYGEN_PRICE,
    FOOD_VALUE, PROBIOTIC_VALUE,
    FOOD_EXPIRE_TIMESTEPS, PROBIOTIC_EXPIRE_TIMESTEPS,
```

```

NH3_EXPIRE_Timesteps,
NATURAL_OXYGEN_SPAWN_RATE, NATURAL_NH3_SPAWN_RATE, OXYGEN_BUBBLES_PER_PUMP,
SINK_SPEED,
SENSITIVE_DISTANCE, SOCIAL_DISTANCE,
FOOD_DRIFT_SPEED, PROBIOTIC_DRIFT_SPEED,
)
from entities import Obstacle, DynObj, Hazard
from helpers import _clamp, _drop_pos, SpatialGrid
from physics import upd_objs, upd_haz, stack_floor_objs
from behavior import decay, eat, pso, cannibal, fecal, death

_WALL = 2.0
_FLOOR_Z = POND_DEPTH - _WALL

class PondSim:
    __slots__ = ('geno', 'runtime', 'max_budget', 'record', 'fskip',
                 'fish', 'n0', 'ts', 'objs', 'hazards', 'obstacles',
                 'oxy_pump', 'frames', 'accum_cost', 'budget_exceeded',
                 'cannibal_events', '_fish_grid', '_obj_grid', '_next_obj_id')

    def __init__(self, geno, fish_templates, runtime, max_budget, record=False, fskip=1):
        self.geno = geno; self.runtime = runtime; self.max_budget = max_budget
        self.record = record; self.fskip = fskip
        self.fish = copy.deepcopy(fish_templates); self.n0 = len(self.fish)
        self.ts = 0; self.objs = []; self.hazards = []; self.obstacles = []
        self.oxy_pump = 0; self.frames = []; self.accum_cost = 0.0
        self.budget_exceeded = False; self.cannibal_events = []
        self._fish_grid = SpatialGrid(cell_size=max(SOCIAL_DISTANCE, SENSITIVE_DISTANCE))
        self._obj_grid = SpatialGrid(cell_size=SENSITIVE_DISTANCE)
        self._make_obs()
        self._next_obj_id = 0

    def _new_obj_id(self):
        self._next_obj_id += 1
        return self._next_obj_id

    def _make_obs(self):
        for _ in range(NUM_OBSTACLES):
            area = random.uniform(*OBSTACLE_AREA_RANGE)
            aspect = random.uniform(*OBSTACLE_ASPECT_RANGE)
            w = math.sqrt(area * aspect)
            h = area / w if w > 0 else math.sqrt(area)
            w = min(w, OBSTACLE_MAX_WIDTH); h = min(h, OBSTACLE_MAX_HEIGHT)
            d = random.uniform(*OBSTACLE_DEPTH_RANGE); d = min(d, OBSTACLE_MAX_DEPTH)
            x = random.uniform(5, POND_WIDTH - 5 - w)
            y = random.uniform(3, POND_HEIGHT - 3 - h)
            z = _FLOOR_Z - d
            self.obstacles.append(Obstacle(x, y, z, w, h, d))

    def _cp(self, x, y, z):
        return (_clamp(x, _WALL, POND_WIDTH - _WALL),
                _clamp(y, _WALL, POND_HEIGHT - _WALL),
                _clamp(z, _WALL, POND_DEPTH - _WALL))

    def _resting_z(self, ox, oy):
        best_z = _FLOOR_Z
        for obs in self.obstacles:
            if obs.xy_contains(ox, oy):
                top = obs.top_z()
                if top < best_z:
                    best_z = top
        return best_z

    def run(self):
        for t in range(self.runtime):
            self.ts = t; self.cannibal_events = []
            self._step()
            if self.record and t % self.fskip == 0:
                self.frames.append(self._frame())
            if self.budget_exceeded: break
            if not any(f.alive for f in self.fish): break
        alive = [f for f in self.fish if f.alive]
        sr = len(alive) / self.n0 if self.n0 else 0
        hlth = sum(f.norm_stats() for f in alive) / len(alive) if alive else 0
        cost = self.accum_cost
        saving = max(0, self.max_budget - cost) if not self.budget_exceeded else 0
        saving_ratio = saving / self.max_budget if self.max_budget > 0 else 0
        yld = len(alive) / MAX_FISH_COUNT
        fit = (W1_YIELD * yld + W2_SAVING * saving_ratio + W3_HEALTHINESS * hlth) if not self.budget_exceeded else 0

```

```

        return {'survival_rate': sr, 'avg_healthiness': hlth, 'saving': saving,
                'fitness': fit, 'cost': cost, 'alive_count': len(alive),
                'initial_count': self.n0, 'yield': yld, 'frames': self.frames,
                'genotype': self.geno.to_dict(), 'budget_exceeded': self.budget_exceeded}

def _step(self):
    t = self.ts
    self._spawn_food(t); self._spawn_prob(t); self._pump_oxy(t); self._nat_spawn()
    upd_objs(self); upd_haz(self)
    decay(self); eat(self); pso(self); cannibal(self); fecal(self); death(self)
    stack_floor_objs(self)

def _add_cost(self, a):
    self.accum_cost += a
    if self.accum_cost > self.max_budget: self.budget_exceeded = True

def _spawn_food(self, t):
    if t % self.geno.food_interval == 0:
        self._add_cost(self.geno.food_quantity * FOOD_PRICE)
        if self.budget_exceeded: return
        for _ in range(self.geno.food_quantity):
            x, y = _drop_pos(self.geno.food_location)
            self.objs.append(DynObj(x, y, _WALL, 'food', FOOD_VALUE,
                                    max_age=FOOD_EXPIRE_TIMESTEPS,
                                    vx=random.uniform(-FOOD_DRIFT_SPEED, FOOD_DRIFT_SPEED),
                                    vy=random.uniform(-FOOD_DRIFT_SPEED, FOOD_DRIFT_SPEED),
                                    vz=SINK_SPEED))

def _spawn_prob(self, t):
    if t % self.geno.probiotic_interval == 0:
        self._add_cost(self.geno.probiotic_quantity * PROBIOTIC_PRICE)
        if self.budget_exceeded: return
        for _ in range(self.geno.probiotic_quantity):
            x, y = _drop_pos(self.geno.probiotic_location)
            self.objs.append(DynObj(x, y, _WALL, 'probiotic', PROBIOTIC_VALUE,
                                    max_age=PROBIOTIC_EXPIRE_TIMESTEPS,
                                    vx=random.uniform(-PROBIOTIC_DRIFT_SPEED, PROBIOTIC_DRIFT_SPEED),
                                    vy=random.uniform(-PROBIOTIC_DRIFT_SPEED, PROBIOTIC_DRIFT_SPEED),
                                    vz=SINK_SPEED))

def _pump_oxy(self, t):
    if t % self.geno.oxygen_interval == 0:
        self.oxy_pump = self.geno.oxygen_duration
    if self.oxy_pump > 0:
        self._add_cost(OXYGEN_PRICE)
        if self.budget_exceeded: return
        for _ in range(OXYGEN_BUBBLES_PER_PUMP):
            x, y = _drop_pos(self.geno.oxygen_location)
            z = random.uniform(_WALL, POND_DEPTH * 0.5)
            self.objs.append(DynObj(x, y, z, 'oxygen', 1.0, max_age=99999,
                                    vx=random.uniform(-OXYGEN_BUBBLE_SPEED, OXYGEN_BUBBLE_SPEED),
                                    vy=random.uniform(-OXYGEN_BUBBLE_SPEED, OXYGEN_BUBBLE_SPEED),
                                    vz=random.uniform(-OXYGEN_BUBBLE_SPEED*0.5, OXYGEN_BUBBLE_SPEED*0.5)))
        self.oxy_pump -= 1

def _nat_spawn(self):
    if random.random() < NATURAL_OXYGEN_SPAWN_RATE:
        x, y = random.uniform(10, POND_WIDTH-10), random.uniform(5, POND_HEIGHT-5)
        z = random.uniform(5, POND_DEPTH-5)
        self.objs.append(DynObj(x, y, z, 'oxygen', 1.0, max_age=99999,
                                vx=random.uniform(-OXYGEN_BUBBLE_SPEED, OXYGEN_BUBBLE_SPEED),
                                vy=random.uniform(-OXYGEN_BUBBLE_SPEED, OXYGEN_BUBBLE_SPEED),
                                vz=random.uniform(-OXYGEN_BUBBLE_SPEED*0.5, OXYGEN_BUBBLE_SPEED*0.5)))
    if random.random() < NATURAL_NH3_SPAWN_RATE:
        x, y = random.uniform(10, POND_WIDTH-10), random.uniform(5, POND_HEIGHT-5)
        z = random.uniform(5, POND_DEPTH-5)
        self.hazards.append(Hazard(x, y, z, random.uniform(*NH3_AREA_RADIUS_RANGE), 'nh3',
                                max_age=NH3_EXPIRE_TIMESTEPS,
                                vx=random.uniform(-NH3_BUBBLE_SPEED, NH3_BUBBLE_SPEED),
                                vy=random.uniform(-NH3_BUBBLE_SPEED, NH3_BUBBLE_SPEED),
                                vz=random.uniform(-NH3_BUBBLE_SPEED*0.5, NH3_BUBBLE_SPEED*0.5)))

def _frame(self):
    alive = [f for f in self.fish if f.alive]
    return {
        't': self.ts, 'day': self.ts//24, 'hour': self.ts%24,
        'fish': [f.snapshot() for f in alive],
        'objects': [{ 'x':round(o.x,1), 'y':round(o.y,1), 'z':round(o.z,1),
                      'type':o.kind, 'value':round(o.value,1)} for o in self.objs if o.alive],
        'hazards': [{ 'x':round(h.x,1), 'y':round(h.y,1), 'z':round(h.z,1),
                      'r':round(h.radius,1), 'type':h.kind, 'is_floor':h.is_floor}
    ]

```

```
        for h in self.hazards],
        'obstacles': [{ 'x':round(o.x,1), 'y':round(o.y,1), 'z':round(o.z,1),
                        'w':round(o.w,1), 'h':round(o.h,1), 'd':round(o.d,1)}
                      for o in self.obstacles],
        'alive_count': len(alive), 'total_count': self.n0,
        'cannibal_events': list(self.cannibal_events)}

'''

# =====
# FILE 6: simulate.py
# =====

SIMULATE_PY = r'''#!/usr/bin/env python3
'''
simulate.py -- Run a single pond simulation.
This is the interface point for any decision-maker (EA, LLM, etc.).
'''

import random

from constants import MAX_BUDGET, FRAME_SKIP, RUNTIME
from entities import PondGenotype
from helpers import _make_fish
from pond import PondSim

def run_single_pond(geno_dict, runtime=None, max_budget=None,
                   record=False, frame_skip=None, seed=None):
    """Run one pond simulation from a genotype dictionary. Returns result dict."""
    if seed is not None:
        random.seed(seed)
    if runtime is None:
        runtime = RUNTIME
    if max_budget is None:
        max_budget = MAX_BUDGET
    if frame_skip is None:
        frame_skip = FRAME_SKIP

    geno = PondGenotype(**geno_dict)
    fishes = [_make_fish(i) for i in range(geno.fish_count)]
    sim = PondSim(geno, fishes, runtime, max_budget, record=record, fskip=frame_skip)
    r = sim.run()
    r['genotype'] = geno_dict
    return r

'''

# =====
# WRITE ALL FILES
# =====

files = {
    "entities.py": ENTITIES_PY,
    "helpers.py": HELPERS_PY,
    "physics.py": PHYSICS_PY,
    "behavior.py": BEHAVIOR_PY,
    "pond.py": POND_PY,
    "simulate.py": SIMULATE_PY,
}

print("\n Writing 6 core files to {}/core/".format(BASE))
for filename, content in files.items():
    path = f"{BASE}/core/{filename}"
    with open(path, "w") as f:
        f.write(content.strip() + "\n")
    size = os.path.getsize(path)
    print(f"    {filename} ({size:,} bytes)")

print(f"\n All 6 core files written. No upload needed!")
```

- Writing 6 core files to /content/AquaLLM/core/
 - entities.py (5,377 bytes)
 - helpers.py (4,395 bytes)
 - physics.py (9,213 bytes)
 - behavior.py (16,615 bytes)
 - pond.py (9,012 bytes)
 - simulate.py (979 bytes)

□ All 6 core files written. No upload needed!

CELL 3: constants.py

In []:

```
# @title
%%writefile /content/AquaLLM/core/constants.py
#!/usr/bin/env python3
"""
constants.py -- All configuration constants for the aquaculture simulation.
AquaLLM version: 5-day simulation, adaptive architecture.
"""

from enum import IntEnum
from pathlib import Path
import math

# =====
# PROJECT PATHS
# =====

PROJECT_ROOT = Path(__file__).resolve().parent.parent
RESULTS_CSV_PATH = str(PROJECT_ROOT / 'logs' / 'results.csv')
SIMULATION_JSON_PATH = str(PROJECT_ROOT / 'logs' / 'simulation_data.json')
PLOTS_DIR = str(PROJECT_ROOT / 'plots')
NUM_WORKERS = None

# =====
# SIMULATION CONFIGURATION
# =====

MAX_BUDGET = 500.0
AQUACULTURE_DAYS = 30
FRAME_SKIP = 1

# =====
# EA POLICY RANGES
# =====

FOOD_QUANTITY_RANGE = (5, 30)
FOOD_INTERVAL_RANGE = (1, 12)
PROBIOTIC_QUANTITY_RANGE = (1, 4)
PROBIOTIC_INTERVAL_STEPS = list(range(24, 169, 12))
OXYGEN_DURATION_RANGE = (1, 4)
OXYGEN_INTERVAL_RANGE = (1, 20)

# =====
# FITNESS WEIGHTS
# =====

W1_YIELD = 0.55
W2_SAVING = 0.33
W3_HEALTHINESS = 0.12

# =====
# POND DIMENSIONS
# =====

POND_WIDTH = 200.0
POND_HEIGHT = 75.0
POND_DEPTH = 50.0

# =====
# FISH POPULATION
# =====

FISH_DENSITY_K = 0.55
MAX_FISH_COUNT = max(10, int(FISH_DENSITY_K * (POND_WIDTH * POND_HEIGHT * POND_DEPTH) ** (1/3)))

# =====
# FISH TRAITS (FIXED AT BIRTH)
# =====

MOUTH_SIZE_RANGE = (3.0, 8.0)
BODY_SIZE_RANGE = (4.0, 10.0)
VELOCITY_RANGE = (4.0, 10.0)

# =====
# FISH STATS (DYNAMIC)
# =====

HEALTH_RANGE = (80.0, 100.0)
FULLNESS_RANGE = (50.0, 100.0)
```

```
IMMUNITY_RANGE = (70.0, 100.0)
OXYGEN_RANGE = (80.0, 100.0)

# //          STAT DECAY (PER TIMESTEP)          //
# //
#

OXYGEN_DECAY = 0.22
OXYGEN_DECAY_NH3_MULT = 3.0
OXYGEN_PASSIVE_REGEN = 0.03
FULLNESS_DECAY = 0.05
FULLNESS_COST_MOVE = 0.10

# //          HEALTH DECAY & REGEN          //
# //
#

HEALTH_DECAY_NO_FULLNESS = 0.8
HEALTH_DECAY_INFECTED = 0.5
HEALTH_DECAY_PARASITE = 0.7
HEALTH_DECAY_IN_NH3 = 0.3
HEALTH_REGEN = 0.02

# //          VELOCITY REDUCTION          //
# //
#

VELOCITY_HEALTH_THRESHOLD = 0.40
VELOCITY_FULLNESS_THRESHOLD = 0.33

# //          DISEASE & PARASITE          //
# //
#

IMMUNITY_DECAY_IN_DISEASE = 18.0
IMMUNITY_DECAY_IN_NH3 = 6.0
IMMUNITY_REGEN = 0.02
DISEASE_SELF_CURE_CHANCE = 0.05
PARASITE_CONTACT_CHANCE = 0.35
PARASITE_FULLNESS_EFFICIENCY = 0.5
PARASITE_EXTRA_FULLNESS_DRAIN = 1.0
PARASITE_VELOCITY_MULT = 0.75

# //          FOOD & CONSUMABLES          //
# //
#

FOOD_VALUE = 5.0
FOOD_FULLNESS_GAIN = 10.0
FOOD_DRIFT_SPEED = 0.15

PROBIOTIC_VALUE = 3.0
PROBIOTIC_IMMUNITY_GAIN = 40.0
PROBIOTIC_DRIFT_SPEED = 0.12

OXYGEN_BUBBLE_GAIN = 20.0
OXYGEN_BUBBLE_SPEED = 0.5
OXYGEN_BUBBLES_PER_PUMP = 10

# //          ECONOMIC (PRICES)          //
# //
#

FOOD_PRICE = 0.05
PROBIOTIC_PRICE = 0.75
OXYGEN_PRICE = 2.50

# //          OBJECT LIFETIMES (TIMESTEPS)          //
# //
#

FOOD_EXPIRE_TIMESTEPS = 48
PROBIOTIC_EXPIRE_TIMESTEPS = 18
FECAL_EXPIRE_TIMESTEPS = 36
DEAD_FISH_DECAY_TIMESTEPS = 24
NH3_EXPIRE_TIMESTEPS = 60
DISEASE_AREA_DECAY = 36
PARASITE_AREA_DECAY = 36
POLLUTANT_TO_HAZARD_TIMESTEPS = 48
DISEASE_AREA_RADIUS_DECAY = 0.990
```


#

SINKING & FLOATING

SINK_SPEED = 0.3
SINK_SPEED_HEAVY = 0.6

#

FECAL

FECAL_DROP_INTERVAL = 3
FECAL_BASE_CHANCE = 0.05
FECAL_VALUE = 3.0
FECAL_SINK_SPEED = 0.4
MAX_SINKING_FECAL = 100

#

DEAD FISH

DEAD_FISH_NH3_RADIUS = 12.0
DEAD_FISH_FLOAT_CHANCE = 0.45
DEAD_FISH_FLOAT_DURATION_RANGE = (12, 72)
DEAD_FISH_FLOAT_SPEED = 0.4
DEAD_FISH_POLLUTANT_MULT = 3.0
INFECTED_FISH_DISEASE_RADIUS_MULT = 5.0

#

CANNIBALISM

CANNIBAL_FULLNESS_THRESHOLD = 0.7
CANNIBAL_BASE_CHANCE = 0.25
CANNIBAL_HUNGER_MULT = 0.80
CANNIBAL_FULLNESS_GAIN_MULT = 3.0
CANNIBAL_COLLISION_RADIUS_MULT = 1.2

#

POLLUTANT TRANSFORMATION

POLLUTANT_TO_NH3_CHANCE = 0.30
POLLUTANT_TO_DISEASE_CHANCE = 0.10
POLLUTANT_TO_PARASITE_CHANCE = 0.10
POLLUTANT_TO_BOTH_CHANCE = 0.05
POLLUTANT_TO_PLANT_CHANCE = 0.10
POLLUTANT_TO_OBSTACLE_CHANCE = 0.20
POLLUTANT_TO_NOTHING_CHANCE = 0.15
POLLUTANT_RADIUS_SCALE = 1.5
POLLUTANT_OBSTACLE_AREA_RANGE = (3, 8)

#

NH3 HAZARD

NH3_BUBBLE_SPEED = 0.4
NH3_AREA_RADIUS_RANGE = (6.0, 14.0)

#

NATURAL SPAWN RATES

NATURAL_OXYGEN_SPAWN_RATE = 0.12
NATURAL_NH3_SPAWN_RATE = 0.04

#

OBSTACLES

OBSTACLE_DENSITY_FACTOR = 0.15
OBSTACLE_MAX_DIM_FRAC = 0.10
OBSTACLE_AREA_RANGE = (30, 150)
OBSTACLE_ASPECT_RANGE = (0.3, 3.0)
OBSTACLE_DEPTH_RANGE = (5.0, 15.0)

#

FLOOR MECHANICS

FLOOR_HAZARD_HEIGHT = 15

```
STACK_RADIUS = 5.0

# =====
# PSO DISTANCE RADII
# =====

SENSITIVE_DISTANCE = 30.0
SOCIAL_DISTANCE = 60.0
SELFISH_DISTANCE = 10.0
FISH_EAT_RANGE = 3.0

# =====
# PSO VECTOR WEIGHTS
# =====

PSO_INERTIA = 0.4

PSO_FOOD_WEIGHT = 3.0
PSO_FOOD_URGENT_MULT = 2.5
PSO_FOOD_URGENT_THRESHOLD = 0.5

PSO_PROBIOTIC_WEIGHT = 1.8

PSO_OXYGEN_WEIGHT = 2.2
PSO_OXYGEN_CRITICAL_MULT = 2.0
PSO_OXYGEN_THRESHOLD = 0.7
PSO_OXYGEN_CRITICAL_THRESHOLD = 0.3
PSO_OXYGEN_INTERCEPT_STEPS = 3

PSO_SOCIAL_WEIGHT = 2.0
PSO_SELFISH_WEIGHT = 1.5

PSO_NH3_WEIGHT = 3.5
PSO_NH3_HUNGRY_OVERRIDE = 0.5
PSO_DISEASE_WEIGHT = 2.2
PSO_PARASITE_WEIGHT = 2.5

PSO_RUN_WEIGHT = 5.0
PSO_OBSTACLE_WEIGHT = 2.0

PSO_SWARM_HUNGRY_WEIGHT = 3.5
PSO_SWARM_HUNGRY_THRESHOLD = 0.7

# =====
# DROP LOCATIONS
# =====

class DropLocation(IntEnum):
    CENTER = 0
    TOP_LEFT = 1
    TOP_RIGHT = 2
    BOT_LEFT = 3
    BOT_RIGHT = 4
    TOP_CENTER = 5
    BOT_CENTER = 6
    LEFT_CENTER = 7
    RIGHT_CENTER = 8
    RANDOM = 9

LOC_NAMES = {
    0: 'Center', 1: 'Top-Left', 2: 'Top-Right', 3: 'Bot-Left', 4: 'Bot-Right',
    5: 'Top-Center', 6: 'Bot-Center', 7: 'Left-Center', 8: 'Right-Center', 9: 'Random'
}

# =====
# DERIVED CONSTANTS (do not edit directly)
# =====

RUNTIME = AQUACULTURE_DAYS * 24

OBSTACLE_MAX_WIDTH = POND_WIDTH * OBSTACLE_MAX_DIM_FRAC
OBSTACLE_MAX_HEIGHT = POND_HEIGHT * OBSTACLE_MAX_DIM_FRAC
OBSTACLE_MAX_DEPTH = POND_DEPTH * OBSTACLE_MAX_DIM_FRAC

NUM_OBSTACLES = max(5, round(math.sqrt(POND_WIDTH * POND_HEIGHT) * OBSTACLE_DENSITY_FACTOR))
```

Overwriting /content/AquaLLM/core/constants.py

CELL 4: simulate_adaptive.py

In []:

```
# @title
%%writefile /content/AquaLLM/core/simulate_adaptive.py
#!/usr/bin/env python3
"""
simulate_adaptive.py -- Adaptive simulation interface.
v3: Added raw hazard data and stat ratios to snapshot for observer.
"""

import random, copy

from constants import (
    MAX_BUDGET, FRAME_SKIP, RUNTIME, AQUACULTURE_DAYS,
    POND_WIDTH, POND_HEIGHT, POND_DEPTH,
    W1_YIELD, W2_SAVING, W3_HEALTHINESS, MAX_FISH_COUNT,
)
from entities import PondGenotype
from helpers import _make_fish
from pond import PondSim

_GRID_COLS = [0, 200/3, 400/3, 200]
_GRID_ROWS = [0, 25, 50, 75]

_ZONE_NAMES = [
    ["Top-Left(1)", "Top-Center(5)", "Top-Right(2)"],
    ["Left-Ctr(7)", "Center(0)", "Right-Ctr(8)"],
    ["Bot-Left(3)", "Bot-Center(6)", "Bot-Right(4)"],
]

def _get_zone(x, y):
    col = 0 if x < _GRID_COLS[1] else (1 if x < _GRID_COLS[2] else 2)
    row = 0 if y < _GRID_ROWS[1] else (1 if y < _GRID_ROWS[2] else 2)
    return row, col

def _count_grid(items_xy):
    grid = [[0]*3 for _ in range(3)]
    for x, y in items_xy:
        r, c = _get_zone(x, y)
        grid[r][c] += 1
    return grid

def take_snapshot(sim, day):
    alive = [f for f in sim.fish if f.alive]
    dead_objs = [o for o in sim.objs if o.alive and o.kind == 'dead_fish']
    n_alive = len(alive)
    n_initial = sim.n0
    n_dead = n_initial - n_alive

    fish_grid = _count_grid([(f.x, f.y) for f in alive])

    if n_alive > 0:
        avg_health_r = sum(f.health / f.max_health for f in alive) / n_alive
        avg_fullness_r = sum(f.fullness / f.max_fullness for f in alive) / n_alive
        avg_oxygen_r = sum(f.oxygen / f.max_oxygen for f in alive) / n_alive
        avg_immunity_r = sum(f.immunity / f.max_immunity for f in alive) / n_alive
        avg_healthiness = sum(f.norm_stats() for f in alive) / n_alive
        n_infected = sum(1 for f in alive if f.is_infected)
        n_parasitized = sum(1 for f in alive if f.has_parasite)
    else:
        avg_health_r = 0
        avg_fullness_r = 0
        avg_oxygen_r = 0
        avg_immunity_r = 0
        avg_healthiness = 0
        n_infected = 0
        n_parasitized = 0

    if n_alive == 0:
        health_impression = "All fish are dead."
    elif avg_health_r > 0.7 and avg_fullness_r > 0.5:
        health_impression = "Fish look healthy and well-fed."
    elif avg_health_r > 0.4:
        health_impression = "Fish look moderately healthy."
    elif avg_health_r > 0.2:
        health_impression = "Fish look stressed."
    else:
        health_impression = "Fish are in critical condition."
```

```

if n_alive > 0:
    if avg_fullness_r > 0.6: fullness_impression = "Fish appear well-fed."
    elif avg_fullness_r > 0.3: fullness_impression = "Some fish look hungry."
    else: fullness_impression = "Fish are clearly starving."
else:
    fullness_impression = ""

if n_alive > 0:
    if avg_oxygen_r > 0.6: oxygen_impression = "Fish are breathing normally."
    elif avg_oxygen_r > 0.3: oxygen_impression = "Some fish are gasping."
    else: oxygen_impression = "Fish are suffocating."
else:
    oxygen_impression = ""

hazard_zones = {"nh3": [], "disease": [], "parasite": []}
raw_hazards = []
for h in sim.hazards:
    if not h.alive:
        continue
    r, c = _get_zone(h.x, h.y)
    zone_name = _ZONE_NAMES[r][c]
    if zone_name not in hazard_zones.get(h.kind, []):
        hazard_zones.setdefault(h.kind, []).append(zone_name)
    raw_hazards.append({"kind": h.kind, "radius": h.radius, "x": h.x, "y": h.y})

dead_fish_grid = _count_grid([(o.x, o.y) for o in dead_objs])
n_pollutants = sum(1 for o in sim.objs if o.alive and o.kind == 'pollutant')

cost_so_far = sim.accum_cost
budget_remaining = max(0, sim.max_budget - cost_so_far)

survival_rate = n_alive / n_initial if n_initial > 0 else 0
yld = n_alive / MAX_FISH_COUNT
saving = max(0, sim.max_budget - cost_so_far) if not sim.budget_exceeded else 0
saving_ratio = saving / sim.max_budget if sim.max_budget > 0 else 0
fitness = (W1_YIELD * yld + W2_SAVING * saving_ratio + W3_HEALTHINESS * avg_healthiness) if not sim.budget_exceeded else 0

geno = sim.geno.to_dict()

return {
    "day": day,
    "alive_count": n_alive,
    "initial_count": n_initial,
    "deaths_total": n_dead,
    "fish_grid": fish_grid,
    "health_impression": health_impression,
    "fullness_impression": fullness_impression,
    "oxygen_impression": oxygen_impression,
    "avg_healthiness": avg_healthiness,
    "n_infected": n_infected,
    "n_parasitized": n_parasitized,
    "hazard_zones": hazard_zones,
    "dead_fish_grid": dead_fish_grid,
    "n_pollutants": n_pollutants,
    "cost_so_far": cost_so_far,
    "budget_remaining": budget_remaining,
    "budget_exceeded": sim.budget_exceeded,
    "genotype": geno,
    "fitness": fitness,
    "yield": yld,
    "saving": saving,
    "saving_ratio": saving_ratio,
    "survival_rate": survival_rate,
    # Internal data for observer
    "_avg_health_ratio": avg_health_r,
    "_avg_fullness_ratio": avg_fullness_r,
    "_avg_oxygen_ratio": avg_oxygen_r,
    "_avg_immunity_ratio": avg_immunity_r,
    "_raw_hazards": raw_hazards,
}

```

```

class AdaptivePondSim:

```

```

    def __init__(self, initial_geno_dict, max_budget=None, seed=None):
        if seed is not None:
            random.seed(seed)
        if max_budget is None:
            max_budget = MAX_BUDGET

```

```

self.geno = PondGenotype(**initial_geno_dict)
self.fish_templates = [_make_fish(i) for i in range(self.geno.fish_count)]
self.max_budget = max_budget
self.total_runtime = RUNTIME

self.sim = PondSim(
    self.geno, self.fish_templates, self.total_runtime,
    max_budget, record=True, fskip=FRAME_SKIP,
)
self.current_ts = 0
self.prev_alive = self.sim.n0

def run_chunk(self, n_timesteps=24):
    steps_run = 0
    for _ in range(n_timesteps):
        if self.current_ts >= self.total_runtime:
            break
        if self.sim.budget_exceeded:
            break
        if not any(f.alive for f in self.sim.fish):
            break
        self.sim.ts = self.current_ts
        self.sim.cannibal_events = []
        self.sim._step()
        if self.sim.record and self.current_ts % self.sim.fskip == 0:
            self.sim.frames.append(self.sim._frame())
        self.current_ts += 1
        steps_run += 1
    return steps_run

def update_policy(self, policy_dict):
    for key, val in policy_dict.items():
        if key == "fish_count":
            continue
        if hasattr(self.sim.geno, key):
            setattr(self.sim.geno, key, val)

def snapshot(self, day):
    deaths_this_day = self.prev_alive - sum(1 for f in self.sim.fish if f.alive)
    snap = take_snapshot(self.sim, day)
    snap["deaths_today"] = max(0, deaths_this_day)
    self.prev_alive = snap["alive_count"]
    return snap

def final_result(self):
    alive = [f for f in self.sim.fish if f.alive]
    sr = len(alive) / self.sim.n0 if self.sim.n0 else 0
    hlth = sum(f.norm_stats() for f in alive) / len(alive) if alive else 0
    cost = self.sim.accum_cost
    saving = max(0, self.max_budget - cost) if not self.sim.budget_exceeded else 0
    saving_ratio = saving / self.max_budget if self.max_budget > 0 else 0
    yld = len(alive) / MAX_FISH_COUNT
    fit = (W1_YIELD * yld + W2_SAVING * saving_ratio + W3_HEALTHINESS * hlth) if not self.sim.budget_exceeded else 0
    return {
        "fitness": fit, "survival_rate": sr, "avg_healthiness": hlth,
        "saving": saving, "cost": cost, "alive_count": len(alive),
        "initial_count": self.sim.n0, "yield": yld, "saving_ratio": saving_ratio,
        "budget_exceeded": self.sim.budget_exceeded,
        "genotype": self.sim.geno.to_dict(), "frames": self.sim.frames,
    }

def is_done(self):
    if self.current_ts >= self.total_runtime: return True
    if self.sim.budget_exceeded: return True
    if not any(f.alive for f in self.sim.fish): return True
    return False

```

Overwriting /content/AquaLLM/core/simulate_adaptive.py

CELL 5: agent/agent_config.py

In []:

```

# @title
%%writefile /content/AquaLLM/agent/agent_config.py
#!/usr/bin/env python3
"""
agent_config.py -- Configuration for ablation studies.

```

Each boolean toggles a component of the LLM agent's "brain".

```
"""

from dataclasses import dataclass

@dataclass
class AgentConfig:
    """Controls which cognitive modules are active in the LLM agent."""

    use_observer: bool = True
    """True = Natural language pond summary. False = Raw JSON state dump."""

    use_memory: bool = True
    """True = Keep history of past rounds. False = Amnesiac (current round only)."""

    use_reflection: bool = True
    """True = Generate periodic high-level insights. False = Raw memory only."""

    use_calculator: bool = True
    """True = LLM can call budget calculator tool. False = LLM must guess math."""

    use_planning: bool = True
    """True = Plan-and-Execute (CoT reasoning first). False = Zero-shot JSON output."""

    def label(self) -> str:
        """Short string label for this config (for logging/filenames)."""
        flags = []
        if self.use_observer: flags.append("Obs")
        if self.use_memory: flags.append("Mem")
        if self.use_reflection: flags.append("Ref")
        if self.use_calculator: flags.append("Tool")
        if self.use_planning: flags.append("Plan")
        return "+".join(flags) if flags else "Minimal"

# =====
# PRESET CONFIGURATIONS FOR ABLATION STUDIES
# =====

FULL_AGENT = AgentConfig(
    use_observer=True, use_memory=True, use_reflection=True,
    use_calculator=True, use_planning=True,
)

NO_PLANNING = AgentConfig(
    use_observer=True, use_memory=True, use_reflection=True,
    use_calculator=True, use_planning=False,
)

NO_MEMORY = AgentConfig(
    use_observer=True, use_memory=False, use_reflection=False,
    use_calculator=True, use_planning=True,
)

NO_TOOLS = AgentConfig(
    use_observer=True, use_memory=True, use_reflection=True,
    use_calculator=False, use_planning=True,
)

RAW_STATE = AgentConfig(
    use_observer=False, use_memory=True, use_reflection=False,
    use_calculator=True, use_planning=True,
)

MINIMAL = AgentConfig(
    use_observer=False, use_memory=False, use_reflection=False,
    use_calculator=False, use_planning=False,
)

ALL_CONFIGS = {
    "full": FULL_AGENT,
    "no_planning": NO_PLANNING,
    "no_memory": NO_MEMORY,
    "no_tools": NO_TOOLS,
    "no_obs": RAW_STATE,
    "minimal": MINIMAL,
}
```

Overwriting /content/AquaLLM/agent/agent_config.py

CELL 6: agent/schemas.py

In []:

```
# @title
%%writefile /content/AquaLLM/agent/schemas.py
#!/usr/bin/env python3
"""
schemas.py -- Pydantic models for the LLM's structured output.
v2: Added early_quit field for strategic termination.
"""

from pydantic import BaseModel, Field

class PondPolicy(BaseModel):
    """The aquaculture management policy the LLM agent must output."""

    fish_count: int = Field(ge=10, le=50, description="Number of fish to stock (10-50)")
    food_interval: int = Field(ge=1, le=12, description="Hours between food drops (1-12)")
    food_quantity: int = Field(ge=5, le=30, description="Pellets per food drop (5-30)")
    food_location: int = Field(ge=0, le=9, description="Drop zone ID (0=Center, 9=Random)")
    probiotic_interval: int = Field(ge=24, le=168, description="Hours between probiotic drops (24-168)")
    probiotic_quantity: int = Field(ge=1, le=4, description="Pellets per probiotic drop (1-4)")
    probiotic_location: int = Field(ge=0, le=9, description="Drop zone ID (0=Center, 9=Random)")
    oxygen_interval: int = Field(ge=1, le=20, description="Hours between O2 pump activations (1-20)")
    oxygen_duration: int = Field(ge=1, le=4, description="Hours O2 pump stays on per activation (1-4)")
    oxygen_location: int = Field(ge=0, le=9, description="Drop zone ID (0=Center, 9=Random)")
    early_quit: bool = Field(default=False, description="Set true to abandon this generation and start fresh")

    def to_genotype_dict(self) -> dict:
        """Convert to the dict format expected by run_single_pond()."""
        d = self.model_dump()
        d.pop("early_quit", None)
        return d
```

Overwriting /content/AquaLLM/agent/schemas.py

CELL 7: agent/llm_client.py

In []:

```
# @title
%%writefile /content/AquaLLM/agent/llm_client.py
#!/usr/bin/env python3
"""
llm_client.py -- Unified LLM client for local Ollama.
v5: Returns (response_text, metadata) tuple with tool call tracking.
"""

import json
import requests
from typing import Optional, Tuple, List, Dict

OLLAMA_URL = "http://localhost:11434/api/chat"
DEFAULT_MODEL = "qwen2.5:7b"

_JSON_REMINDER = (
    "You have checked the budget. Now output your FINAL policy as a JSON object. "
    "Output ONLY the JSON, nothing else."
)

def chat(
    messages,
    tools=None,
    tool_dispatcher=None,
    model=DEFAULT_MODEL,
    temperature=0.3,
    max_tool_rounds=5,
) -> Tuple[str, Dict]:
    """
    Send messages to Ollama. Returns (response_text, metadata).
    metadata = {"tool_calls": [{"name": ..., "args": ..., "result": ...}, ...]}
    """
    if tool_dispatcher is None:
```

```

tool_dispatcher = {}

working_messages = list(messages)
used_tools = False
meta = {"tool_calls": []}

for round_idx in range(max_tool_rounds):
    include_tools = tools if (tools and round_idx < max_tool_rounds - 1) else None

    payload = {
        "model": model,
        "messages": working_messages,
        "stream": False,
        "options": {"temperature": temperature},
    }
    if include_tools:
        payload["tools"] = include_tools

    try:
        resp = requests.post(OLLAMA_URL, json=payload, timeout=180)
        resp.raise_for_status()
    except requests.exceptions.RequestException as e:
        return "[LLM_ERROR] Request failed: {}".format(e), meta

    data = resp.json()
    msg = data.get("message", {})

    tool_calls = msg.get("tool_calls")
    if tool_calls and include_tools:
        working_messages.append(msg)
        used_tools = True

    for tc in tool_calls:
        fn_name = tc["function"]["name"]
        fn_args = tc["function"]["arguments"]

        if fn_name in tool_dispatcher:
            try:
                result = tool_dispatcher[fn_name](**fn_args)
                result_str = json.dumps(result) if not isinstance(result, str) else result
            except Exception as e:
                result_str = "[TOOL_ERROR] {}: {}".format(fn_name, e)
                result = str(e)
            else:
                result_str = "[TOOL_ERROR] Unknown function: {}".format(fn_name)
                result = "unknown function"

            meta["tool_calls"].append({
                "name": fn_name,
                "args": fn_args,
                "result": result if isinstance(result, (dict, list)) else result_str,
            })

            working_messages.append({
                "role": "tool",
                "content": result_str,
            })
            continue

    content = msg.get("content", "")
    if content.strip():
        return content, meta

    if used_tools:
        working_messages.append(msg)
        working_messages.append({
            "role": "user",
            "content": _JSON_REMINDER,
        })
        payload2 = {
            "model": model,
            "messages": working_messages,
            "stream": False,
            "options": {"temperature": temperature},
        }
        try:
            resp2 = requests.post(OLLAMA_URL, json=payload2, timeout=180)
            resp2.raise_for_status()
            data2 = resp2.json()
            return data2.get("message", {}).get("content", ""), meta
        except:

```

```

        return "[LLM_ERROR] Final request failed", meta

    return content, meta

working_messages.append({
    "role": "user",
    "content": _JSON_REMINDER,
})
payload_final = {
    "model": model,
    "messages": working_messages,
    "stream": False,
    "options": {"temperature": temperature},
}
try:
    resp_final = requests.post(OLLAMA_URL, json=payload_final, timeout=180)
    resp_final.raise_for_status()
    data_final = resp_final.json()
    return data_final.get("message", {}).get("content", ""), meta
except:
    return "[LLM_ERROR] Final request failed", meta

```

Overwriting /content/AquaLLM/agent/llm_client.py

CELL 8: agent/observer.py

In []:

```

# @title
%%writefile /content/AquaLLM/agent/observer.py
"""
observer.py -- Farmer's perception module.
v7: Description-only. No prescriptive ACTION strings (moved to agent system prompt).
"""

import json
import sys
sys.path.insert(0, "/content/AquaLLM/core")
from constants import AQUACULTURE_DAYS, MAX_BUDGET

_ZONE = [
    ["TL(1)", "TC(5)", "TR(2)"],
    ["LC(7)", "C(0)", "RC(8)"],
    ["BL(3)", "BC(6)", "BR(4)"],
]

def observe(snapshot, use_observer):
    return _farmer_view(snapshot) if use_observer else _raw_json(snapshot)

def compute_warning_points(snapshot):
    pts, bd = 0, []
    for key, mult, label in [
        ("deaths_today", 10, "dead"),
        ("n_infected", 5, "inf"),
        ("n_parasitized", 4, "par"),
        ("n_pollutants", 1, "poll"),
    ]:
        n = snapshot.get(key, 0)
        if n > 0:
            p = n * mult
            pts += p
            bd.append("{}x{}={}".format(n, label, p))
    for h in snapshot.get("_raw_hazards", []):
        r = h.get("radius", 0)
        p = 3 if r > 10 else (2 if r >= 4 else 1)
        pts += p
        bd.append("{}={}".format(h["kind"][:3], p))
    return pts, bd

def _zone_name(r, c):
    return _ZONE[r][c]

def _get_zone(x, y):
    col = 0 if x < 66.7 else (1 if x < 133.3 else 2)
    row = 0 if y < 25 else (1 if y < 50 else 2)

```

```

return row, col

def _grid_summary(items_xy, label):
    grid = {}
    for x, y in items_xy:
        r, c = _get_zone(x, y)
        z = _zone_name(r, c)
        grid[z] = grid.get(z, 0) + 1
    if not grid:
        return ""
    return "{}: {}".format(label, " ".join("{}:{}".format(k, v) for k, v in grid.items()))

def _farmer_view(s):
    day = s.get("day", 0)
    alive = s.get("alive_count", 0)
    init = s.get("initial_count", 0)
    dt = s.get("deaths_today", 0)
    cost = s.get("cost_so_far", 0)
    left = s.get("budget_remaining", 0)
    fit = s.get("fitness", 0)
    yld = s.get("yield", 0)
    sav = s.get("saving_ratio", 0)
    hlth = s.get("avg_healthiness", 0)

    hr = s.get("_avg_health_ratio", 0)
    fr = s.get("_avg_fullness_ratio", 0)
    oxr = s.get("_avg_oxygen_ratio", 0)
    ir = s.get("_avg_immunity_ratio", 0)

    L = []
    # Line 1: Scoreboard
    L.append("D{} Fitness:{:.3f} Yield:{:.3f} Saving:{:.3f} Health:{:.3f}".format(day, fit, yld, sav, hlt
h))

    # Line 2: Population
    if alive == 0:
        L.append("ALL DEAD")
    elif dt > 0:
        L.append("{} / {} alive (-{})".format(alive, init, dt))
    else:
        L.append("{} / {} alive".format(alive, init))

    # Line 3: Stats (ratios)
    if alive > 0:
        stats = {"h": hr, "f": fr, "o": oxr, "i": ir}
        weakest = min(stats, key=stats.get)
        L.append("h={:.2f} f={:.2f} o={:.2f} i={:.2f} weak={}".format(
            hr, fr, oxr, ir, weakest))

    # Line 4: Sick fish
    ni, np = s.get("n_infected", 0), s.get("n_parasitized", 0)
    if ni or np:
        L.append("sick: {}inf {}par".format(ni, np))

    # Line 5: Hazards
    hz = s.get("hazard_zones", {})
    hz_parts = []
    for kind, zones in hz.items():
        if zones:
            hz_parts.append("{}@{}".format(kind[:3], ",".join(z.split("(")[0] for z in zones[:3])))
    if hz_parts:
        L.append("haz: {}".format(" ".join(hz_parts)))

    # Line 6: Budget
    if s.get("budget_exceeded"):
        L.append("OVER BUDGET")
    else:
        days_left = AQUACULTURE_DAYS - day
        if days_left > 0:
            rate = cost / max(1, day)
            L.append("${:.1f} spent ${:.1f} left {:.1f}$/d".format(cost, left, rate))

    # Line 7: Warning points
    wp, wb = compute_warning_points(s)
    if wp > 0:
        L.append("warn:{} ({} )".format(wp, " ".join(wb[:4])))

    return "\n".join(L)

```

```
def _raw_json(s):
    compact = {k: v for k, v in s.items() if not k.startswith("_") and k != "frames"}
    return json.dumps(compact)
```

Overwriting /content/AquaLLM/agent/observer.py

CELL 9: agent/tools.py

In []:

```
# @title
%%writefile /content/AquaLLM/agent/tools.py
#!/usr/bin/env python3
"""
tools.py -- Tools for the LLM agent.
v3: Added get_pond_info tool. Reads AQUACULTURE_DAYS from constants.
"""

import sys
sys.path.insert(0, "/content/AquaLLM/core")
from constants import AQUACULTURE_DAYS, POND_WIDTH, POND_HEIGHT, POND_DEPTH, MAX_FISH_COUNT

FOOD_PRICE = 0.05
PROBIOTIC_PRICE = 0.75
OXYGEN_PRICE = 2.50
MAX_BUDGET = 500.0

def calculate_budget(
    food_interval,
    food_quantity,
    probiotic_interval,
    probiotic_quantity,
    oxygen_interval,
    oxygen_duration,
):
    """Calculate total cost of a pond policy over the simulation period."""
    daily_food = (24 / max(1, food_interval)) * food_quantity * FOOD_PRICE
    daily_probiotic = (24 / max(1, probiotic_interval)) * probiotic_quantity * PROBIOTIC_PRICE
    daily_oxygen = (24 / max(1, oxygen_interval)) * oxygen_duration * OXYGEN_PRICE

    daily_cost = daily_food + daily_probiotic + daily_oxygen
    total_cost = daily_cost * AQUACULTURE_DAYS
    remaining = MAX_BUDGET - total_cost

    return {
        "simulation_days": AQUACULTURE_DAYS,
        "daily_cost": round(daily_cost, 2),
        "total_cost": round(total_cost, 2),
        "budget_remaining": round(remaining, 2),
        "within_budget": total_cost <= MAX_BUDGET,
        "breakdown": {
            "food_daily": round(daily_food, 2),
            "probiotic_daily": round(daily_probiotic, 2),
            "oxygen_daily": round(daily_oxygen, 2),
        }
    }

def get_pond_info():
    """Get pond dimensions and fish capacity."""
    return {
        "pond_width": POND_WIDTH,
        "pond_height": POND_HEIGHT,
        "pond_depth": POND_DEPTH,
        "max_fish_count": MAX_FISH_COUNT,
        "simulation_days": AQUACULTURE_DAYS,
        "total_budget": MAX_BUDGET,
    }

BUDGET_TOOL_SCHEMA = {
    "type": "function",
    "function": {
        "name": "calculate_budget",
        "description": (
            "Calculate the total cost of a pond management policy over {} days. "
            "Call this BEFORE proposing a policy to verify it stays within the $500 budget. "
        )
    }
}
```

```
        "Prices: food=$0.05/pellet, probiotic=$0.75/pellet, oxygen=$2.50/hour."
    ).format(AQUACULTURE_DAYS),
    "parameters": {
        "type": "object",
        "properties": {
            "food_interval": {
                "type": "integer",
                "description": "Hours between food drops (1-12)"
            },
            "food_quantity": {
                "type": "integer",
                "description": "Number of food pellets per drop (5-30)"
            },
            "probiotic_interval": {
                "type": "integer",
                "description": "Hours between probiotic drops (24-168)"
            },
            "probiotic_quantity": {
                "type": "integer",
                "description": "Number of probiotic pellets per drop (1-4)"
            },
            "oxygen_interval": {
                "type": "integer",
                "description": "Hours between O2 pump activations (1-20)"
            },
            "oxygen_duration": {
                "type": "integer",
                "description": "Hours O2 pump stays on per activation (1-4)"
            },
        },
        "required": [
            "food_interval", "food_quantity",
            "probiotic_interval", "probiotic_quantity",
            "oxygen_interval", "oxygen_duration",
        ]
    }
}

POND_INFO_TOOL_SCHEMA = {
    "type": "function",
    "function": {
        "name": "get_pond_info",
        "description": (
            "Get the pond dimensions, maximum fish capacity, simulation length, and budget. "
            "Call this on the first day to understand the pond before stocking fish."
        ),
        "parameters": {
            "type": "object",
            "properties": {},
            "required": []
        }
    }
}

ALL_TOOL_SCHEMAS = [BUDGET_TOOL_SCHEMA, POND_INFO_TOOL_SCHEMA]

TOOL_DISPATCHER = {
    "calculate_budget": calculate_budget,
    "get_pond_info": get_pond_info,
}
```

Overwriting /content/AquaLLM/agent/tools.py

CELL 10: agent/memory.py

In []:

```
# @title
%%writefile /content/AquaLLM/agent/memory.py
#!/usr/bin/env python3
"""
memory.py -- Memory and reflection module.
v12: Updated for chat() returning (text, metadata) tuple.
"""

import json
import sys
sys.path.insert(0, "/content/AquaLLM/core")
```

```
sys.path.insert(0, "/content/AquaLLM/agent")
```

```
from llm_client import chat
from constants import AQUACULTURE_DAYS
```

```
BASE_THRESHOLD = 30
SPIKE_MULTIPLIER = 1.5
```

```
_CONSTRAINT = (
    "Adjustable params (integers only): "
    "food_interval(1-12h), food_quantity(5-30), food_location(0-9), "
    "probiotic_interval(24-168h), probiotic_quantity(1-4), probiotic_location(0-9), "
    "oxygen_interval(1-20h), oxygen_duration(1-4h), oxygen_location(0-9). "
    "fish_count is LOCKED after Day 0. "
    "Saving=33% of Fitness. Spend LESS = higher Fitness. "
    "State exact param=value. Changes apply tomorrow."
)
```

```
class MemoryStream:
```

```
    def __init__(self):
        self.daily_entries = []
        self.gen_reflections = []
        self.gen_summaries = []
        self.mid_gen_reflections = []
        self.all_mid_reflections = {}
        self.gen_day_fitness = {}
        self.current_gen = 0
        self._reflected_days = set()
        self.warning_points = 0
        self._daily_points_history = []
        self._last_reflection_points = 0
        self.gen_early_fitness_sum = {}
        self.restart_reflections = []

    def add(self, day, observation, policy_dict, result_summary, fitness=0, snapshot=None):
        self.daily_entries.append({
            "day": day, "obs": observation, "policy": policy_dict,
            "result": result_summary, "fitness": fitness,
        })
        if self.current_gen not in self.gen_day_fitness:
            self.gen_day_fitness[self.current_gen] = {}
        self.gen_day_fitness[self.current_gen][day] = fitness

        if snapshot:
            from observer import compute_warning_points
            day_pts, _ = compute_warning_points(snapshot)
            self._daily_points_history.append(day_pts)
            self.warning_points += day_pts

    def _get_dynamic_threshold(self):
        if not self._daily_points_history:
            return BASE_THRESHOLD
        avg = sum(self._daily_points_history) / len(self._daily_points_history)
        return BASE_THRESHOLD + avg * SPIKE_MULTIPLIER

    def get_early_fitness_sum(self):
        cp = max(1, AQUACULTURE_DAYS // 5)
        gd = self.gen_day_fitness.get(self.current_gen, {})
        return sum(gd.get(d, 0) for d in range(1, cp + 1))

    def get_best_early_fitness_sum(self):
        cp = max(1, AQUACULTURE_DAYS // 5)
        best = 0
        for gn, dd in self.gen_day_fitness.items():
            if gn == self.current_gen:
                continue
            s = sum(dd.get(d, 0) for d in range(1, cp + 1))
            if s > best:
                best = s
        return best

    def store_early_fitness(self):
        self.gen_early_fitness_sum[self.current_gen] = self.get_early_fitness_sum()

    def generate_restart_reflection(self, reason, context=""):
        msg = [
            {"role": "system", "content":
                "Gen restarted: {}. {} 1 sentence: what to change.".format(reason, _CONSTRAINT)},
            {"role": "user", "content": context or "No context."},
```

```

]
r, _ = chat(msg, temperature=0.2)
if r and not r.startswith("[LLM_ERROR]"):
    r = r.strip()
    self.restart_reflections.append(r)
    return r
return None

def get_history_text(self, n=3):
    recent = self.daily_entries[-n:]
    if not recent:
        return ""
    return "\n".join("D{}: {}".format(e["day"], e["result"]) for e in recent)

def get_gen_reflections_text(self):
    if not self.gen_reflections:
        return ""
    return "\n".join("G{}: {}".format(i+1, r) for i, r in enumerate(self.gen_reflections[-3:]))

def get_gen_summaries_text(self):
    return "\n".join(self.gen_summaries[-3:]) if self.gen_summaries else ""

def get_mid_reflections_text(self):
    return "\n".join(self.mid_gen_reflections[-2:]) if self.mid_gen_reflections else ""

def get_restart_reflections_text(self):
    return "\n".join(self.restart_reflections[-2:]) if self.restart_reflections else ""

def get_cross_gen_comparison(self, current_day):
    cur = self.gen_day_fitness.get(self.current_gen, {}).get(current_day)
    if cur is None:
        return ""
    parts = []
    for gn in sorted(self.gen_day_fitness):
        if gn == self.current_gen:
            continue
        pf = self.gen_day_fitness[gn].get(current_day)
        if pf is not None:
            parts.append("G{}:{:.3f}".format(gn, pf))
    if not parts:
        return ""
    return "D{} now:{:.3f} prev:{}".format(current_day, cur, " ".join(parts))

def get_reflection_checkpoints(self, total_days):
    if total_days <= 0:
        return []
    iv = max(1, total_days // 5)
    return list(range(iv, total_days + 1, iv))

def maybe_reflect_mid_gen(self, use_reflection, current_day, total_days, snapshot=None):
    if not use_reflection or total_days <= 0 or len(self.daily_entries) < 2:
        return None
    if current_day in self._reflected_days:
        return None

    cps = self.get_reflection_checkpoints(total_days)
    is_cp = current_day in cps
    pts_since = self.warning_points - self._last_reflection_points
    thresh = self._get_dynamic_threshold()
    is_spike = pts_since >= thresh

    if not is_cp and not is_spike:
        return None

    self._reflected_days.add(current_day)
    self._last_reflection_points = self.warning_points

    trigger = "warn:{:.0f}/{:.0f}".format(pts_since, thresh) if is_spike else "{}%".format(
        int(current_day / total_days * 100))

    recent = self.daily_entries[-2:]
    hist = "\n".join("D{}: {}".format(e["day"], e["result"]) for e in recent)

    from observer import compute_warning_points
    warn_text = ""
    if snapshot:
        _, wb = compute_warning_points(snapshot)
        if wb:
            warn_text = " warn:{}".format(", ".join(wb[:3]))

    msg = [

```

```

        {"role": "system", "content":
            "{} D{}/{}} {} 1 sentence: what number to change.".format(
                trigger, current_day, total_days, warn_text, _CONSTRAINT)},
        {"role": "user", "content": hist},
    ]

    r, _ = chat(msg, temperature=0.2)
    if r and not r.startswith("[LLM_ERROR]"):
        r = r.strip()
        self.mid_gen_reflections.append("D{ }({}): {}".format(current_day, trigger[:15], r))
        return r
    return None

def start_generation(self, gen_num):
    self.current_gen = gen_num
    self.daily_entries = []
    self.mid_gen_reflections = []
    self._reflected_days = set()
    self.warning_points = 0
    self._daily_points_history = []
    self._last_reflection_points = 0

def end_generation(self, gen_num, final_result, use_reflection):
    fit = final_result.get("fitness", 0)
    yld = final_result.get("yield", 0)
    sav = final_result.get("saving_ratio", 0)
    hlth = final_result.get("avg_healthiness", 0)
    surv = final_result.get("survival_rate", 0) * 100
    cost = final_result.get("cost", 0)
    alive = final_result.get("alive_count", 0)
    init = final_result.get("initial_count", 0)
    exceeded = final_result.get("budget_exceeded", False)
    eq = final_result.get("early_quit", False)

    tags = []
    if eq: tags.append("Q")
    if exceeded: tags.append("$")
    tag = "[{}]".format("".join(tags)) if tags else ""

    summary = "G{ } F:{:.3f} Y:{:.3f} S:{:.3f} H:{:.3f} {:.0f}%({}/{}) ${:.0f} w:{}".format(
        gen_num, tag, fit, yld, sav, hlth, surv, alive, init, cost, self.warning_points)
    self.gen_summaries.append(summary)
    self.all_mid_reflections[gen_num] = list(self.mid_gen_reflections)
    self.store_early_fitness()

    if use_reflection and self.daily_entries:
        ref = self._generate_gen_reflection(gen_num, final_result)
        if ref:
            self.gen_reflections.append(ref)

    self.daily_entries = []
    self.mid_gen_reflections = []
    self._reflected_days = set()
    self.warning_points = 0
    self._daily_points_history = []
    self._last_reflection_points = 0

def _generate_gen_reflection(self, gen_num, final_result):
    exceeded = final_result.get("budget_exceeded", False)
    eq = final_result.get("early_quit", False)
    last_day = self.daily_entries[-1]["day"] if self.daily_entries else 0
    cost = final_result.get("cost", 0)
    fit = final_result.get("fitness", 0)

    if exceeded:
        ctx = "FAIL:budget D{} ${:.0f}".format(last_day, cost)
    elif eq:
        ctx = "QUIT D{}".format(last_day)
    elif fit > 0:
        ctx = "Done {}d F:{:.3f}".format(last_day, fit)
    else:
        ctx = "FAIL F:0"

    if self.mid_gen_reflections:
        body = "\n".join(self.mid_gen_reflections[-3:]) + "\n" + ctx
    else:
        entries = self.daily_entries
        if len(entries) > 4:
            sampled = entries[:1] + entries[-2:]
        else:
            sampled = entries

```

```
body = "\n".join("D{}: {}".format(e["day"], e["result"])) for e in sampled)
body += "\n" + ctx

msg = [
    {"role": "system", "content":
        "Synthesize gen. {} 2 sentences: #1 cause, one number change.".format(_CONSTRAINT)},
    {"role": "user", "content": body},
]
r, _ = chat(msg, temperature=0.2)
if r and not r.startswith("[LLM_ERROR]"):
    return r.strip()
return None

def export_json(self, path):
    data = {
        "gen_summaries": self.gen_summaries,
        "gen_reflections": self.gen_reflections,
        "restart_reflections": self.restart_reflections,
        "mid_gen_reflections": {str(k): v for k, v in self.all_mid_reflections.items()},
        "gen_day_fitness": {str(k): v for k, v in self.gen_day_fitness.items()},
        "gen_early_fitness_sum": {str(k): v for k, v in self.gen_early_fitness_sum.items()},
    }
    with open(path, "w") as f:
        json.dump(data, f, indent=2)
```

Overwriting /content/AquaLLM/agent/memory.py

CELL 11: agent/agent.py

In []:

```
# @title
%%writefile /content/AquaLLM/agent/agent.py
#!/usr/bin/env python3
"""
agent.py -- FarmManagerAgent.
v19: Trace recording for per-day logging. chat() returns (text, meta).
"""

import json
import re
import sys

sys.path.insert(0, "/content/AquaLLM/core")
sys.path.insert(0, "/content/AquaLLM/agent")

from constants import AQUACULTURE_DAYS, MAX_BUDGET, MAX_FISH_COUNT

from agent_config import AgentConfig
from schemas import PondPolicy
from observer import observe
from memory import MemoryStream
from tools import ALL_TOOL_SCHEMAS, TOOL_DISPATCHER
from llm_client import chat as _raw_chat

_DAILY_BUDGET = MAX_BUDGET / AQUACULTURE_DAYS

# =====
# MODULAR PROMPT BLOCKS
# =====

_CORE = (
    "You manage an aquaculture fish pond (raising fish to keep alive, not catching). "
    "{days}d ${budget} budget.\n"
    "Fitness = 0.55*Yield + 0.33*Saving + 0.12*Health (Saving is 3x more important than Health!)\n"
    "Yield=alive/max. Saving=(500-cost)/500. Over budget or all dead=Fitness 0.\n"
    "CRITICAL: Spending less money DIRECTLY increases your Fitness. "
    "Every $16.67 saved = +0.01 Fitness. Prefer higher intervals and lower quantities.\n"
    "All policy values are integers. Focus on what to do next, not the past.\n"
    "Locations: 0=Center 1=TL 2=TR 3=BL 4=BR 5=TC 6=BC 7=LC 8=RC 9=Random(best)\n"
).format(days=AQUACULTURE_DAYS, budget=int(MAX_BUDGET))

_PLANNING_BLOCK = (
    "Think step-by-step before outputting JSON.\n"
    "1. Read stats. 2. Identify weakest. 3. Pick fix. 4. Check budget. 5. Output JSON.\n"
)

_TOOLS_BLOCK = (
    "Cost/day: food=(24/intv)*qty*$0.05 prob=(24/intv)*qty*$0.75 O2=(24/intv)*dur*$2.50. "
```

```
"Target: ${daily:.2f}/day. Use calculate_budget tool to verify.\n"
).format(daily=_DAILY_BUDGET)

_COUNTER_BLOCK = (
    "Fixes: low_fullness->less food_interval|more food_qty. "
    "low_oxygen->less O2_interval|more O2_dur. "
    "low_immunity->less prob_interval|more prob_qty. "
    "NH3/disease/parasite->move food/O2 away, reduce food_qty. "
    "Over budget->increase O2_interval first, then food_interval. "
    "Leftover food->pollution->NH3(30%). Start cheap, increase only if stat<0.5.\n"
)

_FIRST_ROUND_BLOCK = (
    "Day 0: Stock fish + set initial policy.\n"
    "Call get_pond_info for max capacity (max_fish_count={max_fish}).\n"
    "Yield=alive/max_fish_count. MORE fish stocked = higher possible yield.\n"
    "Yield has the biggest weight (0.55). Try 40-50 fish if budget allows.\n"
    "But more fish = more food needed = higher cost. Find the balance.\n"
    "Call calculate_budget to verify. Output JSON with fish_count + 9 policy fields.\n"
).format(max_fish=MAX_FISH_COUNT)

_ADJUST_BLOCK = (
    "Read observation. If stat<0.5: fix it. If all OK: keep policy. "
    "Output JSON (9 fields + early_quit). Only call budget tool if changing.\n"
)

# =====

_FIELD_ALIASES = {
    "fish_count": "fish_count", "fishcount": "fish_count",
    "food_interval": "food_interval", "foodinterval": "food_interval",
    "food_quantity": "food_quantity", "foodquantity": "food_quantity",
    "food_location": "food_location", "foodlocation": "food_location",
    "probiotic_interval": "probiotic_interval", "probioticinterval": "probiotic_interval",
    "probiotic_quantity": "probiotic_quantity", "probioticquantity": "probiotic_quantity",
    "probiotic_location": "probiotic_location", "probioticlocation": "probiotic_location",
    "oxygen_interval": "oxygen_interval", "oxygeninterval": "oxygen_interval",
    "oxygen_duration": "oxygen_duration", "oxygenduration": "oxygen_duration",
    "oxygen_location": "oxygen_location", "oxygenlocation": "oxygen_location",
    "early_quit": "early_quit", "earlyquit": "early_quit",
}

def _clamp(val, lo, hi):
    try:
        return max(lo, min(hi, int(float(val))))
    except (TypeError, ValueError):
        return lo

def _fix_json(s):
    s = s.replace("'", '"')
    s = re.sub(r'\s*([}\]])', r'\1', s)
    s = re.sub(r'//.*$', '', s, flags=re.MULTILINE)
    s = re.sub(r'\bTrue\b', 'true', s)
    s = re.sub(r'\bFalse\b', 'false', s)
    return s

def _parse_bullets(response):
    data = {}
    for line in response.split("\n"):
        line = line.strip().lstrip("-*\u2022 ").replace("`", "").replace("***", "")
        match = re.match(r'([a-z_\s]+?)\s*[:=]\s*(\d+|true|false)', line, re.IGNORECASE)
        if match:
            raw_key = match.group(1).strip().lower().replace(" ", "_")
            value = match.group(2)
            canonical = _FIELD_ALIASES.get(raw_key)
            if canonical:
                if value.lower() in ("true", "false"):
                    data[canonical] = value.lower() == "true"
                else:
                    data[canonical] = int(value)
    return data if len(data) >= 3 else None

def _extract_json(response):
    match = re.search(r'````(?:json)?\s*(\{.*?\})\s*```', response, re.DOTALL)
    if match:
        try:
            data = json.loads(_fix_json(match.group(1)))
```

```

        if any(k in data for k in ["fish_count", "food_interval", "food_quantity"]):
            return data
        except json.JSONDecodeError:
            pass
candidates = re.findall(r'\{[^\}]*\}', response, re.DOTALL)
for c in sorted(candidates, key=len, reverse=True):
    try:
        data = json.loads(_fix_json(c))
        if any(k in data for k in ["fish_count", "food_interval", "food_quantity"]):
            return data
        except json.JSONDecodeError:
            continue
    try:
        return json.loads(_fix_json(response.strip()))
    except json.JSONDecodeError:
        pass
return _parse_bullets(response)

```

```

def _to_policy(data, include_fish_count=True):
    kwargs = dict(
        food_interval=_clamp(data.get("food_interval", 8), 1, 12),
        food_quantity=_clamp(data.get("food_quantity", 10), 5, 30),
        food_location=_clamp(data.get("food_location", 9), 0, 9),
        probiotic_interval=_clamp(data.get("probiotic_interval", 48), 24, 168),
        probiotic_quantity=_clamp(data.get("probiotic_quantity", 1), 1, 4),
        probiotic_location=_clamp(data.get("probiotic_location", 9), 0, 9),
        oxygen_interval=_clamp(data.get("oxygen_interval", 20), 1, 20),
        oxygen_duration=_clamp(data.get("oxygen_duration", 1), 1, 4),
        oxygen_location=_clamp(data.get("oxygen_location", 9), 0, 9),
        early_quit=bool(data.get("early_quit", False)),
    )
    if include_fish_count:
        kwargs["fish_count"] = _clamp(data.get("fish_count", 30), 10, 50)
    else:
        kwargs["fish_count"] = data.get("fish_count", 30)
    return PondPolicy(**kwargs)

```

```

class FarmManagerAgent:

```

```

    def __init__(self, config):
        self.config = config
        self.memory = MemoryStream()
        self.day = 0
        self.gen_num = 0
        self.fish_count = None
        self._last_day1_fitness = 0
        self._last_gen_result = None
        self._prev_mid_count = 0
        self._restart_hint = None
        self._target_fish_count = None
        self._last_trace = None # trace data from last call

```

```

    def _build_system(self, phase="adjust"):
        parts = [_CORE]
        if self.config.use_observer:
            parts.append(_COUNTER_BLOCK)
        if self.config.use_calculator:
            parts.append(_TOOLS_BLOCK)
        if self.config.use_planning:
            parts.append(_PLANNING_BLOCK)
        else:
            parts.append("Output ONLY JSON.\n")
        if phase == "first":
            parts.append(_FIRST_ROUND_BLOCK)
        else:
            parts.append(_ADJUST_BLOCK)
        return "".join(parts)

```

```

    def start_generation(self, gen_num):
        self.gen_num = gen_num
        self.day = 0
        self.fish_count = None
        self.memory.start_generation(gen_num)

```

```

    def end_generation(self, final_result):
        self.memory.end_generation(
            self.gen_num, final_result, self.config.use_reflection)

```

```

    def set_restart_hint(self, hint, target_fish=None):

```

```

self._restart_hint = hint
self._target_fish_count = target_fish

def reflect_on_restart(self, reason, context=""):
    if self.config.use_reflection:
        return self.memory.generate_restart_reflection(reason, context)
    return None

def first_round(self):
    self.day = 0
    system = self._build_system("first")

    parts = ["G{} Day0.".format(self.gen_num)]

    if self._restart_hint:
        parts.append("*** {} ***".format(self._restart_hint))

    if self.config.use_memory and self.memory.gen_summaries:
        parts.append("Prev:\n" + self.memory.get_gen_summaries_text())

    if self.config.use_reflection and self.memory.gen_reflections:
        parts.append("Lessons:\n" + self.memory.get_gen_reflections_text())

    if self.config.use_reflection and self.memory.restart_reflections:
        parts.append("Restart:\n" + self.memory.get_restart_reflections_text())

    tools = ALL_TOOL_SCHEMAS if self.config.use_calculator else None
    dispatcher = TOOL_DISPATCHER if self.config.use_calculator else None

    custom_reminder = None
    if self._restart_hint and self._target_fish_count:
        custom_reminder = (
            "Output JSON. fish_count must be >= {}. JSON only.".format(
                self._target_fish_count))

    response, meta = self._chat_with_reminder(
        [{"role": "system", "content": system},
         {"role": "user", "content": "\n".join(parts)}],
        tools=tools, dispatcher=dispatcher, custom_reminder=custom_reminder)

    policy = self._parse(response, include_fish_count=True)
    self.fish_count = policy.fish_count

    # Build trace for Day 0
    self._last_trace = {
        "day": 0,
        "obs_lines": 0,
        "obs_text": None,
        "mem_entries": 0,
        "mem_history": None,
        "ref": None,
        "tool_calls": [tc["name"] for tc in meta.get("tool_calls", [])],
        "tool_details": meta.get("tool_calls", []),
        "plan": self.config.use_planning,
        "plan_reasoning": response if self.config.use_planning else None,
        "policy_output": policy.to_genotype_dict(),
        "module_tag": self._module_tag(None),
    }

    self._restart_hint = None
    self._target_fish_count = None
    return policy

def step(self, snapshot):
    self.day = snapshot.get("day", self.day + 1)
    fitness = snapshot.get("fitness", 0)

    observation = observe(snapshot, self.config.use_observer)
    result_summary = "d={} {}/{ } F:{:.3f} ${:.0f}".format(
        self.day, snapshot.get("alive_count", 0), snapshot.get("initial_count", 0),
        fitness, snapshot.get("cost_so_far", 0))

    mem_history = None
    if self.config.use_memory:
        self.memory.add(self.day, observation, snapshot.get("genotype", {}),
                        result_summary, fitness, snapshot)
        mem_history = self.memory.get_history_text(2)

    mid_ref = None
    if self.config.use_reflection and self.config.use_memory:
        mid_ref = self.memory.maybe_reflect_mid_gen(

```

```

        True, self.day, AQUACULTURE_DAYS, snapshot)

system = self._build_system("adjust")

parts = ["G{} D{}".format(self.gen_num, self.day)]
parts.append(observation)

if self.config.use_memory and self.memory.daily_entries:
    h = self.memory.get_history_text(2)
    if h:
        parts.append(h)

comp = self.memory.get_cross_gen_comparison(self.day) if self.config.use_memory else ""
if comp:
    parts.append(comp)

if mid_ref:
    parts.append("Ref: " + mid_ref)
elif self.config.use_reflection and self.memory.mid_gen_reflections:
    parts.append(self.memory.get_mid_reflections_text())
elif self.config.use_reflection and self.memory.gen_reflections:
    parts.append(self.memory.get_gen_reflections_text())

parts.append("JSON now.")

tools = [ALL_TOOL_SCHEMAS[0]] if self.config.use_calculator else None
dispatcher = TOOL_DISPATCHER if self.config.use_calculator else None

response, meta = self._chat_with_reminder(
    [{"role": "system", "content": system},
     {"role": "user", "content": "\n".join(parts)}],
    tools=tools, dispatcher=dispatcher)

policy = self._parse(response, include_fish_count=False)
d = policy.to_genotype_dict()
d["fish_count"] = self.fish_count
d["early_quit"] = policy.early_quit
d["_module_tag"] = self._module_tag(mid_ref)

# Build trace for this day
obs_text = observation
obs_lines = obs_text.count("\n") + 1 if obs_text else 0

self._last_trace = {
    "day": self.day,
    "obs_lines": obs_lines,
    "obs_text": obs_text,
    "mem_entries": len(self.memory.daily_entries) if self.config.use_memory else 0,
    "mem_history": mem_history if self.config.use_memory else None,
    "ref": mid_ref,
    "tool_calls": [tc["name"] for tc in meta.get("tool_calls", [])],
    "tool_details": meta.get("tool_calls", []),
    "plan": self.config.use_planning,
    "plan_reasoning": response if self.config.use_planning else None,
    "policy_output": d.copy(),
    "module_tag": d.get("_module_tag", ""),
}
# Clean internal keys from policy_output
self._last_trace["policy_output"].pop("_module_tag", None)
self._last_trace["policy_output"].pop("early_quit", None)

return d

def get_last_trace(self):
    return self._last_trace

def _chat_with_reminder(self, messages, tools=None, dispatcher=None, custom_reminder=None):
    if custom_reminder:
        import llm_client
        old = llm_client._JSON_REMINDER
        llm_client._JSON_REMINDER = custom_reminder
        try:
            return _raw_chat(messages, tools=tools, tool_dispatcher=dispatcher,
                             temperature=0.3, max_tool_rounds=5)
        finally:
            llm_client._JSON_REMINDER = old
    else:
        return _raw_chat(messages, tools=tools, tool_dispatcher=dispatcher,
                          temperature=0.3, max_tool_rounds=3 if tools and len(tools) == 1 else 5)

def _parse(self, response, include_fish_count=True):

```

```
data = _extract_json(response)
if data:
    try:
        return _to_policy(data, include_fish_count)
    except Exception as e:
        print(" Warn: {}. Fallback.".format(e))
else:
    print(" Warn: No JSON. Fallback.")
return self._fallback(include_fish_count)

def _fallback(self, include_fish_count=True):
    fc = self._target_fish_count or self.fish_count or 30
    return PondPolicy(
        fish_count=fc, food_interval=8, food_quantity=10, food_location=9,
        probiotic_interval=48, probiotic_quantity=1, probiotic_location=9,
        oxygen_interval=20, oxygen_duration=1, oxygen_location=9,
        early_quit=False,
    )

def _module_tag(self, mid_ref):
    parts = []
    if self.config.use_observer:
        parts.append("Obs")
    if self.config.use_memory:
        parts.append("Mem:{}".format(len(self.memory.daily_entries)))
    if self.config.use_reflection:
        parts.append("Ref:Y" if mid_ref else "Ref:-")
    if self.config.use_calculator:
        parts.append("Tool")
    if self.config.use_planning:
        parts.append("Plan")
    return "[{}]".format(" ".join(parts))
```

Overwriting /content/AquaLLM/agent/agent.py

CELL 12: run_llm.py

In []:

```
# @title
%%writefile /content/AquaLLM/run_llm.py
#!/usr/bin/env python3
"""
run_llm.py -- AquaLLM adaptive entry point.
v11: Per-config trace recording with full module logging.
"""

import os, sys, csv, time, json
sys.path.insert(0, "/content/AquaLLM/core")
sys.path.insert(0, "/content/AquaLLM/agent")

from constants import (
    MAX_BUDGET, AQUACULTURE_DAYS, MAX_FISH_COUNT,
    POND_WIDTH, POND_HEIGHT, POND_DEPTH,
    SIMULATION_JSON_PATH,
)
from simulate_adaptive import AdaptivePondSim
from agent_config import AgentConfig, ALL_CONFIGS
from agent import FarmManagerAgent
from memory import MemoryStream

TIMESTEPS_PER_DAY = 24
NUM_GENERATIONS = 5
MAX_RESTARTS_PER_GEN = 2
DAY1_FITNESS_THRESHOLD = 0.90
EARLY_FITNESS_DROP_PCT = 0.10
CONFIGS_TO_TEST = ["full", "no_planning", "no_memory", "no_tools", "no_obs", "minimal"]
NUM_RUNS = 1

CSV_PATH = "/content/AquaLLM/logs/llm_results.csv"
TRACE_DIR = "/content/AquaLLM/logs"

CSV_HEADER = [
    "config", "run", "generation", "attempt", "day",
    "fitness", "yield", "saving_ratio", "healthiness",
    "survival_pct", "alive_count", "initial_count", "deaths_today",
    "cost", "saving",
    "food_interval", "food_quantity", "food_location",
```

```

        "probiotic_interval", "probiotic_quantity", "probiotic_location",
        "oxygen_interval", "oxygen_duration", "oxygen_location",
        "early_quit",
    ]

    _LOC = {
        0: "Center", 1: "Top-Left", 2: "Top-Right", 3: "Bot-Left", 4: "Bot-Right",
        5: "Top-Ctr", 6: "Bot-Ctr", 7: "Left-Ctr", 8: "Right-Ctr", 9: "Random",
    }

def _fp(g):
    fl = _LOC.get(g.get("food_location", -1), "?")
    pl = _LOC.get(g.get("probiotic_location", -1), "?")
    ol = _LOC.get(g.get("oxygen_location", -1), "?")
    return "food={x}/{h}@{}  prob={x}/{h}@{}  O2={h}/{h}@{}".format(
        g.get("food_quantity", "?"), g.get("food_interval", "?"), fl,
        g.get("probiotic_quantity", "?"), g.get("probiotic_interval", "?"), pl,
        g.get("oxygen_duration", "?"), g.get("oxygen_interval", "?"), ol)

def _legend():
    print("  Location Legend:")
    print("    0=Center    1=Top-Left    2=Top-Right")
    print("    3=Bot-Left   4=Bot-Right   5=Top-Ctr")
    print("    6=Bot-Ctr    7=Left-Ctr    8=Right-Ctr  9=Random")

def _checkpoints():
    m = MemoryStream()
    cps = m.get_reflection_checkpoints(AQUACULTURE_DAYS)
    print("  Reflection Checkpoints: {} (every 20% of {} days)".format(
        ", ".join("Day {}".format(d) for d in cps), AQUACULTURE_DAYS))

def run_generation(agent, gen_num, config_name, run_idx, trace_gens):
    """Run a generation with possible restarts. Appends to trace_gens."""
    for attempt in range(1, MAX_RESTARTS_PER_GEN + 2):
        restart_needed, restart_reason, gen_trace = _run_attempt(
            agent, gen_num, attempt, config_name, run_idx)

        trace_gens.append(gen_trace)

        if not restart_needed:
            return agent._last_gen_result

        prev_fish = agent.fish_count or 30
        target_fish = min(MAX_FISH_COUNT, prev_fish + 10)

        print("\n  *** RESTARTING Gen {} (attempt {} -> {}) ***".format(
            gen_num, attempt, attempt + 1))
        print("  Reason: {}".format(restart_reason))

        ref = agent.reflect_on_restart(restart_reason,
            "Stocked {} fish, got fitness {:.4f}. Max capacity: {}. Try {}".format(
                prev_fish, agent._last_day1_fitness, MAX_FISH_COUNT, target_fish))
        if ref:
            print("  Reflection: {}".format(ref))

        agent.set_restart_hint(
            "Previous attempt: {} fish -> fitness {:.4f} (need >= {:.1f}). "
            "Max capacity is {}. Set fish_count to {} or higher.".format(
                prev_fish, agent._last_day1_fitness, DAY1_FITNESS_THRESHOLD,
                MAX_FISH_COUNT, target_fish),
            target_fish=target_fish
        )
        print("")

        if attempt >= MAX_RESTARTS_PER_GEN + 1:
            print("  Max restarts reached. Continuing with last attempt.\n")
            return agent._last_gen_result

    return agent._last_gen_result

def _run_attempt(agent, gen_num, attempt, config_name, run_idx):
    """Run one attempt of a generation. Returns (restart_needed, reason, gen_trace)."""
    if attempt == 1:
        print("\n  " + "~" * 66)
        print("  Generation {}/{ } (attempt {})".format(gen_num, NUM_GENERATIONS, attempt))
        print("  " + "~" * 66)
    else:
        print("  " + ". " * 33)
        print("  Generation {}/{ } (attempt {})".format(gen_num, NUM_GENERATIONS, attempt))

```

```

print(" " + ". " * 33)

agent.start_generation(gen_num)
csv_rows = []
early_quit = False
day_traces = []

# — Day 0: First round —
t0 = time.time()
policy = agent.first_round()
initial_dict = policy.to_genotype_dict()
llm_time = time.time() - t0

sim = AdaptivePondSim(initial_dict, max_budget=MAX_BUDGET,
                      seed=gen_num * 1000 + run_idx * 100 + attempt)

print(" Day 0 | Stocked {} fish | {} | LLM:{:.1f}s".format(
    initial_dict["fish_count"], _fp(initial_dict), llm_time))

# Record Day 0 trace
day0_trace = agent.get_last_trace()
if day0_trace:
    day0_trace["fitness"] = None
    day0_trace["alive"] = initial_dict["fish_count"]
    day0_trace["deaths_today"] = 0
    day0_trace["cost"] = 0.0
    day0_trace["llm_time"] = round(llm_time, 1)
    day_traces.append(day0_trace)

agent._last_day1_fitness = 0
agent._last_gen_result = None
checkpoint_day = max(1, AQUACULTURE_DAYS // 5)

# — Day 1..N —
for day in range(AQUACULTURE_DAYS):
    t_sim = time.time()
    sim.run_chunk(TIMESTEPS_PER_DAY)
    sim_time = time.time() - t_sim

    snap = sim.snapshot(day + 1)
    alive = snap["alive_count"]
    initial = snap["initial_count"]
    deaths_today = snap["deaths_today"]
    fit = snap["fitness"]
    yld = snap["yield"]
    sav_r = snap["saving_ratio"]
    hlth = snap["avg_healthiness"]
    surv = snap["survival_rate"] * 100
    cost = snap["cost_so_far"]
    saving = snap["budget_remaining"]
    geno = snap.get("genotype", {})

    tag = ""
    if snap["budget_exceeded"]:
        tag = " [OVER$]"
    elif alive == 0:
        tag = " [DEAD]"
    elif sim.is_done():
        tag = " [END]"

    llm_str = ""
    llm_time = 0
    if not sim.is_done() and day < AQUACULTURE_DAYS - 1:
        t_llm = time.time()
        new_policy = agent.step(snap)
        llm_time = time.time() - t_llm

        if new_policy.get("early_quit", False):
            early_quit = True
            tag = " [EARLY QUIT]"
            llm_str = " LLM:{:.1f}s".format(llm_time)
        else:
            sim.update_policy(new_policy)
            geno = new_policy
            llm_str = " LLM:{:.1f}s".format(llm_time)

    print(" Day {:>2} | Fit:{:.4f} Yld:{:.4f} Sav:{:.4f} Hlth:{:.4f} | "
          "Surv:{:>5.1f}% ({} / {}) | Cost:${:>7.2f} Save:${:>7.2f} | "
          "Sim:{:.1f}s{}".format(
            day + 1, fit, yld, sav_r, hlth,
            surv, alive, initial,

```

```

cost, saving,
sim_time, llm_str, tag))

if llm_str and not early_quit:
    mtag = geno.get("_module_tag", "")
    print("          | {} {}".format(_fp(geno), mtag))

current_mid_count = len(agent.memory.mid_gen_reflections)
if current_mid_count > agent._prev_mid_count:
    latest = agent.memory.mid_gen_reflections[-1]
    print("")
    print("          >>> REFLECTION: {}".format(latest[:280]))
    print("")
agent._prev_mid_count = current_mid_count

# Record day trace
day_trace = agent.get_last_trace()
if day_trace:
    day_trace["fitness"] = fit
    day_trace["alive"] = alive
    day_trace["deaths_today"] = deaths_today
    day_trace["cost"] = cost
    day_trace["llm_time"] = round(llm_time, 1)
    day_traces.append(day_trace)
else:
    # Last day or done - no LLM call, record minimal trace
    day_traces.append({
        "day": day + 1,
        "fitness": fit,
        "alive": alive,
        "deaths_today": deaths_today,
        "cost": cost,
        "llm_time": 0,
        "obs_lines": 0,
        "obs_text": None,
        "mem_entries": 0,
        "mem_history": None,
        "ref": None,
        "tool_calls": [],
        "tool_details": [],
        "plan": False,
        "plan_reasoning": None,
        "policy_output": None,
        "module_tag": "",
    })

# === SMART RESTART CHECK 1: Day 1 fitness ===
if day == 0:
    agent._last_day1_fitness = fit
    if fit < DAY1_FITNESS_THRESHOLD and attempt <= MAX_RESTARTS_PER_GEN:
        result = sim.final_result()
        result["early_quit"] = False
        agent._last_gen_result = {"result": result, "csv_rows": csv_rows}
        agent.end_generation(result)

        gen_trace = _build_gen_trace(gen_num, attempt, "abandoned",
            "Day 1 fitness {:.4f} < {:.1f}. Had {} fish (max {}).".format(
                fit, DAY1_FITNESS_THRESHOLD, initial_dict["fish_count"], MAX_FISH_COUNT),
            initial_dict["fish_count"], result, day_traces)

        return True, gen_trace["abandon_reason"], gen_trace

# === SMART RESTART CHECK 2: First 20% accumulated fitness ===
if day + 1 == checkpoint_day and attempt <= MAX_RESTARTS_PER_GEN:
    current_sum = agent.memory.get_early_fitness_sum()
    best_prev_sum = agent.memory.get_best_early_fitness_sum()
    if best_prev_sum > 0:
        drop = (best_prev_sum - current_sum) / best_prev_sum
        if drop >= EARLY_FITNESS_DROP_PCT:
            result = sim.final_result()
            result["early_quit"] = False
            agent._last_gen_result = {"result": result, "csv_rows": csv_rows}
            agent.end_generation(result)

            reason = "First {}d sum {:.2f} is {:.0f}% below best ({:.2f}).".format(
                checkpoint_day, current_sum, drop * 100, best_prev_sum)
            gen_trace = _build_gen_trace(gen_num, attempt, "abandoned",
                reason, initial_dict["fish_count"], result, day_traces)

            return True, reason, gen_trace

```

```

geno.pop("_module_tag", None)

csv_rows.append([
    config_name, run_idx + 1, gen_num, attempt, day + 1,
    "{:.4f}".format(fit), "{:.4f}".format(yld),
    "{:.4f}".format(sav_r), "{:.4f}".format(hlth),
    "{:.1f}".format(surv), alive, initial, deaths_today,
    "{:.2f}".format(cost), "{:.2f}".format(saving),
    geno.get("food_interval", ""), geno.get("food_quantity", ""),
    geno.get("food_location", ""), geno.get("probiotic_interval", ""),
    geno.get("probiotic_quantity", ""), geno.get("probiotic_location", ""),
    geno.get("oxygen_interval", ""), geno.get("oxygen_duration", ""),
    geno.get("oxygen_location", ""),
    early_quit,
])

if sim.is_done() or early_quit:
    break

# — Generation complete —
result = sim.final_result()
if early_quit:
    result["fitness"] = 0
    result["early_quit"] = True
else:
    result["early_quit"] = False

status = "early_quit" if early_quit else "completed"

print("")
print("  Gen {} FINAL (attempt {}):".format(gen_num, attempt))
print("    Fit:{:.4f} | Yield:{:.4f} | SavingRate:{:.4f} | Healthiness:{:.4f}".format(
    result["fitness"], result["yield"], result.get("saving_ratio", 0), result["avg_healthiness"]))
print("    Surv:{:.1f}% ({} / {}) | Cost:${:.2f} | Save:${:.2f}{}".format(
    result["survival_rate"] * 100, result["alive_count"], result["initial_count"],
    result["cost"], result["saving"],
    " [EARLY QUIT]" if early_quit else ""))

agent.end_generation(result)

if agent.memory.gen_reflections:
    print("")
    print("  >>> END-OF-GEN REFLECTION:")
    print("    {}".format(agent.memory.gen_reflections[-1][:300]))
    print("")

gen_trace = _build_gen_trace(gen_num, attempt, status, None,
    initial_dict["fish_count"], result, day_traces)

agent._last_gen_result = {"result": result, "csv_rows": csv_rows}
return False, "", gen_trace

def _build_gen_trace(gen_num, attempt, status, abandon_reason, fish_count, result, day_traces):
    """Build the generation trace dict."""
    return {
        "gen": gen_num,
        "attempt": attempt,
        "status": status,
        "abandon_reason": abandon_reason,
        "fish_count": fish_count,
        "final_fitness": round(result.get("fitness", 0), 4),
        "final_yield": round(result.get("yield", 0), 4),
        "final_saving_ratio": round(result.get("saving_ratio", 0), 4),
        "final_healthiness": round(result.get("avg_healthiness", 0), 4),
        "final_cost": round(result.get("cost", 0), 2),
        "survival_rate": round(result.get("survival_rate", 0), 4),
        "alive_count": result.get("alive_count", 0),
        "initial_count": result.get("initial_count", 0),
        "budget_exceeded": result.get("budget_exceeded", False),
        "days": day_traces,
    }

def _save_trace(config_name, label, runs_data):
    """Save trace file for one config."""
    trace = {
        "config": config_name,
        "label": label,
        "settings": {
            "aquaculture_days": AQUACULTURE_DAYS,

```

```

        "max_budget": MAX_BUDGET,
        "max_fish_count": MAX_FISH_COUNT,
        "num_generations": NUM_GENERATIONS,
        "num_runs": NUM_RUNS,
        "day1_fitness_threshold": DAY1_FITNESS_THRESHOLD,
        "early_fitness_drop_pct": EARLY_FITNESS_DROP_PCT,
        "max_restarts_per_gen": MAX_RESTARTS_PER_GEN,
    },
    "runs": runs_data,
}
path = "{}/trace_{}.json".format(TRACE_DIR, config_name)
with open(path, "w") as f:
    json.dump(trace, f, indent=2)
size = os.path.getsize(path)
print("  Saved trace: {} ({} bytes)".format(path, size))
return path

def main():
    import os
    os.makedirs(TRACE_DIR, exist_ok=True)

    wall_start = time.time()

    print("\n" + "#" * 70)
    print("  AQUALLM -- ADAPTIVE LLM AQUACULTURE MANAGER")
    print("#" * 70)
    print("  Model: qwen2.5:7b (local Ollama)")
    print("  Mode: Adaptive (observe every {} timesteps)".format(TIMESTEPS_PER_DAY))
    print("  Days: {} | Budget: ${} | Generations: {}".format(
        AQUACULTURE_DAYS, MAX_BUDGET, NUM_GENERATIONS))
    print("  Smart Restart: Day1 < {:.0f}% fitness or first-20% drop > {:.0f}%".format(
        DAY1_FITNESS_THRESHOLD * 100, EARLY_FITNESS_DROP_PCT * 100))
    print("  Max restarts per gen: {}".format(MAX_RESTARTS_PER_GEN))
    print("  Configs: {}".format(CONFIGS_TO_TEST))
    print()
    _legend()
    _checkpoints()
    print("  Module Tags:")
    print("    Obs      = Observer active (natural language state)")
    print("    Mem:NH   = Memory active, N = number of history entries stored")
    print("    Ref:Y    = Reflection triggered this day")
    print("    Ref:-    = No reflection this day")
    print("    Tool     = Budget calculator tool available")
    print("    Plan     = Step-by-step planning active")
    print("    (missing tag = module disabled in this config)")
    print()

    all_csv_rows = []
    best_result = None
    best_config = None

    for config_name in CONFIGS_TO_TEST:
        if config_name not in ALL_CONFIGS:
            print("  Warning: Unknown config '{}', skipping.".format(config_name))
            continue

        config = ALL_CONFIGS[config_name]
        config_runs = []

        for run_idx in range(NUM_RUNS):
            print("\n" + "=" * 70)
            print("  Config: {} ({} ) | Run {}/{}".format(
                config_name, config.label(), run_idx + 1, NUM_RUNS))
            print("  Days: {} | Budget: ${} | Generations: {}".format(
                AQUACULTURE_DAYS, MAX_BUDGET, NUM_GENERATIONS))
            print("=" * 70)

            agent = FarmManagerAgent(config)
            run_trace = {"run": run_idx + 1, "generations": []}

            for gen in range(1, NUM_GENERATIONS + 1):
                trace_gens = []
                output = run_generation(agent, gen, config_name, run_idx, trace_gens)
                all_csv_rows.extend(output["csv_rows"])

                run_trace["generations"].extend(trace_gens)

                result = output["result"]
                if best_result is None or result["fitness"] > best_result["fitness"]:
                    best_result = result

```

```
best_config = "{}_gen{}".format(config_name, gen)

# Save per-config memory
memory_path = "{}/memory_{}.json".format(TRACE_DIR, config_name)
agent.memory.export_json(memory_path)
print("\n Saved memory: {}".format(memory_path))

config_runs.append(run_trace)

# Save per-config trace
_save_trace(config_name, config.label(), config_runs)

# — Save CSV —
with open(CSV_PATH, "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(CSV_HEADER)
    writer.writerows(all_csv_rows)
print(" Saved CSV: {} ({} rows)".format(CSV_PATH, len(all_csv_rows)))

# — Save best visualization —
if best_result and best_result.get("frames"):
    viz = {
        "pond_width": POND_WIDTH,
        "pond_height": POND_HEIGHT,
        "pond_depth": POND_DEPTH,
        "genotype": best_result["genotype"],
        "fitness": best_result["fitness"],
        "survival_rate": best_result["survival_rate"],
        "avg_healthiness": best_result.get("avg_healthiness", 0),
        "cost": best_result["cost"],
        "saving": best_result.get("saving", 0),
        "yield": best_result.get("yield", 0),
        "aquaculture_days": AQUACULTURE_DAYS,
        "frames": best_result["frames"],
    }
    with open(SIMULATION_JSON_PATH, "w") as fp:
        json.dump(viz, fp)
    print(" Saved viz: {} ({} frames) [best: {}]".format(
        SIMULATION_JSON_PATH, len(best_result["frames"]), best_config))

# — Final summary —
print("\n" + "=" * 70)
if best_result:
    print(" BEST: {}".format(best_config))
    print(" Fit:{:.4f} | Yield:{:.4f} | SavingRate:{:.4f} | Healthiness:{:.4f}".format(
        best_result["fitness"], best_result["yield"],
        best_result.get("saving_ratio", 0), best_result["avg_healthiness"]))
print("=" * 70)

wall_elapsed = time.time() - wall_start
print("\n Total wall time: {:.1f}s ({:.1f} min)".format(wall_elapsed, wall_elapsed / 60))

if __name__ == "__main__":
    main()
```

Overwriting /content/AquaLLM/run_llm.py

CELL 13: LLM Agent Loop

In [58]:

```
# @title
!cd /content/AquaLLM && python run_llm.py
```

```
#####
AQUALLM -- ADAPTIVE LLM AQUACULTURE MANAGER
#####
Model: qwen2.5:7b (local Ollama)
Mode: Adaptive (observe every 24 timesteps)
Days: 30 | Budget: $500.0 | Generations: 5
Smart Restart: Day1 < 90% fitness or first-20% drop > 10%
Max restarts per gen: 2
Configs: ['full', 'no_planning', 'no_memory', 'no_tools', 'no_obs', 'minimal']

Location Legend:
0=Center      1=Top-Left    2=Top-Right
3=Bot-Left    4=Bot-Right    5=Top-Ctr
6=Bot-Ctr     7=Left-Ctr     8=Right-Ctr  9=Random
Reflection Checkpoints: Day 6, Day 12, Day 18, Day 24, Day 30 (every 20% of 30 days)
```

```
Module Tags:
  Obs      = Observer active (natural language state)
  Mem:NH    = Memory active, N = number of history entries stored
  Ref:Y     = Reflection triggered this day
  Ref:-     = No reflection this day
  Tool      = Budget calculator tool available
  Plan      = Step-by-step planning active
  (missing tag = module disabled in this config)

=====
Config: full (Obs+Mem+Ref+Tool+Plan) | Run 1/1
Days: 30 | Budget: $500.0 | Generations: 5
=====

~~~~~
Generation 1/5 (attempt 1)
~~~~~
Day 0 | Stocked 30 fish | food=10x/12h@Random prob=1x/24h@Random O2=1h/8h@Random | LLM:4.7s
Warn: No JSON. Fallback.
Day 1 | Fit:0.7725 Yld:0.6122 Sav:0.9815 Hlth:0.9323 | Surv:100.0% (30/30) | Cost:$ 9.25 Sav
e:$ 490.75 | Sim:0.0s LLM:2.0s
      | food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:1H Ref:- Tool Plan]

*** RESTARTING Gen 1 (attempt 1 -> 2) ***
Reason: Day 1 fitness 0.7725 < 0.9. Had 30 fish (max 49).
Reflection: Increase the `fish_count` to 40 by setting it directly since it's currently locked, but for
future adjustments, consider increasing the `food_interval`, `food_quantity`, or `oxygen_interval` and `ox
ygen_duration` to better support more fish without decreasing fitness.

. . . . .
Generation 1/5 (attempt 2)
. . . . .
Day 0 | Stocked 40 fish | food=10x/6h@Random prob=1x/24h@Random O2=2h/15h@Random | LLM:6.1s
Day 1 | Fit:0.8862 Yld:0.8163 Sav:0.9745 Hlth:0.9639 | Surv:100.0% (40/40) | Cost:$ 12.75 Sav
e:$ 487.25 | Sim:0.1s LLM:4.0s
      | food=15x/8h@Random prob=2x/24h@Random O2=3h/4h@Random [Obs Mem:1H Ref:- Tool Plan]

*** RESTARTING Gen 1 (attempt 2 -> 3) ***
Reason: Day 1 fitness 0.8862 < 0.9. Had 40 fish (max 49).
Reflection: Increase the food interval to 3 and set the food quantity to 25. This might help reach the m
aximum fish count of 49 and potentially improve fitness.

. . . . .
Generation 1/5 (attempt 3)
. . . . .
Day 0 | Stocked 49 fish | food=15x/6h@Random prob=1x/48h@Random O2=3h/12h@Random | LLM:5.9s
Day 1 | Fit:0.9831 Yld:1.0000 Sav:0.9625 Hlth:0.9620 | Surv:100.0% (49/49) | Cost:$ 18.75 Sav
e:$ 481.25 | Sim:0.1s LLM:2.3s
      | food=20x/6h@Random prob=2x/24h@Random O2=3h/5h@Random [Obs Mem:1H Ref:- Tool Plan]
Day 2 | Fit:0.9544 Yld:1.0000 Sav:0.8765 Hlth:0.9594 | Surv:100.0% (49/49) | Cost:$ 61.75 Sav
e:$ 438.25 | Sim:0.1s LLM:2.6s
      | food=24x/6h@Random prob=1x/24h@Random O2=3h/6h@Random [Obs Mem:2H Ref:- Tool Plan]
Day 3 | Fit:0.9305 Yld:1.0000 Sav:0.8054 Hlth:0.9559 | Surv:100.0% (49/49) | Cost:$ 97.30 Sav
e:$ 402.70 | Sim:0.1s LLM:2.6s
      | food=10x/6h@Random prob=1x/24h@Random O2=3h/7h@Random [Obs Mem:3H Ref:- Tool Plan]
Day 4 | Fit:0.9118 Yld:1.0000 Sav:0.7549 Hlth:0.9388 | Surv:100.0% (49/49) | Cost:$ 122.55 Sav
e:$ 377.45 | Sim:0.1s LLM:2.2s
      | food=10x/8h@Random prob=1x/24h@Random O2=1h/4h@Random [Obs Mem:4H Ref:- Tool Plan]
Day 5 | Fit:0.8869 Yld:0.9796 Sav:0.7204 Hlth:0.9196 | Surv: 98.0% (48/49) | Cost:$ 139.80 Sav
e:$ 360.20 | Sim:0.1s LLM:2.4s
      | food=5x/10h@Random prob=1x/24h@Random O2=2h/8h@Random [Obs Mem:5H Ref:Y Tool Plan]

>>> REFLECTION: D5(warn:74/52): To improve your fitness, consider increasing the `food_interval`
to 6 hours and decreasing the `food_quantity` to 15. This might help balance the fish's diet and potential
ly increase your fitness score. Changes will apply tomorrow.

Day 6 | Fit:0.8480 Yld:0.9388 Sav:0.6874 Hlth:0.8740 | Surv: 93.9% (46/49) | Cost:$ 156.30 Sav
e:$ 343.70 | Sim:0.1s LLM:12.4s
      | food=20x/5h@Random prob=4x/24h@Random O2=3h/6h@Random [Obs Mem:6H Ref:Y Tool Plan]

>>> REFLECTION: D6(20%): To increase fitness, you could try reducing the food interval to 2 hours
(food_interval=2). This might help boost your fish's health and fitness slightly.

Day 7 | Fit:0.8149 Yld:0.9184 Sav:0.6114 Hlth:0.9000 | Surv: 91.8% (45/49) | Cost:$ 194.30 Sav
e:$ 305.70 | Sim:0.2s LLM:2.0s
      | food=10x/8h@Random prob=1x/24h@Random O2=2h/8h@Random [Obs Mem:7H Ref:- Tool Plan]
Day 8 | Fit:0.8026 Yld:0.9184 Sav:0.5769 Hlth:0.8926 | Surv: 91.8% (45/49) | Cost:$ 211.55 Sav
e:$ 288.45 | Sim:0.2s LLM:2.8s
      | food=15x/2h@Random prob=2x/24h@Random O2=2h/6h@Random [Obs Mem:8H Ref:Y Tool Plan]

>>> REFLECTION: D8(warn:106/72): To increase your fitness score, consider reducing the `food inte
```

rval` to a lower value, such as 2 hours. This change might help in optimizing feeding times and potentially improving your fish's health and fitness score.

Day 9 | Fit:0.7756 Yld:0.8980 Sav:0.5159 Hlth:0.9286 | Surv: 89.8% (44/49) | Cost:\$ 242.05 Save:\$ 257.95 | Sim:0.2s LLM:3.0s
| food=10x/3h@Random prob=1x/48h@Random O2=2h/9h@Random [Obs Mem:9H Ref:Y Tool Plan]

>>> REFLECTION: D9(warn:87/82): To increase your fitness score, consider reducing the `food\_interval` to a lower value (e.g., from 1-2 hours to 1 hour) and decreasing `food\_quantity` slightly (e.g., from 25 to 20). This can help in spending less while maintaining or improving your fish health.

Day 10 | Fit:0.7651 Yld:0.8980 Sav:0.4779 Hlth:0.9456 | Surv: 89.8% (44/49) | Cost:\$ 261.05 Save:\$ 238.95 | Sim:0.2s LLM:2.7s
| food=15x/1h@Random prob=2x/24h@Random O2=2h/3h@Random [Obs Mem:10H Ref:- Tool Plan]

Day 11 | Fit:0.7293 Yld:0.8980 Sav:0.3589 Hlth:0.9750 | Surv: 89.8% (44/49) | Cost:\$ 320.55 Save:\$ 179.45 | Sim:0.3s LLM:3.0s
| food=15x/4h@Random prob=2x/24h@Random O2=2h/12h@Random [Obs Mem:11H Ref:Y Tool Plan]

>>> REFLECTION: D11(warn:206/101): To increase your fitness score, consider reducing the `food\_interval` to a lower value, such as 2 hours. This might help in spending less and increasing your Fitness score. For example:

```
...
food_interval=2
...
```

Day 12 | Fit:0.7180 Yld:0.8980 Sav:0.3269 Hlth:0.9689 | Surv: 89.8% (44/49) | Cost:\$ 336.55 Save:\$ 163.45 | Sim:0.3s LLM:9.2s
| food=8x/3h@Random prob=2x/24h@Random O2=2h/8h@Random [Obs Mem:12H Ref:Y Tool Plan]

>>> REFLECTION: D12(warn:136/112): To increase your fitness score, consider reducing the `food\_interval` to a lower value (e.g., from 1-6 hours) and decreasing `food\_quantity` slightly (e.g., from 5-20). This can help you save more resources, potentially increasing your fitness score.

Warn: No JSON. Fallback.
Day 13 | Fit:0.7035 Yld:0.8980 Sav:0.2875 Hlth:0.9563 | Surv: 89.8% (44/49) | Cost:\$ 356.25 Save:\$ 143.75 | Sim:0.4s LLM:2.3s
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:13H Ref:Y Tool Plan]

>>> REFLECTION: D13(warn:147/123): To increase your fitness, consider reducing the `food\_interval` to a lower value, such as `food\_interval=2`. This will decrease the time between feedings, potentially improving your fish's health and fitness.

Day 14 | Fit:0.6992 Yld:0.8980 Sav:0.2795 Hlth:0.9427 | Surv: 89.8% (44/49) | Cost:\$ 360.25 Save:\$ 139.75 | Sim:0.4s LLM:2.3s
| food=10x/2h@Random prob=2x/24h@Random O2=2h/8h@Random [Obs Mem:14H Ref:- Tool Plan]

Day 15 | Fit:0.6870 Yld:0.8980 Sav:0.2345 Hlth:0.9642 | Surv: 89.8% (44/49) | Cost:\$ 382.75 Save:\$ 117.25 | Sim:0.3s LLM:2.8s
| food=10x/2h@Random prob=1x/24h@Random O2=2h/8h@Random [Obs Mem:15H Ref:Y Tool Plan]

>>> REFLECTION: D15(warn:261/136): To increase your fitness, consider adjusting the `food\_interval` to a lower value (e.g., `food\_interval=2`) to ensure your fish receive food more frequently. This change might help maintain or improve your fish's health and fitness score.

Day 16 | Fit:0.6718 Yld:0.8980 Sav:0.1910 Hlth:0.9576 | Surv: 89.8% (44/49) | Cost:\$ 404.50 Save:\$ 95.50 | Sim:0.4s LLM:3.1s
| food=30x/5h@Random prob=1x/24h@Random O2=2h/16h@Random [Obs Mem:16H Ref:Y Tool Plan]

>>> REFLECTION: D16(warn:153/144): To increase your fitness score, you might consider reducing the `food\_interval` to a lower value (e.g., from 12h to 5h) since spending less on food can boost your Fitness score. Change it to `food\_interval=5`.

Day 17 | Fit:0.6590 Yld:0.8980 Sav:0.1545 Hlth:0.9509 | Surv: 89.8% (44/49) | Cost:\$ 422.75 Save:\$ 77.25 | Sim:0.4s LLM:5.2s
| food=20x/12h@Random prob=3x/24h@Random O2=4h/16h@Random [Obs Mem:17H Ref:Y Tool Plan]

>>> REFLECTION: D17(warn:168/152): To increase your fitness score, you might consider decreasing the `food\_interval` to a lower value (e.g., from 12h to 8h) and reducing the `food\_quantity` slightly (e.g., from 30 to 25). This could help in spending less while maintaining or improving your fitness.

Day 18 | Fit:0.6350 Yld:0.8776 Sav:0.1260 Hlth:0.9231 | Surv: 87.8% (43/49) | Cost:\$ 437.00 Save:\$ 63.00 | Sim:0.5s LLM:2.9s
| food=10x/4h@Random prob=2x/24h@Random O2=3h/6h@Random [Obs Mem:18H Ref:Y Tool Plan]

>>> REFLECTION: D18(warn:183/161): To increase your fitness, consider reducing the `food\_interval` to a lower value, such as 2 hours. This will allow your fish to eat more frequently, potentially improving their health and increasing your fitness score.

Day 19 | Fit:0.6111 Yld:0.8776 Sav:0.0570 Hlth:0.9132 | Surv: 87.8% (43/49) | Cost:\$ 471.50 Save:\$ 28.50 | Sim:0.4s LLM:2.7s
| food=10x/6h@Random prob=1x/24h@Random O2=4h/8h@Random [Obs Mem:19H Ref:Y Tool Plan]

>>> REFLECTION: D19(warn:237/172): To increase your fitness score, consider reducing the `food\_interval` to a lower value (e.g., from 24 hours to 6 hours) and decreasing `spending` by choosing a higher `food\_quantity` within its range. For example, you could set `food\_interval=6`, `food\_quantity`

Day 20 | Fit:0.0000 Yld:0.8776 Sav:0.0000 Hlth:0.8979 | Surv: 87.8% (43/49) | Cost:\$ 501.75 Save:\$ 0.00 | Sim:0.3s [OVER\$]

Gen 1 FINAL (attempt 3):
Fit:0.0000 | Yield:0.8776 | SavingRate:0.0000 | Healthiness:0.8979
Surv:87.8% (43/49) | Cost:\$501.75 | Save:\$0.00

>>> END-OF-GEN REFLECTION:
D17(warn:168/152): To increase your fitness score, you might consider decreasing the `food\_interval` to a lower value (e.g., from 12h to 8h) and reducing the `food\_quantity` slightly (e.g., from 30 to 25). This could help in spending less while maintaining or improving your fitness level.

D19(warn:

~~~~~  
Generation 2/5 (attempt 1)  
~~~~~  
Day 0 | Stocked 30 fish | food=20x/12h@Random prob=2x/24h@Random O2=2h/4h@Random | LLM:5.0s
Warn: No JSON. Fallback.
Day 1 | Fit:0.7607 Yld:0.6122 Sav:0.9330 Hlth:0.9669 | Surv:100.0% (30/30) | Cost:\$ 33.50 Save:\$ 466.50 | Sim:0.1s LLM:2.3s
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:1H Ref:- Tool Plan]

*** RESTARTING Gen 2 (attempt 1 -> 2) ***
Reason: Day 1 fitness 0.7607 < 0.9. Had 30 fish (max 49).
Reflection: Increase the food_interval to 8h and set fish_count to 40.

.
Generation 2/5 (attempt 2)
.
Day 0 | Stocked 30 fish | food=18x/8h@Random prob=1x/24h@Random O2=3h/12h@Random | LLM:5.4s
Warn: No JSON. Fallback.
Day 1 | Fit:0.7711 Yld:0.6122 Sav:0.9631 Hlth:0.9716 | Surv:100.0% (30/30) | Cost:\$ 18.45 Save:\$ 481.55 | Sim:0.0s LLM:2.4s
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:1H Ref:- Tool Plan]

*** RESTARTING Gen 2 (attempt 2 -> 3) ***
Reason: Day 1 fitness 0.7711 < 0.9. Had 30 fish (max 49).
Reflection: Increase the `food\_interval` to 8h and set `fish\_count` to 40.

.
Generation 2/5 (attempt 3)
.
Day 0 | Stocked 40 fish | food=25x/8h@Random prob=1x/24h@Random O2=2h/10h@Random | LLM:1.8s
Day 1 | Fit:0.8818 Yld:0.8163 Sav:0.9610 Hlth:0.9642 | Surv:100.0% (40/40) | Cost:\$ 19.50 Save:\$ 480.50 | Sim:0.1s LLM:4.4s
| food=18x/5h@Random prob=1x/144h@Random O2=2h/8h@Random [Obs Mem:1H Ref:- Tool Plan]
Day 2 | Fit:0.8678 Yld:0.8163 Sav:0.9220 Hlth:0.9544 | Surv:100.0% (40/40) | Cost:\$ 39.00 Save:\$ 461.00 | Sim:0.1s LLM:2.7s
| food=25x/6h@Random prob=3x/144h@Random O2=1h/8h@Random [Obs Mem:2H Ref:- Tool Plan]
Day 3 | Fit:0.8583 Yld:0.8163 Sav:0.8970 Hlth:0.9445 | Surv:100.0% (40/40) | Cost:\$ 51.50 Save:\$ 448.50 | Sim:0.1s LLM:3.0s
| food=15x/3h@Random prob=3x/144h@Random O2=2h/8h@Random [Obs Mem:3H Ref:- Tool Plan]
Warn: No JSON. Fallback.
Day 4 | Fit:0.8439 Yld:0.8163 Sav:0.8550 Hlth:0.9399 | Surv:100.0% (40/40) | Cost:\$ 72.50 Save:\$ 427.50 | Sim:0.1s LLM:2.2s
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:4H Ref:- Tool Plan]
Day 5 | Fit:0.8397 Yld:0.8163 Sav:0.8455 Hlth:0.9307 | Surv:100.0% (40/40) | Cost:\$ 77.25 Save:\$ 422.75 | Sim:0.1s LLM:6.6s
| food=20x/12h@Random prob=1x/24h@Random O2=4h/12h@Random [Obs Mem:5H Ref:- Tool Plan]
Day 6 | Fit:0.8251 Yld:0.8163 Sav:0.8000 Hlth:0.9341 | Surv:100.0% (40/40) | Cost:\$ 100.00 Save:\$ 400.00 | Sim:0.1s LLM:2.8s
| food=20x/8h@Random prob=1x/24h@Random O2=2h/10h@Random [Obs Mem:6H Ref:Y Tool Plan]

>>> REFLECTION: D6(warn:55/44): To increase your fitness score, you might consider reducing the food interval to 8 hours (`food\_interval=8`) and decreasing the food quantity to 20 (`food\_quantity=20`). This could help save more of your resources, potentially increasing your fitness score.

Day 7 | Fit:0.8147 Yld:0.8163 Sav:0.7725 Hlth:0.9231 | Surv:100.0% (40/40) | Cost:\$ 113.75 Save:\$ 386.25 | Sim:0.1s LLM:3.6s
| food=20x/8h@Random prob=1x/24h@Random O2=3h/6h@Random [Obs Mem:7H Ref:- Tool Plan]
Warn: No JSON. Fallback.
Day 8 | Fit:0.7719 Yld:0.7755 Sav:0.7050 Hlth:0.9394 | Surv: 95.0% (38/40) | Cost:\$ 147.50 Save:\$ 352.50 | Sim:0.1s LLM:1.9s
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:8H Ref:Y Tool Plan]

>>> REFLECTION: D8(warn:93/58): To increase your fitness score, consider reducing the `food\_interval` to a lower value, such as 2 hours. This will allow you to feed your fish more frequently, potentially improving their health and increasing your fitness score. For example:

...
food_interval=2

Day 9 | Fit:0.7594 Yld:0.7551 Sav:0.6955 Hlth:0.9547 | Surv: 92.5% (37/40) | Cost:\$ 152.25 Save:\$ 347.75 | Sim:0.1s LLM:1.9s
| food=10x/1h@Random prob=2x/24h@Random O2=2h/8h@Random [Obs Mem:9H Ref:Y Tool Plan]

>>> REFLECTION: D9(warn:73/67): To increase your fitness score, consider reducing the `food\_interval` to a lower value, such as 2 hours. This might help in optimizing the feeding schedule and potentially improving your fish's health and fitness.

Day 10 | Fit:0.7433 Yld:0.7551 Sav:0.6385 Hlth:0.9770 | Surv: 92.5% (37/40) | Cost:\$ 180.75 Save:\$ 319.25 | Sim:0.2s LLM:2.5s
| food=10x/4h@Random prob=3x/24h@Random O2=4h/8h@Random [Obs Mem:10H Ref:Y Tool Plan]

>>> REFLECTION: D10(warn:83/76): To increase your fitness score, consider reducing the `food\_interval` to a lower value, such as 2 hours. This will allow you to feed your fish more frequently, potentially improving their health and increasing your fitness score. Change `food\_interval=2`.

Day 11 | Fit:0.7198 Yld:0.7551 Sav:0.5680 Hlth:0.9754 | Surv: 92.5% (37/40) | Cost:\$ 216.00 Save:\$ 284.00 | Sim:0.2s LLM:2.7s
| food=10x/6h@Random prob=2x/24h@Random O2=2h/6h@Random [Obs Mem:11H Ref:Y Tool Plan]

>>> REFLECTION: D11(warn:116/87): To increase your fitness score, consider reducing the `food\_interval` to a lower value, such as 2 hours. This will allow you to feed your fish more frequently, potentially improving their health and increasing your fitness score.

Day 12 | Fit:0.7044 Yld:0.7551 Sav:0.5210 Hlth:0.9766 | Surv: 92.5% (37/40) | Cost:\$ 239.50 Save:\$ 260.50 | Sim:0.2s LLM:2.0s
| food=18x/6h@Random prob=1x/48h@Random O2=2h/15h@Random [Obs Mem:12H Ref:Y Tool Plan]

>>> REFLECTION: D12(warn:151/101): To increase your fitness score, consider reducing the `food\_interval` to a lower value (e.g., from 1-2 hours to 1 hour) and decreasing `food\_quantity` slightly (e.g., from 25 to 20). This might help in spending less while maintaining or improving your fitness score.

Day 13 | Fit:0.6969 Yld:0.7551 Sav:0.5023 Hlth:0.9655 | Surv: 92.5% (37/40) | Cost:\$ 248.85 Save:\$ 251.15 | Sim:0.3s LLM:3.1s
| food=20x/4h@Random prob=3x/24h@Random O2=4h/16h@Random [Obs Mem:13H Ref:Y Tool Plan]

>>> REFLECTION: D13(warn:147/113): To increase your fitness score, consider reducing the `food\_interval` to a lower value (e.g., from 12h to 8h) and decreasing the `food\_quantity` slightly (e.g., from 30 to 25). This might help in spending less while maintaining or improving your fish's health.

Day 14 | Fit:0.6859 Yld:0.7551 Sav:0.4658 Hlth:0.9739 | Surv: 92.5% (37/40) | Cost:\$ 267.10 Save:\$ 232.90 | Sim:0.3s LLM:2.7s
| food=20x/3h@Random prob=2x/24h@Random O2=1h/4h@Random [Obs Mem:14H Ref:Y Tool Plan]

>>> REFLECTION: D14(warn:136/122): To increase your fitness score, consider adjusting the `food\_interval` to a lower value (e.g., from 1-5 hours) to ensure more frequent feeding, which can help maintain or improve the fish's health and thus the fitness score.

Day 15 | Fit:0.6702 Yld:0.7551 Sav:0.4168 Hlth:0.9775 | Surv: 92.5% (37/40) | Cost:\$ 291.60 Save:\$ 208.40 | Sim:0.3s LLM:7.0s
| food=20x/8h@Random prob=2x/24h@Random O2=3h/16h@Random [Obs Mem:15H Ref:- Tool Plan]

Day 16 | Fit:0.6618 Yld:0.7551 Sav:0.3928 Hlth:0.9738 | Surv: 92.5% (37/40) | Cost:\$ 303.60 Save:\$ 196.40 | Sim:0.3s LLM:3.0s
| food=15x/2h@Random prob=1x/24h@Random O2=3h/6h@Random [Obs Mem:16H Ref:Y Tool Plan]

>>> REFLECTION: D16(warn:266/135): To increase your fitness score, you could try reducing the `food\_interval` to a lower value, such as 2 hours, and decreasing `food\_quantity` to 15. This might help in spending less while maintaining or improving your fish health. Changes will apply tomorrow.

Day 17 | Fit:0.6363 Yld:0.7551 Sav:0.3133 Hlth:0.9800 | Surv: 92.5% (37/40) | Cost:\$ 343.35 Save:\$ 156.65 | Sim:0.3s LLM:2.9s
| food=10x/3h@Random prob=1x/24h@Random O2=2h/2h@Random [Obs Mem:17H Ref:Y Tool Plan]

>>> REFLECTION: D17(warn:142/141): To increase your fitness score, consider adjusting the `food\_interval` to a lower value (e.g., from 1-2 hours to 1 hour) and the `food\_quantity` to a higher value (e.g., from 5 to 20). This might help in providing more frequent and substantial feeding, which could

Day 18 | Fit:0.5927 Yld:0.7551 Sav:0.1838 Hlth:0.9726 | Surv: 92.5% (37/40) | Cost:\$ 408.10 Save:\$ 91.90 | Sim:0.4s LLM:23.2s
| food=7x/12h@Random prob=3x/24h@Random O2=2h/12h@Random [Obs Mem:18H Ref:Y Tool Plan]

>>> REFLECTION: D18(warn:168/149): To increase your fitness score, you might consider reducing the `food\_interval` to a lower value (e.g., from 24 hours to 6 hours) and decreasing the `food\_quantity` slightly (e.g., from 10 to 7). This could help in spending less while maintaining or improving your

```
gently (e.g., from 10 to 7). This could help in spending less while maintaining or improving y

Warn: No JSON. Fallback.
Day 19 | Fit:0.5822 Yld:0.7551 Sav:0.1579 Hlth:0.9562 | Surv: 92.5% (37/40) | Cost:$ 421.05 Sav
e:$ 78.95 | Sim:0.4s LLM:2.1s
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:19H Ref:Y Tool Plan]

>>> REFLECTION: D19(warn:187/158): To increase your fitness score, consider reducing the `food_in
terval` to a lower value (e.g., from 1-6 hours) and increasing the `food_quantity` slightly (e.g., from 10
-20). This might help in providing more frequent and adequate nutrition to the fish, potentia

Day 20 | Fit:0.5791 Yld:0.7551 Sav:0.1499 Hlth:0.9528 | Surv: 92.5% (37/40) | Cost:$ 425.05 Sav
e:$ 74.95 | Sim:0.4s LLM:3.3s
| food=15x/2h@Random prob=2x/24h@Random O2=2h/12h@Random [Obs Mem:20H Ref:Y Tool Plan]

>>> REFLECTION: D20(warn:203/166): To increase your fitness score, you could try reducing the `fo
od_interval` to a lower value, such as 2 hours. This might help your fish receive more frequent feedings,
potentially improving their health and increasing your fitness score.

Day 21 | Fit:0.5665 Yld:0.7551 Sav:0.1089 Hlth:0.9605 | Surv: 92.5% (37/40) | Cost:$ 445.55 Sav
e:$ 54.45 | Sim:0.5s LLM:3.7s
| food=10x/2h@Random prob=1x/24h@Random O2=4h/6h@Random [Obs Mem:21H Ref:Y Tool Plan]

>>> REFLECTION: D21(warn:221/176): To increase your fitness score, consider adjusting the `food_i
nterval` to a lower value (e.g., from 1-2 hours to 1 hour) and the `food_quantity` to a higher value (e.g.
, from 5 to 20). This can help boost your fish's energy levels more frequently, potentially i

Day 22 | Fit:0.5365 Yld:0.7551 Sav:0.0154 Hlth:0.9672 | Surv: 92.5% (37/40) | Cost:$ 492.30 Sav
e:$ 7.70 | Sim:0.5s LLM:2.8s
| food=15x/1h@Random prob=1x/24h@Random O2=2h/20h@Random [Obs Mem:22H Ref:Y Tool Plan]

>>> REFLECTION: D22(warn:204/183): To increase your fitness score, consider reducing the `food_in
terval` to a lower value (e.g., from 1-2 hours to 1 hour). This might help in spending less and increasing
your Fitness score. Changes will apply tomorrow.

Day 23 | Fit:0.0000 Yld:0.7551 Sav:0.0000 Hlth:0.9681 | Surv: 92.5% (37/40) | Cost:$ 500.55 Sav
e:$ 0.00 | Sim:0.2s [OVER$]

Gen 2 FINAL (attempt 3):
Fit:0.0000 | Yield:0.7551 | SavingRate:0.0000 | Healthiness:0.9681
Surv:92.5% (37/40) | Cost:$500.55 | Save:$0.00

>>> END-OF-GEN REFLECTION:
To increase your fitness score by reducing expenses, decrease the `food_interval` to 2 hours startin
g tomorrow. This will allow for more frequent feedings while potentially lowering costs.

To further improve your fitness score and reduce spending, change the `food_interval` to 1 hour and the `f
ood_

~~~~~
Generation 3/5 (attempt 1)
~~~~~
Day 0 | Stocked 40 fish | food=15x/6h@Random prob=1x/24h@Random O2=3h/5h@Random | LLM:4.8s
Warn: No JSON. Fallback.
Day 1 | Fit:0.8681 Yld:0.8163 Sav:0.9175 Hlth:0.9696 | Surv:100.0% (40/40) | Cost:$ 41.25 Sav
e:$ 458.75 | Sim:0.1s LLM:2.1s
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:1H Ref:- Tool Plan]

*** RESTARTING Gen 3 (attempt 1 -> 2) ***
Reason: Day 1 fitness 0.8681 < 0.9. Had 40 fish (max 49).
Reflection: Change the `fish_count` to 49 by setting `fish_count=49`. This will maximize the number of f
ish in your system.

. . . . .
Generation 3/5 (attempt 2)
. . . . .
Day 0 | Stocked 49 fish | food=10x/12h@Random prob=1x/24h@Random O2=2h/8h@Random | LLM:150.5s
Warn: No JSON. Fallback.
Day 1 | Fit:0.9809 Yld:1.0000 Sav:0.9665 Hlth:0.9329 | Surv:100.0% (49/49) | Cost:$ 16.75 Sav
e:$ 483.25 | Sim:0.1s LLM:2.3s
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:1H Ref:- Tool Plan]
Warn: No JSON. Fallback.
Day 2 | Fit:0.9788 Yld:1.0000 Sav:0.9585 Hlth:0.9374 | Surv:100.0% (49/49) | Cost:$ 20.75 Sav
e:$ 479.25 | Sim:0.1s LLM:2.5s
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:2H Ref:- Tool Plan]
Day 3 | Fit:0.9757 Yld:1.0000 Sav:0.9490 Hlth:0.9375 | Surv:100.0% (49/49) | Cost:$ 25.50 Sav
e:$ 474.50 | Sim:0.1s LLM:2.7s
| food=20x/6h@Random prob=3x/24h@Random O2=1h/4h@Random [Obs Mem:3H Ref:- Tool Plan]
Day 4 | Fit:0.9636 Yld:1.0000 Sav:0.9065 Hlth:0.9537 | Surv:100.0% (49/49) | Cost:$ 46.75 Sav
e:$ 453.25 | Sim:0.1s LLM:2.9s
| food=5x/2h@Random prob=1x/24h@Random O2=4h/8h@Random [Obs Mem:4H Ref:Y Tool Plan]
```

>>> REFLECTION: D4(warn:58/52): To increase your fitness score, consider reducing the `food\_interval` to a lower value, such as 2 hours. This will help you spend less and potentially increase your fitness score. For example:

...

food_interval=2

...

Warn: No JSON. Fallback.

Day 5		Fit:0.9423	Yld:1.0000	Sav:0.8390	Hlth:0.9618		Surv:100.0%	(49/49)		Cost:\$ 80.50	Sav e:\$ 419.50		Sim:0.1s	LLM:2.5s
food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:5H Ref:- Tool Plan]														
Day 6		Fit:0.9356	Yld:1.0000	Sav:0.8260	Hlth:0.9415		Surv:100.0%	(49/49)		Cost:\$ 87.00	Sav e:\$ 413.00		Sim:0.1s	LLM:3.1s
food=25x/3h@Random prob=4x/24h@Random O2=4h/10h@Random [Obs Mem:6H Ref:Y Tool Plan]														

>>> REFLECTION: D6(20%): To increase fitness, consider reducing the food interval to 2 hours (food_interval=2). This change applies tomorrow.

Day 7		Fit:0.9166	Yld:1.0000	Sav:0.7600	Hlth:0.9654		Surv:100.0%	(49/49)		Cost:\$ 120.00	Sav e:\$ 380.00		Sim:0.1s	LLM:53.2s
food=15x/2h@Random prob=2x/24h@Random O2=3h/8h@Random [Obs Mem:7H Ref:- Tool Plan]														
Day 8		Fit:0.8947	Yld:1.0000	Sav:0.6940	Hlth:0.9641		Surv:100.0%	(49/49)		Cost:\$ 153.00	Sav e:\$ 347.00		Sim:0.2s	LLM:3.7s
food=10x/6h@Random prob=1x/24h@Random O2=2h/8h@Random [Obs Mem:8H Ref:Y Tool Plan]														

>>> REFLECTION: D8(warn:94/67): To increase your fitness score, consider reducing the `food\_interval` to a lower value (e.g., from 1-6 hours) and decreasing `food\_quantity` slightly (e.g., from 20 to 15). This might help in spending less while maintaining fish health.

Day 9		Fit:0.8805	Yld:1.0000	Sav:0.6585	Hlth:0.9431		Surv:100.0%	(49/49)		Cost:\$ 170.75	Sav e:\$ 329.25		Sim:0.2s	LLM:4.3s
food=10x/2h@Random prob=1x/24h@Random O2=2h/8h@Random [Obs Mem:9H Ref:- Tool Plan]														
Day 10		Fit:0.8575	Yld:0.9796	Sav:0.6150	Hlth:0.9648		Surv: 98.0%	(48/49)		Cost:\$ 192.50	Sav e:\$ 307.50		Sim:0.2s	LLM:2.9s
food=10x/1h@Random prob=1x/24h@Random O2=2h/8h@Random [Obs Mem:10H Ref:Y Tool Plan]														

>>> REFLECTION: D10(warn:133/80): To increase your fitness score, you could try reducing the `food\_interval` to a lower value, such as 2 hours. This might encourage more frequent feeding, potentially improving your fish's health and fitness score.

Day 11		Fit:0.8405	Yld:0.9796	Sav:0.5595	Hlth:0.9755		Surv: 98.0%	(48/49)		Cost:\$ 220.25	Sav e:\$ 279.75		Sim:0.2s	LLM:2.7s
food=15x/4h@Random prob=1x/24h@Random O2=2h/12h@Random [Obs Mem:11H Ref:- Tool Plan]														
Day 12		Fit:0.8304	Yld:0.9796	Sav:0.5290	Hlth:0.9758		Surv: 98.0%	(48/49)		Cost:\$ 235.50	Sav e:\$ 264.50		Sim:0.3s	LLM:2.5s
food=25x/8h@Random prob=3x/24h@Random O2=2h/12h@Random [Obs Mem:12H Ref:Y Tool Plan]														

>>> REFLECTION: D12(warn:161/91): To increase your fitness score, consider reducing the `food\_interval` to a lower value (e.g., from 12h to 8h) and decreasing `spending` by choosing a lower `food\_quantity`. For example, you could change `food\_quantity` from 30 to 25. This might help boost your f

Day 13		Fit:0.8186	Yld:0.9796	Sav:0.4970	Hlth:0.9649		Surv: 98.0%	(48/49)		Cost:\$ 251.50	Sav e:\$ 248.50		Sim:0.3s	LLM:11.1s
food=25x/12h@Random prob=2x/24h@Random O2=4h/16h@Random [Obs Mem:13H Ref:- Tool Plan]														
Warn: No JSON. Fallback.														
Day 14		Fit:0.8088	Yld:0.9796	Sav:0.4690	Hlth:0.9607		Surv: 98.0%	(48/49)		Cost:\$ 265.50	Sav e:\$ 234.50		Sim:0.3s	LLM:181.8s
food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:14H Ref:Y Tool Plan]														

>>> REFLECTION: D14(warn:196/104): To increase your fitness score, consider reducing the `food\_interval` to a lower value (e.g., from 1-3 hours) and decreasing `food\_quantity` slightly (e.g., to 25). This might help in saving more resources while maintaining fish health.

Day 15		Fit:0.8042	Yld:0.9796	Sav:0.4595	Hlth:0.9482		Surv: 98.0%	(48/49)		Cost:\$ 270.25	Sav e:\$ 229.75		Sim:0.3s	LLM:3.3s
food=10x/3h@Random prob=2x/24h@Random O2=2h/8h@Random [Obs Mem:15H Ref:Y Tool Plan]														

>>> REFLECTION: D15(warn:128/112): To increase your fitness, consider reducing the `food\_interval` to a lower value, such as `food\_interval=3`. This will decrease the time between feedings, potentially increasing your fish's energy and fitness.

Day 16		Fit:0.7902	Yld:0.9796	Sav:0.4185	Hlth:0.9444		Surv: 98.0%	(48/49)		Cost:\$ 290.75	Sav e:\$ 209.25		Sim:0.3s	LLM:2.6s
food=10x/4h@Random prob=2x/24h@Random O2=2h/2h@Random [Obs Mem:16H Ref:Y Tool Plan]														

>>> REFLECTION: D16(warn:172/123): To increase your fitness, consider reducing the `food\_interval` to a lower value, such as 2 hours. This might help in maintaining higher fish health and fitness levels.

Day 17		Fit:0.7470	Yld:0.9796	Sav:0.2895	Hlth:0.9392		Surv: 98.0%	(48/49)		Cost:\$ 355.25	Sav e:\$ 144.75		Sim:0.3s	LLM:3.9s
--------	--	------------	------------	------------	-------------	--	-------------	---------	--	----------------	-----------------	--	----------	----------

```
e:$ 111.75 | Sim:0.3s LLM:3.3s
| food=15x/6h@Random prob=1x/24h@Random O2=2h/10h@Random [Obs Mem:17H Ref:Y Tool Plan]

>>> REFLECTION: D17(warn:187/134): To increase your fitness, you might consider reducing the `food_interval` to a lower value (e.g., from 1-12h to something like 2-6h) and decreasing the `food_quantity` slightly (e.g., from 5-30 to around 15-20). This could help in spending less while maintainin

Day 18 | Fit:0.7343 Yld:0.9796 Sav:0.2520 Hlth:0.9362 | Surv: 98.0% (48/49) | Cost:$ 374.00 Save:$ 126.00 | Sim:0.4s LLM:3.3s
| food=15x/6h@Random prob=2x/24h@Random O2=2h/4h@Random [Obs Mem:18H Ref:Y Tool Plan]

>>> REFLECTION: D18(warn:199/144): To increase your fitness score, consider reducing the `food_interval` to a lower value, such as 2 hours. This might help in boosting your fish's health and increasing the fitness score. For example:

...
food_interval=2
...

Day 19 | Fit:0.7098 Yld:0.9796 Sav:0.1830 Hlth:0.9221 | Surv: 98.0% (48/49) | Cost:$ 408.50 Save:$ 91.50 | Sim:0.5s LLM:3.5s
| food=5x/1h@Random prob=1x/24h@Random O2=2h/8h@Random [Obs Mem:19H Ref:Y Tool Plan]

>>> REFLECTION: D19(warn:196/154): To increase your fitness score, you might consider reducing the `food_interval` to a lower value, such as 2 hours, and decreasing the `food_quantity` to a lower amount, like 10. This could help you save more resources, potentially increasing your fitness score.

Day 20 | Fit:0.6988 Yld:0.9796 Sav:0.1395 Hlth:0.9497 | Surv: 98.0% (48/49) | Cost:$ 430.25 Save:$ 69.75 | Sim:0.4s LLM:2.6s
| food=15x/4h@Random prob=2x/24h@Random O2=2h/8h@Random [Obs Mem:20H Ref:Y Tool Plan]

>>> REFLECTION: D20(warn:198/163): To increase your fitness, consider reducing the `food_interval` to 2 hours. This will allow for more frequent feeding, potentially improving your fish's health and fitness score.

Day 21 | Fit:0.6753 Yld:0.9592 Sav:0.0975 Hlth:0.9634 | Surv: 95.9% (47/49) | Cost:$ 451.25 Save:$ 48.75 | Sim:0.5s LLM:2.1s
| food=10x/4h@Random prob=2x/24h@Random O2=2h/12h@Random [Obs Mem:21H Ref:Y Tool Plan]

>>> REFLECTION: D21(warn:210/171): To increase your fitness score, consider reducing the `food_interval` to a lower value, such as 2 hours. This could potentially boost your fish's health and fitness more effectively.

Day 22 | Fit:0.6631 Yld:0.9592 Sav:0.0685 Hlth:0.9416 | Surv: 95.9% (47/49) | Cost:$ 465.75 Save:$ 34.25 | Sim:0.5s LLM:2.2s
| food=15x/8h@Random prob=1x/24h@Random O2=2h/4h@Random [Obs Mem:22H Ref:Y Tool Plan]

>>> REFLECTION: D22(warn:181/177): To increase your fitness score, consider reducing the `food_interval` to a lower value (e.g., from 1-2 hours to 1 hour). This will help you spend less and potentially increase your fitness score.

Warn: No JSON. Fallback.
Day 23 | Fit:0.6293 Yld:0.9388 Sav:0.0025 Hlth:0.9344 | Surv: 93.9% (46/49) | Cost:$ 498.75 Save:$ 1.25 | Sim:0.5s LLM:181.1s
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:23H Ref:- Tool Plan]
Day 24 | Fit:0.0000 Yld:0.9388 Sav:0.0000 Hlth:0.9282 | Surv: 93.9% (46/49) | Cost:$ 502.25 Save:$ 0.00 | Sim:0.2s [OVER$]

Gen 3 FINAL (attempt 2):
Fit:0.0000 | Yield:0.9388 | SavingRate:0.0000 | Healthiness:0.9282
Surv:93.9% (46/49) | Cost:$502.25 | Save:$0.00

>>> END-OF-GEN REFLECTION:
To increase your fitness, reduce the `food_interval` to 2 hours. This allows for more frequent feeding, which can improve health and fitness.

For better management of resources, set `food_interval` to 1 hour starting tomorrow. This will help you spend less and boost your fitness score.

~~~~~
Generation 4/5 (attempt 1)
~~~~~
Day 0 | Stocked 49 fish | food=20x/1h@Random prob=1x/24h@Random O2=3h/5h@Random | LLM:3.8s
Day 1 | Fit:0.9561 Yld:1.0000 Sav:0.8755 Hlth:0.9770 | Surv:100.0% (49/49) | Cost:$ 62.25 Save:$ 437.75 | Sim:0.1s LLM:2.6s
| food=5x/6h@Random prob=1x/24h@Random O2=3h/12h@Random [Obs Mem:1H Ref:- Tool Plan]
Warn: No JSON. Fallback.
Day 2 | Fit:0.9450 Yld:1.0000 Sav:0.8420 Hlth:0.9762 | Surv:100.0% (49/49) | Cost:$ 79.00 Save:$ 421.00 | Sim:0.2s LLM:2.0s
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:2H Ref:- Tool Plan]
Day 3 | Fit:0.9416 Yld:1.0000 Sav:0.8325 Hlth:0.9740 | Surv:100.0% (49/49) | Cost:$ 83.75 Save:
```

e:\$ 416.25 | Sim:0.2s LLM:3.0s
| food=20x/2h@Right-Ctr prob=1x/24h@Random O2=2h/4h@Random [Obs Mem:3H Ref:- Tool Plan]
Day 4 | Fit:0.9136 Yld:1.0000 Sav:0.7470 Hlth:0.9754 | Surv:100.0% (49/49) | Cost:\$ 126.50 Sav
e:\$ 373.50 | Sim:0.2s LLM:2.8s
| food=10x/2h@Random prob=1x/24h@Random O2=2h/6h@Random [Obs Mem:4H Ref:- Tool Plan]
Day 5 | Fit:0.8961 Yld:1.0000 Sav:0.6935 Hlth:0.9774 | Surv:100.0% (49/49) | Cost:\$ 153.25 Sav
e:\$ 346.75 | Sim:0.2s LLM:3.5s
| food=5x/4h@Random prob=2x/24h@Random O2=2h/8h@Random [Obs Mem:5H Ref:Y Tool Plan]

>>> REFLECTION: D5(warn:43/43): To increase your fitness score, consider reducing the food interval to 2 hours (`food\_interval=2`). This might help in maintaining a higher fish health and thus improve the fitness score.

Day 6 | Fit:0.8825 Yld:1.0000 Sav:0.6575 Hlth:0.9624 | Surv:100.0% (49/49) | Cost:\$ 171.25 Sav
e:\$ 328.75 | Sim:0.2s LLM:24.1s
| food=12x/1h@Random prob=2x/24h@Random O2=4h/1h@Random [Obs Mem:6H Ref:Y Tool Plan]

>>> REFLECTION: D6(20%): To increase your fitness, consider reducing the food interval to 2 hours (food_interval=2). This change might help conserve more of your fish's energy, potentially increasing your fitness score.

Day 7 | Fit:0.8321 Yld:1.0000 Sav:0.5057 Hlth:0.9603 | Surv:100.0% (49/49) | Cost:\$ 247.15 Sav
e:\$ 252.85 | Sim:0.2s LLM:2.6s
| food=15x/2h@Random prob=1x/24h@Random O2=1h/16h@Random [Obs Mem:7H Ref:- Tool Plan]
Day 8 | Fit:0.8055 Yld:0.9796 Sav:0.4662 Hlth:0.9407 | Surv: 98.0% (48/49) | Cost:\$ 266.90 Sav
e:\$ 233.10 | Sim:0.3s LLM:2.1s
| food=15x/3h@Random prob=2x/24h@Random O2=2h/6h@Random [Obs Mem:8H Ref:Y Tool Plan]

>>> REFLECTION: D8(warn:146/73): To increase your fitness, consider reducing the `food\_interval` to 2 hours (`food\_interval=2`). This might help in maintaining a higher food supply for your fish, potentially improving their health and increasing your fitness score.

Day 9 | Fit:0.7882 Yld:0.9796 Sav:0.4112 Hlth:0.9476 | Surv: 98.0% (48/49) | Cost:\$ 294.40 Sav
e:\$ 205.60 | Sim:0.3s LLM:2.2s
| food=10x/3h@Random prob=2x/24h@Random O2=2h/5h@Random [Obs Mem:9H Ref:Y Tool Plan]

>>> REFLECTION: D9(warn:129/90): To increase your fitness, consider reducing the `food\_interval` to a lower value (e.g., `food\_interval=3`) to ensure your fish are fed more frequently. This can help maintain their health and potentially boost your fitness score.

Day 10 | Fit:0.7720 Yld:0.9796 Sav:0.3602 Hlth:0.9533 | Surv: 98.0% (48/49) | Cost:\$ 319.90 Sav
e:\$ 180.10 | Sim:0.4s LLM:9.2s
| food=15x/10h@Random prob=1x/24h@Random O2=4h/6h@Random [Obs Mem:10H Ref:Y Tool Plan]

>>> REFLECTION: D10(warn:170/109): To increase your fitness, consider reducing the `food\_interval` to a lower value, such as `food\_interval=6`. This might help in optimizing feeding times and potentially increasing your fish's fitness.

Day 11 | Fit:0.7419 Yld:0.9796 Sav:0.2742 Hlth:0.9383 | Surv: 98.0% (48/49) | Cost:\$ 362.90 Sav
e:\$ 137.10 | Sim:0.4s LLM:1.6s
| food=20x/6h@Random prob=3x/24h@Random O2=2h/8h@Random [Obs Mem:11H Ref:Y Tool Plan]

>>> REFLECTION: D11(warn:183/127): To increase your fitness score, consider reducing the `food\_interval` to a lower value (e.g., from 1-12h to 1-5h) and decreasing `oxygen\_duration` to a shorter duration (e.g., from 1-4h to 1-2h). This will help you spend less while maintaining or improving your

Day 12 | Fit:0.7168 Yld:0.9592 Sav:0.2317 Hlth:0.9397 | Surv: 95.9% (47/49) | Cost:\$ 384.15 Sav
e:\$ 115.85 | Sim:0.5s LLM:3.8s
| food=12x/4h@Random prob=2x/24h@Random O2=2h/8h@Random [Obs Mem:12H Ref:Y Tool Plan]

>>> REFLECTION: D12(warn:214/146): To increase your fitness score, consider decreasing the `food\_interval` to a lower value (e.g., `food\_interval=5`). This might encourage more frequent feeding, potentially improving your fish's health and fitness score.

Day 13 | Fit:0.6937 Yld:0.9388 Sav:0.1915 Hlth:0.9513 | Surv: 93.9% (46/49) | Cost:\$ 404.25 Sav
e:\$ 95.75 | Sim:0.5s LLM:3.3s
| food=10x/6h@Random prob=2x/24h@Random O2=2h/6h@Random [Obs Mem:13H Ref:Y Tool Plan]

>>> REFLECTION: D13(warn:201/160): To increase your fitness score, consider adjusting the `food\_interval` to a lower value (e.g., 2-5 hours) since higher frequency feeding might help maintain better fish health and performance.

Day 14 | Fit:0.6776 Yld:0.9388 Sav:0.1445 Hlth:0.9469 | Surv: 93.9% (46/49) | Cost:\$ 427.75 Sav
e:\$ 72.25 | Sim:0.4s LLM:2.1s
| food=15x/2h@Random prob=2x/24h@Random O2=3h/6h@Random [Obs Mem:14H Ref:Y Tool Plan]

>>> REFLECTION: D14(warn:197/172): To increase your fitness score, you might consider adjusting the `food\_interval` to a lower value (e.g., from 1-3 hours) to ensure your fish receive food more frequently, which can improve their health and thus your fitness score.

Day 15 | Fit:0.6520 Yld:0.9388 Sav:0.0635 Hlth:0.9556 | Surv: 93.9% (46/49) | Cost:\$ 468.25 Sav
e:\$ 31.75 | Sim:0.4s LLM:3.0s

| food=30x/12h@Random prob=1x/24h@Random O2=4h/15h@Random [Obs Mem:15H Ref:Y Tool Plan]

>>> REFLECTION: D15(warn:266/189): To increase your fitness score, consider reducing the `food\_interval` to a higher value (e.g., from 1-12h to 7-10h) and decreasing `spending` by optimizing other parameters like `food\_quantity`, `probiotic\_quantity`, etc. For example, you could set `food_interv

Day 16 | Fit:0.6340 Yld:0.9388 Sav:0.0160 Hlth:0.9369 | Surv: 93.9% (46/49) | Cost:\$ 492.00 Save:\$ 8.00 | Sim:0.5s LLM:2.4s
| food=30x/4h@Random prob=1x/24h@Random O2=2h/12h@Random [Obs Mem:16H Ref:Y Tool Plan]

>>> REFLECTION: D16(warn:257/203): To increase your fitness score, you might consider decreasing the `food\_interval` to a lower value (e.g., from 12h to 5h) since it's currently set higher than necessary and could be optimized further. Changes apply tomorrow.

Day 17 | Fit:0.0000 Yld:0.9388 Sav:0.0000 Hlth:0.9432 | Surv: 93.9% (46/49) | Cost:\$ 500.75 Save:\$ 0.00 | Sim:0.1s [OVER\$]

Gen 4 FINAL (attempt 1):
Fit:0.0000 | Yield:0.9388 | SavingRate:0.0000 | Healthiness:0.9432
Surv:93.9% (46/49) | Cost:\$500.75 | Save:\$0.00

>>> END-OF-GEN REFLECTION:
D14(warn:197/172): To increase your fitness score, you might consider adjusting the `food\_interval` to a lower value (e.g., from 6 hours to 3 hours) to ensure your fish receive food more frequently, which can improve their health and thus your fitness score.

D15(warn:266/189): To increase your fitn

~~~~~  
Generation 5/5 (attempt 1)  
~~~~~  
Day 0 | Stocked 30 fish | food=15x/6h@Random prob=1x/168h@Random O2=4h/10h@Random | LLM:2.1s
Day 1 | Fit:0.7609 Yld:0.6122 Sav:0.9325 Hlth:0.9702 | Surv:100.0% (30/30) | Cost:\$ 33.75 Save:\$ 466.25 | Sim:0.1s LLM:6.0s
| food=20x/1h@Random prob=1x/24h@Random O2=2h/10h@Random [Obs Mem:1H Ref:- Tool Plan]

*** RESTARTING Gen 5 (attempt 1 -> 2) ***
Reason: Day 1 fitness 0.7609 < 0.9. Had 30 fish (max 49).
Reflection: Increase the food_interval to 8h and set the fish_count to 40.

.
Generation 5/5 (attempt 2)
.
Day 0 | Stocked 49 fish | food=25x/8h@Random prob=1x/48h@Random O2=3h/10h@Random | LLM:2.7s
Day 1 | Fit:0.9787 Yld:1.0000 Sav:0.9460 Hlth:0.9711 | Surv:100.0% (49/49) | Cost:\$ 27.00 Save:\$ 473.00 | Sim:0.1s LLM:1.5s
| food=15x/4h@Random prob=1x/24h@Random O2=3h/6h@Random [Obs Mem:1H Ref:- Tool Plan]
Day 2 | Fit:0.9439 Yld:0.9796 Sav:0.8755 Hlth:0.9681 | Surv: 98.0% (48/49) | Cost:\$ 62.25 Save:\$ 437.75 | Sim:0.1s LLM:3.6s
| food=20x/4h@Random prob=1x/48h@Random O2=2h/4h@Random [Obs Mem:2H Ref:- Tool Plan]
Day 3 | Fit:0.9189 Yld:0.9796 Sav:0.8020 Hlth:0.9621 | Surv: 98.0% (48/49) | Cost:\$ 99.00 Save:\$ 401.00 | Sim:0.1s LLM:1.9s
| food=20x/8h@Random prob=1x/48h@Random O2=2h/10h@Random [Obs Mem:3H Ref:- Tool Plan]
Day 4 | Fit:0.9094 Yld:0.9796 Sav:0.7760 Hlth:0.9547 | Surv: 98.0% (48/49) | Cost:\$ 112.00 Save:\$ 388.00 | Sim:0.1s LLM:1.1s
| food=25x/2h@Random prob=1x/48h@Random O2=4h/5h@Random [Obs Mem:4H Ref:- Tool Plan]
Day 5 | Fit:0.8724 Yld:0.9796 Sav:0.6645 Hlth:0.9527 | Surv: 98.0% (48/49) | Cost:\$ 167.75 Save:\$ 332.25 | Sim:0.2s LLM:3.0s
| food=10x/4h@Random prob=3x/24h@Random O2=2h/8h@Random [Obs Mem:5H Ref:Y Tool Plan]

>>> REFLECTION: D5(warn:51/45): To increase your fitness from 0.872 to closer to 0.909, consider adjusting the `food\_interval` to a lower value, such as 2 hours, to ensure more frequent feeding which can help maintain or boost your fish's health and fitness.

Day 6 | Fit:0.8581 Yld:0.9796 Sav:0.6240 Hlth:0.9454 | Surv: 98.0% (48/49) | Cost:\$ 188.00 Save:\$ 312.00 | Sim:0.2s LLM:5.1s
| food=15x/6h@Random prob=2x/24h@Random O2=2h/12h@Random [Obs Mem:6H Ref:Y Tool Plan]

>>> REFLECTION: D6(20%): To increase fitness, consider reducing the food interval to 6 hours and decreasing the food quantity to 15. This will help save more of your fish's energy, potentially increasing your fitness score. Param values: `food\_interval=6h`, `food\_quantity=15`.

Day 7 | Fit:0.8477 Yld:0.9796 Sav:0.5950 Hlth:0.9378 | Surv: 98.0% (48/49) | Cost:\$ 202.50 Save:\$ 297.50 | Sim:0.2s LLM:2.1s
| food=20x/8h@Random prob=2x/24h@Random O2=2h/10h@Random [Obs Mem:7H Ref:- Tool Plan]
Day 8 | Fit:0.8346 Yld:0.9796 Sav:0.5560 Hlth:0.9358 | Surv: 98.0% (48/49) | Cost:\$ 222.00 Save:\$ 278.00 | Sim:0.2s LLM:2.4s
| food=10x/12h@Random prob=2x/24h@Random O2=2h/4h@Random [Obs Mem:8H Ref:Y Tool Plan]

>>> REFLECTION: D8(warn:96/61): To improve your fitness score, consider reducing the `food_interv

al to a higher value (e.g., from 1-2 hours to 6-7 hours) and decreasing the `food\_quantity` slightly (e.g., from 25 to 20). This might help in saving more resources without significantly impacting y

Day 9 | Fit:0.8027 Yld:0.9592 Sav:0.4910 Hlth:0.9425 | Surv: 95.9% (47/49) | Cost:\$ 254.50 Save:\$ 245.50 | Sim:0.2s LLM:2.5s
| food=15x/6h@Random prob=2x/24h@Random O2=2h/12h@Random [Obs Mem:9H Ref:Y Tool Plan]

>>> REFLECTION: D9(warn:92/73): To increase your fitness score, you could try reducing the `food\_interval` to a lower value, such as 2 hours. This might encourage more frequent feeding, potentially improving your fish's health and fitness score.

Day 10 | Fit:0.7920 Yld:0.9592 Sav:0.4620 Hlth:0.9331 | Surv: 95.9% (47/49) | Cost:\$ 269.00 Save:\$ 231.00 | Sim:0.3s LLM:2.7s
| food=15x/3h@Random prob=2x/24h@Random O2=2h/8h@Random [Obs Mem:10H Ref:Y Tool Plan]

>>> REFLECTION: D10(warn:104/84): To increase your fitness score, you could try adjusting the `food\_interval` to a lower value, such as `food\_interval=2`. This might help in improving your fish's health and fitness more effectively.

Day 11 | Fit:0.7786 Yld:0.9592 Sav:0.4170 Hlth:0.9453 | Surv: 95.9% (47/49) | Cost:\$ 291.50 Save:\$ 208.50 | Sim:0.3s LLM:2.4s
| food=10x/2h@Random prob=1x/48h@Random O2=1h/2h@Random [Obs Mem:11H Ref:Y Tool Plan]

>>> REFLECTION: D11(warn:150/100): To increase your fitness score, you might consider reducing the `food\_interval` to a lower value, such as 2 hours, and decreasing `oxygen\_duration` to 1 hour. This could help in spending less while increasing fitness. For example:

...
food_interval=2h
oxygen_d

Day 12 | Fit:0.7531 Yld:0.9592 Sav:0.3450 Hlth:0.9305 | Surv: 95.9% (47/49) | Cost:\$ 327.50 Save:\$ 172.50 | Sim:0.3s LLM:2.2s
| food=20x/4h@Random prob=2x/24h@Random O2=3h/6h@Random [Obs Mem:12H Ref:Y Tool Plan]

>>> REFLECTION: D12(warn:168/115): To increase your fitness score, you might consider adjusting the `food\_interval` to a lower value (e.g., from 1-6 hours) to ensure your fish receive food more frequently, or adjust the `probiotic\_interval` and `oxygen\_interval` similarly. For example, you could

Day 13 | Fit:0.7273 Yld:0.9592 Sav:0.2700 Hlth:0.9219 | Surv: 95.9% (47/49) | Cost:\$ 365.00 Save:\$ 135.00 | Sim:0.4s LLM:3.0s
| food=8x/4h@Random prob=1x/24h@Random O2=1h/12h@Random [Obs Mem:13H Ref:Y Tool Plan]

>>> REFLECTION: D13(warn:212/133): To increase your fitness score, you might consider adjusting the `food\_interval` to a lower value, such as 2 hours, and reducing `food\_quantity` to 10, to save more resources. This could look like:

...
food_interval=2, food_quantity=10
...

Day 14 | Fit:0.7104 Yld:0.9388 Sav:0.2537 Hlth:0.9197 | Surv: 93.9% (46/49) | Cost:\$ 373.15 Save:\$ 126.85 | Sim:0.4s LLM:3.6s
| food=30x/8h@Random prob=1x/24h@Random O2=4h/6h@Random [Obs Mem:14H Ref:Y Tool Plan]

>>> REFLECTION: D14(warn:234/151): To increase your fitness score, you could try adjusting the `food\_interval` to a lower value, such as `food\_interval=8`. This might help in improving the fish's health and thus increasing the fitness score.

Warn: No JSON. Fallback.
Day 15 | Fit:0.6799 Yld:0.9388 Sav:0.1632 Hlth:0.9140 | Surv: 93.9% (46/49) | Cost:\$ 418.40 Save:\$ 81.60 | Sim:0.5s LLM:181.8s
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:15H Ref:Y Tool Plan]

>>> REFLECTION: D15(warn:250/168): To increase your fitness score, you might consider adjusting the `food\_interval` to a lower value (e.g., from 24h to 12h) and increasing the `food\_quantity` to a higher value (e.g., from 5 to 30). This could help boost your fish's health and fitness.

Day 16 | Fit:0.6746 Yld:0.9388 Sav:0.1502 Hlth:0.9055 | Surv: 93.9% (46/49) | Cost:\$ 424.90 Save:\$ 75.10 | Sim:0.5s LLM:3.2s
| food=10x/6h@Random prob=1x/24h@Random O2=2h/12h@Random [Obs Mem:16H Ref:Y Tool Plan]

>>> REFLECTION: D16(warn:253/183): To increase your fitness score, you might consider reducing the `food\_interval` to a lower value, such as 2 hours, and decreasing the `food\_quantity` to a lower amount, like 10. This could help in spending less while maintaining or improving your fitness score.

Day 17 | Fit:0.6485 Yld:0.8980 Sav:0.1247 Hlth:0.9452 | Surv: 89.8% (44/49) | Cost:\$ 437.65 Save:\$ 62.35 | Sim:0.5s LLM:3.5s
| food=5x/4h@Random prob=1x/24h@Random O2=2h/8h@Random [Obs Mem:17H Ref:Y Tool Plan]

>>> REFLECTION: D17(warn:202/191): To increase your fitness score, you might consider adjusting the `food\_interval` to a lower value (e.g., from 1-2 hours to 1 hour) and decreasing `food\_quantity` slight

ly (e.g., from 5 to 4). This could help in optimizing feeding times and quantities to boost y

Day 18 | Fit:0.6271 Yld:0.8776 Sav:0.0902 Hlth:0.9556 | Surv: 87.8% (43/49) | Cost:\$ 454.90 Sav
e:\$ 45.10 | Sim:0.4s LLM:4.3s [EARLY QUIT]

>>> REFLECTION: D18(60%): To increase fitness, consider reducing the food interval to 2 hours and
decreasing the food quantity to 10. This will help save more of your fitness budget. Param values: `food_i
nterval=2`, `food_quantity=10`.

Gen 5 FINAL (attempt 2):
Fit:0.0000 | Yield:0.8776 | SavingRate:0.0902 | Healthiness:0.9556
Surv:87.8% (43/49) | Cost:\$454.90 | Save:\$45.10 [EARLY QUIT]

>>> END-OF-GEN REFLECTION:
To increase your fitness score, you might consider reducing the `food\_interval` to a lower value, su
ch as 2 hours, and decreasing the `food\_quantity` to a lower amount, like 10. This could help in spending
less while maintaining or improving your fitness score.

For example:

...
food_interval=2
foo

Saved memory: /content/AquaLLM/logs/memory_full.json
Saved trace: /content/AquaLLM/logs/trace_full.json (736,047 bytes)

=====
Config: no_planning (Obs+Mem+Ref+Tool) | Run 1/1
Days: 30 | Budget: \$500.0 | Generations: 5
=====

~~~~~  
Generation 1/5 (attempt 1)  
~~~~~  
Day 0 | Stocked 30 fish | food=20x/12h@Random prob=1x/72h@Random O2=4h/20h@Random | LLM:1.6s
Day 1 | Fit:0.7660 Yld:0.6122 Sav:0.9545 Hlth:0.9525 | Surv:100.0% (30/30) | Cost:\$ 22.75 Sav
e:\$ 477.25 | Sim:0.0s LLM:1.1s
| food=15x/6h@Random prob=1x/24h@Random O2=3h/8h@Random [Obs Mem:1H Ref:- Tool]

*** RESTARTING Gen 1 (attempt 1 -> 2) ***
Reason: Day 1 fitness 0.7660 < 0.9. Had 30 fish (max 49).
Reflection: Increase the food_interval to 8h and set the fish count to 40.

.
Generation 1/5 (attempt 2)
.
Day 0 | Stocked 40 fish | food=20x/8h@Random prob=2x/48h@Random O2=2h/10h@Random | LLM:2.8s
Day 1 | Fit:0.8814 Yld:0.8163 Sav:0.9610 Hlth:0.9605 | Surv:100.0% (40/40) | Cost:\$ 19.50 Sav
e:\$ 480.50 | Sim:0.1s LLM:1.3s
| food=20x/6h@Random prob=3x/48h@Random O2=2h/12h@Random [Obs Mem:1H Ref:- Tool]

*** RESTARTING Gen 1 (attempt 2 -> 3) ***
Reason: Day 1 fitness 0.8814 < 0.9. Had 40 fish (max 49).
Reflection: Increase the food interval to 6 hours and set the oxygen interval to 12 hours. This might he
lp in improving your fitness score by providing more consistent care without overworking the system. Param
values: food_interval=6, oxygen_interval=12.

.
Generation 1/5 (attempt 3)
.
Day 0 | Stocked 30 fish | food=25x/6h@Random prob=1x/48h@Random O2=2h/12h@Random | LLM:2.5s
Day 1 | Fit:0.7738 Yld:0.6122 Sav:0.9685 Hlth:0.9790 | Surv:100.0% (30/30) | Cost:\$ 15.75 Sav
e:\$ 484.25 | Sim:0.1s LLM:2.9s
| food=25x/6h@Random prob=1x/48h@Random O2=2h/12h@Random [Obs Mem:1H Ref:- Tool]
Day 2 | Fit:0.7641 Yld:0.6122 Sav:0.9385 Hlth:0.9802 | Surv:100.0% (30/30) | Cost:\$ 30.75 Sav
e:\$ 469.25 | Sim:0.1s LLM:1.9s
| food=15x/8h@Random prob=1x/24h@Random O2=4h/8h@Random [Obs Mem:2H Ref:- Tool]
Day 3 | Fit:0.7420 Yld:0.6122 Sav:0.8725 Hlth:0.9776 | Surv:100.0% (30/30) | Cost:\$ 63.75 Sav
e:\$ 436.25 | Sim:0.1s LLM:1.1s
| food=15x/6h@Random prob=2x/48h@Random O2=4h/12h@Random [Obs Mem:3H Ref:- Tool]
Day 4 | Fit:0.7274 Yld:0.6122 Sav:0.8265 Hlth:0.9825 | Surv:100.0% (30/30) | Cost:\$ 86.75 Sav
e:\$ 413.25 | Sim:0.1s LLM:1.2s
| food=25x/6h@Random prob=1x/48h@Random O2=2h/20h@Random [Obs Mem:4H Ref:- Tool]
Day 5 | Fit:0.7208 Yld:0.6122 Sav:0.8050 Hlth:0.9865 | Surv:100.0% (30/30) | Cost:\$ 97.50 Sav
e:\$ 402.50 | Sim:0.1s LLM:1.1s
| food=10x/6h@Random prob=2x/48h@Random O2=4h/12h@Random [Obs Mem:5H Ref:- Tool]
Day 6 | Fit:0.7057 Yld:0.6122 Sav:0.7610 Hlth:0.9817 | Surv:100.0% (30/30) | Cost:\$ 119.50 Sav
e:\$ 380.50 | Sim:0.1s LLM:1.5s
| food=5x/2h@Random prob=1x/48h@Random O2=2h/8h@Random [Obs Mem:6H Ref:- Tool]

```
| food=5x/2h@Random prob=1x/48h@Random O2=2h/8h@Random [Obs Mem:6H Ref:Y Tool]

>>> REFLECTION: D6(warn:50/42): To increase your fitness score, you might consider reducing the food interval to 1-2 hours and decreasing the food quantity to 5-10 units. This could help save more resources, potentially increasing your fitness score. For example:

...

food_interval=1, food_quant

Day 7 | Fit:0.6930 Yld:0.6122 Sav:0.7235 Hlth:0.9794 | Surv:100.0% (30/30) | Cost:$ 138.25 Save:$ 361.75 | Sim:0.1s LLM:1.1s
| food=5x/2h@Random prob=1x/48h@Random O2=2h/8h@Random [Obs Mem:7H Ref:- Tool]
Day 8 | Fit:0.6807 Yld:0.6122 Sav:0.6875 Hlth:0.9755 | Surv:100.0% (30/30) | Cost:$ 156.25 Save:$ 343.75 | Sim:0.1s LLM:1.3s
| food=8x/2h@Random prob=1x/24h@Random O2=2h/6h@Random [Obs Mem:8H Ref:Y Tool]

>>> REFLECTION: D8(warn:69/52): To increase your fitness score, consider reducing the food interval to 2 hours (`food_interval=2`). This might help in maintaining a higher fish health level more consistently.

Day 9 | Fit:0.6636 Yld:0.6122 Sav:0.6364 Hlth:0.9740 | Surv:100.0% (30/30) | Cost:$ 181.80 Save:$ 318.20 | Sim:0.1s LLM:1.4s
| food=15x/8h@Random prob=2x/32h@Random O2=2h/16h@Random [Obs Mem:9H Ref:Y Tool]

>>> REFLECTION: D9(warn:68/61): To increase your fitness score, consider reducing the `food_interval` to a lower value, such as 2 hours. This change might help in optimizing the feeding schedule and potentially improving the fish's health, thereby increasing the fitness score.

Day 10 | Fit:0.6556 Yld:0.6122 Sav:0.6189 Hlth:0.9556 | Surv:100.0% (30/30) | Cost:$ 190.55 Save:$ 309.45 | Sim:0.2s LLM:1.4s
| food=30x/6h@Random prob=1x/48h@Random O2=2h/8h@Random [Obs Mem:10H Ref:Y Tool]

>>> REFLECTION: D10(warn:76/69): To increase your fitness score, consider reducing the `food_interval` to a lower value, such as 2 hours. This will allow you to feed your fish more frequently, potentially improving their health and increasing your fitness score.

Day 11 | Fit:0.6427 Yld:0.6122 Sav:0.5754 Hlth:0.9673 | Surv:100.0% (30/30) | Cost:$ 212.30 Save:$ 287.70 | Sim:0.2s LLM:0.9s
| food=30x/1h@Random prob=1x/24h@Random O2=2h/5h@Random [Obs Mem:11H Ref:Y Tool]

>>> REFLECTION: D11(warn:80/77): To increase your fitness score, consider reducing the `food_interval` to a lower value (e.g., from 1-2 hours to 1 hour) and decreasing `spending` by choosing a higher `food_quantity` within its limits. For example, you could set `food_interval=1`, `food_quantity`=

Day 12 | Fit:0.6022 Yld:0.6122 Sav:0.4519 Hlth:0.9696 | Surv:100.0% (30/30) | Cost:$ 274.05 Save:$ 225.95 | Sim:0.2s LLM:1.5s
| food=25x/4h@Random prob=1x/48h@Random O2=2h/8h@Random [Obs Mem:12H Ref:Y Tool]

>>> REFLECTION: D12(warn:104/86): To increase your fitness score, consider reducing the `food_interval` to a lower value, such as 2 hours. This will allow you to feed your fish more frequently, potentially improving their health and increasing your fitness score. Change `food_interval=2`.

Day 13 | Fit:0.5859 Yld:0.6122 Sav:0.4054 Hlth:0.9618 | Surv:100.0% (30/30) | Cost:$ 297.30 Save:$ 202.70 | Sim:0.3s LLM:1.4s
| food=5x/2h@Random prob=1x/24h@Random O2=3h/8h@Random [Obs Mem:13H Ref:Y Tool]

>>> REFLECTION: D13(warn:115/95): To increase your fitness score, consider reducing the `food_interval` to a lower value, such as 2 hours. This will allow you to feed your fish more frequently, potentially improving their health and increasing your fitness score. Change it to `food_interval=2`.

Day 14 | Fit:0.5694 Yld:0.6122 Sav:0.3529 Hlth:0.9687 | Surv:100.0% (30/30) | Cost:$ 323.55 Save:$ 176.45 | Sim:0.3s LLM:1.5s
| food=10x/8h@Random prob=1x/48h@Random O2=2h/16h@Random [Obs Mem:14H Ref:Y Tool]

>>> REFLECTION: D14(warn:139/105): To increase your fitness score, consider reducing the `food_interval` to a lower value, such as 2 hours. This will allow you to feed your fish more frequently, potentially improving their health and increasing your fitness score. Change `food_interval=2`.

Day 15 | Fit:0.5602 Yld:0.6122 Sav:0.3284 Hlth:0.9595 | Surv:100.0% (30/30) | Cost:$ 335.80 Save:$ 164.20 | Sim:0.4s LLM:1.4s
| food=9x/8h@Random prob=2x/24h@Random O2=3h/6h@Random [Obs Mem:15H Ref:Y Tool]

>>> REFLECTION: D15(warn:146/115): To increase your fitness, consider reducing the `food_interval` to 2 hours and decreasing `food_quantity` to 15. This will help you spend less while maintaining nutrition, potentially boosting your fitness score.

Day 16 | Fit:0.5372 Yld:0.6122 Sav:0.2627 Hlth:0.9485 | Surv:100.0% (30/30) | Cost:$ 368.65 Save:$ 131.35 | Sim:0.4s LLM:1.5s
| food=10x/12h@Random prob=1x/24h@Random O2=4h/15h@Random [Obs Mem:16H Ref:Y Tool]

>>> REFLECTION: D16(warn:152/124): To increase your fitness score, consider reducing the `food_quantity` to spend less and improve your Fitness. For example, you could change it from `food_quantity=28` to
```

o `food\_quantity=20`. This might help boost your Fitness score on Day 17.

Day 17 | Fit:0.5257 Yld:0.6122 Sav:0.2242 Hlth:0.9582 | Surv:100.0% (30/30) | Cost:\$ 387.90 Save:\$ 112.10 | Sim:0.4s LLM:1.4s
| food=10x/4h@Random prob=2x/24h@Random O2=4h/8h@Random [Obs Mem:17H Ref:Y Tool]

>>> REFLECTION: D17(warn:164/133): To increase your fitness, consider reducing the `food\_interval` to 2 hours (`food\_interval=2`). This will allow you to feed your fish more frequently, potentially improving their health and increasing your fitness score.

Day 18 | Fit:0.5016 Yld:0.6122 Sav:0.1552 Hlth:0.9469 | Surv:100.0% (30/30) | Cost:\$ 422.40 Save:\$ 77.60 | Sim:0.4s LLM:1.5s
| food=30x/2h@Random prob=1x/168h@Random O2=2h/6h@Random [Obs Mem:18H Ref:Y Tool]

>>> REFLECTION: D18(warn:187/142): To increase your fitness score, consider reducing the `food\_interval` to a lower value (e.g., from 1-2 hours to 1 hour). This will allow you to feed your fish more frequently, potentially improving their health and increasing your fitness score.

Day 19 | Fit:0.4786 Yld:0.6122 Sav:0.0792 Hlth:0.9645 | Surv:100.0% (30/30) | Cost:\$ 460.40 Save:\$ 39.60 | Sim:0.5s LLM:1.5s
| food=8x/2h@Random prob=2x/24h@Random O2=3h/6h@Random [Obs Mem:19H Ref:Y Tool]

>>> REFLECTION: D19(warn:232/155): To increase your fitness, consider reducing the `food\_interval` to a lower value (e.g., from 1-12h to 1-5h) and decreasing the `food\_quantity` slightly (e.g., from 5-30 to 5-20). This might help in spending less while maintaining or improving your fitness.

Day 20 | Fit:0.4542 Yld:0.6122 Sav:0.0066 Hlth:0.9606 | Surv:100.0% (30/30) | Cost:\$ 496.70 Save:\$ 3.30 | Sim:0.5s LLM:1.4s
| food=25x/4h@Random prob=1x/48h@Random O2=2h/8h@Random [Obs Mem:20H Ref:Y Tool]

>>> REFLECTION: D20(warn:334/174): To increase your fitness, consider reducing the `food\_interval` to a lower value, such as `food\_interval=2`. This will encourage more frequent feeding, which might help improve your fish's health and fitness slightly.

Day 21 | Fit:0.0000 Yld:0.6122 Sav:0.0000 Hlth:0.9594 | Surv:100.0% (30/30) | Cost:\$ 501.20 Save:\$ 0.00 | Sim:0.0s [OVER\$]

Gen 1 FINAL (attempt 3):
Fit:0.0000 | Yield:0.6122 | SavingRate:0.0000 | Healthiness:0.9594
Surv:100.0% (30/30) | Cost:\$501.20 | Save:\$0.00

>>> END-OF-GEN REFLECTION:
To increase your fitness score, reduce the `food\_interval` to 1 hour. This will allow for more frequent feedings, potentially improving your fish's health and increasing your fitness score.

For better management of resources, decrease both the `food\_interval` to 3 hours and the `food\_quantity` to 1

~~~~~  
Generation 2/5 (attempt 1)  
~~~~~

Day 0 | Stocked 30 fish | food=25x/6h@Random prob=1x/48h@Random O2=2h/12h@Random | LLM:1.5s
Day 1 | Fit:0.7739 Yld:0.6122 Sav:0.9685 Hlth:0.9796 | Surv:100.0% (30/30) | Cost:\$ 15.75 Save:\$ 484.25 | Sim:0.1s LLM:1.6s [EARLY QUIT]

*** RESTARTING Gen 2 (attempt 1 -> 2) ***
Reason: Day 1 fitness 0.7739 < 0.9. Had 30 fish (max 49).
Reflection: Increase the food_interval to 8h and set the fish_count to 40.

.....
Generation 2/5 (attempt 2)
.....

Day 0 | Stocked 40 fish | food=15x/8h@Random prob=2x/24h@Random O2=2h/12h@Random | LLM:1.8s
Day 1 | Fit:0.8848 Yld:0.8163 Sav:0.9725 Hlth:0.9574 | Surv:100.0% (40/40) | Cost:\$ 13.75 Save:\$ 486.25 | Sim:0.1s LLM:1.1s
| food=15x/6h@Random prob=1x/24h@Random O2=4h/8h@Random [Obs Mem:1H Ref:- Tool]

*** RESTARTING Gen 2 (attempt 2 -> 3) ***
Reason: Day 1 fitness 0.8848 < 0.9. Had 40 fish (max 49).
Reflection: Increase the food interval to 2 hours and set the food quantity to 30. This might help reach the maximum fish count of 49 and potentially improve fitness.

.....
Generation 2/5 (attempt 3)
.....

Day 0 | Stocked 30 fish | food=20x/4h@Random prob=1x/168h@Random O2=2h/4h@Random | LLM:1.7s
Day 1 | Fit:0.7606 Yld:0.6122 Sav:0.9265 Hlth:0.9840 | Surv:100.0% (30/30) | Cost:\$ 36.75 Save:\$ 463.25 | Sim:0.1s LLM:1.2s
| food=15x/3h@Random prob=1x/24h@Random O2=4h/6h@Random [Obs Mem:1H Ref:- Tool]
Day 2 | Fit:0.7302 Yld:0.6122 Sav:0.8330 Hlth:0.9882 | Surv:100.0% (30/30) | Cost:\$ 83.50 Save:\$ 416.50 | Sim:0.1s LLM:1.1s

```
Day 3 | Fit:0.7005 Yld:0.6122 Sav:0.7435 Hlth:0.9865 | Surv:100.0% (30/30) | Cost:$ 128.25 Save:$ 371.75 | Sim:0.1s LLM:1.4s
| food=10x/3h@Random prob=1x/24h@Random O2=4h/6h@Random [Obs Mem:2H Ref:- Tool]
Day 4 | Fit:0.6865 Yld:0.6122 Sav:0.7080 Hlth:0.9678 | Surv:100.0% (30/30) | Cost:$ 146.00 Save:$ 354.00 | Sim:0.1s LLM:1.1s
| food=10x/6h@Random prob=1x/24h@Random O2=2h/8h@Random [Obs Mem:3H Ref:- Tool]
Day 5 | Fit:0.6738 Yld:0.6122 Sav:0.6705 Hlth:0.9653 | Surv:100.0% (30/30) | Cost:$ 164.75 Save:$ 335.25 | Sim:0.1s LLM:1.4s
| food=8x/2h@Random prob=1x/32h@Random O2=4h/16h@Random [Obs Mem:5H Ref:Y Tool]

>>> REFLECTION: D5(warn:58/47): To increase your fitness, consider reducing the `food_interval` to a lower value, such as `food_interval=2h`. This might help in boosting your fish's health and fitness more effectively.

Day 6 | Fit:0.6531 Yld:0.5918 Sav:0.6394 Hlth:0.9717 | Surv: 96.7% (29/30) | Cost:$ 180.30 Save:$ 319.70 | Sim:0.1s LLM:1.4s
| food=15x/4h@Random prob=1x/48h@Random O2=2h/8h@Random [Obs Mem:6H Ref:Y Tool]

>>> REFLECTION: D6(20%): To increase your fitness, consider reducing the food interval to 2 hours (food_interval=2). This change might help boost your fish's health and fitness slightly.

Day 7 | Fit:0.6412 Yld:0.5918 Sav:0.5989 Hlth:0.9838 | Surv: 96.7% (29/30) | Cost:$ 200.55 Save:$ 299.45 | Sim:0.1s LLM:1.1s
| food=15x/6h@Random prob=1x/30h@Random O2=2h/8h@Random [Obs Mem:7H Ref:- Tool]
Day 8 | Fit:0.6290 Yld:0.5918 Sav:0.5614 Hlth:0.9852 | Surv: 96.7% (29/30) | Cost:$ 219.30 Save:$ 280.70 | Sim:0.1s LLM:1.5s
| food=30x/10h@Random prob=1x/48h@Random O2=4h/6h@Random [Obs Mem:8H Ref:Y Tool]

>>> REFLECTION: D8(warn:74/62): To improve your fitness score, consider reducing the food interval to increase spending less while maintaining health. For example, you could change `food_interval` from its current value to a higher number within the allowed range (1-12h). Try setting it to 8 hours.

Day 9 | Fit:0.5994 Yld:0.5918 Sav:0.4739 Hlth:0.9790 | Surv: 96.7% (29/30) | Cost:$ 263.05 Save:$ 236.95 | Sim:0.1s LLM:1.4s
| food=14x/9h@Random prob=1x/24h@Random O2=2h/6h@Random [Obs Mem:9H Ref:Y Tool]

>>> REFLECTION: D9(warn:70/70): To increase your fitness, consider reducing the `food_interval` to 7 hours and decreasing the `food_quantity` to 20. This might help you save more while maintaining your fish's health. Changes will apply tomorrow.

Day 10 | Fit:0.5829 Yld:0.5918 Sav:0.4282 Hlth:0.9673 | Surv: 96.7% (29/30) | Cost:$ 285.90 Save:$ 214.10 | Sim:0.2s LLM:1.5s
| food=30x/4h@Random prob=1x/24h@Random O2=3h/8h@Random [Obs Mem:10H Ref:Y Tool]

>>> REFLECTION: D10(warn:114/83): To increase your fitness score, consider adjusting the `food_interval` to a lower value (e.g., 2 hours) and decreasing `food_quantity` slightly (e.g., to 20). This might help in optimizing feeding times and quantities to boost your fish's health and fitness.

Day 11 | Fit:0.5635 Yld:0.5918 Sav:0.3637 Hlth:0.9833 | Surv: 96.7% (29/30) | Cost:$ 318.15 Save:$ 181.85 | Sim:0.2s LLM:1.7s
| food=15x/6h@Random prob=1x/24h@Random O2=4h/15h@Random [Obs Mem:11H Ref:Y Tool]

>>> REFLECTION: D11(warn:147/98): To increase your fitness score, you might consider adjusting the `food_interval` to a lower value (e.g., from 1-2 hours to 1 hour) and decreasing `food_quantity` slightly (e.g., from 25 to 20). This could help in optimizing feeding frequency without overfeeding.

Day 12 | Fit:0.5497 Yld:0.5918 Sav:0.3212 Hlth:0.9847 | Surv: 96.7% (29/30) | Cost:$ 339.40 Save:$ 160.60 | Sim:0.3s LLM:1.5s
| food=25x/8h@Random prob=4x/24h@Random O2=2h/16h@Random [Obs Mem:12H Ref:Y Tool]

>>> REFLECTION: D12(warn:193/116): To increase your fitness score, you might consider adjusting the `food_interval` to a lower value, such as 2 hours, and decreasing `food_quantity` to 15. This could help in spending less while maintaining or improving your fitness level. Changes will apply tomorrow.

Day 13 | Fit:0.5361 Yld:0.5918 Sav:0.2877 Hlth:0.9640 | Surv: 96.7% (29/30) | Cost:$ 356.15 Save:$ 143.85 | Sim:0.3s LLM:1.5s
| food=5x/12h@Random prob=1x/48h@Random O2=4h/16h@Random [Obs Mem:13H Ref:Y Tool]

>>> REFLECTION: D13(warn:176/130): To increase your fitness score, you might consider reducing the `food_interval` to a lower value (e.g., from 24 hours to 12 hours) and decreasing the `food_quantity` slightly (e.g., from 5 to 4). This could help in spending less while maintaining or improving your fitness.

Day 14 | Fit:0.5269 Yld:0.5918 Sav:0.2667 Hlth:0.9449 | Surv: 96.7% (29/30) | Cost:$ 366.65 Save:$ 133.35 | Sim:0.3s LLM:1.6s
| food=30x/12h@Random prob=1x/24h@Random O2=4h/20h@Random [Obs Mem:14H Ref:Y Tool]

>>> REFLECTION: D14(warn:182/142): To increase your fitness score, you might consider adjusting the `food_interval` to a lower value (e.g., from 1-2 hours to 1 hour) and reducing `oxygen_duration` to its minimum (1 hour), as these changes can help boost your fish's health and fitness.
```

```
Day 15 | Fit:0.5190 Yld:0.5918 Sav:0.2392 Hlth:0.9542 | Surv: 96.7% (29/30) | Cost:$ 380.40 Sav
e:$ 119.60 | Sim:0.3s LLM:1.6s
| food=15x/8h@Random prob=1x/24h@Random O2=2h/15h@Random [Obs Mem:15H Ref:Y Tool]

>>> REFLECTION: D15(warn:179/153): To increase your fitness score, you might consider adjusting t
he `food_interval` to a lower value (e.g., from 1-2 hours to 1 hour) and reducing the `food_quantity` slig
htly (e.g., from 25 to 20). This could help in spending less while maintaining or improving y

Day 16 | Fit:0.5105 Yld:0.5918 Sav:0.2132 Hlth:0.9550 | Surv: 96.7% (29/30) | Cost:$ 393.40 Sav
e:$ 106.60 | Sim:0.3s LLM:1.6s
| food=18x/6h@Random prob=1x/24h@Random O2=2h/12h@Random [Obs Mem:16H Ref:Y Tool]

>>> REFLECTION: D16(warn:183/162): To increase your fitness score, you might consider adjusting t
he `food_interval` to a lower value (e.g., from 1-2 hours to 1 hour) and increasing the `food_quantity` sl
ightly (e.g., from 15 to 20). This could help in providing more frequent feeding, which might

Day 17 | Fit:0.5036 Yld:0.5918 Sav:0.1845 Hlth:0.9768 | Surv: 96.7% (29/30) | Cost:$ 407.75 Sav
e:$ 92.25 | Sim:0.4s LLM:1.4s
| food=17x/6h@Random prob=3x/24h@Random O2=2h/8h@Random [Obs Mem:17H Ref:Y Tool]

>>> REFLECTION: D17(warn:186/171): To increase your fitness score, you might consider adjusting t
he `food_interval` to a lower value (e.g., from 1-6 hours) to ensure more frequent feeding, which can help
maintain or improve your fish's health and thus boost your fitness score.

Day 18 | Fit:0.4889 Yld:0.5918 Sav:0.1432 Hlth:0.9676 | Surv: 96.7% (29/30) | Cost:$ 428.40 Sav
e:$ 71.60 | Sim:0.4s LLM:1.4s
| food=15x/3h@Random prob=1x/24h@Random O2=2h/6h@Random [Obs Mem:18H Ref:Y Tool]

>>> REFLECTION: D18(warn:184/178): To increase your fitness score, you might consider adjusting t
he `food_interval` to a lower value (e.g., from 1-6 hours) to ensure more frequent feeding, which can help
maintain or improve your fish's health and thus boost your fitness score.

Day 19 | Fit:0.4730 Yld:0.5918 Sav:0.0897 Hlth:0.9823 | Surv: 96.7% (29/30) | Cost:$ 455.15 Sav
e:$ 44.85 | Sim:0.4s LLM:1.6s
| food=25x/8h@Random prob=1x/48h@Random O2=2h/8h@Random [Obs Mem:19H Ref:Y Tool]

>>> REFLECTION: D19(warn:198/186): To increase your fitness score, you might consider adjusting t
he `food_interval` to a lower value (e.g., from 1-6 hours) to ensure more frequent feeding, which can help
maintain or improve your fish's health and thus boost fitness. Additionally, you could try i

Day 20 | Fit:0.4608 Yld:0.5918 Sav:0.0522 Hlth:0.9841 | Surv: 96.7% (29/30) | Cost:$ 473.90 Sav
e:$ 26.10 | Sim:0.3s LLM:1.4s
| food=7x/8h@Random prob=1x/48h@Random O2=2h/16h@Random [Obs Mem:20H Ref:Y Tool]

>>> REFLECTION: D20(warn:210/194): To increase your fitness score, consider reducing the `food_in
terval` to a lower value, such as 2 hours. This will allow you to feed your fish more frequently, potentia
lly improving their health and increasing your fitness score.

Day 21 | Fit:0.4512 Yld:0.5918 Sav:0.0286 Hlth:0.9688 | Surv: 96.7% (29/30) | Cost:$ 485.70 Sav
e:$ 14.30 | Sim:0.4s LLM:3.5s
| food=30x/8h@Random prob=1x/24h@Random O2=1h/5h@Random [Obs Mem:21H Ref:Y Tool]

>>> REFLECTION: D21(warn:239/204): To increase your fitness score, you might consider reducing th
e `food_interval` to a lower value, such as 2 hours, and decreasing `oxygen_duration` to 1 hour. This coul
d help in spending less while increasing fitness. For example:

...
food_interval=2
oxygen_du

Day 22 | Fit:0.0000 Yld:0.5918 Sav:0.0000 Hlth:0.9875 | Surv: 96.7% (29/30) | Cost:$ 500.95 Sav
e:$ 0.00 | Sim:0.3s [OVER$]

Gen 2 FINAL (attempt 3):
Fit:0.0000 | Yield:0.5918 | SavingRate:0.0000 | Healthiness:0.9875
Surv:96.7% (29/30) | Cost:$500.95 | Save:$0.00

>>> END-OF-GEN REFLECTION:
To increase your fitness score, you might consider reducing the `food_interval` to a lower value, su
ch as 2 hours. This will allow you to feed your fish more frequently, potentially improving their health a
nd increasing your fitness score.

For Day 21, to address the budget issue, reduce both `food_

~~~~~
Generation 3/5 (attempt 1)
~~~~~
Day 0 | Stocked 30 fish | food=25x/2h@Random prob=1x/24h@Random O2=3h/10h@Random | LLM:1.7s
Day 1 | Fit:0.7602 Yld:0.6122 Sav:0.9235 Hlth:0.9895 | Surv:100.0% (30/30) | Cost:$ 38.25 Sav
e:$ 461.75 | Sim:0.1s LLM:1.1s
| food=30x/4h@Random prob=1x/24h@Random O2=4h/20h@Random [Obs Mem:1H Ref:- Tool]
```

```
*** RESTARTING Gen 3 (attempt 1 -> 2) ***
Reason: Day 1 fitness 0.7602 < 0.9. Had 30 fish (max 49).
Reflection: Increase the food_interval to 8h and set the fish count to 40.

. . . . .
Generation 3/5 (attempt 2)
. . . . .
Day 0 | Stocked 40 fish | food=25x/8h@Random prob=1x/24h@Random O2=2h/10h@Random | LLM:1.4s
Day 1 | Fit:0.8811 Yld:0.8163 Sav:0.9610 Hlth:0.9585 | Surv:100.0% (40/40) | Cost:$ 19.50 Sav
e:$ 480.50 | Sim:0.1s LLM:1.1s
      | food=25x/6h@Random prob=1x/24h@Random O2=2h/4h@Random [Obs Mem:1H Ref:- Tool]

*** RESTARTING Gen 3 (attempt 2 -> 3) ***
Reason: Day 1 fitness 0.8811 < 0.9. Had 40 fish (max 49).
Reflection: Increase the food interval to 6 and set the food quantity to 25. This might help reach a hig
her fitness level by ensuring the fish are well-fed without overfeeding.

. . . . .
Generation 3/5 (attempt 3)
. . . . .
Day 0 | Stocked 49 fish | food=25x/8h@Random prob=1x/168h@Random O2=2h/12h@Random | LLM:1.9s
Day 1 | Fit:0.9865 Yld:1.0000 Sav:0.9710 Hlth:0.9671 | Surv:100.0% (49/49) | Cost:$ 14.50 Sav
e:$ 485.50 | Sim:0.1s LLM:1.1s
      | food=20x/3h@Random prob=1x/24h@Random O2=2h/4h@Random [Obs Mem:1H Ref:- Tool]
Day 2 | Fit:0.9621 Yld:1.0000 Sav:0.8935 Hlth:0.9767 | Surv:100.0% (49/49) | Cost:$ 53.25 Sav
e:$ 446.75 | Sim:0.1s LLM:1.1s
      | food=20x/2h@Random prob=2x/24h@Random O2=1h/5h@Random [Obs Mem:2H Ref:- Tool]
Day 3 | Fit:0.9456 Yld:1.0000 Sav:0.8415 Hlth:0.9825 | Surv:100.0% (49/49) | Cost:$ 79.25 Sav
e:$ 420.75 | Sim:0.1s LLM:1.1s
      | food=20x/2h@Random prob=1x/24h@Random O2=1h/5h@Random [Obs Mem:3H Ref:- Tool]
Day 4 | Fit:0.9291 Yld:1.0000 Sav:0.7910 Hlth:0.9842 | Surv:100.0% (49/49) | Cost:$ 104.50 Sav
e:$ 395.50 | Sim:0.2s LLM:1.1s
      | food=20x/3h@Random prob=1x/24h@Random O2=1h/4h@Random [Obs Mem:4H Ref:- Tool]
Day 5 | Fit:0.9127 Yld:1.0000 Sav:0.7435 Hlth:0.9776 | Surv:100.0% (49/49) | Cost:$ 128.25 Sav
e:$ 371.75 | Sim:0.2s LLM:1.1s
      | food=20x/2h@Random prob=1x/24h@Random O2=1h/5h@Random [Obs Mem:5H Ref:- Tool]
Day 6 | Fit:0.8971 Yld:1.0000 Sav:0.6930 Hlth:0.9870 | Surv:100.0% (49/49) | Cost:$ 153.50 Sav
e:$ 346.50 | Sim:0.2s LLM:1.4s
      | food=25x/4h@Random prob=1x/48h@Random O2=2h/8h@Random [Obs Mem:6H Ref:Y Tool]

>>> REFLECTION: D6(warn:56/44): To increase your fitness score, consider reducing the `food_inter
val` to a lower value, such as 2 hours. This might help in maintaining a higher fish health and fitness le
vel more consistently.

Day 7 | Fit:0.8815 Yld:1.0000 Sav:0.6465 Hlth:0.9842 | Surv:100.0% (49/49) | Cost:$ 176.75 Sav
e:$ 323.25 | Sim:0.2s LLM:1.1s
      | food=25x/4h@Random prob=1x/96h@Random O2=2h/8h@Random [Obs Mem:7H Ref:- Tool]
Day 8 | Fit:0.8662 Yld:1.0000 Sav:0.6015 Hlth:0.9805 | Surv:100.0% (49/49) | Cost:$ 199.25 Sav
e:$ 300.75 | Sim:0.2s LLM:1.4s
      | food=25x/4h@Random prob=1x/48h@Random O2=2h/8h@Random [Obs Mem:8H Ref:Y Tool]

>>> REFLECTION: D8(warn:79/55): To increase your fitness score, you might consider reducing the f
ood interval to make it more frequent (e.g., `food_interval=2`). This could help maintain a higher fish he
alth level throughout the day, potentially boosting your fitness score.

Day 9 | Fit:0.8510 Yld:1.0000 Sav:0.5550 Hlth:0.9821 | Surv:100.0% (49/49) | Cost:$ 222.50 Sav
e:$ 277.50 | Sim:0.3s LLM:1.5s
      | food=15x/6h@Random prob=2x/24h@Random O2=2h/10h@Random [Obs Mem:9H Ref:Y Tool]

>>> REFLECTION: D9(warn:69/64): To increase your fitness score, you might consider reducing the f
ood interval to make it more frequent (e.g., `food_interval=3`). This could help maintain a higher fish he
alth level throughout the day, potentially boosting your fitness score.

Day 10 | Fit:0.8401 Yld:1.0000 Sav:0.5260 Hlth:0.9713 | Surv:100.0% (49/49) | Cost:$ 237.00 Sav
e:$ 263.00 | Sim:0.2s LLM:1.5s
      | food=6x/8h@Random prob=1x/36h@Random O2=4h/12h@Random [Obs Mem:10H Ref:Y Tool]

>>> REFLECTION: D10(warn:109/77): To increase your fitness score, you might consider reducing the
`food_interval` to a lower value, such as 2 hours, and decreasing the `food_quantity` to a lower number, l
ike 5. This could help you save more resources, potentially increasing your fitness score. F

Day 11 | Fit:0.8247 Yld:1.0000 Sav:0.4827 Hlth:0.9615 | Surv:100.0% (49/49) | Cost:$ 258.65 Sav
e:$ 241.35 | Sim:0.3s LLM:1.5s
      | food=15x/8h@Random prob=2x/24h@Random O2=2h/10h@Random [Obs Mem:11H Ref:Y Tool]

>>> REFLECTION: D11(warn:158/94): To increase your fitness score, you might consider reducing the
food interval to make it more frequent (e.g., `food_interval=6`). This could help maintain a higher food a
vailability for your fish, potentially improving their health and thus increasing your fitne

Day 12 | Fit:0.8152 Yld:1.0000 Sav:0.4552 Hlth:0.9584 | Surv:100.0% (49/49) | Cost:$ 272.40 Sav
e:$ 227.60 | Sim:0.3s LLM:1.4s
```

e:\$ 227.60 | Sim:0.3s LLM:1.4s
| food=25x/8h@Random prob=1x/48h@Random O2=2h/8h@Random [Obs Mem:12H Ref:Y Tool]

>>> REFLECTION: D12(warn:214/116): To increase your fitness score, consider reducing the `food\_interval` to a lower value, such as 2 hours. This will allow you to feed your fish more frequently, potentially improving their health and increasing your fitness score. Change `food\_interval=2`.

Day 13 | Fit:0.7926 Yld:0.9796 Sav:0.4162 Hlth:0.9710 | Surv: 98.0% (48/49) | Cost:\$ 291.90 Save:\$ 208.10 | Sim:0.4s LLM:1.5s
| food=25x/6h@Random prob=1x/24h@Random O2=2h/15h@Random [Obs Mem:13H Ref:Y Tool]

>>> REFLECTION: D13(warn:277/141): To increase your fitness score, you might consider reducing the `food\_interval` to make feedings more frequent, which could improve your fish's health and fitness. For example, you could set `food\_interval=5`. This change would apply tomorrow.

Day 14 | Fit:0.7838 Yld:0.9796 Sav:0.3847 Hlth:0.9837 | Surv: 98.0% (48/49) | Cost:\$ 307.65 Save:\$ 192.35 | Sim:0.4s LLM:0.8s
| food=25x/4h@Random prob=3x/24h@Random O2=2h/8h@Random [Obs Mem:14H Ref:Y Tool]

>>> REFLECTION: D14(warn:304/166): To increase your fitness score, consider reducing the `food\_interval` to a lower value (e.g., from 1-6 hours) and increasing the `food\_quantity` to a higher value (e.g., up to 25). This might help in boosting your fish's health and fitness more effectively.

Day 15 | Fit:0.7686 Yld:0.9796 Sav:0.3352 Hlth:0.9933 | Surv: 98.0% (48/49) | Cost:\$ 332.40 Save:\$ 167.60 | Sim:0.4s LLM:1.5s
| food=25x/8h@Random prob=3x/24h@Random O2=4h/20h@Random [Obs Mem:15H Ref:Y Tool]

>>> REFLECTION: D15(warn:292/186): To increase your fitness score, you might consider adjusting the `food\_interval` to a lower value (e.g., `food\_interval=2h`) to ensure your fish receive food more frequently, which could help maintain or improve their health and thus your fitness score.

Day 16 | Fit:0.7507 Yld:0.9796 Sav:0.2832 Hlth:0.9873 | Surv: 98.0% (48/49) | Cost:\$ 358.40 Save:\$ 141.60 | Sim:0.4s LLM:0.8s
| food=20x/1h@Random prob=2x/24h@Random O2=2h/4h@Random [Obs Mem:16H Ref:Y Tool]

>>> REFLECTION: D16(warn:291/203): To increase your fitness score, consider decreasing the `food\_interval` to a lower value (e.g., from 1-2 hours to 1 hour) and increasing the `food\_quantity` slightly (e.g., from 15 to 20). This can help boost your fish's energy levels more frequently.

Day 17 | Fit:0.7145 Yld:0.9796 Sav:0.1722 Hlth:0.9909 | Surv: 98.0% (48/49) | Cost:\$ 413.90 Save:\$ 86.10 | Sim:0.5s LLM:1.6s
| food=25x/8h@Random prob=1x/24h@Random O2=2h/10h@Random [Obs Mem:17H Ref:Y Tool]

>>> REFLECTION: D17(warn:294/219): To increase your fitness score, consider adjusting the `food\_interval` to a lower value (e.g., from 12h to 6h) and the `food\_quantity` to a higher value (e.g., from 30 to 25). This can help in providing more frequent and substantial feeding, which might improve

Day 18 | Fit:0.7000 Yld:0.9796 Sav:0.1332 Hlth:0.9774 | Surv: 98.0% (48/49) | Cost:\$ 433.40 Save:\$ 66.60 | Sim:0.6s LLM:1.5s
| food=5x/12h@Random prob=1x/24h@Random O2=4h/12h@Random [Obs Mem:18H Ref:Y Tool]

>>> REFLECTION: D18(warn:274/231): To increase your fitness score, consider reducing the `food\_interval` to a lower value (e.g., from 24 hours to 12 hours) and decreasing `food\_quantity` slightly (e.g., from 5 to 4). This might help in spending less while maintaining or improving your fitness score

Day 19 | Fit:0.6862 Yld:0.9796 Sav:0.0907 Hlth:0.9788 | Surv: 98.0% (48/49) | Cost:\$ 454.65 Save:\$ 45.35 | Sim:0.6s LLM:1.2s
| food=20x/8h@Random prob=2x/24h@Random O2=2h/12h@Random [Obs Mem:19H Ref:- Tool]

Day 20 | Fit:0.6770 Yld:0.9796 Sav:0.0617 Hlth:0.9822 | Surv: 98.0% (48/49) | Cost:\$ 469.15 Save:\$ 30.85 | Sim:0.6s LLM:1.4s
| food=25x/8h@Random prob=2x/24h@Random O2=2h/8h@Random [Obs Mem:20H Ref:Y Tool]

>>> REFLECTION: D20(warn:422/243): To increase your fitness score, consider decreasing the `food\_interval` to a lower value, such as 2 hours. This will allow you to feed your fish more frequently and potentially improve their health, thereby increasing your fitness score.

Day 21 | Fit:0.6631 Yld:0.9796 Sav:0.0212 Hlth:0.9777 | Surv: 98.0% (48/49) | Cost:\$ 489.40 Save:\$ 10.60 | Sim:0.5s LLM:1.1s
| food=5x/12h@Random prob=1x/24h@Random O2=2h/6h@Random [Obs Mem:21H Ref:- Tool]

Day 22 | Fit:0.0000 Yld:0.9796 Sav:0.0000 Hlth:0.9632 | Surv: 98.0% (48/49) | Cost:\$ 500.40 Save:\$ 0.00 | Sim:0.2s [OVER\$]

Gen 3 FINAL (attempt 3):
Fit:0.0000 | Yield:0.9796 | SavingRate:0.0000 | Healthiness:0.9632
Surv:98.0% (48/49) | Cost:\$500.40 | Save:\$0.00

>>> END-OF-GEN REFLECTION:
D21(warn:498/333): To increase your fitness score, consider decreasing the `food\_interval` to 6 hours and reducing the `food\_quantity` to 20. This will help in spending less while still providing adequate nutrition.

FAIL:budget D21 \$500

D21(warn:498/333): To increase your fitness score, consider d

~~~~~  
Generation 4/5 (attempt 1)  
~~~~~  
Day 0 | Stocked 30 fish | food=20x/6h@Random prob=3x/24h@Random O2=2h/5h@Random | LLM:1.7s
Day 1 | Fit:0.7626 Yld:0.6122 Sav:0.9375 Hlth:0.9706 | Surv:100.0% (30/30) | Cost:\$ 31.25 Sav
e:\$ 468.75 | Sim:0.1s LLM:1.1s
| food=10x/12h@Random prob=3x/24h@Random O2=2h/8h@Random [Obs Mem:1H Ref:- Tool]

*** RESTARTING Gen 4 (attempt 1 -> 2) ***
Reason: Day 1 fitness 0.7626 < 0.9. Had 30 fish (max 49).
Reflection: Increase `fish\_count` to 40.

.
Generation 4/5 (attempt 2)
.
Day 0 | Stocked 30 fish | food=10x/6h@Random prob=3x/24h@Random O2=3h/5h@Random | LLM:1.7s
Day 1 | Fit:0.7541 Yld:0.6122 Sav:0.9165 Hlth:0.9578 | Surv:100.0% (30/30) | Cost:\$ 41.75 Sav
e:\$ 458.25 | Sim:0.1s LLM:1.1s
| food=15x/6h@Random prob=1x/24h@Random O2=2h/5h@Random [Obs Mem:1H Ref:- Tool]

*** RESTARTING Gen 4 (attempt 2 -> 3) ***
Reason: Day 1 fitness 0.7541 < 0.9. Had 30 fish (max 49).
Reflection: Increase the food_interval to 8h and set the fish_count to 40.

.
Generation 4/5 (attempt 3)
.
Day 0 | Stocked 30 fish | food=6x/8h@Random prob=2x/24h@Random O2=3h/12h@Random | LLM:1.7s
Day 1 | Fit:0.7695 Yld:0.6122 Sav:0.9652 Hlth:0.9520 | Surv:100.0% (30/30) | Cost:\$ 17.40 Sav
e:\$ 482.60 | Sim:0.0s LLM:1.1s
| food=15x/6h@Random prob=2x/24h@Random O2=2h/4h@Random [Obs Mem:1H Ref:- Tool]
Day 2 | Fit:0.7495 Yld:0.6122 Sav:0.8962 Hlth:0.9753 | Surv:100.0% (30/30) | Cost:\$ 51.90 Sav
e:\$ 448.10 | Sim:0.1s LLM:1.1s
| food=15x/6h@Random prob=1x/24h@Random O2=1h/4h@Random [Obs Mem:2H Ref:- Tool]
Day 3 | Fit:0.7378 Yld:0.6122 Sav:0.8587 Hlth:0.9808 | Surv:100.0% (30/30) | Cost:\$ 70.65 Sav
e:\$ 429.35 | Sim:0.1s LLM:1.1s
| food=15x/6h@Random prob=2x/24h@Random O2=2h/4h@Random [Obs Mem:3H Ref:- Tool]
Day 4 | Fit:0.7140 Yld:0.6122 Sav:0.7897 Hlth:0.9726 | Surv:100.0% (30/30) | Cost:\$ 105.15 Sav
e:\$ 394.85 | Sim:0.1s LLM:1.1s
| food=15x/6h@Random prob=2x/24h@Random O2=2h/4h@Random [Obs Mem:4H Ref:- Tool]
Day 5 | Fit:0.6924 Yld:0.6122 Sav:0.7207 Hlth:0.9822 | Surv:100.0% (30/30) | Cost:\$ 139.65 Sav
e:\$ 360.35 | Sim:0.1s LLM:2.2s
| food=15x/6h@Random prob=1x/24h@Random O2=1h/4h@Random [Obs Mem:5H Ref:- Tool]
Day 6 | Fit:0.6791 Yld:0.6122 Sav:0.6832 Hlth:0.9746 | Surv:100.0% (30/30) | Cost:\$ 158.40 Sav
e:\$ 341.60 | Sim:0.1s LLM:1.5s
| food=15x/6h@Random prob=1x/24h@Random O2=4h/8h@Random [Obs Mem:6H Ref:Y Tool]

>>> REFLECTION: D6(20%): To increase fitness, consider reducing the food interval to 6 hours and decreasing the food quantity to 20. This will help save more of your fish's energy, potentially increasing its fitness. Param values: `food\_interval=6h`, `food\_quantity=20`.

Day 7 | Fit:0.6558 Yld:0.6122 Sav:0.6157 Hlth:0.9661 | Surv:100.0% (30/30) | Cost:\$ 192.15 Sav
e:\$ 307.85 | Sim:0.1s LLM:1.1s
| food=15x/6h@Random prob=1x/24h@Random O2=4h/8h@Random [Obs Mem:7H Ref:- Tool]
Day 8 | Fit:0.6339 Yld:0.6122 Sav:0.5482 Hlth:0.9690 | Surv:100.0% (30/30) | Cost:\$ 225.90 Sav
e:\$ 274.10 | Sim:0.1s LLM:1.4s
| food=25x/8h@Random prob=2x/24h@Random O2=2h/8h@Random [Obs Mem:8H Ref:Y Tool]

>>> REFLECTION: D8(warn:64/47): To increase your fitness score, consider reducing the `food_inter
val` to a lower value, such as 2 hours. This might help in maintaining a higher fish health level more con
sistently. Change it to `food\_interval=2`.

Day 9 | Fit:0.6225 Yld:0.6122 Sav:0.5077 Hlth:0.9853 | Surv:100.0% (30/30) | Cost:\$ 246.15 Sav
e:\$ 253.85 | Sim:0.1s LLM:1.4s
| food=20x/8h@Random prob=3x/24h@Random O2=4h/6h@Random [Obs Mem:9H Ref:Y Tool]

>>> REFLECTION: D9(warn:55/54): To increase your fitness score, you might consider reducing the f
ood interval to make it more frequent (e.g., `food\_interval=5`). This could help maintain a higher energy
level for your fish, potentially improving their performance and your fitness score.

Day 10 | Fit:0.5917 Yld:0.6122 Sav:0.4172 Hlth:0.9771 | Surv:100.0% (30/30) | Cost:\$ 291.40 Sav
e:\$ 208.60 | Sim:0.2s LLM:1.5s
| food=20x/6h@Random prob=2x/24h@Random O2=2h/12h@Random [Obs Mem:10H Ref:Y Tool]

>>> REFLECTION: D10(warn:75/63): To increase your fitness score, consider reducing the `food_inte
rval` to a lower value, such as 2 hours. This will allow you to feed your fish more frequently, potentiall
y improving their health and increasing your fitness score.

Day 11 | Fit:0.5824 Yld:0.6122 Sav:0.3862 Hlth:0.9852 | Surv:100.0% (30/30) | Cost:\$ 306.90 Save:\$ 193.10 | Sim:0.3s LLM:1.6s
| food=30x/4h@Random prob=2x/24h@Random O2=2h/5h@Random [Obs Mem:11H Ref:Y Tool]

>>> REFLECTION: D11(warn:90/72): To increase your fitness score, consider reducing the `food\_interval` to a lower value (e.g., from 1-6 hours) and decreasing `spending` by choosing a higher `food\_quantity` within its range (e.g., closer to 30). For example, you could change `food\_interval=4` to

Day 12 | Fit:0.5590 Yld:0.6122 Sav:0.3152 Hlth:0.9854 | Surv:100.0% (30/30) | Cost:\$ 342.40 Save:\$ 157.60 | Sim:0.2s LLM:2.1s
| food=25x/3h@Random prob=1x/24h@Random O2=2h/8h@Random [Obs Mem:12H Ref:Y Tool]

>>> REFLECTION: D12(warn:84/79): To increase your fitness score, consider reducing the `food\_interval` to a lower value (e.g., from 1-6 hours) and decreasing `food\_quantity` slightly (e.g., to 25). This might help in spending less while maintaining or improving your fitness.

Day 13 | Fit:0.5416 Yld:0.6122 Sav:0.2637 Hlth:0.9821 | Surv:100.0% (30/30) | Cost:\$ 368.15 Save:\$ 131.85 | Sim:0.2s LLM:1.4s
| food=25x/8h@Random prob=2x/24h@Random O2=2h/8h@Random [Obs Mem:13H Ref:Y Tool]

>>> REFLECTION: D13(warn:109/88): To increase your fitness score, consider reducing the `food\_interval` to a lower value, such as 2 hours. This will allow you to feed your fish more frequently, potentially improving their health and increasing your fitness score. Change `food\_interval=2`.

Day 14 | Fit:0.5258 Yld:0.6122 Sav:0.2232 Hlth:0.9618 | Surv:100.0% (30/30) | Cost:\$ 388.40 Save:\$ 111.60 | Sim:0.3s LLM:1.4s
| food=20x/4h@Random prob=1x/48h@Random O2=3h/6h@Random [Obs Mem:14H Ref:Y Tool]

>>> REFLECTION: D14(warn:137/99): To improve your fitness score, you might consider reducing the food interval to increase spending less while maintaining health. Try setting `food\_interval` to 2 hours on Day 14. This could help boost your fitness score by saving more resources.

Day 15 | Fit:0.4980 Yld:0.6122 Sav:0.1497 Hlth:0.9320 | Surv:100.0% (30/30) | Cost:\$ 425.15 Save:\$ 74.85 | Sim:0.3s LLM:1.5s
| food=20x/4h@Random prob=1x/24h@Random O2=3h/8h@Random [Obs Mem:15H Ref:Y Tool]

>>> REFLECTION: D15(warn:143/108): To increase your fitness from 0.498 to closer to 0.526, consider adjusting the `food\_interval` parameter to a lower value (e.g., 2 hours) since you are already feeding at maximum quantity and location. This might encourage more frequent feeding, which could help

Day 16 | Fit:0.4795 Yld:0.6122 Sav:0.0912 Hlth:0.9388 | Surv:100.0% (30/30) | Cost:\$ 454.40 Save:\$ 45.60 | Sim:0.4s LLM:2.1s [EARLY QUIT]

>>> REFLECTION: D16(warn:140/117): To increase your fitness score, you might consider decreasing the `food\_interval` to a lower value (e.g., from 1-2 hours to 1 hour) and reducing the `food\_quantity` slightly (e.g., from 30 to 25). This could help in spending less while maintaining or improving

Gen 4 FINAL (attempt 3):
Fit:0.0000 | Yield:0.6122 | SavingRate:0.0912 | Healthiness:0.9388
Surv:100.0% (30/30) | Cost:\$454.40 | Save:\$45.60 [EARLY QUIT]

>>> END-OF-GEN REFLECTION:
D14(warn:137/99): To improve your fitness score, you might consider reducing the food interval to increase spending less while maintaining health. Try setting `food\_interval` to 2 hours on Day 14. This could help boost your fitness score by saving more resources.

D15(warn:143/108): To increase your

~~~~~  
Generation 5/5 (attempt 1)  
~~~~~

Day 0 | Stocked 40 fish | food=15x/8h@Random prob=1x/24h@Random O2=1h/20h@Random | LLM:1.4s
Day 1 | Fit:0.8871 Yld:0.8163 Sav:0.9840 Hlth:0.9449 | Surv:100.0% (40/40) | Cost:\$ 8.00 Save:\$ 492.00 | Sim:0.1s LLM:1.1s
| food=15x/2h@Random prob=1x/24h@Random O2=1h/4h@Random [Obs Mem:1H Ref:- Tool]

*** RESTARTING Gen 5 (attempt 1 -> 2) ***
Reason: Day 1 fitness 0.8871 < 0.9. Had 40 fish (max 49).
Reflection: Change `fish\_count` to 49 by setting `food\_interval` to a value that ensures adequate feeding, such as `6`. This will help maximize your fitness score by utilizing the full tank capacity. Parameters: `food\_interval=6`.

.....
Generation 5/5 (attempt 2)

.....
Day 0 | Stocked 30 fish | food=10x/4h@Random prob=1x/24h@Random O2=2h/6h@Random | LLM:3.0s
Day 1 | Fit:0.7678 Yld:0.6122 Sav:0.9525 Hlth:0.9726 | Surv:100.0% (30/30) | Cost:\$ 23.75 Save:\$ 476.25 | Sim:0.1s LLM:1.2s
| food=10x/8h@Random prob=1x/24h@Random O2=2h/10h@Random [Obs Mem:1H Ref:- Tool]

```
*** RESTARTING Gen 5 (attempt 2 -> 3) ***
Reason: Day 1 fitness 0.7678 < 0.9. Had 30 fish (max 49).
Reflection: Increase the `food_interval` to 8h and set `fish_count` to 40.

. . . . .
Generation 5/5 (attempt 3)
. . . . .
Day 0 | Stocked 30 fish | food=25x/10h@Random prob=2x/24h@Random O2=3h/4h@Random | LLM:7.2s
Day 1 | Fit:0.7509 Yld:0.6122 Sav:0.8995 Hlth:0.9776 | Surv:100.0% (30/30) | Cost:$ 50.25 Sav
e:$ 449.75 | Sim:0.1s LLM:1.1s
    | food=20x/3h@Random prob=1x/24h@Random O2=4h/20h@Random [Obs Mem:1H Ref:- Tool]
Day 2 | Fit:0.7399 Yld:0.6122 Sav:0.8620 Hlth:0.9891 | Surv:100.0% (30/30) | Cost:$ 69.00 Sav
e:$ 431.00 | Sim:0.1s LLM:1.1s
    | food=30x/8h@Random prob=1x/24h@Random O2=4h/16h@Random [Obs Mem:2H Ref:- Tool]
Day 3 | Fit:0.7227 Yld:0.6122 Sav:0.8115 Hlth:0.9848 | Surv:100.0% (30/30) | Cost:$ 94.25 Sav
e:$ 405.75 | Sim:0.1s LLM:1.1s
    | food=15x/6h@Random prob=1x/24h@Random O2=3h/10h@Random [Obs Mem:3H Ref:- Tool]
Day 4 | Fit:0.7084 Yld:0.6122 Sav:0.7740 Hlth:0.9689 | Surv:100.0% (30/30) | Cost:$ 113.00 Sav
e:$ 387.00 | Sim:0.1s LLM:1.2s
    | food=25x/8h@Random prob=1x/24h@Random O2=3h/16h@Random [Obs Mem:4H Ref:- Tool]
Day 5 | Fit:0.6957 Yld:0.6122 Sav:0.7350 Hlth:0.9699 | Surv:100.0% (30/30) | Cost:$ 132.50 Sav
e:$ 367.50 | Sim:0.1s LLM:1.4s
    | food=20x/4h@Random prob=3x/24h@Random O2=4h/6h@Random [Obs Mem:5H Ref:Y Tool]

>>> REFLECTION: D5(warn:46/44): To increase your fitness, consider reducing the `food_interval` t
o a lower value, such as `food_interval=2h`. This might help in maintaining higher fish health and fitness
levels.

Day 6 | Fit:0.6651 Yld:0.6122 Sav:0.6385 Hlth:0.9807 | Surv:100.0% (30/30) | Cost:$ 180.75 Sav
e:$ 319.25 | Sim:0.1s LLM:1.4s
    | food=20x/4h@Random prob=1x/24h@Random O2=4h/10h@Random [Obs Mem:6H Ref:Y Tool]

>>> REFLECTION: D6(20%): To increase your fitness, consider reducing the `food_interval` to 2h an
d decreasing `food_quantity` to 10. This will help you save more while maintaining health.

Day 7 | Fit:0.6478 Yld:0.6122 Sav:0.5850 Hlth:0.9833 | Surv:100.0% (30/30) | Cost:$ 207.50 Sav
e:$ 292.50 | Sim:0.1s LLM:0.4s
    | food=10x/2h@Random prob=1x/24h@Random O2=3h/5h@Random [Obs Mem:7H Ref:- Tool]
Day 8 | Fit:0.6211 Yld:0.6122 Sav:0.5015 Hlth:0.9906 | Surv:100.0% (30/30) | Cost:$ 249.25 Sav
e:$ 250.75 | Sim:0.2s LLM:1.5s
    | food=25x/4h@Random prob=3x/24h@Random O2=4h/20h@Random [Obs Mem:8H Ref:Y Tool]

>>> REFLECTION: D8(warn:75/57): To improve your fitness score, you might consider reducing the `f
ood_interval` to a lower value, such as 2 hours, and decreasing the `food_quantity` to 15. This could help
in spending less while maintaining or improving your fitness level. Change these parameters

Day 9 | Fit:0.6053 Yld:0.6122 Sav:0.4570 Hlth:0.9816 | Surv:100.0% (30/30) | Cost:$ 271.50 Sav
e:$ 228.50 | Sim:0.2s LLM:1.1s
    | food=15x/4h@Random prob=1x/48h@Random O2=4h/8h@Random [Obs Mem:9H Ref:- Tool]
Day 10 | Fit:0.5821 Yld:0.6122 Sav:0.3880 Hlth:0.9780 | Surv:100.0% (30/30) | Cost:$ 306.00 Sav
e:$ 194.00 | Sim:0.2s LLM:1.5s
    | food=15x/6h@Random prob=2x/24h@Random O2=2h/8h@Random [Obs Mem:10H Ref:Y Tool]

>>> REFLECTION: D10(warn:169/77): To increase your fitness score, consider reducing the `food_int
erval` to a lower value, such as 2 hours. This will allow you to feed your fish more frequently, potential
ly improving their health and increasing your fitness score. For example:

...
food_interval=

Day 11 | Fit:0.5712 Yld:0.6122 Sav:0.3490 Hlth:0.9945 | Surv:100.0% (30/30) | Cost:$ 325.50 Sav
e:$ 174.50 | Sim:0.2s LLM:1.4s
    | food=20x/8h@Random prob=3x/24h@Random O2=4h/16h@Random [Obs Mem:11H Ref:Y Tool]

>>> REFLECTION: D11(warn:128/90): To increase your fitness, consider reducing the `food_interval`
to 2 hours and decreasing `food_quantity` to 10. This will help you spend less while maintaining your fish
health. Changes apply tomorrow.

Day 12 | Fit:0.5453 Yld:0.6122 Sav:0.3185 Hlth:0.8618 | Surv:100.0% (30/30) | Cost:$ 340.75 Sav
e:$ 159.25 | Sim:0.3s LLM:1.5s
    | food=15x/6h@Random prob=2x/24h@Random O2=3h/8h@Random [Obs Mem:12H Ref:Y Tool]

>>> REFLECTION: D12(warn:154/104): To increase your fitness, consider reducing the `food_interval
` to a lower value, such as 2 hours. This can help boost your fish's energy and potentially improve fitnes
s. For example:

...
food_interval=2
...

This change will apply tomorrow.
```

Day 13 | Fit:0.5095 Yld:0.5918 Sav:0.2645 Hlth:0.8061 | Surv: 96.7% (29/30) | Cost:\$ 367.75 Save:\$ 132.25 | Sim:0.3s LLM:2.5s
| food=25x/2h@Random prob=2x/24h@Random O2=2h/6h@Random [Obs Mem:13H Ref:Y Tool]

>>> REFLECTION: D13(warn:177/119): To increase your fitness score, you might consider reducing the `food\_interval` to a lower value, such as 2 hours, and decreasing the `food\_quantity` to 25. This could help in spending less while maintaining or improving your fitness level. Here's how you can s

Day 14 | Fit:0.4876 Yld:0.5918 Sav:0.1915 Hlth:0.8243 | Surv: 96.7% (29/30) | Cost:\$ 404.25 Save:\$ 95.75 | Sim:0.3s LLM:0.8s
| food=20x/3h@Random prob=1x/24h@Random O2=2h/5h@Random [Obs Mem:14H Ref:Y Tool]

>>> REFLECTION: D14(warn:192/133): To increase your fitness score, you might consider adjusting the `food\_interval` to a lower value (e.g., from 1-5 hours) to ensure your fish receive food more frequently, which can boost their health and, consequently, your fitness score.

Day 15 | Fit:0.4720 Yld:0.5918 Sav:0.1340 Hlth:0.8526 | Surv: 96.7% (29/30) | Cost:\$ 433.00 Save:\$ 67.00 | Sim:0.3s LLM:1.6s
| food=15x/12h@Random prob=1x/24h@Random O2=3h/20h@Random [Obs Mem:15H Ref:Y Tool]

>>> REFLECTION: D15(warn:208/147): To increase your fitness score, consider adjusting the `food\_interval` to a lower value (e.g., `food\_interval=6`) and the `food\_quantity` to a higher value (e.g., `food\_quantity=25`). This can help in providing more frequent and substantial food intake, potenti

Day 16 | Fit:0.4640 Yld:0.5918 Sav:0.0995 Hlth:0.8806 | Surv: 96.7% (29/30) | Cost:\$ 450.25 Save:\$ 49.75 | Sim:0.4s LLM:2.2s
| food=20x/3h@Random prob=1x/24h@Random O2=1h/4h@Random [Obs Mem:16H Ref:Y Tool]

>>> REFLECTION: D16(warn:271/165): To increase your fitness, you could try reducing the `food\_interval` to 2 hours and decreasing `oxygen\_duration` to 1 hour. This might help by ensuring more frequent feeding and shorter oxygen exposure periods, potentially leading to better fish health and high

Day 17 | Fit:0.4491 Yld:0.5918 Sav:0.0520 Hlth:0.8867 | Surv: 96.7% (29/30) | Cost:\$ 474.00 Save:\$ 26.00 | Sim:0.4s LLM:1.5s
| food=25x/2h@Random prob=2x/24h@Random O2=4h/4h@Random [Obs Mem:17H Ref:Y Tool]

>>> REFLECTION: D17(warn:285/182): To increase your fitness score, consider adjusting the `food\_interval` to a lower value (e.g., `food\_interval=2`) and the `food\_quantity` to a higher value (e.g., `food\_quantity=25`). This combination might help boost your fish's health and fitness.

Day 18 | Fit:0.0000 Yld:0.5918 Sav:0.0000 Hlth:0.8977 | Surv: 96.7% (29/30) | Cost:\$ 500.50 Save:\$ 0.00 | Sim:0.2s [OVER\$]

Gen 5 FINAL (attempt 3):
Fit:0.0000 | Yield:0.5918 | SavingRate:0.0000 | Healthiness:0.8977
Surv:96.7% (29/30) | Cost:\$500.50 | Save:\$0.00

>>> END-OF-GEN REFLECTION:
To increase your fitness score, consider adjusting the `food\_interval` to 6 hours and the `food\_quantity` to 25. This can help in providing more frequent and substantial food intake, potentially boosting your fish's health and fitness.

To further improve your fitness, reduce the `food\_interval` to

Saved memory: /content/AquaLLM/logs/memory_no_planning.json
Saved trace: /content/AquaLLM/logs/trace_no_planning.json (408,173 bytes)

=====
Config: no_memory (Obs+Tool+Plan) | Run 1/1
Days: 30 | Budget: \$500.0 | Generations: 5
=====

~~~~~  
Generation 1/5 (attempt 1)  
~~~~~  
Warn: No JSON. Fallback.
Day 0 | Stocked 30 fish | food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random | LLM:3.2s
Day 1 | Fit:0.7754 Yld:0.6122 Sav:0.9855 Hlth:0.9451 | Surv:100.0% (30/30) | Cost:\$ 7.25 Save:\$ 492.75 | Sim:0.0s LLM:2.9s
| food=25x/8h@Random prob=3x/24h@Random O2=2h/6h@Random [Obs Tool Plan]

*** RESTARTING Gen 1 (attempt 1 -> 2) ***
Reason: Day 1 fitness 0.7754 < 0.9. Had 30 fish (max 49).

.
Generation 1/5 (attempt 2)
.
Warn: No JSON. Fallback.
Day 0 | Stocked 40 fish | food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random | LLM:4.3s
Day 1 | Fit:0.8885 Yld:0.6162 Sav:0.9855 Hlth:0.9501 | Surv:100.0% (40/40) | Cost:\$ 7.25 Save:\$ 492.75 | Sim:0.0s LLM:2.9s
| food=25x/8h@Random prob=3x/24h@Random O2=2h/6h@Random [Obs Tool Plan]

Day 1 | Fit:0.8885 Yld:1.0000 Sav:0.9855 Hlth:0.9521 | Surv:100.0% (40/40) | Cost:\$ 7.25 Sav
e:\$ 492.75 | Sim:0.1s LLM:2.8s
| food=20x/12h@Random prob=3x/24h@Random O2=2h/8h@Random [Obs Tool Plan]

*** RESTARTING Gen 1 (attempt 2 -> 3) ***
Reason: Day 1 fitness 0.8885 < 0.9. Had 40 fish (max 49).

Generation 1/5 (attempt 3)

Day 0 | Stocked 49 fish | food=15x/12h@Random prob=3x/24h@Random O2=1h/8h@Random | LLM:2.8s
Day 1 | Fit:0.9853 Yld:1.0000 Sav:0.9775 Hlth:0.9393 | Surv:100.0% (49/49) | Cost:\$ 11.25 Sav
e:\$ 488.75 | Sim:0.1s LLM:6.6s
| food=15x/6h@Random prob=1x/24h@Random O2=3h/10h@Random [Obs Tool Plan]

Day 2 | Fit:0.9628 Yld:0.9796 Sav:0.9400 Hlth:0.9482 | Surv: 98.0% (48/49) | Cost:\$ 30.00 Sav
e:\$ 470.00 | Sim:0.1s LLM:3.4s
| food=10x/12h@Random prob=1x/24h@Random O2=1h/10h@Random [Obs Tool Plan]

Day 3 | Fit:0.9561 Yld:0.9796 Sav:0.9215 Hlth:0.9437 | Surv: 98.0% (48/49) | Cost:\$ 39.25 Sav
e:\$ 460.75 | Sim:0.1s LLM:5.1s
| food=10x/6h@Random prob=2x/24h@Random O2=3h/4h@Random [Obs Tool Plan]

Warn: No JSON. Fallback.
Day 4 | Fit:0.9228 Yld:0.9796 Sav:0.8245 Hlth:0.9325 | Surv: 98.0% (48/49) | Cost:\$ 87.75 Sav
e:\$ 412.25 | Sim:0.1s LLM:1.8s
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Tool Plan]

Warn: No JSON. Fallback.
Day 5 | Fit:0.9204 Yld:0.9796 Sav:0.8150 Hlth:0.9391 | Surv: 98.0% (48/49) | Cost:\$ 92.50 Sav
e:\$ 407.50 | Sim:0.1s LLM:183.0s
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Tool Plan]

Day 6 | Fit:0.9128 Yld:0.9796 Sav:0.8020 Hlth:0.9110 | Surv: 98.0% (48/49) | Cost:\$ 99.00 Sav
e:\$ 401.00 | Sim:0.1s LLM:5.4s
| food=25x/8h@Random prob=2x/24h@Random O2=3h/4h@Random [Obs Tool Plan]

Day 7 | Fit:0.8681 Yld:0.9592 Sav:0.7015 Hlth:0.9092 | Surv: 95.9% (47/49) | Cost:\$ 149.25 Sav
e:\$ 350.75 | Sim:0.1s LLM:5.0s
| food=15x/10h@Random prob=2x/24h@Random O2=3h/4h@Random [Obs Tool Plan]

Day 8 | Fit:0.8238 Yld:0.9388 Sav:0.6040 Hlth:0.9013 | Surv: 93.9% (46/49) | Cost:\$ 198.00 Sav
e:\$ 302.00 | Sim:0.2s LLM:3.7s
| food=20x/12h@Random prob=3x/24h@Random O2=2h/6h@Random [Obs Tool Plan]

Day 9 | Fit:0.8057 Yld:0.9388 Sav:0.5555 Hlth:0.8835 | Surv: 93.9% (46/49) | Cost:\$ 222.25 Sav
e:\$ 277.75 | Sim:0.2s LLM:4.3s
| food=15x/6h@Random prob=3x/24h@Random O2=1h/8h@Random [Obs Tool Plan]

Day 10 | Fit:0.7898 Yld:0.9184 Sav:0.5300 Hlth:0.9149 | Surv: 91.8% (45/49) | Cost:\$ 235.00 Sav
e:\$ 265.00 | Sim:0.2s LLM:5.1s
| food=5x/12h@Random prob=1x/24h@Random O2=3h/16h@Random [Obs Tool Plan]

Day 11 | Fit:0.7741 Yld:0.9184 Sav:0.4975 Hlth:0.8738 | Surv: 91.8% (45/49) | Cost:\$ 251.25 Sav
e:\$ 248.75 | Sim:0.3s LLM:6.9s
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Tool Plan]

Day 12 | Fit:0.7677 Yld:0.9184 Sav:0.4895 Hlth:0.8426 | Surv: 91.8% (45/49) | Cost:\$ 255.25 Sav
e:\$ 244.75 | Sim:0.3s LLM:4.1s
| food=29x/5h@Random prob=2x/24h@Random O2=4h/5h@Random [Obs Tool Plan]

Day 13 | Fit:0.7174 Yld:0.8776 Sav:0.3820 Hlth:0.9053 | Surv: 87.8% (43/49) | Cost:\$ 309.00 Sav
e:\$ 191.00 | Sim:0.3s LLM:3.2s
| food=15x/8h@Random prob=2x/24h@Random O2=3h/5h@Random [Obs Tool Plan]

Warn: No JSON. Fallback.
Day 14 | Fit:0.6811 Yld:0.8571 Sav:0.2995 Hlth:0.9232 | Surv: 85.7% (42/49) | Cost:\$ 350.25 Sav
e:\$ 149.75 | Sim:0.3s LLM:3.8s
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Tool Plan]

Day 15 | Fit:0.6637 Yld:0.8367 Sav:0.2800 Hlth:0.9259 | Surv: 83.7% (41/49) | Cost:\$ 360.00 Sav
e:\$ 140.00 | Sim:0.3s LLM:6.9s
| food=10x/4h@Random prob=3x/24h@Random O2=1h/2h@Random [Obs Tool Plan]

Day 16 | Fit:0.6425 Yld:0.8367 Sav:0.2095 Hlth:0.9433 | Surv: 83.7% (41/49) | Cost:\$ 395.25 Sav
e:\$ 104.75 | Sim:0.4s LLM:3.6s
| food=10x/12h@Random prob=1x/24h@Random O2=1h/6h@Random [Obs Tool Plan]

Day 17 | Fit:0.6341 Yld:0.8367 Sav:0.1860 Hlth:0.9375 | Surv: 83.7% (41/49) | Cost:\$ 407.00 Sav
e:\$ 93.00 | Sim:0.3s LLM:4.4s
| food=15x/9h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Tool Plan]

Day 18 | Fit:0.6300 Yld:0.8367 Sav:0.1780 Hlth:0.9257 | Surv: 83.7% (41/49) | Cost:\$ 411.00 Sav
e:\$ 89.00 | Sim:0.3s LLM:5.3s
| food=10x/8h@Random prob=1x/24h@Random O2=1h/15h@Random [Obs Tool Plan]

Day 19 | Fit:0.6124 Yld:0.8163 Sav:0.1635 Hlth:0.9124 | Surv: 81.6% (40/49) | Cost:\$ 418.25 Sav
e:\$ 81.75 | Sim:0.3s LLM:3.1s
| food=11x/4h@Random prob=1x/48h@Random O2=4h/3h@Random [Obs Tool Plan]

Day 20 | Fit:0.5596 Yld:0.7959 Sav:0.0369 Hlth:0.9143 | Surv: 79.6% (39/49) | Cost:\$ 481.55 Sav
e:\$ 18.45 | Sim:0.4s LLM:7.5s
| food=15x/10h@Random prob=3x/24h@Random O2=2h/15h@Random [Obs Tool Plan]

Day 21 | Fit:0.5512 Yld:0.7959 Sav:0.0079 Hlth:0.9240 | Surv: 79.6% (39/49) | Cost:\$ 496.05 Sav
e:\$ 3.95 | Sim:0.4s LLM:5.2s
| food=10x/10h@Random prob=1x/24h@Random O2=1h/10h@Random [Obs Tool Plan]

Day 22 | Fit:0.0000 Yld:0.7959 Sav:0.0000 Hlth:0.9363 | Surv: 79.6% (39/49) | Cost:\$ 502.80 Sav
e:\$ 0.00 | Sim:0.3s [OVER\$]

Gen 1 FINAL (attempt 3):
Fit:0.0000 | Yield:0.7959 | SavingRate:0.0000 | Healthiness:0.9363

Surv:79.6% (39/49) | Cost:\$502.80 | Save:\$0.00

~~~~~

Generation 2/5 (attempt 1)

~~~~~

Day 0 | Stocked 30 fish | food=10x/12h@Random prob=1x/24h@Random O2=3h/8h@Random | LLM:3.4s
Warn: No JSON. Fallback.

Day 1 | Fit:0.7647 Yld:0.6122 Sav:0.9515 Hlth:0.9496 | Surv:100.0% (30/30) | Cost:\$ 24.25 Sav
e:\$ 475.75 | Sim:0.0s LLM:2.0s
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Tool Plan]

*** RESTARTING Gen 2 (attempt 1 -> 2) ***
Reason: Day 1 fitness 0.7647 < 0.9. Had 30 fish (max 49).

.....

Generation 2/5 (attempt 2)

.....

Day 0 | Stocked 40 fish | food=8x/12h@Random prob=3x/24h@Random O2=2h/16h@Random | LLM:5.4s
Day 1 | Fit:0.8817 Yld:0.8163 Sav:0.9739 Hlth:0.9274 | Surv:100.0% (40/40) | Cost:\$ 13.05 Sav
e:\$ 486.95 | Sim:0.1s LLM:2.2s
| food=5x/12h@Random prob=3x/24h@Random O2=2h/5h@Random [Obs Tool Plan]

*** RESTARTING Gen 2 (attempt 2 -> 3) ***
Reason: Day 1 fitness 0.8817 < 0.9. Had 40 fish (max 49).

.....

Generation 2/5 (attempt 3)

.....

Day 0 | Stocked 49 fish | food=25x/10h@Random prob=3x/48h@Random O2=1h/16h@Random | LLM:3.2s
Day 1 | Fit:0.9886 Yld:1.0000 Sav:0.9780 Hlth:0.9651 | Surv:100.0% (49/49) | Cost:\$ 11.00 Sav
e:\$ 489.00 | Sim:0.1s LLM:4.9s
| food=24x/8h@Random prob=2x/24h@Random O2=1h/20h@Random [Obs Tool Plan]

Day 2 | Fit:0.9829 Yld:1.0000 Sav:0.9628 Hlth:0.9594 | Surv:100.0% (49/49) | Cost:\$ 18.60 Sav
e:\$ 481.40 | Sim:0.1s LLM:4.6s
| food=10x/10h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Tool Plan]

Day 3 | Fit:0.9776 Yld:1.0000 Sav:0.9533 Hlth:0.9414 | Surv:100.0% (49/49) | Cost:\$ 23.35 Sav
e:\$ 476.65 | Sim:0.1s LLM:3.8s
| food=5x/12h@Random prob=1x/24h@Random O2=3h/4h@Random [Obs Tool Plan]

Day 4 | Fit:0.9438 Yld:1.0000 Sav:0.8608 Hlth:0.9144 | Surv:100.0% (49/49) | Cost:\$ 69.60 Sav
e:\$ 430.40 | Sim:0.1s LLM:3.5s
| food=19x/8h@Random prob=3x/24h@Random O2=4h/5h@Random [Obs Tool Plan]

Day 5 | Fit:0.9054 Yld:0.9796 Sav:0.7706 Hlth:0.9362 | Surv: 98.0% (48/49) | Cost:\$ 114.70 Sav
e:\$ 385.30 | Sim:0.1s LLM:3.2s
| food=15x/12h@Random prob=3x/24h@Random O2=2h/4h@Random [Obs Tool Plan]

Day 6 | Fit:0.8807 Yld:0.9796 Sav:0.7031 Hlth:0.9156 | Surv: 98.0% (48/49) | Cost:\$ 148.45 Sav
e:\$ 351.55 | Sim:0.1s LLM:7.6s
| food=5x/1h@Random prob=1x/24h@Random O2=1h/1h@Random [Obs Tool Plan]

Warn: No JSON. Fallback.

Day 7 | Fit:0.8288 Yld:0.9592 Sav:0.5696 Hlth:0.9443 | Surv: 95.9% (47/49) | Cost:\$ 215.20 Sav
e:\$ 284.80 | Sim:0.2s LLM:183.4s
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Tool Plan]

Day 8 | Fit:0.8235 Yld:0.9592 Sav:0.5616 Hlth:0.9220 | Surv: 95.9% (47/49) | Cost:\$ 219.20 Sav
e:\$ 280.80 | Sim:0.2s LLM:5.3s
| food=25x/8h@Random prob=1x/48h@Random O2=4h/10h@Random [Obs Tool Plan]

Day 9 | Fit:0.8094 Yld:0.9592 Sav:0.5126 Hlth:0.9393 | Surv: 95.9% (47/49) | Cost:\$ 243.70 Sav
e:\$ 256.30 | Sim:0.2s LLM:5.5s
| food=7x/8h@Random prob=2x/24h@Random O2=1h/6h@Random [Obs Tool Plan]

Warn: No JSON. Fallback.

Day 10 | Fit:0.7999 Yld:0.9592 Sav:0.4875 Hlth:0.9289 | Surv: 95.9% (47/49) | Cost:\$ 256.25 Sav
e:\$ 243.75 | Sim:0.2s LLM:183.1s
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Tool Plan]

Day 11 | Fit:0.7841 Yld:0.9388 Sav:0.4730 Hlth:0.9306 | Surv: 93.9% (46/49) | Cost:\$ 263.50 Sav
e:\$ 236.50 | Sim:0.2s LLM:2.4s
| food=5x/12h@Random prob=1x/24h@Random O2=4h/20h@Random [Obs Tool Plan]

Day 12 | Fit:0.7671 Yld:0.9184 Sav:0.4505 Hlth:0.9448 | Surv: 91.8% (45/49) | Cost:\$ 274.75 Sav
e:\$ 225.25 | Sim:0.3s LLM:5.9s
| food=10x/12h@Random prob=1x/24h@Random O2=3h/6h@Random [Obs Tool Plan]

Day 13 | Fit:0.7448 Yld:0.9184 Sav:0.3870 Hlth:0.9333 | Surv: 91.8% (45/49) | Cost:\$ 306.50 Sav
e:\$ 193.50 | Sim:0.2s LLM:3.7s
| food=20x/12h@Random prob=2x/24h@Random O2=3h/4h@Random [Obs Tool Plan]

Day 14 | Fit:0.7152 Yld:0.9184 Sav:0.2900 Hlth:0.9536 | Surv: 91.8% (45/49) | Cost:\$ 355.00 Sav
e:\$ 145.00 | Sim:0.3s LLM:5.4s
| food=10x/12h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Tool Plan]

Warn: No JSON. Fallback.

Day 15 | Fit:0.7109 Yld:0.9184 Sav:0.2815 Hlth:0.9410 | Surv: 91.8% (45/49) | Cost:\$ 359.25 Sav
e:\$ 140.75 | Sim:0.3s LLM:3.1s
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Tool Plan]

Day 16 | Fit:0.7043 Yld:0.9184 Sav:0.2685 Hlth:0.9212 | Surv: 91.8% (45/49) | Cost:\$ 365.75 Sav
e:\$ 134.25 | Sim:0.3s LLM:5.2s
| food=25x/7h@Random prob=2x/24h@Random O2=4h/5h@Random [Obs Tool Plan]

Warn: No JSON. Fallback.

Day 17 | Fit:0.6662 Yld:0.9184 Sav:0.1605 Hlth:0.9010 | Surv: 91.8% (45/49) | Cost:\$ 419.75 Sav

Day 17 | Fit:0.6662 Yld:0.9184 Sav:0.1605 Hlth:0.9010 | Surv: 91.8% (45/49) | Cost:\$ 419.75 Sav
e:\$ 80.25 | Sim:0.3s LLM:17.6s
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Tool Plan]
Day 18 | Fit:0.6630 Yld:0.9184 Sav:0.1475 Hlth:0.9100 | Surv: 91.8% (45/49) | Cost:\$ 426.25 Sav
e:\$ 73.75 | Sim:0.3s LLM:24.8s
| food=5x/8h@Random prob=3x/24h@Random O2=2h/6h@Random [Obs Tool Plan]
Day 19 | Fit:0.6455 Yld:0.9184 Sav:0.1015 Hlth:0.8911 | Surv: 91.8% (45/49) | Cost:\$ 449.25 Sav
e:\$ 50.75 | Sim:0.3s LLM:3.7s
| food=25x/12h@Random prob=1x/24h@Random O2=4h/10h@Random [Obs Tool Plan]
Day 20 | Fit:0.6201 Yld:0.8980 Sav:0.0550 Hlth:0.9008 | Surv: 89.8% (44/49) | Cost:\$ 472.50 Sav
e:\$ 27.50 | Sim:0.3s LLM:4.6s
| food=18x/12h@Random prob=1x/24h@Random O2=1h/20h@Random [Obs Tool Plan]
Day 21 | Fit:0.6031 Yld:0.8776 Sav:0.0399 Hlth:0.8941 | Surv: 87.8% (43/49) | Cost:\$ 480.05 Sav
e:\$ 19.95 | Sim:0.4s LLM:3.7s
| food=10x/12h@Random prob=1x/24h@Random O2=3h/4h@Random [Obs Tool Plan]
Day 22 | Fit:0.0000 Yld:0.8776 Sav:0.0000 Hlth:0.8790 | Surv: 87.8% (43/49) | Cost:\$ 501.30 Sav
e:\$ 0.00 | Sim:0.2s [OVER\$]

Gen 2 FINAL (attempt 3):
Fit:0.0000 | Yield:0.8776 | SavingRate:0.0000 | Healthiness:0.8790
Surv:87.8% (43/49) | Cost:\$501.30 | Save:\$0.00

~~~~~  
Generation 3/5 (attempt 1)  
~~~~~  
Day 0 | Stocked 25 fish | food=10x/8h@Random prob=3x/24h@Random O2=2h/12h@Random | LLM:3.4s
Day 1 | Fit:0.7182 Yld:0.5102 Sav:0.9725 Hlth:0.9718 | Surv:100.0% (25/25) | Cost:\$ 13.75 Sav
e:\$ 486.25 | Sim:0.0s LLM:4.1s
| food=15x/8h@Random prob=2x/24h@Random O2=3h/10h@Random [Obs Tool Plan]

*** RESTARTING Gen 3 (attempt 1 -> 2) ***
Reason: Day 1 fitness 0.7182 < 0.9. Had 25 fish (max 49).

.
Generation 3/5 (attempt 2)
.
Warn: No JSON. Fallback.
Day 0 | Stocked 35 fish | food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random | LLM:3.5s
Day 1 | Fit:0.8307 Yld:0.7143 Sav:0.9855 Hlth:0.9383 | Surv:100.0% (35/35) | Cost:\$ 7.25 Sav
e:\$ 492.75 | Sim:0.1s LLM:2.2s
| food=25x/12h@Random prob=3x/24h@Random O2=2h/8h@Random [Obs Tool Plan]

*** RESTARTING Gen 3 (attempt 2 -> 3) ***
Reason: Day 1 fitness 0.8307 < 0.9. Had 35 fish (max 49).

.
Generation 3/5 (attempt 3)
.
Warn: No JSON. Fallback.
Day 0 | Stocked 45 fish | food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random | LLM:3.9s
Day 1 | Fit:0.9437 Yld:0.9184 Sav:0.9855 Hlth:0.9450 | Surv:100.0% (45/45) | Cost:\$ 7.25 Sav
e:\$ 492.75 | Sim:0.1s LLM:3.3s
| food=8x/3h@Random prob=2x/24h@Random O2=4h/15h@Random [Obs Tool Plan]
Warn: No JSON. Fallback.
Day 2 | Fit:0.9326 Yld:0.9184 Sav:0.9411 Hlth:0.9746 | Surv:100.0% (45/45) | Cost:\$ 29.45 Sav
e:\$ 470.55 | Sim:0.1s LLM:1.9s
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Tool Plan]
Day 3 | Fit:0.9263 Yld:0.9184 Sav:0.9266 Hlth:0.9622 | Surv:100.0% (45/45) | Cost:\$ 36.70 Sav
e:\$ 463.30 | Sim:0.1s LLM:4.3s
| food=10x/12h@Random prob=2x/24h@Random O2=3h/4h@Random [Obs Tool Plan]
Day 4 | Fit:0.8937 Yld:0.9184 Sav:0.8316 Hlth:0.9512 | Surv:100.0% (45/45) | Cost:\$ 84.20 Sav
e:\$ 415.80 | Sim:0.1s LLM:2.0s
| food=20x/6h@Random prob=1x/24h@Random O2=3h/8h@Random [Obs Tool Plan]
Warn: No JSON. Fallback.
Day 5 | Fit:0.8787 Yld:0.9184 Sav:0.7771 Hlth:0.9760 | Surv:100.0% (45/45) | Cost:\$ 111.45 Sav
e:\$ 388.55 | Sim:0.1s LLM:3.1s
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Tool Plan]
Day 6 | Fit:0.8742 Yld:0.9184 Sav:0.7641 Hlth:0.9746 | Surv:100.0% (45/45) | Cost:\$ 117.95 Sav
e:\$ 382.05 | Sim:0.1s LLM:3.6s
| food=10x/12h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Tool Plan]
Day 7 | Fit:0.8687 Yld:0.9184 Sav:0.7556 Hlth:0.9524 | Surv:100.0% (45/45) | Cost:\$ 122.20 Sav
e:\$ 377.80 | Sim:0.1s LLM:4.5s
| food=7x/8h@Random prob=2x/24h@Random O2=2h/6h@Random [Obs Tool Plan]
Day 8 | Fit:0.8520 Yld:0.9184 Sav:0.7105 Hlth:0.9372 | Surv:100.0% (45/45) | Cost:\$ 144.75 Sav
e:\$ 355.25 | Sim:0.1s LLM:24.9s
| food=15x/12h@Random prob=2x/48h@Random O2=3h/12h@Random [Obs Tool Plan]
Warn: No JSON. Fallback.
Day 9 | Fit:0.8402 Yld:0.9184 Sav:0.6745 Hlth:0.9376 | Surv:100.0% (45/45) | Cost:\$ 162.75 Sav
e:\$ 337.25 | Sim:0.1s LLM:183.4s
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Tool Plan]
Day 10 | Fit:0.8370 Yld:0.9184 Sav:0.6665 Hlth:0.9328 | Surv:100.0% (45/45) | Cost:\$ 166.75 Sav
e:\$ 333.25 | Sim:0.1s LLM:1.7s

```
| food=30x/10h@Random prob=3x/24h@Random O2=2h/7h@Random [Obs Tool Plan]
Warn: No JSON. Fallback.
Day 11 | Fit:0.8255 Yld:0.9184 Sav:0.6230 Hlth:0.9566 | Surv:100.0% (45/45) | Cost:$ 188.50 Sav
e:$ 311.50 | Sim:0.2s LLM:2.6s
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Tool Plan]
Day 12 | Fit:0.8237 Yld:0.9184 Sav:0.6150 Hlth:0.9639 | Surv:100.0% (45/45) | Cost:$ 192.50 Sav
e:$ 307.50 | Sim:0.2s LLM:4.4s
| food=12x/12h@Random prob=3x/24h@Random O2=3h/5h@Random [Obs Tool Plan]
Day 13 | Fit:0.7974 Yld:0.9184 Sav:0.5381 Hlth:0.9562 | Surv:100.0% (45/45) | Cost:$ 230.95 Sav
e:$ 269.05 | Sim:0.2s LLM:7.5s
| food=10x/10h@Random prob=1x/24h@Random O2=3h/4h@Random [Obs Tool Plan]
Day 14 | Fit:0.7539 Yld:0.8980 Sav:0.4446 Hlth:0.9439 | Surv: 97.8% (44/45) | Cost:$ 277.70 Sav
e:$ 222.30 | Sim:0.2s LLM:7.2s
| food=10x/12h@Random prob=1x/24h@Random O2=1h/10h@Random [Obs Tool Plan]
Warn: No JSON. Fallback.
Day 15 | Fit:0.7464 Yld:0.8980 Sav:0.4311 Hlth:0.9190 | Surv: 97.8% (44/45) | Cost:$ 284.45 Sav
e:$ 215.55 | Sim:0.3s LLM:183.1s
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Tool Plan]
Day 16 | Fit:0.7441 Yld:0.8980 Sav:0.4181 Hlth:0.9351 | Surv: 97.8% (44/45) | Cost:$ 290.95 Sav
e:$ 209.05 | Sim:0.3s LLM:4.7s
| food=5x/12h@Random prob=1x/24h@Random O2=2h/6h@Random [Obs Tool Plan]
Day 17 | Fit:0.7169 Yld:0.8776 Sav:0.3756 Hlth:0.9191 | Surv: 95.6% (43/45) | Cost:$ 312.20 Sav
e:$ 187.80 | Sim:0.2s LLM:49.1s
| food=5x/12h@Random prob=1x/168h@Random O2=1h/2h@Random [Obs Tool Plan]
Day 18 | Fit:0.6941 Yld:0.8776 Sav:0.3146 Hlth:0.8966 | Surv: 95.6% (43/45) | Cost:$ 342.70 Sav
e:$ 157.30 | Sim:0.2s LLM:5.6s
| food=20x/12h@Random prob=1x/24h@Random O2=1h/20h@Random [Obs Tool Plan]
Day 19 | Fit:0.6923 Yld:0.8776 Sav:0.3041 Hlth:0.9105 | Surv: 95.6% (43/45) | Cost:$ 347.95 Sav
e:$ 152.05 | Sim:0.3s LLM:4.8s
| food=15x/3h@Random prob=2x/24h@Random O2=3h/6h@Random [Obs Tool Plan]
Day 20 | Fit:0.6735 Yld:0.8776 Sav:0.2291 Hlth:0.9604 | Surv: 95.6% (43/45) | Cost:$ 385.45 Sav
e:$ 114.55 | Sim:0.3s LLM:5.2s
| food=25x/12h@Random prob=1x/24h@Random O2=3h/6h@Random [Obs Tool Plan]
Day 21 | Fit:0.6537 Yld:0.8776 Sav:0.1626 Hlth:0.9786 | Surv: 95.6% (43/45) | Cost:$ 418.70 Sav
e:$ 81.30 | Sim:0.3s LLM:3.4s
| food=10x/12h@Random prob=1x/36h@Random O2=3h/8h@Random [Obs Tool Plan]
Day 22 | Fit:0.6355 Yld:0.8776 Sav:0.1141 Hlth:0.9601 | Surv: 95.6% (43/45) | Cost:$ 442.95 Sav
e:$ 57.05 | Sim:0.3s LLM:2.2s
| food=25x/8h@Random prob=2x/24h@Random O2=3h/10h@Random [Obs Tool Plan]
Day 23 | Fit:0.6215 Yld:0.8776 Sav:0.0636 Hlth:0.9822 | Surv: 95.6% (43/45) | Cost:$ 468.20 Sav
e:$ 31.80 | Sim:0.3s LLM:2.2s
| food=25x/8h@Random prob=1x/168h@Random O2=3h/4h@Random [Obs Tool Plan]
Day 24 | Fit:0.0000 Yld:0.8776 Sav:0.0000 Hlth:0.9863 | Surv: 95.6% (43/45) | Cost:$ 500.70 Sav
e:$ 0.00 | Sim:0.2s [OVER$]
```

Gen 3 FINAL (attempt 3):
Fit:0.0000 | Yield:0.8776 | SavingRate:0.0000 | Healthiness:0.9863
Surv:95.6% (43/45) | Cost:\$500.70 | Save:\$0.00

~~~~~  
Generation 4/5 (attempt 1)  
~~~~~

```
Day 0 | Stocked 45 fish | food=10x/8h@Random prob=1x/24h@Random O2=2h/12h@Random | LLM:3.2s
Day 1 | Fit:0.9397 Yld:0.9184 Sav:0.9755 Hlth:0.9388 | Surv:100.0% (45/45) | Cost:$ 12.25 Sav
e:$ 487.75 | Sim:0.1s LLM:2.5s
| food=15x/8h@Random prob=1x/24h@Random O2=1h/15h@Random [Obs Tool Plan]
Day 2 | Fit:0.9242 Yld:0.8980 Sav:0.9595 Hlth:0.9470 | Surv: 97.8% (44/45) | Cost:$ 20.25 Sav
e:$ 479.75 | Sim:0.1s LLM:7.3s
| food=20x/12h@Random prob=2x/24h@Random O2=4h/16h@Random [Obs Tool Plan]
Day 3 | Fit:0.9090 Yld:0.8980 Sav:0.9125 Hlth:0.9499 | Surv: 97.8% (44/45) | Cost:$ 43.75 Sav
e:$ 456.25 | Sim:0.1s LLM:3.0s
| food=25x/10h@Random prob=2x/24h@Random O2=3h/10h@Random [Obs Tool Plan]
Day 4 | Fit:0.8965 Yld:0.8980 Sav:0.8745 Hlth:0.9505 | Surv: 97.8% (44/45) | Cost:$ 62.75 Sav
e:$ 437.25 | Sim:0.1s LLM:4.9s
| food=10x/3h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Tool Plan]
Day 5 | Fit:0.8920 Yld:0.8980 Sav:0.8600 Hlth:0.9528 | Surv: 97.8% (44/45) | Cost:$ 70.00 Sav
e:$ 430.00 | Sim:0.1s LLM:3.9s
| food=15x/8h@Random prob=3x/24h@Random O2=2h/10h@Random [Obs Tool Plan]
Day 6 | Fit:0.8796 Yld:0.8980 Sav:0.8210 Hlth:0.9568 | Surv: 97.8% (44/45) | Cost:$ 89.50 Sav
e:$ 410.50 | Sim:0.1s LLM:3.8s
| food=10x/12h@Random prob=1x/24h@Random O2=3h/5h@Random [Obs Tool Plan]
Day 7 | Fit:0.8520 Yld:0.8980 Sav:0.7425 Hlth:0.9425 | Surv: 97.8% (44/45) | Cost:$ 128.75 Sav
e:$ 371.25 | Sim:0.1s LLM:5.2s
| food=10x/8h@Random prob=1x/48h@Random O2=1h/10h@Random [Obs Tool Plan]
Day 8 | Fit:0.8460 Yld:0.8980 Sav:0.7245 Hlth:0.9424 | Surv: 97.8% (44/45) | Cost:$ 137.75 Sav
e:$ 362.25 | Sim:0.1s LLM:2.4s
| food=29x/4h@Random prob=4x/168h@Random O2=3h/15h@Random [Obs Tool Plan]
Warn: No JSON. Fallback.
Day 9 | Fit:0.8342 Yld:0.8980 Sav:0.6771 Hlth:0.9737 | Surv: 97.8% (44/45) | Cost:$ 161.45 Sav
e:$ 338.55 | Sim:0.2s LLM:182.8s
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Tool Plan]
Day 10 | Fit:0.8308 Yld:0.8980 Sav:0.6691 Hlth:0.9677 | Surv: 97.8% (44/45) | Cost:$ 165.45 Sav
```

Day 10 | Fit:0.7931 Yld:0.8980 Sav:0.6551 Hlth:0.9671 | Surv: 97.8% (44/45) | Cost:\$ 183.15 Sav
e:\$ 334.55 | Sim:0.2s LLM:6.0s
| food=15x/6h@Random prob=1x/24h@Random O2=1h/20h@Random [Obs Tool Plan]
Day 11 | Fit:0.8261 Yld:0.8980 Sav:0.6516 Hlth:0.9763 | Surv: 97.8% (44/45) | Cost:\$ 174.20 Sav
e:\$ 325.80 | Sim:0.2s LLM:3.9s
| food=20x/12h@Random prob=2x/24h@Random O2=2h/3h@Random [Obs Tool Plan]
Day 12 | Fit:0.7960 Yld:0.8980 Sav:0.5646 Hlth:0.9654 | Surv: 97.8% (44/45) | Cost:\$ 217.70 Sav
e:\$ 282.30 | Sim:0.2s LLM:4.8s
| food=10x/10h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Tool Plan]
Day 13 | Fit:0.7931 Yld:0.8980 Sav:0.5551 Hlth:0.9671 | Surv: 97.8% (44/45) | Cost:\$ 222.45 Sav
e:\$ 277.55 | Sim:0.2s LLM:3.7s
| food=15x/8h@Random prob=2x/24h@Random O2=2h/12h@Random [Obs Tool Plan]
Day 14 | Fit:0.7841 Yld:0.8980 Sav:0.5276 Hlth:0.9677 | Surv: 97.8% (44/45) | Cost:\$ 236.20 Sav
e:\$ 263.80 | Sim:0.2s LLM:4.2s
| food=25x/4h@Random prob=1x/144h@Random O2=2h/8h@Random [Obs Tool Plan]
Day 15 | Fit:0.7693 Yld:0.8980 Sav:0.4826 Hlth:0.9679 | Surv: 97.8% (44/45) | Cost:\$ 258.70 Sav
e:\$ 241.30 | Sim:0.3s LLM:3.4s
| food=15x/4h@Random prob=2x/24h@Random O2=4h/5h@Random [Obs Tool Plan]
Day 16 | Fit:0.7329 Yld:0.8980 Sav:0.3706 Hlth:0.9725 | Surv: 97.8% (44/45) | Cost:\$ 314.70 Sav
e:\$ 185.30 | Sim:0.3s LLM:3.5s
| food=10x/12h@Random prob=2x/24h@Random O2=3h/12h@Random [Obs Tool Plan]
Day 17 | Fit:0.7079 Yld:0.8776 Sav:0.3356 Hlth:0.9539 | Surv: 95.6% (43/45) | Cost:\$ 332.20 Sav
e:\$ 167.80 | Sim:0.3s LLM:5.1s
| food=5x/6h@Random prob=2x/24h@Random O2=2h/12h@Random [Obs Tool Plan]
Day 18 | Fit:0.6866 Yld:0.8571 Sav:0.3106 Hlth:0.9391 | Surv: 93.3% (42/45) | Cost:\$ 344.70 Sav
e:\$ 155.30 | Sim:0.3s LLM:4.5s
| food=10x/12h@Random prob=3x/24h@Random O2=4h/5h@Random [Obs Tool Plan]
Day 19 | Fit:0.6547 Yld:0.8571 Sav:0.2191 Hlth:0.9246 | Surv: 93.3% (42/45) | Cost:\$ 390.45 Sav
e:\$ 109.55 | Sim:0.3s LLM:1.5s
| food=20x/8h@Random prob=2x/24h@Random O2=3h/10h@Random [Obs Tool Plan]
Day 20 | Fit:0.6382 Yld:0.8571 Sav:0.1651 Hlth:0.9357 | Surv: 93.3% (42/45) | Cost:\$ 417.45 Sav
e:\$ 82.55 | Sim:0.4s LLM:5.8s
| food=25x/4h@Random prob=1x/24h@Random O2=3h/5h@Random [Obs Tool Plan]
Day 21 | Fit:0.6073 Yld:0.8571 Sav:0.0736 Hlth:0.9300 | Surv: 93.3% (42/45) | Cost:\$ 463.20 Sav
e:\$ 36.80 | Sim:0.4s LLM:7.4s
| food=15x/12h@Random prob=4x/36h@Random O2=4h/18h@Random [Obs Tool Plan]
Day 22 | Fit:0.5806 Yld:0.8367 Sav:0.0246 Hlth:0.9353 | Surv: 91.1% (41/45) | Cost:\$ 487.70 Sav
e:\$ 12.30 | Sim:0.4s LLM:3.6s
| food=10x/12h@Random prob=1x/24h@Random O2=2h/6h@Random [Obs Tool Plan]
Day 23 | Fit:0.0000 Yld:0.8367 Sav:0.0000 Hlth:0.9410 | Surv: 91.1% (41/45) | Cost:\$ 501.95 Sav
e:\$ 0.00 | Sim:0.2s [OVER\$]

Gen 4 FINAL (attempt 1):
Fit:0.0000 | Yield:0.8367 | SavingRate:0.0000 | Healthiness:0.9410
Surv:91.1% (41/45) | Cost:\$501.95 | Save:\$0.00

~~~~~  
Generation 5/5 (attempt 1)  
~~~~~

Warn: No JSON. Fallback.
Day 0 | Stocked 30 fish | food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random | LLM:3.3s
Warn: No JSON. Fallback.
Day 1 | Fit:0.7758 Yld:0.6122 Sav:0.9855 Hlth:0.9489 | Surv:100.0% (30/30) | Cost:\$ 7.25 Sav
e:\$ 492.75 | Sim:0.0s LLM:3.3s
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Tool Plan]

*** RESTARTING Gen 5 (attempt 1 -> 2) ***
Reason: Day 1 fitness 0.7758 < 0.9. Had 30 fish (max 49).

.
Generation 5/5 (attempt 2)
.

Day 0 | Stocked 40 fish | food=15x/8h@Random prob=2x/24h@Random O2=2h/10h@Random | LLM:2.8s
Day 1 | Fit:0.8699 Yld:0.7959 Sav:0.9625 Hlth:0.9545 | Surv: 97.5% (39/40) | Cost:\$ 18.75 Sav
e:\$ 481.25 | Sim:0.1s LLM:4.0s
| food=10x/12h@Random prob=2x/36h@Random O2=3h/8h@Random [Obs Tool Plan]

*** RESTARTING Gen 5 (attempt 2 -> 3) ***
Reason: Day 1 fitness 0.8699 < 0.9. Had 40 fish (max 49).

.
Generation 5/5 (attempt 3)
.

Day 0 | Stocked 49 fish | food=10x/8h@Random prob=3x/24h@Random O2=2h/16h@Random | LLM:5.2s
Day 1 | Fit:0.9850 Yld:1.0000 Sav:0.9725 Hlth:0.9509 | Surv:100.0% (49/49) | Cost:\$ 13.75 Sav
e:\$ 486.25 | Sim:0.1s LLM:4.5s
| food=10x/5h@Random prob=1x/24h@Random O2=3h/8h@Random [Obs Tool Plan]
Day 2 | Fit:0.9666 Yld:1.0000 Sav:0.9210 Hlth:0.9390 | Surv:100.0% (49/49) | Cost:\$ 39.50 Sav
e:\$ 460.50 | Sim:0.1s LLM:8.7s
| food=10x/12h@Random prob=2x/24h@Random O2=3h/4h@Random [Obs Tool Plan]
Day 3 | Fit:0.9356 Yld:1.0000 Sav:0.8260 Hlth:0.9421 | Surv:100.0% (49/49) | Cost:\$ 87.00 Sav
e:\$ 413.00 | Sim:0.1s LLM:4.8s

| food=30x/12h@Random prob=1x/168h@Random O2=3h/12h@Random [Obs Tool Plan]
Day 4 | Fit:0.9233 Yld:1.0000 Sav:0.7900 Hlth:0.9387 | Surv:100.0% (49/49) | Cost:\$ 105.00 Sav
e:\$ 395.00 | Sim:0.1s LLM:6.0s
| food=12x/8h@Random prob=1x/24h@Random O2=3h/5h@Random [Obs Tool Plan]
Day 5 | Fit:0.9006 Yld:1.0000 Sav:0.7249 Hlth:0.9281 | Surv:100.0% (49/49) | Cost:\$ 137.55 Sav
e:\$ 362.45 | Sim:0.1s LLM:3.5s
| food=10x/12h@Random prob=2x/24h@Random O2=2h/6h@Random [Obs Tool Plan]
Day 6 | Fit:0.8826 Yld:1.0000 Sav:0.6799 Hlth:0.9015 | Surv:100.0% (49/49) | Cost:\$ 160.05 Sav
e:\$ 339.95 | Sim:0.1s LLM:4.2s
| food=7x/8h@Random prob=2x/24h@Random O2=3h/10h@Random [Obs Tool Plan]
Day 7 | Fit:0.8687 Yld:1.0000 Sav:0.6448 Hlth:0.8829 | Surv:100.0% (49/49) | Cost:\$ 177.60 Sav
e:\$ 322.40 | Sim:0.1s LLM:3.6s
| food=20x/12h@Random prob=2x/30h@Random O2=3h/10h@Random [Obs Tool Plan]
Day 8 | Fit:0.8325 Yld:0.9592 Sav:0.5978 Hlth:0.8975 | Surv: 95.9% (47/49) | Cost:\$ 201.10 Sav
e:\$ 298.90 | Sim:0.1s LLM:3.4s
| food=10x/7h@Random prob=1x/24h@Random O2=4h/7h@Random [Obs Tool Plan]
Day 9 | Fit:0.8105 Yld:0.9592 Sav:0.5283 Hlth:0.9052 | Surv: 95.9% (47/49) | Cost:\$ 235.85 Sav
e:\$ 264.15 | Sim:0.2s LLM:5.0s
| food=10x/8h@Random prob=3x/24h@Random O2=3h/12h@Random [Obs Tool Plan]
Day 10 | Fit:0.7745 Yld:0.9184 Sav:0.4908 Hlth:0.8953 | Surv: 91.8% (45/49) | Cost:\$ 254.60 Sav
e:\$ 245.40 | Sim:0.2s LLM:4.5s
| food=10x/12h@Random prob=1x/168h@Random O2=4h/4h@Random [Obs Tool Plan]
Day 11 | Fit:0.7078 Yld:0.8776 Sav:0.3688 Hlth:0.8618 | Surv: 87.8% (43/49) | Cost:\$ 315.60 Sav
e:\$ 184.40 | Sim:0.2s LLM:5.4s
| food=25x/10h@Random prob=1x/48h@Random O2=4h/6h@Random [Obs Tool Plan]
Day 12 | Fit:0.6820 Yld:0.8776 Sav:0.2838 Hlth:0.8807 | Surv: 87.8% (43/49) | Cost:\$ 358.10 Sav
e:\$ 141.90 | Sim:0.3s LLM:4.4s
| food=15x/2h@Random prob=2x/24h@Random O2=3h/8h@Random [Obs Tool Plan]
Day 13 | Fit:0.6653 Yld:0.8776 Sav:0.2178 Hlth:0.9228 | Surv: 87.8% (43/49) | Cost:\$ 391.10 Sav
e:\$ 108.90 | Sim:0.3s LLM:3.5s
| food=25x/5h@Random prob=1x/24h@Random O2=2h/6h@Random [Obs Tool Plan]
Day 14 | Fit:0.6481 Yld:0.8776 Sav:0.1638 Hlth:0.9283 | Surv: 87.8% (43/49) | Cost:\$ 418.10 Sav
e:\$ 81.90 | Sim:0.3s LLM:3.5s
| food=12x/10h@Random prob=1x/24h@Random O2=3h/14h@Random [Obs Tool Plan]
Warn: No JSON. Fallback.
Day 15 | Fit:0.6177 Yld:0.8367 Sav:0.1299 Hlth:0.9554 | Surv: 83.7% (41/49) | Cost:\$ 435.05 Sav
e:\$ 64.95 | Sim:0.3s LLM:3.0s
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Tool Plan]
Day 16 | Fit:0.6148 Yld:0.8367 Sav:0.1169 Hlth:0.9666 | Surv: 83.7% (41/49) | Cost:\$ 441.55 Sav
e:\$ 58.45 | Sim:0.4s LLM:4.8s
| food=10x/6h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Tool Plan]
Warn: No JSON. Fallback.
Day 17 | Fit:0.6109 Yld:0.8367 Sav:0.1064 Hlth:0.9633 | Surv: 83.7% (41/49) | Cost:\$ 446.80 Sav
e:\$ 53.20 | Sim:0.3s LLM:3.5s
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Tool Plan]
Day 18 | Fit:0.6070 Yld:0.8367 Sav:0.0984 Hlth:0.9527 | Surv: 83.7% (41/49) | Cost:\$ 450.80 Sav
e:\$ 49.20 | Sim:0.3s LLM:3.9s
| food=5x/6h@Random prob=2x/24h@Random O2=3h/8h@Random [Obs Tool Plan]
Day 19 | Fit:0.5878 Yld:0.8367 Sav:0.0484 Hlth:0.9304 | Surv: 83.7% (41/49) | Cost:\$ 475.80 Sav
e:\$ 24.20 | Sim:0.3s LLM:3.7s
| food=20x/12h@Random prob=2x/24h@Random O2=2h/4h@Random [Obs Tool Plan]
Day 20 | Fit:0.0000 Yld:0.8367 Sav:0.0000 Hlth:0.9309 | Surv: 83.7% (41/49) | Cost:\$ 501.80 Sav
e:\$ 0.00 | Sim:0.3s [OVER\$]

Gen 5 FINAL (attempt 3):
Fit:0.0000 | Yield:0.8367 | SavingRate:0.0000 | Healthiness:0.9309
Surv:83.7% (41/49) | Cost:\$501.80 | Save:\$0.00

Saved memory: /content/AquaLLM/logs/memory_no_memory.json
Saved trace: /content/AquaLLM/logs/trace_no_memory.json (611,116 bytes)

=====
Config: no_tools (Obs+Mem+Ref+Plan) | Run 1/1
Days: 30 | Budget: \$500.0 | Generations: 5
=====

~~~~~  
Generation 1/5 (attempt 1)  
~~~~~

Day 0 | Stocked 45 fish | food=10x/2h@Random prob=1x/48h@Random O2=1h/1h@Random | LLM:1.0s
Day 1 | Fit:0.9074 Yld:0.9184 Sav:0.8665 Hlth:0.9693 | Surv:100.0% (45/45) | Cost:\$ 66.75 Sav
e:\$ 433.25 | Sim:0.1s LLM:2.4s
| food=10x/1h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:1H Ref:- Plan]
Day 2 | Fit:0.8988 Yld:0.9184 Sav:0.8375 Hlth:0.9781 | Surv:100.0% (45/45) | Cost:\$ 81.25 Sav
e:\$ 418.75 | Sim:0.1s LLM:1.7s
| food=10x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:2H Ref:- Plan]
Warn: No JSON. Fallback.
Day 3 | Fit:0.8922 Yld:0.9184 Sav:0.8190 Hlth:0.9736 | Surv:100.0% (45/45) | Cost:\$ 90.50 Sav
e:\$ 409.50 | Sim:0.1s LLM:1.9s
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:3H Ref:- Plan]
Day 4 | Fit:0.8890 Yld:0.9184 Sav:0.8110 Hlth:0.9692 | Surv:100.0% (45/45) | Cost:\$ 94.50 Sav
e:\$ 405.50 | Sim:0.1s LLM:1.2s

```
| food=10x/4h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:4H Ref:- Plan]
Day 5 | Fit:0.8841 Yld:0.9184 Sav:0.7985 Hlth:0.9625 | Surv:100.0% (45/45) | Cost:$ 100.75 Sav
e:$ 399.25 | Sim:0.1s LLM:0.7s
| food=10x/2h@Random prob=1x/48h@Random O2=4h/4h@Random [Obs Mem:5H Ref:Y Plan]

>>> REFLECTION: D5(warn:55/46): To increase your fitness, consider reducing the `food_interval` t
o 2 hours and decreasing `food_quantity` to 10. This will help you spend less while maintaining good healt
h for your fish. Changes apply tomorrow.

Day 6 | Fit:0.8407 Yld:0.9184 Sav:0.6665 Hlth:0.9639 | Surv:100.0% (45/45) | Cost:$ 166.75 Sav
e:$ 333.25 | Sim:0.1s LLM:1.1s
| food=10x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:6H Ref:Y Plan]

>>> REFLECTION: D6(20%): To increase your fitness, consider reducing the `food_interval` to 2 hou
rs and decreasing the `food_quantity` to 10. This will help you save more while maintaining adequate nutri
tion for your fish population.

Warn: No JSON. Fallback.
Day 7 | Fit:0.8336 Yld:0.9184 Sav:0.6480 Hlth:0.9555 | Surv:100.0% (45/45) | Cost:$ 176.00 Sav
e:$ 324.00 | Sim:0.2s LLM:0.5s
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:7H Ref:- Plan]
Day 8 | Fit:0.8286 Yld:0.9184 Sav:0.6400 Hlth:0.9357 | Surv:100.0% (45/45) | Cost:$ 180.00 Sav
e:$ 320.00 | Sim:0.2s LLM:1.6s
| food=10x/2h@Random prob=1x/48h@Random O2=1h/4h@Random [Obs Mem:8H Ref:Y Plan]

>>> REFLECTION: D8(warn:93/61): To increase your fitness score, you might consider reducing the `
food_interval` to a lower value, such as 2 hours, and decreasing `oxygen_duration` to 1 hour. This could h
elp in spending less while increasing fitness. For example:

...
food_interval=2h
oxygen_dura

Day 9 | Fit:0.8155 Yld:0.9184 Sav:0.5965 Hlth:0.9466 | Surv:100.0% (45/45) | Cost:$ 201.75 Sav
e:$ 298.25 | Sim:0.2s LLM:0.4s
| food=10x/2h@Random prob=1x/48h@Random O2=2h/3h@Random [Obs Mem:9H Ref:- Plan]
Day 10 | Fit:0.7886 Yld:0.9184 Sav:0.5045 Hlth:0.9749 | Surv:100.0% (45/45) | Cost:$ 247.75 Sav
e:$ 252.25 | Sim:0.2s LLM:1.6s
| food=10x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:10H Ref:Y Plan]

>>> REFLECTION: D10(warn:191/84): To increase your fitness score, consider reducing the `food_int
erval` to a lower value, such as 2 hours. This will allow you to feed your fish more frequently, potential
ly improving their health and increasing your fitness score.

Day 11 | Fit:0.7809 Yld:0.9184 Sav:0.4810 Hlth:0.9758 | Surv:100.0% (45/45) | Cost:$ 259.50 Sav
e:$ 240.50 | Sim:0.3s LLM:2.2s
| food=10x/3h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:11H Ref:Y Plan]

>>> REFLECTION: D11(warn:149/99): To increase your fitness score, you might consider reducing the
food interval to make it more frequent (e.g., `food_interval=3`). This could potentially boost your fish's
health and fitness slightly.

Day 12 | Fit:0.7758 Yld:0.9184 Sav:0.4680 Hlth:0.9688 | Surv:100.0% (45/45) | Cost:$ 266.00 Sav
e:$ 234.00 | Sim:0.3s LLM:1.8s
| food=10x/6h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:12H Ref:Y Plan]

>>> REFLECTION: D12(warn:173/115): To increase your fitness score, you might consider reducing th
e `food_interval` to a lower value, such as 2 hours, to spend less and potentially boost your fitness furt
her. For example:

...
food_interval=2
...

This change will apply tomorrow.

Day 13 | Fit:0.7716 Yld:0.9184 Sav:0.4575 Hlth:0.9626 | Surv:100.0% (45/45) | Cost:$ 271.25 Sav
e:$ 228.75 | Sim:0.4s LLM:1.9s
| food=10x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:13H Ref:Y Plan]

>>> REFLECTION: D13(warn:183/130): To increase your fitness score, you might consider reducing th
e `food_interval` to a lower value, such as 2 hours, to spend less and potentially increase your fitness.
For example:

...
food_interval=2
...

Day 14 | Fit:0.7654 Yld:0.9184 Sav:0.4405 Hlth:0.9574 | Surv:100.0% (45/45) | Cost:$ 279.75 Sav
e:$ 220.25 | Sim:0.3s LLM:1.6s
| food=10x/3h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:14H Ref:Y Plan]
```

>>> REFLECTION: D14(warn:157/139): To increase your fitness score, you might consider adjusting the `food\_interval` to a lower value (e.g., `food\_interval=3`) to ensure your fish receive food more frequently, which can improve their health and thus your fitness score.

Warn: No JSON. Fallback.
Day 15 | Fit:0.7595 Yld:0.9184 Sav:0.4260 Hlth:0.9485 | Surv:100.0% (45/45) | Cost:\$ 287.00 Save:\$ 213.00 | Sim:0.3s LLM:1.2s
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:15H Ref:Y Plan]

>>> REFLECTION: D15(warn:166/149): To increase your fitness score, consider reducing the `food\_interval` to a lower value, such as 2 hours. This will allow you to feed your fish more frequently, potentially improving their health and increasing your fitness score. Change the parameter by setting

Day 16 | Fit:0.7547 Yld:0.9184 Sav:0.4130 Hlth:0.9445 | Surv:100.0% (45/45) | Cost:\$ 293.50 Save:\$ 206.50 | Sim:0.3s LLM:0.9s
| food=10x/3h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:16H Ref:Y Plan]

>>> REFLECTION: D16(warn:184/158): To increase your fitness score, you might consider reducing the food interval to make it more frequent (e.g., from 1-12h to a lower number like 3h). This could help boost your fish's health and fitness slightly. Change `food\_interval=3`.

Day 17 | Fit:0.7501 Yld:0.9184 Sav:0.3985 Hlth:0.9458 | Surv:100.0% (45/45) | Cost:\$ 300.75 Save:\$ 199.25 | Sim:0.3s LLM:2.1s
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:17H Ref:Y Plan]

>>> REFLECTION: D17(warn:206/169): To increase your fitness score, you could try adjusting the `food\_interval` to a lower value, such as `food\_interval=8`. This might help in boosting your fish's health and fitness slightly.

Day 18 | Fit:0.7450 Yld:0.9184 Sav:0.3905 Hlth:0.9256 | Surv:100.0% (45/45) | Cost:\$ 304.75 Save:\$ 195.25 | Sim:0.4s LLM:0.8s
| food=10x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:18H Ref:Y Plan]

>>> REFLECTION: D18(warn:179/176): To increase your fitness, consider reducing the `food\_interval` to a lower value, such as `food\_interval=2`. This will increase the frequency of feeding, potentially improving your fish's health and fitness.

Day 19 | Fit:0.7410 Yld:0.9184 Sav:0.3720 Hlth:0.9427 | Surv:100.0% (45/45) | Cost:\$ 314.00 Save:\$ 186.00 | Sim:0.3s LLM:0.5s
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:19H Ref:- Plan]
Day 20 | Fit:0.7368 Yld:0.9184 Sav:0.3640 Hlth:0.9302 | Surv:100.0% (45/45) | Cost:\$ 318.00 Save:\$ 182.00 | Sim:0.3s LLM:1.3s
| food=10x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:20H Ref:Y Plan]

>>> REFLECTION: D20(warn:366/189): To increase your fitness, consider reducing the `food\_interval` to 2 hours. This will allow you to feed your fish more frequently, potentially improving their health and increasing your fitness score.

Day 21 | Fit:0.7202 Yld:0.8980 Sav:0.3405 Hlth:0.9495 | Surv: 97.8% (44/45) | Cost:\$ 329.75 Save:\$ 170.25 | Sim:0.3s LLM:1.2s
| food=10x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:21H Ref:- Plan]
Day 22 | Fit:0.7137 Yld:0.8980 Sav:0.3235 Hlth:0.9419 | Surv: 97.8% (44/45) | Cost:\$ 338.25 Save:\$ 161.75 | Sim:0.4s LLM:1.6s
| food=10x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:22H Ref:Y Plan]

>>> REFLECTION: D22(warn:377/200): To increase your fitness score, consider reducing the `food\_interval` to a lower value, such as 2 hours. This will help you save more than 33% of your fish's fitness, potentially improving your score. For example:

...
food_interval=2
...

Day 23 | Fit:0.7085 Yld:0.8980 Sav:0.3050 Hlth:0.9501 | Surv: 97.8% (44/45) | Cost:\$ 347.50 Save:\$ 152.50 | Sim:0.4s LLM:1.3s
| food=10x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:23H Ref:- Plan]
Day 24 | Fit:0.7022 Yld:0.8980 Sav:0.2880 Hlth:0.9438 | Surv: 97.8% (44/45) | Cost:\$ 356.00 Save:\$ 144.00 | Sim:0.4s LLM:1.6s
| food=10x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:24H Ref:Y Plan]

>>> REFLECTION: D24(warn:326/206): To increase your fitness score, you could try reducing the `food\_interval` to a lower value, such as `food\_interval=2h`. This might help in boosting your fish's health and fitness more effectively.

Day 25 | Fit:0.6954 Yld:0.8980 Sav:0.2695 Hlth:0.9383 | Surv: 97.8% (44/45) | Cost:\$ 365.25 Save:\$ 134.75 | Sim:0.4s LLM:1.8s
| food=10x/3h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:25H Ref:- Plan]
Day 26 | Fit:0.6792 Yld:0.8776 Sav:0.2515 Hlth:0.9461 | Surv: 95.6% (43/45) | Cost:\$ 374.25 Save:\$ 125.75 | Sim:0.4s LLM:1.6s
| food=25x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:26H Ref:Y Plan]

>>> REFLECTION: D26(warn:365/214): To increase fitness, consider reducing the `food\_interval` to a lower value (e.g., `food\_interval=2`) and increasing the `food\_quantity` to a higher value (e.g., `food\_quantity=25`). This will ensure your fish receive more frequent and larger portions of food,

Day 27 | Fit:0.6684 Yld:0.8776 Sav:0.2150 Hlth:0.9563 | Surv: 95.6% (43/45) | Cost:\$ 392.50 Save:\$ 107.50 | Sim:0.4s LLM:2.4s
| food=15x/3h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:27H Ref:- Plan]
Warn: No JSON. Fallback.
Day 28 | Fit:0.6514 Yld:0.8571 Sav:0.1980 Hlth:0.9550 | Surv: 93.3% (42/45) | Cost:\$ 401.00 Save:\$ 99.00 | Sim:0.5s LLM:2.2s
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:28H Ref:Y Plan]

>>> REFLECTION: D28(warn:395/222): To increase your fitness score, consider adjusting the `food\_interval` to a lower value (e.g., 2-3 hours) to ensure your fish receive food more frequently, which can improve their health and boost your fitness score.

Day 29 | Fit:0.6370 Yld:0.8367 Sav:0.1885 Hlth:0.9548 | Surv: 91.1% (41/45) | Cost:\$ 405.75 Save:\$ 94.25 | Sim:0.5s LLM:1.8s
| food=20x/6h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:29H Ref:- Plan]
Day 30 | Fit:0.6337 Yld:0.8367 Sav:0.1755 Hlth:0.9628 | Surv: 91.1% (41/45) | Cost:\$ 412.25 Save:\$ 87.75 | Sim:0.5s [END]

Gen 1 FINAL (attempt 1):
Fit:0.6337 | Yield:0.8367 | SavingRate:0.1755 | Healthiness:0.9628
Surv:91.1% (41/45) | Cost:\$412.25 | Save:\$87.75

>>> END-OF-GEN REFLECTION:
To increase your fitness score, you could try reducing the `food\_interval` to a lower value, such as `food\_interval=2h`. This might help in boosting your fish's health and fitness more effectively.

For further improvement, consider setting both `food\_interval=2h` and increasing the `food\_quantity`

~~~~~  
Generation 2/5 (attempt 1)  
~~~~~  
Day 0 | Stocked 45 fish | food=20x/2h@Random prob=1x/48h@Random O2=4h/3h@Random | LLM:2.6s
Day 1 | Fit:0.9052 Yld:0.9184 Sav:0.8545 Hlth:0.9845 | Surv:100.0% (45/45) | Cost:\$ 72.75 Save:\$ 427.25 | Sim:0.1s LLM:1.1s
| food=25x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:1H Ref:- Plan]
Day 2 | Fit:0.8927 Yld:0.9184 Sav:0.8145 Hlth:0.9899 | Surv:100.0% (45/45) | Cost:\$ 92.75 Save:\$ 407.25 | Sim:0.1s LLM:1.1s
| food=10x/3h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:2H Ref:- Plan]
Day 3 | Fit:0.8876 Yld:0.9184 Sav:0.8000 Hlth:0.9875 | Surv:100.0% (45/45) | Cost:\$ 100.00 Save:\$ 400.00 | Sim:0.1s LLM:0.5s
| food=25x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:3H Ref:- Plan]
Day 4 | Fit:0.8764 Yld:0.9184 Sav:0.7650 Hlth:0.9900 | Surv:100.0% (45/45) | Cost:\$ 117.50 Save:\$ 382.50 | Sim:0.1s LLM:0.5s
| food=25x/2h@Random prob=1x/48h@Random O2=3h/4h@Random [Obs Mem:4H Ref:- Plan]
Day 5 | Fit:0.8363 Yld:0.9184 Sav:0.6435 Hlth:0.9908 | Surv:100.0% (45/45) | Cost:\$ 178.25 Save:\$ 321.75 | Sim:0.2s LLM:1.1s
| food=25x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:5H Ref:- Plan]
Day 6 | Fit:0.8234 Yld:0.9184 Sav:0.6035 Hlth:0.9928 | Surv:100.0% (45/45) | Cost:\$ 198.25 Save:\$ 301.75 | Sim:0.2s LLM:1.6s
| food=10x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:6H Ref:Y Plan]

>>> REFLECTION: D6(warn:51/43): To increase your fitness score, consider reducing the `food\_interval` to a lower value, such as 2 hours. This might help in maintaining a higher food availability for your fish, potentially boosting your fitness score.

Warn: No JSON. Fallback.
Day 7 | Fit:0.8175 Yld:0.9184 Sav:0.5850 Hlth:0.9944 | Surv:100.0% (45/45) | Cost:\$ 207.50 Save:\$ 292.50 | Sim:0.2s LLM:1.1s
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:7H Ref:- Plan]
Day 8 | Fit:0.8145 Yld:0.9184 Sav:0.5770 Hlth:0.9913 | Surv:100.0% (45/45) | Cost:\$ 211.50 Save:\$ 288.50 | Sim:0.2s LLM:1.6s
| food=10x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:8H Ref:Y Plan]

>>> REFLECTION: D8(warn:79/54): To improve your fitness score, you might consider reducing the food interval to increase spending while maintaining performance. For example, you could change `food\_interval` to 2 hours instead of the current value. This would help you save more, potentially increase

Day 9 | Fit:0.8079 Yld:0.9184 Sav:0.5585 Hlth:0.9877 | Surv:100.0% (45/45) | Cost:\$ 220.75 Save:\$ 279.25 | Sim:0.2s LLM:0.8s
| food=10x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:9H Ref:Y Plan]

>>> REFLECTION: D9(warn:85/66): To improve your fitness score, you might consider reducing the food interval to increase spending while maintaining a reasonable survival rate. For example, you could change `food\_interval` to 2 hours:

...
food_interval=2h

food_interval=2h
```

This will decrease your spending a

Day 10 | Fit:0.8010 Yld:0.9184 Sav:0.5415 Hlth:0.9764 | Surv:100.0% (45/45) | Cost:\$ 229.25 Save:\$ 270.75 | Sim:0.3s LLM:1.5s  
| food=10x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:10H Ref:Y Plan]

>>> REFLECTION: D10(warn:146/84): To increase your fitness score, consider reducing the `food\_interval` to a lower value, such as 2 hours. This will allow you to feed your fish more frequently, potentially improving their health and increasing your fitness score.

Day 11 | Fit:0.7929 Yld:0.9184 Sav:0.5180 Hlth:0.9735 | Surv:100.0% (45/45) | Cost:\$ 241.00 Save:\$ 259.00 | Sim:0.3s LLM:2.2s  
| food=10x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:11H Ref:Y Plan]

>>> REFLECTION: D11(warn:178/104): To increase your fitness score, consider reducing the `food\_interval` to a lower value, such as 2 hours. This will decrease spending and potentially raise your fitness score. For example:

```  
food_interval=2
```

This change applies tomorrow.

Day 12 | Fit:0.7877 Yld:0.9184 Sav:0.5010 Hlth:0.9776 | Surv:100.0% (45/45) | Cost:\$ 249.50 Save:\$ 250.50 | Sim:0.3s LLM:1.6s  
| food=10x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:12H Ref:Y Plan]

>>> REFLECTION: D12(warn:227/126): To increase your fitness score, consider reducing the `food\_interval` to a lower value, such as 2 hours. This will allow your fish to be fed more frequently, potentially improving their health and increasing your fitness score.

Day 13 | Fit:0.7817 Yld:0.9184 Sav:0.4825 Hlth:0.9783 | Surv:100.0% (45/45) | Cost:\$ 258.75 Save:\$ 241.25 | Sim:0.3s LLM:1.9s  
| food=10x/3h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:13H Ref:Y Plan]

>>> REFLECTION: D13(warn:241/146): To increase your fitness score, you might consider reducing the `food\_interval` to a lower value, such as 2 hours, to spend less and potentially boost your fitness further. For example:

```  
food_interval=2
```

Day 14 | Fit:0.7775 Yld:0.9184 Sav:0.4695 Hlth:0.9792 | Surv:100.0% (45/45) | Cost:\$ 265.25 Save:\$ 234.75 | Sim:0.3s LLM:1.5s  
| food=10x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:14H Ref:Y Plan]

>>> REFLECTION: D14(warn:265/166): To increase your fitness score, consider reducing the `food\_interval` to a lower value, such as 2 hours. This will allow you to feed your fish more frequently, potentially improving their health and increasing your fitness score. Change `food\_interval=2`.

Day 15 | Fit:0.7708 Yld:0.9184 Sav:0.4510 Hlth:0.9738 | Surv:100.0% (45/45) | Cost:\$ 274.50 Save:\$ 225.50 | Sim:0.3s LLM:1.5s  
| food=10x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:15H Ref:Y Plan]

>>> REFLECTION: D15(warn:282/185): To increase your fitness score, consider reducing the `food\_interval` to a lower value, such as 2 hours. This will allow you to feed your fish more frequently, potentially improving their health and increasing your fitness score. Change `food\_interval=2`.

Day 16 | Fit:0.7620 Yld:0.9184 Sav:0.4290 Hlth:0.9614 | Surv:100.0% (45/45) | Cost:\$ 285.50 Save:\$ 214.50 | Sim:0.4s LLM:1.2s  
| food=20x/3h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:16H Ref:Y Plan]

>>> REFLECTION: D16(warn:264/200): To increase your fitness score, you might consider reducing the food interval to make it more frequent (e.g., from 1-12h to a lower value like 3h) and decreasing the food quantity slightly (e.g., from 5-30 to around 20). This could help boost your fish's health

Day 17 | Fit:0.7541 Yld:0.9184 Sav:0.4065 Hlth:0.9571 | Surv:100.0% (45/45) | Cost:\$ 296.75 Save:\$ 203.25 | Sim:0.4s LLM:2.3s  
| food=10x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:17H Ref:Y Plan]

>>> REFLECTION: D17(warn:222/210): To increase your fitness score, consider reducing the `food\_interval` to a lower value, such as 2 hours. This will allow you to feed your fish more frequently, potentially improving their health and increasing your fitness score. Change `food\_interval=2`.

Day 18 | Fit:0.7477 Yld:0.9184 Sav:0.3895 Hlth:0.9505 | Surv:100.0% (45/45) | Cost:\$ 305.25 Save:\$ 194.75 | Sim:0.4s LLM:0.7s  
| food=10x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:18H Ref:Y Plan]

>>> REFLECTION: D18(60%): To increase your fitness, consider reducing the food interval to 2 hours (food\_interval=2). This change might help boost your fish's health and fitness slightly.

Day 19 | Fit:0.7410 Yld:0.9184 Sav:0.3710 Hlth:0.9457 | Surv:100.0% (45/45) | Cost:\$ 314.50 Save:\$ 185.50 | Sim:0.4s LLM:1.9s  
| food=10x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:19H Ref:- Plan]  
Warn: No JSON. Fallback.  
Day 20 | Fit:0.7239 Yld:0.8980 Sav:0.3540 Hlth:0.9437 | Surv: 97.8% (44/45) | Cost:\$ 323.00 Save:\$ 177.00 | Sim:0.4s LLM:1.6s  
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:20H Ref:Y Plan]

>>> REFLECTION: D20(warn:468/233): To increase your fitness, consider reducing the `food\_interval` to 2 hours (`food\_interval=2`). This might help in maintaining a higher food availability for your fish, potentially improving their health and thus increasing your fitness score.

Day 21 | Fit:0.7192 Yld:0.8980 Sav:0.3395 Hlth:0.9438 | Surv: 97.8% (44/45) | Cost:\$ 330.25 Save:\$ 169.75 | Sim:0.4s LLM:2.0s  
| food=10x/4h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:21H Ref:- Plan]  
Day 22 | Fit:0.7159 Yld:0.8980 Sav:0.3285 Hlth:0.9465 | Surv: 97.8% (44/45) | Cost:\$ 335.75 Save:\$ 164.25 | Sim:0.5s LLM:2.1s  
| food=10x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:22H Ref:Y Plan]

>>> REFLECTION: D22(warn:420/243): To increase your fitness score, consider reducing the `food\_interval` to a lower value, such as 2 hours. This might help in spending less and increasing your Fitness score. For example:

...  
food\_interval=2  
...

Day 23 | Fit:0.7105 Yld:0.8980 Sav:0.3100 Hlth:0.9524 | Surv: 97.8% (44/45) | Cost:\$ 345.00 Save:\$ 155.00 | Sim:0.5s LLM:2.2s  
| food=10x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:23H Ref:- Plan]  
Day 24 | Fit:0.6926 Yld:0.8776 Sav:0.2930 Hlth:0.9440 | Surv: 95.6% (43/45) | Cost:\$ 353.50 Save:\$ 146.50 | Sim:0.5s LLM:0.8s  
| food=15x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:24H Ref:Y Plan]

>>> REFLECTION: D24(warn:472/255): To increase your fitness, consider reducing the `food\_interval` to 8 hours and decreasing `food\_quantity` to 15. This might help in optimizing feeding times and quantities to boost your fish's health and fitness further.

Day 25 | Fit:0.6788 Yld:0.8571 Sav:0.2820 Hlth:0.9524 | Surv: 93.3% (42/45) | Cost:\$ 359.00 Save:\$ 141.00 | Sim:0.5s LLM:2.6s  
| food=10x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:25H Ref:Y Plan]

>>> REFLECTION: D25(warn:274/262): To increase your fitness score, consider reducing the `food\_interval` to a lower value, such as 2 hours. This might help your fish receive more frequent feedings, potentially improving their health and increasing your fitness score.

Day 26 | Fit:0.6717 Yld:0.8571 Sav:0.2600 Hlth:0.9543 | Surv: 93.3% (42/45) | Cost:\$ 370.00 Save:\$ 130.00 | Sim:0.5s LLM:2.1s  
| food=10x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:26H Ref:Y Plan]

>>> REFLECTION: D26(warn:274/269): To increase your fitness score, consider reducing the `food\_interval` to a lower value, such as 2 hours. This could help in spending less and potentially increasing your fitness score. For example, you can change it to:

`food\_interval=2`

Warn: No JSON. Fallback.  
Day 27 | Fit:0.6647 Yld:0.8571 Sav:0.2415 Hlth:0.9464 | Surv: 93.3% (42/45) | Cost:\$ 379.25 Save:\$ 120.75 | Sim:0.5s LLM:2.3s  
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:27H Ref:Y Plan]

>>> REFLECTION: D27(warn:278/276): To increase your fitness score, consider reducing the `food\_interval` to a lower value (e.g., from 1-2 hours to 1 hour). This might help in boosting your fish's health and increasing your fitness score.

Day 28 | Fit:0.6581 Yld:0.8571 Sav:0.2335 Hlth:0.9134 | Surv: 93.3% (42/45) | Cost:\$ 383.25 Save:\$ 116.75 | Sim:0.5s LLM:2.0s  
| food=10x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:28H Ref:- Plan]  
Warn: No JSON. Fallback.  
Day 29 | Fit:0.6556 Yld:0.8571 Sav:0.2150 Hlth:0.9432 | Surv: 93.3% (42/45) | Cost:\$ 392.50 Save:\$ 107.50 | Sim:0.5s LLM:1.0s  
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:29H Ref:Y Plan]

>>> REFLECTION: D29(warn:439/281): To increase your fitness, consider reducing the `food\_interval` to 10 hours or decreasing `food\_quantity` to 25. This might help you save more while maintaining fish health.

Day 30 | Fit:0.6505 Yld:0.8571 Sav:0.2070 Hlth:0.9228 | Surv: 93.3% (42/45) | Cost:\$ 396.50 Save:

```
Day 0 | Fit:0.6505 | Yield:0.8571 | SavingRate:0.2070 | Healthiness:0.9228
e:$ 103.50 | Sim:0.5s [END]

Gen 2 FINAL (attempt 1):
Fit:0.6505 | Yield:0.8571 | SavingRate:0.2070 | Healthiness:0.9228
Surv:93.3% (42/45) | Cost:$396.50 | Save:$103.50

>>> END-OF-GEN REFLECTION:
To increase your fitness score, reduce the `food_interval` to 2 hours. This change will help in spending less and potentially increasing your fitness score.

For tomorrow, set:

`food_interval=2`

This adjustment should help in optimizing your parameters for better fitness.

~~~~~
Generation 3/5 (attempt 1)
~~~~~
Day 0 | Stocked 45 fish | food=15x/2h@Random prob=1x/48h@Random O2=3h/6h@Random | LLM:2.1s
Warn: No JSON. Fallback.
Day 1 | Fit:0.9251 Yld:0.9184 Sav:0.9205 Hlth:0.9684 | Surv:100.0% (45/45) | Cost:$ 39.75 Save:$ 460.25 | Sim:0.1s LLM:0.9s
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:1H Ref:- Plan]
Day 2 | Fit:0.9216 Yld:0.9184 Sav:0.9125 Hlth:0.9612 | Surv:100.0% (45/45) | Cost:$ 43.75 Save:$ 456.25 | Sim:0.1s LLM:0.5s
| food=10x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:2H Ref:- Plan]
Warn: No JSON. Fallback.
Day 3 | Fit:0.9151 Yld:0.9184 Sav:0.8940 Hlth:0.9581 | Surv:100.0% (45/45) | Cost:$ 53.00 Save:$ 447.00 | Sim:0.1s LLM:0.7s
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:3H Ref:- Plan]
Day 4 | Fit:0.9099 Yld:0.9184 Sav:0.8860 Hlth:0.9366 | Surv:100.0% (45/45) | Cost:$ 57.00 Save:$ 443.00 | Sim:0.1s LLM:1.0s
| food=15x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:4H Ref:- Plan]
Warn: No JSON. Fallback.
Day 5 | Fit:0.8913 Yld:0.8980 Sav:0.8615 Hlth:0.9427 | Surv: 97.8% (44/45) | Cost:$ 69.25 Save:$ 430.75 | Sim:0.1s LLM:1.3s
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:5H Ref:Y Plan]

>>> REFLECTION: D5(warn:62/49): To increase your fitness score, consider reducing the `food_interval` to a lower value, such as 2 hours. This will allow you to feed your fish more frequently, potentially improving their health and increasing your fitness score. Change `food_interval=2`.

Day 6 | Fit:0.8885 Yld:0.8980 Sav:0.8485 Hlth:0.9549 | Surv: 97.8% (44/45) | Cost:$ 75.75 Save:$ 424.25 | Sim:0.1s LLM:1.1s
| food=10x/2h@Random prob=1x/48h@Random O2=4h/6h@Random [Obs Mem:6H Ref:Y Plan]

>>> REFLECTION: D6(20%): To increase your fitness, consider reducing the `food_interval` to 2 hours and decreasing the `food_quantity` to 10. This will help you save more while maintaining good health for your fish.

Day 7 | Fit:0.8580 Yld:0.8980 Sav:0.7550 Hlth:0.9582 | Surv: 97.8% (44/45) | Cost:$ 122.50 Save:$ 377.50 | Sim:0.2s LLM:1.0s
| food=10x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:7H Ref:- Plan]
Day 8 | Fit:0.8524 Yld:0.8980 Sav:0.7380 Hlth:0.9583 | Surv: 97.8% (44/45) | Cost:$ 131.00 Save:$ 369.00 | Sim:0.2s LLM:0.8s
| food=10x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:8H Ref:Y Plan]

>>> REFLECTION: D8(warn:79/62): To improve your fitness score, you might consider reducing the food interval to increase spending while maintaining a reasonable survival rate. For example, you could change `food_interval` to 2 hours:

...
food_interval=2
...

This will decrease your spending and

Warn: No JSON. Fallback.
Day 9 | Fit:0.8475 Yld:0.8980 Sav:0.7195 Hlth:0.9680 | Surv: 97.8% (44/45) | Cost:$ 140.25 Save:$ 359.75 | Sim:0.2s LLM:2.0s
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:9H Ref:Y Plan]

>>> REFLECTION: D9(warn:73/70): To increase your fitness score, consider reducing the `food_interval` to a lower value, such as 2 hours. This might help in maintaining a higher fish health and fitness level more consistently.

Day 10 | Fit:0.8442 Yld:0.8980 Sav:0.7115 Hlth:0.9630 | Surv: 97.8% (44/45) | Cost:$ 144.25 Save:$ 355.75 | Sim:0.2s LLM:2.0s
| food=10x/2h@Random prob=1x/48h@Random O2=1h/3h@Random [Obs Mem:10H Ref:Y Plan]
```

>>> REFLECTION: D10(warn:99/81): To increase your fitness score, consider reducing the `food\_interval` to a lower value, such as 2 hours. This will allow you to feed your fish more frequently, potentially improving their health and increasing your fitness score. Change `food\_interval=2`.

Warn: No JSON. Fallback.  
Day 11 | Fit:0.8287 Yld:0.8980 Sav:0.6580 Hlth:0.9809 | Surv: 97.8% (44/45) | Cost:\$ 171.00 Save:\$ 329.00 | Sim:0.2s LLM:2.2s  
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:11H Ref:Y Plan]

>>> REFLECTION: D11(warn:136/95): To increase your fitness score, consider reducing the `food\_interval` to a lower value (e.g., from 1-6 hours) and decreasing `food\_quantity` slightly (e.g., to 20). This might help in spending less while maintaining health. Changes will apply tomorrow.

Day 12 | Fit:0.8228 Yld:0.8980 Sav:0.6500 Hlth:0.9533 | Surv: 97.8% (44/45) | Cost:\$ 175.00 Save:\$ 325.00 | Sim:0.3s LLM:2.4s  
| food=10x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:12H Ref:Y Plan]

>>> REFLECTION: D12(warn:160/110): To increase your fitness score, consider reducing the `food\_interval` to a lower value, such as 2 or 3 hours, to spend less and improve your Fitness score. For example:  
...  
food\_interval=2  
...

Day 13 | Fit:0.8172 Yld:0.8980 Sav:0.6315 Hlth:0.9577 | Surv: 97.8% (44/45) | Cost:\$ 184.25 Save:\$ 315.75 | Sim:0.3s LLM:0.9s  
| food=10x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:13H Ref:Y Plan]

>>> REFLECTION: D13(warn:185/125): To increase your fitness score, you might consider reducing the `food\_interval` to a lower value, such as 2 hours, and decreasing the `food\_quantity` to a lower number, like 10. This could help save more of your fish's energy, potentially increasing your fitness

Day 14 | Fit:0.8111 Yld:0.8980 Sav:0.6145 Hlth:0.9540 | Surv: 97.8% (44/45) | Cost:\$ 192.75 Save:\$ 307.25 | Sim:0.3s LLM:1.9s  
| food=10x/4h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:14H Ref:Y Plan]

>>> REFLECTION: D14(warn:199/140): To increase your fitness score, you might consider reducing the `food\_interval` to a lower value, such as 2 hours, to spend less and potentially boost your fitness further. For example:  
...  
food\_interval=2  
...

Day 15 | Fit:0.8064 Yld:0.8980 Sav:0.6020 Hlth:0.9492 | Surv: 97.8% (44/45) | Cost:\$ 199.00 Save:\$ 301.00 | Sim:0.3s LLM:2.4s  
| food=10x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:15H Ref:Y Plan]

>>> REFLECTION: D15(warn:210/153): To increase your fitness score, you might consider decreasing the `food\_interval` to a lower value, such as 2 hours, to spend less and potentially raise your fitness score. For example:  
...  
food\_interval=2  
...

This change will apply tomorrow.

Day 16 | Fit:0.8001 Yld:0.8980 Sav:0.5800 Hlth:0.9569 | Surv: 97.8% (44/45) | Cost:\$ 210.00 Save:\$ 290.00 | Sim:0.3s LLM:2.0s  
| food=10x/4h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:16H Ref:Y Plan]

>>> REFLECTION: D16(warn:204/165): To increase your fitness score, you could try reducing the `food\_interval` to a lower value, such as 2 hours. This might help in spending less and increasing your Fitness score. For example:  
...  
food\_interval=2  
...

Warn: No JSON. Fallback.  
Day 17 | Fit:0.7946 Yld:0.8980 Sav:0.5675 Hlth:0.9450 | Surv: 97.8% (44/45) | Cost:\$ 216.25 Save:\$ 283.75 | Sim:0.3s LLM:0.7s  
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:17H Ref:Y Plan]

>>> REFLECTION: D17(warn:195/174): To increase your fitness score, consider decreasing the `food\_interval` to a lower value, such as 2 hours. This will allow you to feed your fish more frequently, potentially improving their health and increasing your fitness score.

Day 18 | Fit:0.7927 Yld:0.8980 Sav:0.5595 Hlth:0.9513 | Surv: 97.8% (44/45) | Cost:\$ 220.25 Save:\$ 279.75 | Sim:0.3s LLM:1.7s

| food=10x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:18H Ref:Y Plan]

>>> REFLECTION: D18(warn:206/183): To increase your fitness score, you could try reducing the `food\_interval` to a lower value, such as 2 hours. This might help your fish receive more frequent feedings, potentially improving their health and increasing your fitness score.

Day 19 | Fit:0.7876 Yld:0.8980 Sav:0.5410 Hlth:0.9598 | Surv: 97.8% (44/45) | Cost:\$ 229.50 Save:\$ 270.50 | Sim:0.3s LLM:2.0s  
| food=10x/3h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:19H Ref:Y Plan]

>>> REFLECTION: D19(warn:206/191): To increase your fitness score, you could try reducing the `food\_interval` to a lower value, such as 2 hours. This might help boost your fish's health and fitness slightly. For example:

...  
food\_interval=2  
...

This change will apply tomorrow.

Day 20 | Fit:0.7842 Yld:0.8980 Sav:0.5280 Hlth:0.9672 | Surv: 97.8% (44/45) | Cost:\$ 236.00 Save:\$ 264.00 | Sim:0.4s LLM:1.4s  
| food=10x/3h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:20H Ref:- Plan]

Warn: No JSON. Fallback.  
Day 21 | Fit:0.7770 Yld:0.8980 Sav:0.5085 Hlth:0.9607 | Surv: 97.8% (44/45) | Cost:\$ 245.75 Save:\$ 254.25 | Sim:0.3s LLM:1.1s  
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:21H Ref:Y Plan]

>>> REFLECTION: D21(warn:396/204): To increase your fitness score, consider reducing the `food\_interval` to a lower value, such as 2 hours. This might help in boosting your fish's health and increasing your fitness score.

Day 22 | Fit:0.7724 Yld:0.8980 Sav:0.5005 Hlth:0.9451 | Surv: 97.8% (44/45) | Cost:\$ 249.75 Save:\$ 250.25 | Sim:0.3s LLM:1.6s  
| food=10x/6h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:22H Ref:- Plan]

Day 23 | Fit:0.7676 Yld:0.8980 Sav:0.4900 Hlth:0.9338 | Surv: 97.8% (44/45) | Cost:\$ 255.00 Save:\$ 245.00 | Sim:0.3s LLM:2.7s  
| food=20x/1h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:23H Ref:Y Plan]

>>> REFLECTION: D23(warn:390/214): To improve your fitness score, consider increasing the `food\_interval` to a lower value (e.g., from 1-2 hours to 1 hour) or decreasing the `food\_quantity` slightly (e.g., from 25 to 20). This might help in spending less while maintaining health, thus potentiall

Day 24 | Fit:0.7527 Yld:0.8980 Sav:0.4370 Hlth:0.9551 | Surv: 97.8% (44/45) | Cost:\$ 281.50 Save:\$ 218.50 | Sim:0.4s LLM:1.4s  
| food=10x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:24H Ref:Y Plan]

>>> REFLECTION: D24(80%): To increase your fitness, consider reducing the food interval to 2 hours (`food\_interval=2`). This might help in maintaining a higher fish health and fitness level.

Day 25 | Fit:0.7456 Yld:0.8980 Sav:0.4185 Hlth:0.9466 | Surv: 97.8% (44/45) | Cost:\$ 290.75 Save:\$ 209.25 | Sim:0.4s LLM:1.7s  
| food=10x/1h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:25H Ref:- Plan]

Day 26 | Fit:0.7235 Yld:0.8776 Sav:0.3845 Hlth:0.9494 | Surv: 95.6% (43/45) | Cost:\$ 307.75 Save:\$ 192.25 | Sim:0.4s LLM:1.7s  
| food=30x/1h@Random prob=1x/48h@Random O2=4h/4h@Random [Obs Mem:26H Ref:Y Plan]

>>> REFLECTION: D26(warn:348/224): To increase your fitness score, consider reducing the `food\_interval` to a lower value (e.g., `food\_interval=2`) and increasing the `food\_quantity` to a higher value (e.g., `food\_quantity=25`). This might help in boosting the fish's health and fitness more effe

Day 27 | Fit:0.6600 Yld:0.8776 Sav:0.1910 Hlth:0.9527 | Surv: 95.6% (43/45) | Cost:\$ 404.50 Save:\$ 95.50 | Sim:0.6s LLM:2.0s  
| food=20x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:27H Ref:- Plan]

Warn: No JSON. Fallback.  
Day 28 | Fit:0.6500 Yld:0.8776 Sav:0.1620 Hlth:0.9489 | Surv: 95.6% (43/45) | Cost:\$ 419.00 Save:\$ 81.00 | Sim:0.7s LLM:3.1s  
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:28H Ref:Y Plan]

>>> REFLECTION: D28(warn:361/229): To increase your fitness score, consider reducing the `food\_interval` to a lower value (e.g., from 1-2 hours to 1 hour). This can help boost your fitness score since spending less time between feedings increases Fitness.

Day 29 | Fit:0.6453 Yld:0.8776 Sav:0.1525 Hlth:0.9359 | Surv: 95.6% (43/45) | Cost:\$ 423.75 Save:\$ 76.25 | Sim:0.7s LLM:1.8s  
| food=20x/1h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:29H Ref:- Plan]

Day 30 | Fit:0.6276 Yld:0.8776 Sav:0.0995 Hlth:0.9344 | Surv: 95.6% (43/45) | Cost:\$ 450.25 Save:\$ 49.75 | Sim:0.7s [END]

Gen 3 FINAL (attempt 1):  
Fit:0.6276 | Yield:0.8776 | SavingRate:0.0995 | Healthiness:0.9344  
Surv:95.6% (43/45) | Cost:\$450.25 | Save:\$49.75

```
Surv:95.6% (43/45) | Cost:$450.25 | Save:$49.75
>>> END-OF-GEN REFLECTION:
 To increase your fitness, consider reducing the `food_interval` to 2 hours (`food_interval=2`). This
might help in maintaining a higher fish health and fitness level.

To further improve, also decrease the `food_quantity` to a lower value (e.g., `food_quantity=10`) while ke
eping the `food_interval`

~~~~~
Generation 4/5 (attempt 1)
~~~~~
Day 0 | Stocked 45 fish | food=10x/2h@Random prob=1x/48h@Random O2=3h/6h@Random | LLM:1.6s
Day 1 | Fit:0.9269 Yld:0.9184 Sav:0.9265 Hlth:0.9670 | Surv:100.0% (45/45) | Cost:$ 36.75 Sav
e:$ 463.25 | Sim:0.1s LLM:1.1s
 | food=25x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:1H Ref:- Plan]
Day 2 | Fit:0.9171 Yld:0.9184 Sav:0.8915 Hlth:0.9820 | Surv:100.0% (45/45) | Cost:$ 54.25 Sav
e:$ 445.75 | Sim:0.1s LLM:1.1s
 | food=20x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:2H Ref:- Plan]
Day 3 | Fit:0.9074 Yld:0.9184 Sav:0.8610 Hlth:0.9847 | Surv:100.0% (45/45) | Cost:$ 69.50 Sav
e:$ 430.50 | Sim:0.1s LLM:1.8s
 | food=10x/2h@Random prob=1x/48h@Random O2=4h/3h@Random [Obs Mem:3H Ref:- Plan]
Day 4 | Fit:0.8622 Yld:0.9184 Sav:0.7290 Hlth:0.9713 | Surv:100.0% (45/45) | Cost:$ 135.50 Sav
e:$ 364.50 | Sim:0.2s LLM:1.2s
 | food=15x/4h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:4H Ref:- Plan]
Day 5 | Fit:0.8545 Yld:0.9184 Sav:0.7085 Hlth:0.9630 | Surv:100.0% (45/45) | Cost:$ 145.75 Sav
e:$ 354.25 | Sim:0.2s LLM:1.6s
 | food=10x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:5H Ref:Y Plan]

>>> REFLECTION: D5(warn:58/47): To increase your fitness score, consider reducing the `food_inter
val` to a lower value, such as 2 hours. This might help in maintaining a higher fish health and thus impro
ving the fitness score.

Warn: No JSON. Fallback.
Day 6 | Fit:0.8477 Yld:0.9184 Sav:0.6865 Hlth:0.9670 | Surv:100.0% (45/45) | Cost:$ 156.75 Sav
e:$ 343.25 | Sim:0.2s LLM:0.7s
 | food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:6H Ref:Y Plan]

>>> REFLECTION: D6(20%): To increase your fitness, consider reducing the `food_interval` to 2 hou
rs and decreasing the `food_quantity` to 10. This will help you save more while maintaining good health fo
r your fish. Changes apply tomorrow.

Day 7 | Fit:0.8425 Yld:0.9184 Sav:0.6770 Hlth:0.9496 | Surv:100.0% (45/45) | Cost:$ 161.50 Sav
e:$ 338.50 | Sim:0.2s LLM:2.1s
 | food=10x/4h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:7H Ref:- Plan]
Day 8 | Fit:0.8382 Yld:0.9184 Sav:0.6660 Hlth:0.9444 | Surv:100.0% (45/45) | Cost:$ 167.00 Sav
e:$ 333.00 | Sim:0.2s LLM:1.9s
 | food=10x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:8H Ref:Y Plan]

>>> REFLECTION: D8(warn:117/70): To increase your fitness score, consider reducing the `food_inte
rval` to a lower value, such as 2 hours. This could help boost your fish's health and fitness more effecti
vely. Changes will apply tomorrow.

Day 9 | Fit:0.8319 Yld:0.9184 Sav:0.6475 Hlth:0.9431 | Surv:100.0% (45/45) | Cost:$ 176.25 Sav
e:$ 323.75 | Sim:0.2s LLM:1.6s
 | food=10x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:9H Ref:Y Plan]

>>> REFLECTION: D9(warn:92/81): To increase your fitness, you might consider reducing the `food_i
nterval` to a lower value, such as `food_interval=2h`. This could help in maintaining a higher food availa
bility for your fish, potentially improving their health and increasing your fitness score.

Day 10 | Fit:0.8168 Yld:0.8980 Sav:0.6305 Hlth:0.9574 | Surv: 97.8% (44/45) | Cost:$ 184.75 Sav
e:$ 315.25 | Sim:0.3s LLM:2.4s
 | food=10x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:10H Ref:Y Plan]

>>> REFLECTION: D10(warn:153/99): To increase your fitness score, you could try reducing the `foo
d_interval` to a lower value, such as 2 hours. This might encourage more frequent feedings, potentially imp
roving your fish's health and fitness score.

Day 11 | Fit:0.8109 Yld:0.8980 Sav:0.6070 Hlth:0.9725 | Surv: 97.8% (44/45) | Cost:$ 196.50 Sav
e:$ 303.50 | Sim:0.3s LLM:2.0s
 | food=10x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:11H Ref:Y Plan]

>>> REFLECTION: D11(warn:200/120): To increase your fitness score, consider reducing the `food_in
terval` to a lower value, such as `food_interval=2`. This will decrease the time between feedings, potenti
ally improving your fish's health and fitness score.

Day 12 | Fit:0.8055 Yld:0.8980 Sav:0.5900 Hlth:0.9744 | Surv: 97.8% (44/45) | Cost:$ 205.00 Sav
e:$ 295.00 | Sim:0.3s LLM:2.3s
 | food=10x/3h@Random prob=1x/48h@Random O2=2h/4h@Random [Obs Mem:12H Ref:Y Plan]
```

>>> REFLECTION: D12(warn:238/142): To increase your fitness score, you might consider reducing the `food\_interval` to a lower value (e.g., from 1-3 hours) and decreasing the `oxygen\_duration` to a shorter period (e.g., from 1-2 hours). This could help in spending less while maintaining or improving

Day 13 | Fit:0.7827 Yld:0.8980 Sav:0.5205 Hlth:0.9757 | Surv: 97.8% (44/45) | Cost:\$ 239.75 Save:\$ 260.25 | Sim:0.4s LLM:2.2s  
| food=5x/1h@Random prob=1x/48h@Random O2=1h/1h@Random [Obs Mem:13H Ref:Y Plan]

>>> REFLECTION: D13(warn:265/164): To increase your fitness score, you might consider decreasing the `food\_interval` to a lower value, such as 2 hours, to ensure your fish receive more frequent feedings without exceeding the maximum of 12 hours. This could potentially boost your fitness score on

Day 14 | Fit:0.7389 Yld:0.8980 Sav:0.3885 Hlth:0.9736 | Surv: 97.8% (44/45) | Cost:\$ 305.75 Save:\$ 194.25 | Sim:0.4s LLM:2.3s  
| food=5x/12h@Random prob=1x/48h@Random O2=4h/20h@Random [Obs Mem:14H Ref:Y Plan]

>>> REFLECTION: D14(warn:249/181): To increase your fitness score, you might consider decreasing the `food\_interval` to a lower value (e.g., from 24h to 12h) and reducing the `food\_quantity` slightly (e.g., from 5 to 4). This could help in spending less while maintaining or improving your fitness

Day 15 | Fit:0.7291 Yld:0.8980 Sav:0.3660 Hlth:0.9533 | Surv: 97.8% (44/45) | Cost:\$ 317.00 Save:\$ 183.00 | Sim:0.4s LLM:2.9s  
| food=10x/1h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:15H Ref:Y Plan]

>>> REFLECTION: D15(warn:225/194): To increase your fitness score, consider reducing the `food\_interval` to a lower value (e.g., from 1-2 hours to 1 hour). This will help you spend less and potentially increase your fitness score. For example, you can change it to:

...  
food\_interval=1  
...

Day 16 | Fit:0.7176 Yld:0.8980 Sav:0.3320 Hlth:0.9511 | Surv: 97.8% (44/45) | Cost:\$ 334.00 Save:\$ 166.00 | Sim:0.4s LLM:0.8s  
| food=10x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:16H Ref:Y Plan]

>>> REFLECTION: D16(warn:221/204): To increase your fitness score, you might consider reducing the `food\_interval` to a lower value, such as 2 hours, to spend less and potentially boost your fitness further. For example:

...  
food\_interval=2  
...

Day 17 | Fit:0.7116 Yld:0.8980 Sav:0.3135 Hlth:0.9520 | Surv: 97.8% (44/45) | Cost:\$ 343.25 Save:\$ 156.75 | Sim:0.4s LLM:1.6s  
| food=10x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:17H Ref:Y Plan]

>>> REFLECTION: D17(warn:215/213): To increase your fitness score, you might consider adjusting the `food\_interval` to a lower value, such as 2 hours, to ensure your fish receive more frequent feedings. This could potentially boost your fitness score on Day 18.

Day 18 | Fit:0.7047 Yld:0.8980 Sav:0.2965 Hlth:0.9412 | Surv: 97.8% (44/45) | Cost:\$ 351.75 Save:\$ 148.25 | Sim:0.4s LLM:2.5s  
| food=10x/3h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:18H Ref:Y Plan]

>>> REFLECTION: D18(60%): To increase your fitness score, consider reducing the food interval to 2 hours (food\_interval=2). This change might help in optimizing resource utilization and potentially boosting your fitness score.

Warn: No JSON. Fallback.  
Day 19 | Fit:0.7012 Yld:0.8980 Sav:0.2820 Hlth:0.9525 | Surv: 97.8% (44/45) | Cost:\$ 359.00 Save:\$ 141.00 | Sim:0.5s LLM:2.1s  
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:19H Ref:- Plan]  
Day 20 | Fit:0.6996 Yld:0.8980 Sav:0.2740 Hlth:0.9611 | Surv: 97.8% (44/45) | Cost:\$ 363.00 Save:\$ 137.00 | Sim:0.5s LLM:1.5s  
| food=10x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:20H Ref:Y Plan]

>>> REFLECTION: D20(warn:432/234): To increase your fitness score, consider reducing the `food\_interval` to a lower value, such as 2 hours. This could potentially boost your fitness score since spending less on food maintenance can lead to higher overall fitness.

Day 21 | Fit:0.6920 Yld:0.8980 Sav:0.2505 Hlth:0.9622 | Surv: 97.8% (44/45) | Cost:\$ 374.75 Save:\$ 125.25 | Sim:0.5s LLM:2.2s  
| food=10x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:21H Ref:- Plan]  
Day 22 | Fit:0.6860 Yld:0.8980 Sav:0.2335 Hlth:0.9588 | Surv: 97.8% (44/45) | Cost:\$ 383.25 Save:\$ 116.75 | Sim:0.5s LLM:2.3s  
| food=10x/1h@Random prob=1x/48h@Random O2=4h/3h@Random [Obs Mem:22H Ref:Y Plan]

>>> REFLECTION: D22(warn:407/243): To increase your fitness score, consider reducing the `food\_interval` to a lower value (e.g., from 1-2 hours to 1 hour) and decreasing `oxygen\_duration` to a shorter time frame (e.g., from 1-4 hours to 1 hour). This could help boost your fish's health and fitness

Day 23 | Fit:0.6386 Yld:0.8980 Sav:0.0880 Hlth:0.9638 | Surv: 97.8% (44/45) | Cost:\$ 456.00 Sav  
e:\$ 44.00 | Sim:0.5s LLM:2.2s  
| food=10x/2h@Random prob=1x/48h@Random O2=2h/3h@Random [Obs Mem:23H Ref:- Plan]  
Day 24 | Fit:0.0000 Yld:0.8980 Sav:0.0000 Hlth:0.9667 | Surv: 97.8% (44/45) | Cost:\$ 502.00 Sav  
e:\$ 0.00 | Sim:0.5s [OVER\$]  
  
Gen 4 FINAL (attempt 1):  
Fit:0.0000 | Yield:0.8980 | SavingRate:0.0000 | Healthiness:0.9667  
Surv:97.8% (44/45) | Cost:\$502.00 | Save:\$0.00

>>> END-OF-GEN REFLECTION:  
D22(warn:407/243): To increase your fitness score, consider reducing the `food\_interval` to a lower  
value (e.g., from 1-2 hours to 1 hour) and decreasing `oxygen\_duration` to a shorter time frame (e.g., fro  
m 1-4 hours to 1 hour). This could help boost your fish's health and fitness score.

D23(fail:

~~~~~  
Generation 5/5 (attempt 1)  
~~~~~  
Day 0 | Stocked 40 fish | food=10x/2h@Random prob=1x/48h@Random O2=4h/3h@Random | LLM:2.2s  
Day 1 | Fit:0.8512 Yld:0.8163 Sav:0.8665 Hlth:0.9689 | Surv:100.0% (40/40) | Cost:\$ 66.75 Sav  
e:\$ 433.25 | Sim:0.1s LLM:1.3s  
| food=30x/2h@Random prob=1x/48h@Random O2=4h/4h@Random [Obs Mem:1H Ref:- Plan]

\*\*\* RESTARTING Gen 5 (attempt 1 -> 2) \*\*\*  
Reason: Day 1 fitness 0.8512 < 0.9. Had 40 fish (max 49).  
Reflection: Increase the food interval to 6 and set the food quantity to 25. This might help reach a hig  
her fitness level by ensuring the fish are well-fed without overfeeding.

. . . . .  
Generation 5/5 (attempt 2)  
. . . . .  
Day 0 | Stocked 49 fish | food=15x/2h@Random prob=1x/48h@Random O2=3h/3h@Random | LLM:2.4s  
Day 1 | Fit:0.9520 Yld:1.0000 Sav:0.8605 Hlth:0.9836 | Surv:100.0% (49/49) | Cost:\$ 69.75 Sav  
e:\$ 430.25 | Sim:0.1s LLM:1.4s  
| food=10x/2h@Random prob=1x/48h@Random O2=3h/4h@Random [Obs Mem:1H Ref:- Plan]  
Day 2 | Fit:0.9167 Yld:1.0000 Sav:0.7585 Hlth:0.9703 | Surv:100.0% (49/49) | Cost:\$ 120.75 Sav  
e:\$ 379.25 | Sim:0.1s LLM:0.7s  
| food=10x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:2H Ref:- Plan]  
Day 3 | Fit:0.9094 Yld:1.0000 Sav:0.7400 Hlth:0.9603 | Surv:100.0% (49/49) | Cost:\$ 130.00 Sav  
e:\$ 370.00 | Sim:0.1s LLM:0.5s  
| food=10x/2h@Random prob=1x/48h@Random O2=3h/3h@Random [Obs Mem:3H Ref:- Plan]  
Day 4 | Fit:0.8675 Yld:1.0000 Sav:0.6080 Hlth:0.9739 | Surv:100.0% (49/49) | Cost:\$ 196.00 Sav  
e:\$ 304.00 | Sim:0.2s LLM:1.0s  
| food=10x/2h@Random prob=1x/48h@Random O2=2h/4h@Random [Obs Mem:4H Ref:- Plan]  
Warn: No JSON. Fallback.  
Day 5 | Fit:0.8454 Yld:1.0000 Sav:0.5345 Hlth:0.9920 | Surv:100.0% (49/49) | Cost:\$ 232.75 Sav  
e:\$ 267.25 | Sim:0.2s LLM:1.4s  
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:5H Ref:Y Plan]

>>> REFLECTION: D5(warn:52/46): To increase your fitness score, consider reducing the `food\_inter  
val` to a lower value, such as 2 hours. This might help in maintaining a higher fish health and fitness le  
vel more consistently.

Day 6 | Fit:0.8397 Yld:1.0000 Sav:0.5215 Hlth:0.9802 | Surv:100.0% (49/49) | Cost:\$ 239.25 Sav  
e:\$ 260.75 | Sim:0.2s LLM:0.8s  
| food=10x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:6H Ref:Y Plan]

>>> REFLECTION: D6(20%): To increase fitness, consider reducing the food interval to 2 hours and  
decreasing the food quantity to 10. This will help save more of your fish's fitness.

Day 7 | Fit:0.8347 Yld:1.0000 Sav:0.5030 Hlth:0.9894 | Surv:100.0% (49/49) | Cost:\$ 248.50 Sav  
e:\$ 251.50 | Sim:0.2s LLM:2.0s  
| food=10x/7h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:7H Ref:- Plan]  
Day 8 | Fit:0.8317 Yld:1.0000 Sav:0.4940 Hlth:0.9893 | Surv:100.0% (49/49) | Cost:\$ 253.00 Sav  
e:\$ 247.00 | Sim:0.2s LLM:1.7s  
| food=10x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:8H Ref:Y Plan]

>>> REFLECTION: D8(warn:94/62): To potentially improve your fitness score, you might consider red  
ucing the `food\_interval` to a lower value, such as 2 hours, to ensure your fish receive more frequent fee  
dings without exceeding the maximum of 12 hours. This could help maintain or slightly increas

Day 9 | Fit:0.8248 Yld:1.0000 Sav:0.4755 Hlth:0.9823 | Surv:100.0% (49/49) | Cost:\$ 262.25 Sav  
e:\$ 237.75 | Sim:0.2s LLM:2.0s  
| food=10x/9h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:9H Ref:Y Plan]

>>> REFLECTION: D9(warn:81/72): To increase your fitness score, consider reducing the food interv  
al to 1-2 hours or decreasing the food quantity to 5-10 units. This might help you save more resources and

potentially boost your fitness score.

Day 10 | Fit:0.8193 Yld:1.0000 Sav:0.4675 Hlth:0.9584 | Surv:100.0% (49/49) | Cost:\$ 266.25 Save:\$ 233.75 | Sim:0.2s LLM:0.8s  
| food=10x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:10H Ref:Y Plan]

>>> REFLECTION: D10(warn:83/80): To increase your fitness, consider reducing the `food\_interval` to 2 hours (`food\_interval=2`). This might help your fish receive more frequent feedings, potentially improving their health and increasing your fitness score.

Day 11 | Fit:0.8122 Yld:1.0000 Sav:0.4440 Hlth:0.9639 | Surv:100.0% (49/49) | Cost:\$ 278.00 Save:\$ 222.00 | Sim:0.2s LLM:0.8s  
| food=10x/3h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:11H Ref:Y Plan]

>>> REFLECTION: D11(warn:100/89): To increase your fitness score, consider reducing the `food\_interval` to a lower value (e.g., from 1-2 hours to 3-5 hours). This might help you save more while maintaining your fish's health.

Day 12 | Fit:0.8068 Yld:1.0000 Sav:0.4310 Hlth:0.9547 | Surv:100.0% (49/49) | Cost:\$ 284.50 Save:\$ 215.50 | Sim:0.3s LLM:1.4s  
| food=10x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:12H Ref:Y Plan]

>>> REFLECTION: D12(warn:99/97): To increase your fitness score, consider reducing the `food\_interval` to a lower value, such as 2 hours. This will allow your fish to be fed more frequently, potentially improving their health and increasing your fitness score.

Warn: No JSON. Fallback.  
Day 13 | Fit:0.8011 Yld:1.0000 Sav:0.4125 Hlth:0.9577 | Surv:100.0% (49/49) | Cost:\$ 293.75 Save:\$ 206.25 | Sim:0.3s LLM:1.8s  
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:13H Ref:- Plan]

Warn: No JSON. Fallback.  
Day 14 | Fit:0.7944 Yld:1.0000 Sav:0.4045 Hlth:0.9244 | Surv:100.0% (49/49) | Cost:\$ 297.75 Save:\$ 202.25 | Sim:0.3s LLM:1.9s  
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:14H Ref:Y Plan]

>>> REFLECTION: D14(warn:205/109): To increase your fitness score, consider reducing the `food\_interval` to a lower value, such as 2 hours. This might help in spending less and increasing your Fitness score. Changes will apply tomorrow.

Day 15 | Fit:0.7774 Yld:0.9796 Sav:0.3950 Hlth:0.9025 | Surv: 98.0% (48/49) | Cost:\$ 302.50 Save:\$ 197.50 | Sim:0.3s LLM:1.7s  
| food=10x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:15H Ref:- Plan]

Day 16 | Fit:0.7694 Yld:0.9796 Sav:0.3730 Hlth:0.8959 | Surv: 98.0% (48/49) | Cost:\$ 313.50 Save:\$ 186.50 | Sim:0.3s LLM:3.4s  
| food=5x/12h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:16H Ref:Y Plan]

>>> REFLECTION: D16(warn:223/120): To increase your fitness score, consider decreasing the `food\_interval` to a lower value (e.g., from 48h to 24h) and increasing the `food\_quantity` slightly (e.g., from 5 to 7). This might help in boosting your fish's health and fitness more effectively.

Day 17 | Fit:0.7660 Yld:0.9796 Sav:0.3655 Hlth:0.8882 | Surv: 98.0% (48/49) | Cost:\$ 317.25 Save:\$ 182.75 | Sim:0.3s LLM:0.5s  
| food=5x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:17H Ref:- Plan]

Day 18 | Fit:0.7518 Yld:0.9592 Sav:0.3545 Hlth:0.8935 | Surv: 95.9% (47/49) | Cost:\$ 322.75 Save:\$ 177.25 | Sim:0.3s LLM:1.6s  
| food=10x/2h@Random prob=2x/48h@Random O2=1h/20h@Random [Obs Mem:18H Ref:Y Plan]

>>> REFLECTION: D18(warn:253/131): To increase your fitness score, you might consider reducing the `food\_interval` to a lower value, such as 2 hours, and decreasing `probiotic\_quantity` to 2. This could help you save more resources while maintaining adequate nutrition for your fish.

Day 19 | Fit:0.7500 Yld:0.9592 Sav:0.3345 Hlth:0.9342 | Surv: 95.9% (47/49) | Cost:\$ 332.75 Save:\$ 167.25 | Sim:0.3s LLM:1.2s  
| food=10x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:19H Ref:Y Plan]

>>> REFLECTION: D19(warn:154/138): To increase your fitness score, you could try reducing the `food\_interval` to a lower value, such as 2 hours. This might help boost your fish's energy levels more frequently, potentially improving performance. Change it to `food\_interval=2`.

Day 20 | Fit:0.7464 Yld:0.9592 Sav:0.3175 Hlth:0.9504 | Surv: 95.9% (47/49) | Cost:\$ 341.25 Save:\$ 158.75 | Sim:0.4s LLM:0.8s  
| food=10x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:20H Ref:Y Plan]

>>> REFLECTION: D20(warn:181/146): To increase your fitness, consider reducing the `food\_interval` to a lower value, such as `food\_interval=2`. This will ensure your fish receive food more frequently, potentially improving their health and increasing your fitness score.

Day 21 | Fit:0.7375 Yld:0.9592 Sav:0.2940 Hlth:0.9409 | Surv: 95.9% (47/49) | Cost:\$ 353.00 Save:\$ 147.00 | Sim:0.4s LLM:2.2s  
| food=10x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:21H Ref:Y Plan]

>>> REFLECTION: D21(warn:191/154): To increase your fitness score, you might consider reducing th

e `food\_interval` to a lower value, such as 2 hours. This could help in spending less and potentially increasing your fitness score. For example:

```
...
food_interval=2
...
```

Warn: No JSON. Fallback.  
Day 22 | Fit:0.7197 Yld:0.9388 Sav:0.2770 Hlth:0.9334 | Surv: 93.9% (46/49) | Cost:\$ 361.50 Save:\$ 138.50 | Sim:0.4s LLM:1.4s  
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:22H Ref:Y Plan]

>>> REFLECTION: D22(warn:191/162): To increase your fitness score, consider reducing the `food\_interval` to a lower value, such as 2 hours. This could potentially boost your fish's health and fitness score.

Day 23 | Fit:0.7141 Yld:0.9388 Sav:0.2675 Hlth:0.9123 | Surv: 93.9% (46/49) | Cost:\$ 366.25 Save:\$ 133.75 | Sim:0.4s LLM:1.8s  
| food=10x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:23H Ref:Y Plan]

>>> REFLECTION: D23(warn:180/168): To increase your fitness score, consider reducing the `food\_interval` to a lower value, such as 2 hours. This might help in spending less while increasing your fitness slightly. For example:

```
...
food_interval=2
...
```

Day 24 | Fit:0.7009 Yld:0.9184 Sav:0.2505 Hlth:0.9428 | Surv: 91.8% (45/49) | Cost:\$ 374.75 Save:\$ 125.25 | Sim:0.4s LLM:1.0s  
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:24H Ref:Y Plan]

>>> REFLECTION: D24(warn:185/174): To increase your fitness, consider reducing the `food\_interval` to 8 hours (`food\_interval=8`). This might help in optimizing feeding times and potentially improving your fish's health and fitness.

Day 25 | Fit:0.6963 Yld:0.9184 Sav:0.2410 Hlth:0.9309 | Surv: 91.8% (45/49) | Cost:\$ 379.50 Save:\$ 120.50 | Sim:0.4s LLM:1.5s  
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:25H Ref:- Plan]

Day 26 | Fit:0.6800 Yld:0.8980 Sav:0.2280 Hlth:0.9244 | Surv: 89.8% (44/49) | Cost:\$ 386.00 Save:\$ 114.00 | Sim:0.4s LLM:1.0s  
| food=10x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:26H Ref:Y Plan]

>>> REFLECTION: D26(warn:354/183): To increase fitness, consider reducing the `food\_interval` to a lower value, such as `food\_interval=2`. This will ensure your fish receive food more frequently, potentially improving their health and fitness score.

Day 27 | Fit:0.6765 Yld:0.8980 Sav:0.2095 Hlth:0.9458 | Surv: 89.8% (44/49) | Cost:\$ 395.25 Save:\$ 104.75 | Sim:0.4s LLM:0.8s  
| food=10x/2h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:27H Ref:Y Plan]

>>> REFLECTION: D27(warn:188/188): To increase your fitness score, you could try decreasing the `food\_interval` to a lower value, such as 2 hours. This might help your fish receive more frequent feedings, potentially improving their health and increasing your fitness score.

Day 28 | Fit:0.6714 Yld:0.8980 Sav:0.1925 Hlth:0.9501 | Surv: 89.8% (44/49) | Cost:\$ 403.75 Save:\$ 96.25 | Sim:0.5s LLM:2.1s  
| food=10x/3h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:28H Ref:Y Plan]

>>> REFLECTION: D28(warn:220/194): To increase your fitness score, you might consider reducing the `food\_interval` to a lower value, such as 2 hours, to spend less and potentially boost your fitness further. For example:

```
...
food_interval=2
...
```

Warn: No JSON. Fallback.  
Day 29 | Fit:0.6633 Yld:0.8980 Sav:0.1780 Hlth:0.9226 | Surv: 89.8% (44/49) | Cost:\$ 411.00 Save:\$ 89.00 | Sim:0.5s LLM:1.4s  
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Obs Mem:29H Ref:Y Plan]

>>> REFLECTION: D29(warn:268/202): To increase your fitness score, consider reducing the `food\_interval` to a lower value, such as 2 hours. This will allow you to feed your fish more frequently, potentially improving their health and increasing your fitness score.

Day 30 | Fit:0.6562 Yld:0.8980 Sav:0.1700 Hlth:0.8852 | Surv: 89.8% (44/49) | Cost:\$ 415.00 Save:\$ 85.00 | Sim:0.5s [END]

Gen 5 FINAL (attempt 2):  
Fit:0.6562 | Yield:0.8980 | SavingRate:0.1700 | Healthiness:0.8852  
Surv: 89.8% (44/49) | Cost: \$415.00 | Save: \$85.00

>>> END-OF-GEN REFLECTION:

To increase your fitness score, you could try decreasing the `food\_interval` to a lower value, such as 2 hours. This might help your fish receive more frequent feedings, potentially improving their health and increasing your fitness score.

```
...
food_interval=2
...
```

Saved memory: /content/AquaLLM/logs/memory\_no\_tools.json  
Saved trace: /content/AquaLLM/logs/trace\_no\_tools.json (367,370 bytes)

```
=====
Config: no_obs (Mem+Tool+Plan) | Run 1/1
Days: 30 | Budget: $500.0 | Generations: 5
=====
```

```
~~~~~
Generation 1/5 (attempt 1)
~~~~~
Day 0 | Stocked 40 fish | food=5x/12h@Random prob=3x/24h@Random O2=1h/12h@Random | LLM:3.8s
Day 1 | Fit:0.8840 Yld:0.8163 Sav:0.9845 Hlth:0.9175 | Surv:100.0% (40/40) | Cost:$ 7.75 Sav
e:$ 492.25 | Sim:0.1s LLM:3.0s
 | food=10x/6h@Random prob=2x/24h@Random O2=4h/8h@Random [Mem:1H Tool Plan]
```

\*\*\* RESTARTING Gen 1 (attempt 1 -> 2) \*\*\*  
Reason: Day 1 fitness 0.8840 < 0.9. Had 40 fish (max 49).

```
.
Generation 1/5 (attempt 2)
.
Day 0 | Stocked 30 fish | food=10x/10h@Random prob=3x/24h@Random O2=2h/15h@Random | LLM:3.5s
Day 1 | Fit:0.7724 Yld:0.6122 Sav:0.9725 Hlth:0.9560 | Surv:100.0% (30/30) | Cost:$ 13.75 Sav
e:$ 486.25 | Sim:0.0s LLM:3.4s
 | food=24x/6h@Random prob=2x/24h@Random O2=1h/20h@Random [Mem:1H Tool Plan]
```

\*\*\* RESTARTING Gen 1 (attempt 2 -> 3) \*\*\*  
Reason: Day 1 fitness 0.7724 < 0.9. Had 30 fish (max 49).

```
.
Generation 1/5 (attempt 3)
.
Day 0 | Stocked 40 fish | food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random | LLM:3.3s
Day 1 | Fit:0.8877 Yld:0.8163 Sav:0.9855 Hlth:0.9461 | Surv:100.0% (40/40) | Cost:$ 7.25 Sav
e:$ 492.75 | Sim:0.1s LLM:1.8s
 | food=9x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Mem:1H Tool Plan]
Day 2 | Fit:0.8840 Yld:0.8163 Sav:0.9778 Hlth:0.9364 | Surv:100.0% (40/40) | Cost:$ 11.10 Sav
e:$ 488.90 | Sim:0.1s LLM:1.3s
 | food=9x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Mem:2H Tool Plan]
Day 3 | Fit:0.8796 Yld:0.8163 Sav:0.9686 Hlth:0.9248 | Surv:100.0% (40/40) | Cost:$ 15.70 Sav
e:$ 484.30 | Sim:0.1s LLM:3.5s
 | food=9x/10h@Random prob=1x/24h@Random O2=1h/20h@Random [Mem:3H Tool Plan]
Day 4 | Fit:0.8759 Yld:0.8163 Sav:0.9603 Hlth:0.9168 | Surv:100.0% (40/40) | Cost:$ 19.85 Sav
e:$ 480.15 | Sim:0.1s LLM:1.3s
 | food=9x/10h@Random prob=1x/24h@Random O2=1h/20h@Random [Mem:4H Tool Plan]
Day 5 | Fit:0.8738 Yld:0.8163 Sav:0.9520 Hlth:0.9225 | Surv:100.0% (40/40) | Cost:$ 24.00 Sav
e:$ 476.00 | Sim:0.1s LLM:1.2s
 | food=9x/10h@Random prob=1x/24h@Random O2=1h/20h@Random [Mem:5H Tool Plan]
Day 6 | Fit:0.8683 Yld:0.8163 Sav:0.9378 Hlth:0.9152 | Surv:100.0% (40/40) | Cost:$ 31.10 Sav
e:$ 468.90 | Sim:0.1s LLM:3.1s
 | food=9x/10h@Random prob=1x/24h@Random O2=1h/20h@Random [Mem:6H Tool Plan]
Day 7 | Fit:0.8563 Yld:0.7959 Sav:0.9295 Hlth:0.9314 | Surv: 97.5% (39/40) | Cost:$ 35.25 Sav
e:$ 464.75 | Sim:0.1s LLM:2.1s
 | food=9x/10h@Random prob=1x/24h@Random O2=1h/20h@Random [Mem:7H Tool Plan]
Day 8 | Fit:0.8519 Yld:0.7959 Sav:0.9203 Hlth:0.9201 | Surv: 97.5% (39/40) | Cost:$ 39.85 Sav
e:$ 460.15 | Sim:0.1s LLM:7.8s
 | food=9x/8h@Center prob=1x/24h@Random O2=1h/20h@Random [Mem:8H Tool Plan]
Day 9 | Fit:0.8381 Yld:0.7755 Sav:0.9111 Hlth:0.9245 | Surv: 95.0% (38/40) | Cost:$ 44.45 Sav
e:$ 455.55 | Sim:0.1s LLM:3.8s
 | food=9x/6h@Random prob=1x/24h@Random O2=1h/20h@Random [Mem:9H Tool Plan]
Day 10 | Fit:0.8244 Yld:0.7551 Sav:0.9010 Hlth:0.9317 | Surv: 92.5% (37/40) | Cost:$ 49.50 Sav
e:$ 450.50 | Sim:0.1s LLM:1.9s
 | food=9x/6h@Random prob=1x/24h@Random O2=1h/20h@Random [Mem:10H Tool Plan]
Day 11 | Fit:0.8196 Yld:0.7551 Sav:0.8859 Hlth:0.9329 | Surv: 92.5% (37/40) | Cost:$ 57.05 Sav
e:$ 442.95 | Sim:0.1s LLM:2.2s
 | food=9x/6h@Random prob=1x/24h@Random O2=1h/20h@Random [Mem:11H Tool Plan]
Day 12 | Fit:0.8132 Yld:0.7551 Sav:0.8758 Hlth:0.9074 | Surv: 92.5% (37/40) | Cost:$ 62.10 Sav
e:$ 437.90 | Sim:0.1s LLM:3.2s
 | food=9x/5h@Random prob=1x/24h@Random O2=1h/20h@Random [Mem:12H Tool Plan]
```

Day 13 | Fit:0.8081 Yld:0.7551 Sav:0.8648 Hlth:0.8954 | Surv: 92.5% (37/40) | Cost:\$ 67.60 Sav  
e:\$ 432.40 | Sim:0.1s LLM:8.0s  
| food=9x/5h@Random prob=2x/24h@Random O2=1h/20h@Random [Mem:13H Tool Plan]  
Day 14 | Fit:0.8026 Yld:0.7551 Sav:0.8523 Hlth:0.8832 | Surv: 92.5% (37/40) | Cost:\$ 73.85 Sav  
e:\$ 426.15 | Sim:0.1s LLM:2.3s  
| food=9x/5h@Random prob=2x/24h@Random O2=1h/16h@Random [Mem:14H Tool Plan]  
Day 15 | Fit:0.7974 Yld:0.7551 Sav:0.8357 Hlth:0.8857 | Surv: 92.5% (37/40) | Cost:\$ 82.15 Sav  
e:\$ 417.85 | Sim:0.1s LLM:2.4s  
| food=10x/5h@Random prob=2x/24h@Random O2=1h/16h@Random [Mem:15H Tool Plan]  
Day 16 | Fit:0.7951 Yld:0.7551 Sav:0.8227 Hlth:0.9025 | Surv: 92.5% (37/40) | Cost:\$ 88.65 Sav  
e:\$ 411.35 | Sim:0.1s LLM:1.8s  
| food=10x/5h@Random prob=2x/24h@Random O2=1h/16h@Random [Mem:16H Tool Plan]  
Warn: No JSON. Fallback.  
Day 17 | Fit:0.7783 Yld:0.7347 Sav:0.8047 Hlth:0.9054 | Surv: 90.0% (36/40) | Cost:\$ 97.65 Sav  
e:\$ 402.35 | Sim:0.2s LLM:2.5s  
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Mem:17H Tool Plan]  
Day 18 | Fit:0.7743 Yld:0.7347 Sav:0.7967 Hlth:0.8940 | Surv: 90.0% (36/40) | Cost:\$ 101.65 Sav  
e:\$ 398.35 | Sim:0.2s LLM:2.4s  
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Mem:18H Tool Plan]  
Day 19 | Fit:0.7689 Yld:0.7347 Sav:0.7872 Hlth:0.8750 | Surv: 90.0% (36/40) | Cost:\$ 106.40 Sav  
e:\$ 393.60 | Sim:0.2s LLM:3.4s  
| food=9x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Mem:19H Tool Plan]  
Day 20 | Fit:0.7543 Yld:0.7143 Sav:0.7795 Hlth:0.8688 | Surv: 87.5% (35/40) | Cost:\$ 110.25 Sav  
e:\$ 389.75 | Sim:0.2s LLM:2.6s  
| food=9x/9h@Random prob=1x/48h@Random O2=1h/20h@Random [Mem:20H Tool Plan]  
Day 21 | Fit:0.7423 Yld:0.6939 Sav:0.7662 Hlth:0.8989 | Surv: 85.0% (34/40) | Cost:\$ 116.90 Sav  
e:\$ 383.10 | Sim:0.2s LLM:5.3s  
| food=9x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Mem:21H Tool Plan]  
Day 22 | Fit:0.7218 Yld:0.6531 Sav:0.7585 Hlth:0.9356 | Surv: 80.0% (32/40) | Cost:\$ 120.75 Sav  
e:\$ 379.25 | Sim:0.2s LLM:2.0s  
| food=10x/6h@Random prob=1x/48h@Random O2=1h/20h@Random [Mem:22H Tool Plan]  
Day 23 | Fit:0.7181 Yld:0.6531 Sav:0.7480 Hlth:0.9339 | Surv: 80.0% (32/40) | Cost:\$ 126.00 Sav  
e:\$ 374.00 | Sim:0.2s LLM:2.1s  
| food=10x/6h@Random prob=1x/48h@Random O2=1h/20h@Random [Mem:23H Tool Plan]  
Day 24 | Fit:0.7156 Yld:0.6531 Sav:0.7390 Hlth:0.9379 | Surv: 80.0% (32/40) | Cost:\$ 130.50 Sav  
e:\$ 369.50 | Sim:0.2s LLM:2.5s  
| food=10x/6h@Random prob=1x/48h@Random O2=1h/20h@Random [Mem:24H Tool Plan]  
Day 25 | Fit:0.7117 Yld:0.6531 Sav:0.7285 Hlth:0.9345 | Surv: 80.0% (32/40) | Cost:\$ 135.75 Sav  
e:\$ 364.25 | Sim:0.2s LLM:2.1s  
| food=10x/6h@Random prob=1x/36h@Random O2=1h/15h@Random [Mem:25H Tool Plan]  
Day 26 | Fit:0.7055 Yld:0.6531 Sav:0.7130 Hlth:0.9251 | Surv: 80.0% (32/40) | Cost:\$ 143.50 Sav  
e:\$ 356.50 | Sim:0.2s LLM:2.1s  
| food=10x/6h@Random prob=1x/36h@Random O2=2h/8h@Random [Mem:26H Tool Plan]  
Day 27 | Fit:0.6944 Yld:0.6531 Sav:0.6790 Hlth:0.9264 | Surv: 80.0% (32/40) | Cost:\$ 160.50 Sav  
e:\$ 339.50 | Sim:0.2s LLM:4.4s  
| food=12x/8h@Random prob=1x/36h@Random O2=2h/12h@Random [Mem:27H Tool Plan]  
Day 28 | Fit:0.6839 Yld:0.6531 Sav:0.6539 Hlth:0.9074 | Surv: 80.0% (32/40) | Cost:\$ 173.05 Sav  
e:\$ 326.95 | Sim:0.2s LLM:4.0s  
| food=8x/10h@Random prob=1x/48h@Random O2=1h/20h@Random [Mem:28H Tool Plan]  
Day 29 | Fit:0.6801 Yld:0.6531 Sav:0.6458 Hlth:0.8981 | Surv: 80.0% (32/40) | Cost:\$ 177.10 Sav  
e:\$ 322.90 | Sim:0.2s LLM:2.3s  
| food=7x/10h@Random prob=1x/48h@Random O2=1h/20h@Random [Mem:29H Tool Plan]  
Day 30 | Fit:0.6769 Yld:0.6531 Sav:0.6394 Hlth:0.8897 | Surv: 80.0% (32/40) | Cost:\$ 180.30 Sav  
e:\$ 319.70 | Sim:0.2s [END]

Gen 1 FINAL (attempt 3):

Fit:0.6769 | Yield:0.6531 | SavingRate:0.6394 | Healthiness:0.8897  
Surv:80.0% (32/40) | Cost:\$180.30 | Save:\$319.70

~~~~~  
Generation 2/5 (attempt 1)  
~~~~~

Day 0 | Stocked 49 fish | food=10x/12h@Random prob=3x/24h@Random O2=1h/6h@Random | LLM:3.7s  
Day 1 | Fit:0.9829 Yld:1.0000 Sav:0.9735 Hlth:0.9303 | Surv:100.0% (49/49) | Cost:\$ 13.25 Sav  
e:\$ 486.75 | Sim:0.1s LLM:0.4s  
| food=10x/12h@Random prob=3x/24h@Random O2=1h/6h@Random [Mem:1H Tool Plan]  
Day 2 | Fit:0.9735 Yld:1.0000 Sav:0.9470 Hlth:0.9245 | Surv:100.0% (49/49) | Cost:\$ 26.50 Sav  
e:\$ 473.50 | Sim:0.1s LLM:1.9s  
| food=10x/12h@Random prob=3x/24h@Random O2=1h/6h@Random [Mem:2H Tool Plan]  
Day 3 | Fit:0.9624 Yld:1.0000 Sav:0.9205 Hlth:0.9049 | Surv:100.0% (49/49) | Cost:\$ 39.75 Sav  
e:\$ 460.25 | Sim:0.1s LLM:1.5s  
| food=10x/12h@Random prob=3x/24h@Random O2=1h/6h@Random [Mem:3H Tool Plan]  
Day 4 | Fit:0.9297 Yld:0.9592 Sav:0.8940 Hlth:0.8925 | Surv: 95.9% (47/49) | Cost:\$ 53.00 Sav  
e:\$ 447.00 | Sim:0.1s LLM:1.8s  
| food=10x/12h@Random prob=3x/24h@Random O2=1h/6h@Random [Mem:4H Tool Plan]  
Day 5 | Fit:0.9203 Yld:0.9592 Sav:0.8675 Hlth:0.8877 | Surv: 95.9% (47/49) | Cost:\$ 66.25 Sav  
e:\$ 433.75 | Sim:0.1s LLM:4.7s  
| food=10x/12h@Random prob=3x/24h@Random O2=1h/6h@Random [Mem:5H Tool Plan]  
Day 6 | Fit:0.9103 Yld:0.9592 Sav:0.8410 Hlth:0.8767 | Surv: 95.9% (47/49) | Cost:\$ 79.50 Sav  
e:\$ 420.50 | Sim:0.1s LLM:1.7s  
| food=10x/12h@Random prob=3x/24h@Random O2=1h/6h@Random [Mem:6H Tool Plan]  
Day 7 | Fit:0.9018 Yld:0.9592 Sav:0.8145 Hlth:0.8786 | Surv: 95.9% (47/49) | Cost:\$ 92.75 Sav

Day 7 | Fit:0.8914 Yld:0.9592 Sav:0.8145 Hlth:0.8786 | Surv: 95.9% (47/49) | Cost:\$ 92.75 Sav  
e:\$ 407.25 | Sim:0.1s LLM:1.7s  
| food=10x/12h@Random prob=3x/24h@Random O2=1h/6h@Random [Mem:7H Tool Plan]  
Day 8 | Fit:0.8914 Yld:0.9592 Sav:0.7880 Hlth:0.8654 | Surv: 95.9% (47/49) | Cost:\$ 106.00 Sav  
e:\$ 394.00 | Sim:0.1s LLM:2.2s  
| food=8x/12h@Random prob=3x/30h@Random O2=1h/10h@Random [Mem:8H Tool Plan]  
Day 9 | Fit:0.8813 Yld:0.9592 Sav:0.7719 Hlth:0.8252 | Surv: 95.9% (47/49) | Cost:\$ 114.05 Sav  
e:\$ 385.95 | Sim:0.1s LLM:3.3s  
| food=8x/12h@Random prob=3x/30h@Random O2=1h/20h@Random [Mem:9H Tool Plan]  
Day 10 | Fit:0.8755 Yld:0.9592 Sav:0.7653 Hlth:0.7951 | Surv: 95.9% (47/49) | Cost:\$ 117.35 Sav  
e:\$ 382.65 | Sim:0.1s LLM:2.5s  
| food=8x/12h@Random prob=3x/30h@Random O2=1h/20h@Random [Mem:10H Tool Plan]  
Day 11 | Fit:0.8556 Yld:0.9388 Sav:0.7492 Hlth:0.7666 | Surv: 93.9% (46/49) | Cost:\$ 125.40 Sav  
e:\$ 374.60 | Sim:0.2s LLM:4.1s  
| food=8x/6h@Random prob=3x/30h@Random O2=1h/20h@Random [Mem:11H Tool Plan]  
Day 12 | Fit:0.8524 Yld:0.9388 Sav:0.7365 Hlth:0.7756 | Surv: 93.9% (46/49) | Cost:\$ 131.75 Sav  
e:\$ 368.25 | Sim:0.2s LLM:3.3s  
| food=8x/8h@Random prob=3x/30h@Random O2=1h/20h@Random [Mem:12H Tool Plan]  
Day 13 | Fit:0.8280 Yld:0.8980 Sav:0.7246 Hlth:0.7913 | Surv: 89.8% (44/49) | Cost:\$ 137.70 Sav  
e:\$ 362.30 | Sim:0.2s LLM:4.1s  
| food=10x/8h@Random prob=3x/30h@Random O2=1h/20h@Random [Mem:13H Tool Plan]  
Day 14 | Fit:0.8273 Yld:0.8980 Sav:0.7121 Hlth:0.8198 | Surv: 89.8% (44/49) | Cost:\$ 143.95 Sav  
e:\$ 356.05 | Sim:0.2s LLM:3.7s  
| food=8x/9h@Random prob=3x/30h@Random O2=1h/20h@Random [Mem:14H Tool Plan]  
Warn: No JSON. Fallback.  
Day 15 | Fit:0.8220 Yld:0.8980 Sav:0.7055 Hlth:0.7941 | Surv: 89.8% (44/49) | Cost:\$ 147.25 Sav  
e:\$ 352.75 | Sim:0.2s LLM:182.4s  
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Mem:15H Tool Plan]  
Day 16 | Fit:0.7881 Yld:0.8367 Sav:0.6925 Hlth:0.8282 | Surv: 83.7% (41/49) | Cost:\$ 153.75 Sav  
e:\$ 346.25 | Sim:0.2s LLM:4.4s  
| food=15x/8h@Random prob=1x/24h@Random O2=1h/20h@Random [Mem:16H Tool Plan]  
Day 17 | Fit:0.7789 Yld:0.8163 Sav:0.6815 Hlth:0.8750 | Surv: 81.6% (40/49) | Cost:\$ 159.25 Sav  
e:\$ 340.75 | Sim:0.2s LLM:3.4s  
| food=15x/9h@Random prob=1x/24h@Random O2=1h/20h@Random [Mem:17H Tool Plan]  
Day 18 | Fit:0.7736 Yld:0.8163 Sav:0.6720 Hlth:0.8568 | Surv: 81.6% (40/49) | Cost:\$ 164.00 Sav  
e:\$ 336.00 | Sim:0.2s LLM:3.9s  
| food=15x/9h@Random prob=1x/24h@Random O2=1h/20h@Random [Mem:18H Tool Plan]  
Day 19 | Fit:0.7497 Yld:0.7755 Sav:0.6610 Hlth:0.8753 | Surv: 77.6% (38/49) | Cost:\$ 169.50 Sav  
e:\$ 330.50 | Sim:0.2s LLM:2.1s  
| food=15x/9h@Random prob=1x/24h@Random O2=1h/20h@Random [Mem:19H Tool Plan]  
Day 20 | Fit:0.7511 Yld:0.7755 Sav:0.6500 Hlth:0.9173 | Surv: 77.6% (38/49) | Cost:\$ 175.00 Sav  
e:\$ 325.00 | Sim:0.2s LLM:4.1s  
| food=15x/8h@Random prob=1x/24h@Random O2=1h/20h@Random [Mem:20H Tool Plan]  
Day 21 | Fit:0.7470 Yld:0.7755 Sav:0.6340 Hlth:0.9269 | Surv: 77.6% (38/49) | Cost:\$ 183.00 Sav  
e:\$ 317.00 | Sim:0.2s LLM:1.3s  
| food=15x/8h@Random prob=1x/24h@Random O2=1h/20h@Random [Mem:21H Tool Plan]  
Day 22 | Fit:0.7416 Yld:0.7755 Sav:0.6230 Hlth:0.9120 | Surv: 77.6% (38/49) | Cost:\$ 188.50 Sav  
e:\$ 311.50 | Sim:0.2s LLM:4.1s  
| food=14x/8h@Random prob=1x/24h@Random O2=1h/20h@Random [Mem:22H Tool Plan]  
Day 23 | Fit:0.7374 Yld:0.7755 Sav:0.6123 Hlth:0.9071 | Surv: 77.6% (38/49) | Cost:\$ 193.85 Sav  
e:\$ 306.15 | Sim:0.2s LLM:3.1s  
| food=13x/9h@Random prob=1x/24h@Random O2=1h/20h@Random [Mem:23H Tool Plan]  
Day 24 | Fit:0.7341 Yld:0.7755 Sav:0.6032 Hlth:0.9045 | Surv: 77.6% (38/49) | Cost:\$ 198.40 Sav  
e:\$ 301.60 | Sim:0.2s LLM:4.0s  
| food=14x/9h@Random prob=1x/24h@Random O2=1h/20h@Random [Mem:24H Tool Plan]  
Day 25 | Fit:0.7300 Yld:0.7755 Sav:0.5925 Hlth:0.8995 | Surv: 77.6% (38/49) | Cost:\$ 203.75 Sav  
e:\$ 296.25 | Sim:0.2s LLM:1.8s  
| food=14x/9h@Random prob=1x/24h@Random O2=1h/20h@Random [Mem:25H Tool Plan]  
Day 26 | Fit:0.7215 Yld:0.7755 Sav:0.5768 Hlth:0.8720 | Surv: 77.6% (38/49) | Cost:\$ 211.60 Sav  
e:\$ 288.40 | Sim:0.2s LLM:1.7s  
| food=14x/9h@Random prob=1x/24h@Random O2=1h/20h@Random [Mem:26H Tool Plan]  
Day 27 | Fit:0.7063 Yld:0.7551 Sav:0.5675 Hlth:0.8640 | Surv: 75.5% (37/49) | Cost:\$ 216.25 Sav  
e:\$ 283.75 | Sim:0.2s LLM:2.2s  
| food=14x/9h@Random prob=1x/24h@Random O2=1h/20h@Random [Mem:27H Tool Plan]  
Day 28 | Fit:0.7054 Yld:0.7551 Sav:0.5568 Hlth:0.8864 | Surv: 75.5% (37/49) | Cost:\$ 221.60 Sav  
e:\$ 278.40 | Sim:0.2s LLM:1.5s  
| food=14x/9h@Random prob=1x/24h@Random O2=1h/20h@Random [Mem:28H Tool Plan]  
Day 29 | Fit:0.6941 Yld:0.7347 Sav:0.5461 Hlth:0.9150 | Surv: 73.5% (36/49) | Cost:\$ 226.95 Sav  
e:\$ 273.05 | Sim:0.2s LLM:4.1s  
| food=14x/10h@Random prob=1x/48h@Random O2=1h/20h@Random [Mem:29H Tool Plan]  
Day 30 | Fit:0.6898 Yld:0.7347 Sav:0.5383 Hlth:0.9010 | Surv: 73.5% (36/49) | Cost:\$ 230.85 Sav  
e:\$ 269.15 | Sim:0.2s [END]

Gen 2 FINAL (attempt 1):

Fit:0.6898 | Yield:0.7347 | SavingRate:0.5383 | Healthiness:0.9010  
Surv:73.5% (36/49) | Cost:\$230.85 | Save:\$269.15

~~~~~  
Generation 3/5 (attempt 1)  
~~~~~

Day 0 | Stocked 30 fish | food=8x/8h@Random prob=2x/24h@Random O2=2h/10h@Random | LLM:3.1s  
Day 1 | Fit:0.7696 Yld:0.6122 Sav:0.9646 Hlth:0.9543 | Surv:100.0% (30/30) | Cost:\$ 17.70 Sav

```
e:$ 482.30 | Sim:0.0s LLM:2.8s
 | food=10x/8h@Random prob=2x/24h@Random O2=2h/10h@Random [Mem:1H Tool Plan]

*** RESTARTING Gen 3 (attempt 1 -> 2) ***
Reason: Day 1 fitness 0.7696 < 0.9. Had 30 fish (max 49).

.
Generation 3/5 (attempt 2)
.
Day 0 | Stocked 30 fish | food=5x/12h@Random prob=1x/24h@Random O2=1h/8h@Random | LLM:3.2s
Day 1 | Fit:0.7707 Yld:0.6122 Sav:0.9825 Hlth:0.9148 | Surv:100.0% (30/30) | Cost:$ 8.75 Sav
e:$ 491.25 | Sim:0.0s LLM:1.4s
 | food=5x/12h@Random prob=1x/24h@Random O2=1h/8h@Random [Mem:1H Tool Plan]

*** RESTARTING Gen 3 (attempt 2 -> 3) ***
Reason: Day 1 fitness 0.7707 < 0.9. Had 30 fish (max 49).

.
Generation 3/5 (attempt 3)
.
Warn: No JSON. Fallback.
Day 0 | Stocked 40 fish | food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random | LLM:3.8s
Day 1 | Fit:0.8883 Yld:0.8163 Sav:0.9855 Hlth:0.9507 | Surv:100.0% (40/40) | Cost:$ 7.25 Sav
e:$ 492.75 | Sim:0.1s LLM:2.4s
 | food=20x/6h@Random prob=1x/48h@Random O2=1h/15h@Random [Mem:1H Tool Plan]
Day 2 | Fit:0.8830 Yld:0.8163 Sav:0.9675 Hlth:0.9561 | Surv:100.0% (40/40) | Cost:$ 16.25 Sav
e:$ 483.75 | Sim:0.1s LLM:4.6s
 | food=20x/6h@Random prob=1x/48h@Random O2=1h/15h@Random [Mem:2H Tool Plan]
Day 3 | Fit:0.8805 Yld:0.8163 Sav:0.9530 Hlth:0.9754 | Surv:100.0% (40/40) | Cost:$ 23.50 Sav
e:$ 476.50 | Sim:0.1s LLM:1.3s
 | food=20x/6h@Random prob=1x/48h@Random O2=1h/15h@Random [Mem:3H Tool Plan]
Day 4 | Fit:0.8731 Yld:0.8163 Sav:0.9350 Hlth:0.9631 | Surv:100.0% (40/40) | Cost:$ 32.50 Sav
e:$ 467.50 | Sim:0.1s LLM:1.6s
 | food=20x/6h@Random prob=1x/48h@Random O2=1h/15h@Random [Mem:4H Tool Plan]
Day 5 | Fit:0.8688 Yld:0.8163 Sav:0.9205 Hlth:0.9675 | Surv:100.0% (40/40) | Cost:$ 39.75 Sav
e:$ 460.25 | Sim:0.1s LLM:4.2s
 | food=20x/6h@Random prob=1x/48h@Random O2=1h/15h@Random [Mem:5H Tool Plan]
Day 6 | Fit:0.8630 Yld:0.8163 Sav:0.9025 Hlth:0.9680 | Surv:100.0% (40/40) | Cost:$ 48.75 Sav
e:$ 451.25 | Sim:0.1s LLM:5.3s
 | food=20x/9h@Random prob=1x/48h@Random O2=1h/15h@Random [Mem:6H Tool Plan]
Day 7 | Fit:0.8565 Yld:0.8163 Sav:0.8850 Hlth:0.9624 | Surv:100.0% (40/40) | Cost:$ 57.50 Sav
e:$ 442.50 | Sim:0.1s LLM:3.1s
 | food=20x/9h@Random prob=1x/24h@Random O2=1h/15h@Random [Mem:7H Tool Plan]
Day 8 | Fit:0.8524 Yld:0.8163 Sav:0.8725 Hlth:0.9621 | Surv:100.0% (40/40) | Cost:$ 63.75 Sav
e:$ 436.25 | Sim:0.1s LLM:2.1s
 | food=20x/9h@Random prob=1x/24h@Random O2=1h/15h@Random [Mem:8H Tool Plan]
Day 9 | Fit:0.8467 Yld:0.8163 Sav:0.8570 Hlth:0.9576 | Surv:100.0% (40/40) | Cost:$ 71.50 Sav
e:$ 428.50 | Sim:0.1s LLM:3.2s
 | food=20x/9h@Random prob=1x/24h@Random O2=2h/8h@Center [Mem:9H Tool Plan]
Day 10 | Fit:0.8360 Yld:0.8163 Sav:0.8195 Hlth:0.9713 | Surv:100.0% (40/40) | Cost:$ 90.25 Sav
e:$ 409.75 | Sim:0.1s LLM:4.2s
 | food=10x/12h@Random prob=1x/72h@Random O2=2h/8h@Random [Mem:10H Tool Plan]
Day 11 | Fit:0.8205 Yld:0.8163 Sav:0.7875 Hlth:0.9304 | Surv:100.0% (40/40) | Cost:$ 106.25 Sav
e:$ 393.75 | Sim:0.1s LLM:3.7s
 | food=10x/12h@Random prob=1x/72h@Random O2=2h/8h@Random [Mem:11H Tool Plan]
Day 12 | Fit:0.8098 Yld:0.8163 Sav:0.7555 Hlth:0.9289 | Surv:100.0% (40/40) | Cost:$ 122.25 Sav
e:$ 377.75 | Sim:0.2s LLM:2.9s
 | food=6x/8h@Random prob=1x/96h@Random O2=2h/8h@Random [Mem:12H Tool Plan]
Day 13 | Fit:0.7964 Yld:0.8163 Sav:0.7222 Hlth:0.9087 | Surv:100.0% (40/40) | Cost:$ 138.90 Sav
e:$ 361.10 | Sim:0.2s LLM:2.5s
 | food=6x/9h@Random prob=1x/168h@Random O2=2h/8h@Random [Mem:13H Tool Plan]
Day 14 | Fit:0.7849 Yld:0.8163 Sav:0.6904 Hlth:0.9009 | Surv:100.0% (40/40) | Cost:$ 154.80 Sav
e:$ 345.20 | Sim:0.3s LLM:3.2s
 | food=6x/10h@Random prob=1x/168h@Random O2=2h/8h@Random [Mem:14H Tool Plan]
Day 15 | Fit:0.7713 Yld:0.8163 Sav:0.6577 Hlth:0.8773 | Surv:100.0% (40/40) | Cost:$ 171.15 Sav
e:$ 328.85 | Sim:0.3s LLM:3.8s
 | food=5x/12h@Random prob=1x/168h@Random O2=2h/20h@Random [Mem:15H Tool Plan]
Day 16 | Fit:0.7619 Yld:0.8163 Sav:0.6367 Hlth:0.8570 | Surv:100.0% (40/40) | Cost:$ 181.65 Sav
e:$ 318.35 | Sim:0.3s LLM:5.6s
 | food=5x/12h@Random prob=1x/168h@Random O2=2h/20h@Random [Mem:16H Tool Plan]
Day 17 | Fit:0.7523 Yld:0.8163 Sav:0.6257 Hlth:0.8070 | Surv:100.0% (40/40) | Cost:$ 187.15 Sav
e:$ 312.85 | Sim:0.3s LLM:4.2s
 | food=5x/12h@Random prob=1x/168h@Random O2=2h/20h@Random [Mem:17H Tool Plan]
Day 18 | Fit:0.7437 Yld:0.8163 Sav:0.6147 Hlth:0.7653 | Surv:100.0% (40/40) | Cost:$ 192.65 Sav
e:$ 307.35 | Sim:0.3s LLM:2.6s
 | food=5x/8h@Random prob=1x/168h@Random O2=2h/20h@Random [Mem:18H Tool Plan]
Day 19 | Fit:0.7430 Yld:0.8163 Sav:0.6032 Hlth:0.7918 | Surv:100.0% (40/40) | Cost:$ 198.40 Sav
e:$ 301.60 | Sim:0.2s LLM:4.9s
 | food=6x/8h@Random prob=1x/24h@Random O2=2h/15h@Random [Mem:19H Tool Plan]
Day 20 | Fit:0.7408 Yld:0.8163 Sav:0.5899 Hlth:0.8098 | Surv:100.0% (40/40) | Cost:$ 205.05 Sav
e:$ 294.95 | Sim:0.3s LLM:3.0s
 | food=6x/8h@Random prob=1x/30h@Random O2=2h/15h@Random [Mem:20H Tool Plan]
```

Day 21 | Fit:0.7335 Yld:0.8163 Sav:0.5666 Hlth:0.8131 | Surv:100.0% (40/40) | Cost:\$ 216.70 Sav  
e:\$ 283.30 | Sim:0.3s LLM:5.4s  
| food=6x/10h@Random prob=1x/48h@Random O2=2h/15h@Random [Mem:21H Tool Plan]  
Day 22 | Fit:0.7260 Yld:0.8163 Sav:0.5454 Hlth:0.8083 | Surv:100.0% (40/40) | Cost:\$ 227.30 Sav  
e:\$ 272.70 | Sim:0.3s LLM:1.8s  
| food=6x/10h@Random prob=1x/48h@Random O2=2h/15h@Random [Mem:22H Tool Plan]  
Day 23 | Fit:0.7213 Yld:0.8163 Sav:0.5321 Hlth:0.8065 | Surv:100.0% (40/40) | Cost:\$ 233.95 Sav  
e:\$ 266.05 | Sim:0.2s LLM:3.6s  
| food=6x/8h@Random prob=1x/48h@Random O2=2h/15h@Random [Mem:23H Tool Plan]  
Day 24 | Fit:0.7155 Yld:0.8163 Sav:0.5103 Hlth:0.8176 | Surv:100.0% (40/40) | Cost:\$ 244.85 Sav  
e:\$ 255.15 | Sim:0.2s LLM:3.6s  
| food=7x/8h@Random prob=1x/48h@Random O2=2h/15h@Random [Mem:24H Tool Plan]  
Day 25 | Fit:0.7067 Yld:0.8163 Sav:0.4967 Hlth:0.7818 | Surv:100.0% (40/40) | Cost:\$ 251.65 Sav  
e:\$ 248.35 | Sim:0.2s LLM:5.3s  
| food=7x/8h@Random prob=1x/48h@Random O2=2h/15h@Random [Mem:25H Tool Plan]  
Day 26 | Fit:0.6883 Yld:0.7959 Sav:0.4746 Hlth:0.7823 | Surv: 97.5% (39/40) | Cost:\$ 262.70 Sav  
e:\$ 237.30 | Sim:0.3s LLM:1.7s  
| food=7x/8h@Random prob=1x/48h@Random O2=2h/15h@Random [Mem:26H Tool Plan]  
Day 27 | Fit:0.6766 Yld:0.7959 Sav:0.4510 Hlth:0.7502 | Surv: 97.5% (39/40) | Cost:\$ 274.50 Sav  
e:\$ 225.50 | Sim:0.2s LLM:2.8s  
| food=6x/10h@Random prob=1x/48h@Random O2=2h/15h@Random [Mem:27H Tool Plan]  
Day 28 | Fit:0.6595 Yld:0.7755 Sav:0.4392 Hlth:0.7339 | Surv: 95.0% (38/40) | Cost:\$ 280.40 Sav  
e:\$ 219.60 | Sim:0.2s LLM:3.8s  
| food=5x/10h@Random prob=1x/48h@Random O2=2h/15h@Random [Mem:28H Tool Plan]  
Day 29 | Fit:0.6479 Yld:0.7755 Sav:0.4167 Hlth:0.6992 | Surv: 95.0% (38/40) | Cost:\$ 291.65 Sav  
e:\$ 208.35 | Sim:0.3s LLM:3.9s  
| food=5x/8h@Random prob=1x/48h@Random O2=2h/15h@Random [Mem:29H Tool Plan]  
Day 30 | Fit:0.6114 Yld:0.7143 Sav:0.4052 Hlth:0.7065 | Surv: 87.5% (35/40) | Cost:\$ 297.40 Sav  
e:\$ 202.60 | Sim:0.2s [END]

Gen 3 FINAL (attempt 3):  
Fit:0.6114 | Yield:0.7143 | SavingRate:0.4052 | Healthiness:0.7065  
Surv:87.5% (35/40) | Cost:\$297.40 | Save:\$202.60

~~~~~  
Generation 4/5 (attempt 1)  
~~~~~

Warn: No JSON. Fallback.  
Day 0 | Stocked 30 fish | food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random | LLM:6.6s  
Day 1 | Fit:0.7755 Yld:0.6122 Sav:0.9855 Hlth:0.9462 | Surv:100.0% (30/30) | Cost:\$ 7.25 Sav  
e:\$ 492.75 | Sim:0.0s LLM:2.9s  
| food=20x/6h@Random prob=1x/48h@Random O2=1h/15h@Random [Mem:1H Tool Plan]

\*\*\* RESTARTING Gen 4 (attempt 1 -> 2) \*\*\*  
Reason: Day 1 fitness 0.7755 < 0.9. Had 30 fish (max 49).

. . . . .  
Generation 4/5 (attempt 2)

. . . . .  
Day 0 | Stocked 40 fish | food=5x/12h@Random prob=3x/24h@Random O2=2h/6h@Random | LLM:2.7s  
Day 1 | Fit:0.8736 Yld:0.8163 Sav:0.9545 Hlth:0.9136 | Surv:100.0% (40/40) | Cost:\$ 22.75 Sav  
e:\$ 477.25 | Sim:0.1s LLM:2.3s  
| food=5x/6h@Random prob=3x/24h@Random O2=1h/6h@Random [Mem:1H Tool Plan]

\*\*\* RESTARTING Gen 4 (attempt 2 -> 3) \*\*\*  
Reason: Day 1 fitness 0.8736 < 0.9. Had 40 fish (max 49).

. . . . .  
Generation 4/5 (attempt 3)

. . . . .  
Day 0 | Stocked 49 fish | food=8x/6h@Random prob=3x/24h@Random O2=1h/10h@Random | LLM:4.2s  
Day 1 | Fit:0.9852 Yld:1.0000 Sav:0.9773 Hlth:0.9387 | Surv:100.0% (49/49) | Cost:\$ 11.35 Sav  
e:\$ 488.65 | Sim:0.1s LLM:1.3s  
| food=8x/6h@Random prob=3x/24h@Random O2=1h/10h@Random [Mem:1H Tool Plan]  
Day 2 | Fit:0.9784 Yld:1.0000 Sav:0.9596 Hlth:0.9312 | Surv:100.0% (49/49) | Cost:\$ 20.20 Sav  
e:\$ 479.80 | Sim:0.1s LLM:1.7s  
| food=8x/6h@Random prob=3x/24h@Random O2=1h/10h@Random [Mem:2H Tool Plan]  
Day 3 | Fit:0.9729 Yld:1.0000 Sav:0.9369 Hlth:0.9481 | Surv:100.0% (49/49) | Cost:\$ 31.55 Sav  
e:\$ 468.45 | Sim:0.1s LLM:1.9s  
| food=8x/6h@Random prob=3x/24h@Random O2=1h/10h@Random [Mem:3H Tool Plan]  
Day 4 | Fit:0.9653 Yld:1.0000 Sav:0.9192 Hlth:0.9328 | Surv:100.0% (49/49) | Cost:\$ 40.40 Sav  
e:\$ 459.60 | Sim:0.1s LLM:3.9s  
| food=8x/6h@Random prob=2x/24h@Random O2=1h/10h@Random [Mem:4H Tool Plan]  
Day 5 | Fit:0.9497 Yld:0.9796 Sav:0.9030 Hlth:0.9407 | Surv: 98.0% (48/49) | Cost:\$ 48.50 Sav  
e:\$ 451.50 | Sim:0.1s LLM:2.1s  
| food=8x/6h@Random prob=2x/24h@Random O2=1h/10h@Random [Mem:5H Tool Plan]  
Day 6 | Fit:0.9405 Yld:0.9796 Sav:0.8818 Hlth:0.9230 | Surv: 98.0% (48/49) | Cost:\$ 59.10 Sav  
e:\$ 440.90 | Sim:0.1s LLM:3.3s  
| food=8x/6h@Random prob=2x/24h@Random O2=1h/8h@Random [Mem:6H Tool Plan]  
Day 7 | Fit:0.9336 Yld:0.9796 Sav:0.8606 Hlth:0.9235 | Surv: 98.0% (48/49) | Cost:\$ 69.70 Sav  
e:\$ 430.30 | Sim:0.1s LLM:1.2s

Day 8 | Fit:0.9260 Yld:0.9796 Sav:0.8394 Hlth:0.9189 | Surv: 98.0% (48/49) | Cost:\$ 80.30 Save:\$ 419.70 | Sim:0.1s LLM:3.9s  
| food=8x/6h@Random prob=2x/36h@Random O2=1h/8h@Random [Mem:7H Tool Plan]  
Day 9 | Fit:0.9195 Yld:0.9796 Sav:0.8212 Hlth:0.9140 | Surv: 98.0% (48/49) | Cost:\$ 89.40 Save:\$ 410.60 | Sim:0.1s LLM:1.6s  
| food=8x/6h@Random prob=2x/36h@Random O2=1h/8h@Random [Mem:9H Tool Plan]  
Day 10 | Fit:0.9018 Yld:0.9592 Sav:0.8000 Hlth:0.9184 | Surv: 95.9% (47/49) | Cost:\$ 100.00 Save:\$ 400.00 | Sim:0.1s LLM:3.9s  
| food=8x/4h@Random prob=2x/36h@Random O2=1h/8h@Random [Mem:10H Tool Plan]  
Day 11 | Fit:0.8966 Yld:0.9592 Sav:0.7772 Hlth:0.9384 | Surv: 95.9% (47/49) | Cost:\$ 111.40 Save:\$ 388.60 | Sim:0.2s LLM:3.3s  
| food=12x/3h@Random prob=2x/24h@Random O2=1h/8h@Random [Mem:11H Tool Plan]  
Day 12 | Fit:0.8903 Yld:0.9592 Sav:0.7496 Hlth:0.9613 | Surv: 95.9% (47/49) | Cost:\$ 125.20 Save:\$ 374.80 | Sim:0.2s LLM:3.1s  
| food=12x/3h@Random prob=2x/24h@Random O2=1h/8h@Random [Mem:12H Tool Plan]  
Day 13 | Fit:0.8819 Yld:0.9592 Sav:0.7220 Hlth:0.9672 | Surv: 95.9% (47/49) | Cost:\$ 139.00 Save:\$ 361.00 | Sim:0.2s LLM:3.7s  
| food=24x/6h@Random prob=4x/24h@Random O2=1h/8h@Random [Mem:13H Tool Plan]  
Day 14 | Fit:0.8606 Yld:0.9388 Sav:0.6914 Hlth:0.9675 | Surv: 93.9% (46/49) | Cost:\$ 154.30 Save:\$ 345.70 | Sim:0.2s LLM:4.9s  
| food=5x/8h@Random prob=4x/24h@Random O2=1h/8h@Random [Mem:14H Tool Plan]  
Day 15 | Fit:0.8495 Yld:0.9388 Sav:0.6689 Hlth:0.9368 | Surv: 93.9% (46/49) | Cost:\$ 165.55 Save:\$ 334.45 | Sim:0.2s LLM:3.7s  
| food=5x/7h@Random prob=4x/24h@Random O2=1h/8h@Random [Mem:15H Tool Plan]  
Day 16 | Fit:0.8262 Yld:0.9184 Sav:0.6464 Hlth:0.8984 | Surv: 91.8% (45/49) | Cost:\$ 176.80 Save:\$ 323.20 | Sim:0.2s LLM:2.5s  
| food=5x/8h@Random prob=4x/24h@Random O2=1h/8h@Random [Mem:16H Tool Plan]  
Day 17 | Fit:0.8165 Yld:0.9184 Sav:0.6239 Hlth:0.8791 | Surv: 91.8% (45/49) | Cost:\$ 188.05 Save:\$ 311.95 | Sim:0.2s LLM:4.4s  
| food=5x/8h@Random prob=4x/24h@Random O2=1h/8h@Random [Mem:17H Tool Plan]  
Day 18 | Fit:0.8084 Yld:0.9184 Sav:0.6014 Hlth:0.8740 | Surv: 91.8% (45/49) | Cost:\$ 199.30 Save:\$ 300.70 | Sim:0.2s LLM:1.6s  
| food=5x/8h@Random prob=4x/24h@Random O2=1h/8h@Random [Mem:18H Tool Plan]  
Warn: No JSON. Fallback.  
Day 19 | Fit:0.8025 Yld:0.9184 Sav:0.5789 Hlth:0.8862 | Surv: 91.8% (45/49) | Cost:\$ 210.55 Save:\$ 289.45 | Sim:0.2s LLM:4.0s  
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Mem:19H Tool Plan]  
Day 20 | Fit:0.8034 Yld:0.9184 Sav:0.5709 Hlth:0.9162 | Surv: 91.8% (45/49) | Cost:\$ 214.55 Save:\$ 285.45 | Sim:0.2s LLM:1.7s  
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Mem:20H Tool Plan]  
Day 21 | Fit:0.7972 Yld:0.9184 Sav:0.5564 Hlth:0.9039 | Surv: 91.8% (45/49) | Cost:\$ 221.80 Save:\$ 278.20 | Sim:0.2s LLM:4.4s  
| food=7x/8h@Random prob=1x/64h@Random O2=1h/20h@Random [Mem:21H Tool Plan]  
Day 22 | Fit:0.7966 Yld:0.9184 Sav:0.5478 Hlth:0.9224 | Surv: 91.8% (45/49) | Cost:\$ 226.10 Save:\$ 273.90 | Sim:0.3s LLM:4.4s  
| food=7x/9h@Random prob=1x/64h@Random O2=1h/20h@Random [Mem:22H Tool Plan]  
Day 23 | Fit:0.7931 Yld:0.9184 Sav:0.5407 Hlth:0.9131 | Surv: 91.8% (45/49) | Cost:\$ 229.65 Save:\$ 270.35 | Sim:0.3s LLM:2.1s  
| food=7x/9h@Random prob=1x/144h@Random O2=1h/20h@Random [Mem:23H Tool Plan]  
Warn: No JSON. Fallback.  
Day 24 | Fit:0.7649 Yld:0.8776 Sav:0.5343 Hlth:0.8826 | Surv: 87.8% (43/49) | Cost:\$ 232.85 Save:\$ 267.15 | Sim:0.3s LLM:175.8s  
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Mem:24H Tool Plan]  
Day 25 | Fit:0.7508 Yld:0.8571 Sav:0.5248 Hlth:0.8853 | Surv: 85.7% (42/49) | Cost:\$ 237.60 Save:\$ 262.40 | Sim:0.2s LLM:4.7s  
| food=10x/6h@Random prob=1x/48h@Random O2=1h/20h@Random [Mem:25H Tool Plan]  
Day 26 | Fit:0.7471 Yld:0.8571 Sav:0.5108 Hlth:0.8922 | Surv: 85.7% (42/49) | Cost:\$ 244.60 Save:\$ 255.40 | Sim:0.3s LLM:3.4s  
| food=10x/6h@Random prob=1x/48h@Random O2=1h/15h@Random [Mem:26H Tool Plan]  
Day 27 | Fit:0.7444 Yld:0.8571 Sav:0.4953 Hlth:0.9129 | Surv: 85.7% (42/49) | Cost:\$ 252.35 Save:\$ 247.65 | Sim:0.2s LLM:1.7s  
| food=10x/6h@Random prob=1x/48h@Random O2=1h/15h@Random [Mem:27H Tool Plan]  
Day 28 | Fit:0.7419 Yld:0.8571 Sav:0.4863 Hlth:0.9164 | Surv: 85.7% (42/49) | Cost:\$ 256.85 Save:\$ 243.15 | Sim:0.2s LLM:3.3s  
| food=10x/8h@Random prob=1x/48h@Random O2=1h/15h@Random [Mem:28H Tool Plan]  
Day 29 | Fit:0.7375 Yld:0.8571 Sav:0.4718 Hlth:0.9198 | Surv: 85.7% (42/49) | Cost:\$ 264.10 Save:\$ 235.90 | Sim:0.2s LLM:4.1s  
| food=10x/8h@Random prob=1x/24h@Random O2=1h/15h@Random [Mem:29H Tool Plan]  
Day 30 | Fit:0.7335 Yld:0.8571 Sav:0.4623 Hlth:0.9122 | Surv: 85.7% (42/49) | Cost:\$ 268.85 Save:\$ 231.15 | Sim:0.2s [END]

Gen 4 FINAL (attempt 3):

Fit:0.7335 | Yield:0.8571 | SavingRate:0.4623 | Healthiness:0.9122  
Surv:85.7% (42/49) | Cost:\$268.85 | Save:\$231.15

~~~~~  
Generation 5/5 (attempt 1)  
~~~~~

Warn: No JSON. Fallback.

Day 0 | Stocked 30 fish | food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random | LLM:0.2s

Day 1 | Fit:0.7758 Yld:0.6122 Sav:0.9855 Hlth:0.9489 | Surv:100.0% (30/30) | Cost:\$ 7.25 Save:

```
e:$ 492.75 | Sim:0.0s LLM:6.6s
| food=10x/9h@Random prob=1x/48h@Random O2=1h/20h@Random [Mem:1H Tool Plan]

*** RESTARTING Gen 5 (attempt 1 -> 2) ***
Reason: Day 1 fitness 0.7758 < 0.9. Had 30 fish (max 49).

.
Generation 5/5 (attempt 2)
.
Day 0 | Stocked 40 fish | food=5x/12h@Random prob=1x/48h@Random O2=3h/20h@Random | LLM:3.0s
Day 1 | Fit:0.8668 Yld:0.7959 Sav:0.9675 Hlth:0.9148 | Surv: 97.5% (39/40) | Cost:$ 16.25 Sav
e:$ 483.75 | Sim:0.1s LLM:2.8s
| food=5x/12h@Random prob=1x/48h@Random O2=3h/20h@Random [Mem:1H Tool Plan]

*** RESTARTING Gen 5 (attempt 2 -> 3) ***
Reason: Day 1 fitness 0.8668 < 0.9. Had 40 fish (max 49).

.
Generation 5/5 (attempt 3)
.
Warn: No JSON. Fallback.
Day 0 | Stocked 49 fish | food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random | LLM:5.0s
Day 1 | Fit:0.9888 Yld:1.0000 Sav:0.9855 Hlth:0.9467 | Surv:100.0% (49/49) | Cost:$ 7.25 Sav
e:$ 492.75 | Sim:0.1s LLM:2.7s
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Mem:1H Tool Plan]
Day 2 | Fit:0.9866 Yld:1.0000 Sav:0.9775 Hlth:0.9499 | Surv:100.0% (49/49) | Cost:$ 11.25 Sav
e:$ 488.75 | Sim:0.1s LLM:3.9s
| food=10x/9h@Random prob=1x/48h@Random O2=1h/20h@Random [Mem:2H Tool Plan]
Day 3 | Fit:0.9806 Yld:1.0000 Sav:0.9690 Hlth:0.9233 | Surv:100.0% (49/49) | Cost:$ 15.50 Sav
e:$ 484.50 | Sim:0.1s LLM:5.0s
| food=9x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Mem:3H Tool Plan]
Day 4 | Fit:0.9800 Yld:1.0000 Sav:0.9613 Hlth:0.9395 | Surv:100.0% (49/49) | Cost:$ 19.35 Sav
e:$ 480.65 | Sim:0.1s LLM:1.6s
| food=9x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Mem:4H Tool Plan]
Day 5 | Fit:0.9741 Yld:1.0000 Sav:0.9521 Hlth:0.9159 | Surv:100.0% (49/49) | Cost:$ 23.95 Sav
e:$ 476.05 | Sim:0.1s LLM:1.4s
| food=9x/8h@Random prob=1x/48h@Random O2=1h/20h@Random [Mem:5H Tool Plan]
Day 6 | Fit:0.9657 Yld:1.0000 Sav:0.9394 Hlth:0.8809 | Surv:100.0% (49/49) | Cost:$ 30.30 Sav
e:$ 469.70 | Sim:0.1s LLM:3.2s
| food=9x/9h@Random prob=1x/48h@Random O2=1h/20h@Random [Mem:6H Tool Plan]
Day 7 | Fit:0.9521 Yld:0.9796 Sav:0.9302 Hlth:0.8860 | Surv: 98.0% (48/49) | Cost:$ 34.90 Sav
e:$ 465.10 | Sim:0.1s LLM:5.1s
| food=10x/9h@Random prob=1x/24h@Random O2=1h/20h@Random [Mem:7H Tool Plan]
Day 8 | Fit:0.9469 Yld:0.9796 Sav:0.9207 Hlth:0.8692 | Surv: 98.0% (48/49) | Cost:$ 39.65 Sav
e:$ 460.35 | Sim:0.1s LLM:2.2s
| food=10x/9h@Random prob=1x/24h@Random O2=1h/20h@Random [Mem:8H Tool Plan]
Day 9 | Fit:0.9461 Yld:0.9796 Sav:0.9122 Hlth:0.8855 | Surv: 98.0% (48/49) | Cost:$ 43.90 Sav
e:$ 456.10 | Sim:0.1s LLM:2.0s
| food=12x/9h@Random prob=1x/24h@Random O2=1h/20h@Random [Mem:9H Tool Plan]
Day 10 | Fit:0.9430 Yld:0.9796 Sav:0.9021 Hlth:0.8878 | Surv: 98.0% (48/49) | Cost:$ 48.95 Sav
e:$ 451.05 | Sim:0.1s LLM:3.8s
| food=14x/9h@Random prob=1x/24h@Random O2=1h/20h@Random [Mem:10H Tool Plan]
Day 11 | Fit:0.9401 Yld:0.9796 Sav:0.8864 Hlth:0.9065 | Surv: 98.0% (48/49) | Cost:$ 56.80 Sav
e:$ 443.20 | Sim:0.2s LLM:3.4s
| food=10x/9h@Random prob=1x/24h@Random O2=1h/20h@Random [Mem:11H Tool Plan]
Day 12 | Fit:0.9354 Yld:0.9796 Sav:0.8779 Hlth:0.8913 | Surv: 98.0% (48/49) | Cost:$ 61.05 Sav
e:$ 438.95 | Sim:0.2s LLM:1.7s
| food=10x/9h@Random prob=1x/24h@Random O2=1h/20h@Random [Mem:12H Tool Plan]
Day 13 | Fit:0.9340 Yld:0.9796 Sav:0.8684 Hlth:0.9057 | Surv: 98.0% (48/49) | Cost:$ 65.80 Sav
e:$ 434.20 | Sim:0.2s LLM:1.6s
| food=10x/9h@Random prob=1x/24h@Random O2=1h/20h@Random [Mem:13H Tool Plan]
Day 14 | Fit:0.9222 Yld:0.9592 Sav:0.8589 Hlth:0.9266 | Surv: 95.9% (47/49) | Cost:$ 70.55 Sav
e:$ 429.45 | Sim:0.2s LLM:5.7s
| food=8x/9h@Random prob=1x/24h@Random O2=1h/20h@Random [Mem:14H Tool Plan]
Day 15 | Fit:0.9202 Yld:0.9592 Sav:0.8508 Hlth:0.9320 | Surv: 95.9% (47/49) | Cost:$ 74.60 Sav
e:$ 425.40 | Sim:0.2s LLM:3.6s
| food=8x/9h@Random prob=1x/24h@Random O2=1h/10h@Random [Mem:15H Tool Plan]
Day 16 | Fit:0.9123 Yld:0.9592 Sav:0.8319 Hlth:0.9188 | Surv: 95.9% (47/49) | Cost:$ 84.05 Sav
e:$ 415.95 | Sim:0.2s LLM:3.9s
| food=8x/9h@Random prob=1x/24h@Random O2=1h/5h@Random [Mem:16H Tool Plan]
Day 17 | Fit:0.9001 Yld:0.9592 Sav:0.8030 Hlth:0.8962 | Surv: 95.9% (47/49) | Cost:$ 98.50 Sav
e:$ 401.50 | Sim:0.2s LLM:3.1s
| food=7x/9h@Random prob=1x/24h@Random O2=1h/5h@Random [Mem:17H Tool Plan]
Day 18 | Fit:0.8902 Yld:0.9592 Sav:0.7751 Hlth:0.8903 | Surv: 95.9% (47/49) | Cost:$ 112.45 Sav
e:$ 387.55 | Sim:0.2s LLM:3.7s
| food=7x/10h@Random prob=1x/24h@Random O2=1h/6h@Random [Mem:18H Tool Plan]
Day 19 | Fit:0.8805 Yld:0.9592 Sav:0.7522 Hlth:0.8725 | Surv: 95.9% (47/49) | Cost:$ 123.90 Sav
e:$ 376.10 | Sim:0.2s LLM:4.1s
| food=7x/12h@Random prob=1x/24h@Random O2=1h/6h@Random [Mem:19H Tool Plan]
Day 20 | Fit:0.8512 Yld:0.9184 Sav:0.7293 Hlth:0.8787 | Surv: 91.8% (45/49) | Cost:$ 135.35 Sav
e:$ 364.65 | Sim:0.2s LLM:3.8s
| food=7x/12h@Random prob=1x/24h@Random O2=1h/6h@Random [Mem:20H Tool Plan]
```

```

| food=7x/12h@Random prob=1x/24h@Random O2=1h/6h@Random [Mem:20H Tool Plan]
Day 21 | Fit:0.8431 Yld:0.9184 Sav:0.7064 Hlth:0.8739 | Surv: 91.8% (45/49) | Cost:$ 146.80 Sav
e:$ 353.20 | Sim:0.2s LLM:2.8s
| food=7x/12h@Random prob=1x/24h@Random O2=1h/6h@Random [Mem:21H Tool Plan]
Day 22 | Fit:0.8348 Yld:0.9184 Sav:0.6835 Hlth:0.8676 | Surv: 91.8% (45/49) | Cost:$ 158.25 Sav
e:$ 341.75 | Sim:0.2s LLM:2.7s
| food=7x/12h@Random prob=1x/36h@Random O2=1h/6h@Random [Mem:22H Tool Plan]
Day 23 | Fit:0.8039 Yld:0.8776 Sav:0.6606 Hlth:0.8607 | Surv: 87.8% (43/49) | Cost:$ 169.70 Sav
e:$ 330.30 | Sim:0.2s LLM:2.3s
| food=7x/8h@Random prob=1x/24h@Random O2=1h/6h@Random [Mem:23H Tool Plan]
Day 24 | Fit:0.7868 Yld:0.8571 Sav:0.6370 Hlth:0.8765 | Surv: 85.7% (42/49) | Cost:$ 181.50 Sav
e:$ 318.50 | Sim:0.2s LLM:3.3s
| food=7x/8h@Random prob=1x/24h@Random O2=1h/6h@Random [Mem:24H Tool Plan]
Day 25 | Fit:0.7793 Yld:0.8571 Sav:0.6134 Hlth:0.8784 | Surv: 85.7% (42/49) | Cost:$ 193.30 Sav
e:$ 306.70 | Sim:0.2s LLM:2.8s
| food=7x/8h@Random prob=1x/24h@Random O2=1h/6h@Random [Mem:25H Tool Plan]
Day 26 | Fit:0.7707 Yld:0.8571 Sav:0.5898 Hlth:0.8718 | Surv: 85.7% (42/49) | Cost:$ 205.10 Sav
e:$ 294.90 | Sim:0.2s LLM:3.9s
| food=7x/8h@Random prob=1x/24h@Random O2=1h/6h@Random [Mem:26H Tool Plan]
Day 27 | Fit:0.7637 Yld:0.8571 Sav:0.5662 Hlth:0.8783 | Surv: 85.7% (42/49) | Cost:$ 216.90 Sav
e:$ 283.10 | Sim:0.2s LLM:4.0s
| food=7x/9h@Random prob=1x/24h@Random O2=1h/6h@Random [Mem:27H Tool Plan]
Day 28 | Fit:0.7562 Yld:0.8571 Sav:0.5426 Hlth:0.8809 | Surv: 85.7% (42/49) | Cost:$ 228.70 Sav
e:$ 271.30 | Sim:0.2s LLM:3.8s
| food=7x/8h@Random prob=1x/24h@Random O2=1h/6h@Random [Mem:28H Tool Plan]
Day 29 | Fit:0.7362 Yld:0.8367 Sav:0.5190 Hlth:0.8724 | Surv: 83.7% (41/49) | Cost:$ 240.50 Sav
e:$ 259.50 | Sim:0.2s LLM:3.1s
| food=7x/6h@Random prob=1x/24h@Random O2=1h/6h@Random [Mem:29H Tool Plan]
Day 30 | Fit:0.7335 Yld:0.8367 Sav:0.4947 Hlth:0.9169 | Surv: 83.7% (41/49) | Cost:$ 252.65 Sav
e:$ 247.35 | Sim:0.2s [END]

```

Gen 5 FINAL (attempt 3):  
Fit:0.7335 | Yield:0.8367 | SavingRate:0.4947 | Healthiness:0.9169  
Surv:83.7% (41/49) | Cost:\$252.65 | Save:\$247.35

Saved memory: /content/AquaLLM/logs/memory\_no\_obs.json  
Saved trace: /content/AquaLLM/logs/trace\_no\_obs.json (899,399 bytes)

```

=====
Config: minimal (Minimal) | Run 1/1
Days: 30 | Budget: $500.0 | Generations: 5
=====

~~~~~
Generation 1/5 (attempt 1)
~~~~~
Day 0 | Stocked 45 fish | food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random | LLM:0.5s
Day 1 | Fit:0.9440 Yld:0.9184 Sav:0.9855 Hlth:0.9470 | Surv:100.0% (45/45) | Cost:$ 7.25 Sav
e:$ 492.75 | Sim:0.1s LLM:0.5s
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random []
Day 2 | Fit:0.9400 Yld:0.9184 Sav:0.9775 Hlth:0.9358 | Surv:100.0% (45/45) | Cost:$ 11.25 Sav
e:$ 488.75 | Sim:0.1s LLM:0.5s
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random []
Day 3 | Fit:0.9360 Yld:0.9184 Sav:0.9680 Hlth:0.9290 | Surv:100.0% (45/45) | Cost:$ 16.00 Sav
e:$ 484.00 | Sim:0.1s LLM:0.5s
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random []
Day 4 | Fit:0.9343 Yld:0.9184 Sav:0.9600 Hlth:0.9368 | Surv:100.0% (45/45) | Cost:$ 20.00 Sav
e:$ 480.00 | Sim:0.1s LLM:0.5s
| food=10x/8h@Top-Ctr prob=1x/48h@Random O2=1h/20h@Bot-Ctr []
Day 5 | Fit:0.9288 Yld:0.9184 Sav:0.9505 Hlth:0.9165 | Surv:100.0% (45/45) | Cost:$ 24.75 Sav
e:$ 475.25 | Sim:0.1s LLM:0.5s
| food=10x/8h@Top-Ctr prob=1x/48h@Random O2=1h/20h@Bot-Ctr []
Day 6 | Fit:0.9188 Yld:0.9184 Sav:0.9375 Hlth:0.8693 | Surv:100.0% (45/45) | Cost:$ 31.25 Sav
e:$ 468.75 | Sim:0.1s LLM:0.5s
| food=10x/8h@Top-Ctr prob=1x/48h@Random O2=1h/20h@Bot-Ctr []
Day 7 | Fit:0.9094 Yld:0.9184 Sav:0.9280 Hlth:0.8173 | Surv:100.0% (45/45) | Cost:$ 36.00 Sav
e:$ 464.00 | Sim:0.1s LLM:0.5s
| food=10x/8h@Top-Ctr prob=1x/48h@Random O2=1h/20h@Bot-Ctr []
Day 8 | Fit:0.9005 Yld:0.9184 Sav:0.9200 Hlth:0.7647 | Surv:100.0% (45/45) | Cost:$ 40.00 Sav
e:$ 460.00 | Sim:0.1s LLM:0.5s
| food=10x/8h@Top-Ctr prob=1x/48h@Random O2=1h/20h@Bot-Ctr []
Day 9 | Fit:0.8916 Yld:0.9184 Sav:0.9105 Hlth:0.7168 | Surv:100.0% (45/45) | Cost:$ 44.75 Sav
e:$ 455.25 | Sim:0.1s LLM:0.5s
| food=10x/8h@Top-Ctr prob=1x/48h@Random O2=1h/20h@Bot-Ctr []
Day 10 | Fit:0.8730 Yld:0.8980 Sav:0.9025 Hlth:0.6773 | Surv: 97.8% (44/45) | Cost:$ 48.75 Sav
e:$ 451.25 | Sim:0.1s LLM:0.6s
| food=15x/8h@Top-Ctr prob=1x/48h@Random O2=1h/20h@Bot-Ctr []
Day 11 | Fit:0.8400 Yld:0.8571 Sav:0.8865 Hlth:0.6336 | Surv: 93.3% (42/45) | Cost:$ 56.75 Sav
e:$ 443.25 | Sim:0.2s LLM:0.5s
| food=10x/12h@Left-Ctr prob=1x/48h@Random O2=1h/20h@Bot-Ctr []
Day 12 | Fit:0.8237 Yld:0.8163 Sav:0.8795 Hlth:0.7038 | Surv: 88.9% (40/45) | Cost:$ 60.25 Sav
e:$ 439.75 | Sim:0.2s LLM:0.6s

```

```

| food=10x/12h@Left-Ctr prob=1x/48h@Random O2=1h/20h@Bot-Ctr []
Day 13 | Fit:0.8034 Yld:0.7755 Sav:0.8710 Hlth:0.7451 | Surv: 84.4% (38/45) | Cost:$ 64.50 Sav
e:$ 435.50 | Sim:0.2s LLM:0.5s
| food=15x/12h@Left-Ctr prob=1x/48h@Random O2=1h/20h@Bot-Ctr []
Day 14 | Fit:0.7923 Yld:0.7551 Sav:0.8630 Hlth:0.7682 | Surv: 82.2% (37/45) | Cost:$ 68.50 Sav
e:$ 431.50 | Sim:0.2s LLM:0.5s
| food=15x/12h@Bot-Ctr prob=1x/48h@Random O2=1h/20h@Top-Ctr []
Day 15 | Fit:0.7788 Yld:0.7347 Sav:0.8535 Hlth:0.7759 | Surv: 80.0% (36/45) | Cost:$ 73.25 Sav
e:$ 426.75 | Sim:0.2s LLM:0.5s
| food=15x/12h@Bot-Ctr prob=1x/48h@Random O2=1h/20h@Top-Ctr []
Day 16 | Fit:0.7610 Yld:0.7143 Sav:0.8405 Hlth:0.7568 | Surv: 77.8% (35/45) | Cost:$ 79.75 Sav
e:$ 420.25 | Sim:0.2s LLM:0.6s
| food=15x/12h@Bot-Ctr prob=1x/48h@Random O2=1h/20h@Top-Ctr []
Day 17 | Fit:0.7296 Yld:0.6735 Sav:0.8310 Hlth:0.7084 | Surv: 73.3% (33/45) | Cost:$ 84.50 Sav
e:$ 415.50 | Sim:0.1s LLM:0.5s
| food=20x/12h@Bot-Ctr prob=1x/48h@Random O2=1h/15h@Top-Ctr []
Day 18 | Fit:0.7088 Yld:0.6531 Sav:0.8220 Hlth:0.6534 | Surv: 71.1% (32/45) | Cost:$ 89.00 Sav
e:$ 411.00 | Sim:0.1s LLM:0.5s
| food=25x/12h@Bot-Ctr prob=1x/48h@Random O2=1h/15h@Top-Ctr []
Day 19 | Fit:0.6730 Yld:0.6122 Sav:0.8055 Hlth:0.5875 | Surv: 66.7% (30/45) | Cost:$ 97.25 Sav
e:$ 402.75 | Sim:0.1s LLM:0.6s
| food=30x/12h@Bot-Ctr prob=1x/48h@Random O2=1h/15h@Top-Ctr []
Day 20 | Fit:0.6190 Yld:0.5306 Sav:0.7945 Hlth:0.5411 | Surv: 57.8% (26/45) | Cost:$ 102.75 Sav
e:$ 397.25 | Sim:0.2s LLM:0.6s
| food=30x/12h@Bot-Ctr prob=1x/48h@Random O2=2h/15h@Top-Ctr []
Day 21 | Fit:0.5786 Yld:0.4898 Sav:0.7670 Hlth:0.4671 | Surv: 53.3% (24/45) | Cost:$ 116.50 Sav
e:$ 383.50 | Sim:0.1s LLM:0.5s
| food=30x/12h@Bot-Ctr prob=1x/48h@Random O2=2h/15h@Top-Ctr []
Day 22 | Fit:0.5108 Yld:0.3878 Sav:0.7410 Hlth:0.4416 | Surv: 42.2% (19/45) | Cost:$ 129.50 Sav
e:$ 370.50 | Sim:0.2s LLM:0.5s
| food=30x/12h@Bot-Ctr prob=1x/48h@Random O2=2h/15h@Top-Ctr []
Day 23 | Fit:0.4453 Yld:0.2653 Sav:0.7235 Hlth:0.5050 | Surv: 28.9% (13/45) | Cost:$ 138.25 Sav
e:$ 361.75 | Sim:0.2s LLM:0.6s
| food=30x/12h@Left-Ctr prob=1x/48h@Random O2=2h/10h@Top-Ctr []
Day 24 | Fit:0.4397 Yld:0.2245 Sav:0.6975 Hlth:0.7169 | Surv: 24.4% (11/45) | Cost:$ 151.25 Sav
e:$ 348.75 | Sim:0.1s LLM:0.6s
| food=30x/12h@Left-Ctr prob=1x/48h@Random O2=2h/10h@Top-Ctr []
Day 25 | Fit:0.4191 Yld:0.2041 Sav:0.6700 Hlth:0.7147 | Surv: 22.2% (10/45) | Cost:$ 165.00 Sav
e:$ 335.00 | Sim:0.1s LLM:0.5s
| food=30x/12h@Left-Ctr prob=1x/48h@Random O2=2h/10h@Top-Ctr []
Day 26 | Fit:0.4005 Yld:0.2041 Sav:0.6340 Hlth:0.6588 | Surv: 22.2% (10/45) | Cost:$ 183.00 Sav
e:$ 317.00 | Sim:0.1s LLM:0.6s
| food=30x/12h@Left-Ctr prob=1x/48h@Random O2=2h/10h@Bot-Ctr []
Day 27 | Fit:0.3854 Yld:0.2041 Sav:0.6065 Hlth:0.6087 | Surv: 22.2% (10/45) | Cost:$ 196.75 Sav
e:$ 303.25 | Sim:0.1s LLM:0.6s
| food=30x/12h@Left-Ctr prob=1x/48h@Random O2=2h/10h@Bot-Ctr []
Day 28 | Fit:0.3625 Yld:0.2041 Sav:0.5705 Hlth:0.5167 | Surv: 22.2% (10/45) | Cost:$ 214.75 Sav
e:$ 285.25 | Sim:0.1s LLM:0.6s
| food=30x/12h@Left-Ctr prob=1x/48h@Random O2=2h/6h@Top-Ctr []
Day 29 | Fit:0.3415 Yld:0.2041 Sav:0.5230 Hlth:0.4721 | Surv: 22.2% (10/45) | Cost:$ 238.50 Sav
e:$ 261.50 | Sim:0.1s LLM:0.6s
| food=30x/12h@Left-Ctr prob=1x/48h@Random O2=2h/6h@Top-Ctr []
Day 30 | Fit:0.3090 Yld:0.1837 Sav:0.4770 Hlth:0.4213 | Surv: 20.0% (9/45) | Cost:$ 261.50 Save
:$ 238.50 | Sim:0.1s [END]

Gen 1 FINAL (attempt 1):
Fit:0.3090 | Yield:0.1837 | SavingRate:0.4770 | Healthiness:0.4213
Surv:20.0% (9/45) | Cost:$261.50 | Save:$238.50

~~~~~
Generation 2/5 (attempt 1)
~~~~~
Day 0 | Stocked 45 fish | food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random | LLM:0.5s
Day 1 | Fit:0.9440 Yld:0.9184 Sav:0.9855 Hlth:0.9478 | Surv:100.0% (45/45) | Cost:$ 7.25 Sav
e:$ 492.75 | Sim:0.1s LLM:0.5s
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random []
Day 2 | Fit:0.9399 Yld:0.9184 Sav:0.9775 Hlth:0.9353 | Surv:100.0% (45/45) | Cost:$ 11.25 Sav
e:$ 488.75 | Sim:0.1s LLM:0.5s
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random []
Day 3 | Fit:0.9352 Yld:0.9184 Sav:0.9680 Hlth:0.9223 | Surv:100.0% (45/45) | Cost:$ 16.00 Sav
e:$ 484.00 | Sim:0.1s LLM:0.5s
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random []
Day 4 | Fit:0.9214 Yld:0.8980 Sav:0.9600 Hlth:0.9231 | Surv: 97.8% (44/45) | Cost:$ 20.00 Sav
e:$ 480.00 | Sim:0.1s LLM:0.5s
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random []
Day 5 | Fit:0.9196 Yld:0.8980 Sav:0.9505 Hlth:0.9335 | Surv: 97.8% (44/45) | Cost:$ 24.75 Sav
e:$ 475.25 | Sim:0.1s LLM:0.5s
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random []
Day 6 | Fit:0.9153 Yld:0.8980 Sav:0.9375 Hlth:0.9334 | Surv: 97.8% (44/45) | Cost:$ 31.25 Sav
e:$ 468.75 | Sim:0.1s LLM:0.5s
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random []
Day 7 | Fit:0.9101 Yld:0.8980 Sav:0.9280 Hlth:0.9166 | Surv: 97.8% (44/45) | Cost:$ 36.00 Sav
```

Day 7 | Fit:0.9101 Yld:0.8980 Sav:0.9200 Hlth:0.9100 | Surv: 97.8% (44/45) | Cost:\$ 56.00 Sav  
e:\$ 464.00 | Sim:0.1s LLM:0.5s  
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random []  
Day 8 | Fit:0.9094 Yld:0.8980 Sav:0.9200 Hlth:0.9324 | Surv: 97.8% (44/45) | Cost:\$ 40.00 Sav  
e:\$ 460.00 | Sim:0.1s LLM:0.5s  
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random []  
Day 9 | Fit:0.9087 Yld:0.8980 Sav:0.9105 Hlth:0.9533 | Surv: 97.8% (44/45) | Cost:\$ 44.75 Sav  
e:\$ 455.25 | Sim:0.1s LLM:0.5s  
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random []  
Day 10 | Fit:0.9051 Yld:0.8980 Sav:0.9025 Hlth:0.9448 | Surv: 97.8% (44/45) | Cost:\$ 48.75 Sav  
e:\$ 451.25 | Sim:0.1s LLM:0.6s  
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random []  
Day 11 | Fit:0.9010 Yld:0.8980 Sav:0.8880 Hlth:0.9511 | Surv: 97.8% (44/45) | Cost:\$ 56.00 Sav  
e:\$ 444.00 | Sim:0.1s LLM:0.6s  
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random []  
Day 12 | Fit:0.8955 Yld:0.8980 Sav:0.8800 Hlth:0.9272 | Surv: 97.8% (44/45) | Cost:\$ 60.00 Sav  
e:\$ 440.00 | Sim:0.1s LLM:0.6s  
| food=10x/8h@Top-Ctr prob=1x/48h@Top-Ctr O2=1h/20h@Top-Ctr []  
Day 13 | Fit:0.8939 Yld:0.8980 Sav:0.8705 Hlth:0.9394 | Surv: 97.8% (44/45) | Cost:\$ 64.75 Sav  
e:\$ 435.25 | Sim:0.1s LLM:0.5s  
| food=10x/8h@Top-Ctr prob=1x/48h@Top-Ctr O2=1h/20h@Top-Ctr []  
Day 14 | Fit:0.8871 Yld:0.8980 Sav:0.8625 Hlth:0.9052 | Surv: 97.8% (44/45) | Cost:\$ 68.75 Sav  
e:\$ 431.25 | Sim:0.1s LLM:0.5s  
| food=10x/8h@Top-Ctr prob=1x/48h@Top-Ctr O2=1h/20h@Top-Ctr []  
Day 15 | Fit:0.8784 Yld:0.8980 Sav:0.8530 Hlth:0.8586 | Surv: 97.8% (44/45) | Cost:\$ 73.50 Sav  
e:\$ 426.50 | Sim:0.2s LLM:0.5s  
| food=10x/8h@Top-Ctr prob=1x/48h@Top-Ctr O2=1h/20h@Top-Ctr []  
Day 16 | Fit:0.8713 Yld:0.8980 Sav:0.8400 Hlth:0.8354 | Surv: 97.8% (44/45) | Cost:\$ 80.00 Sav  
e:\$ 420.00 | Sim:0.2s LLM:0.5s  
| food=10x/8h@Top-Ctr prob=1x/48h@Top-Ctr O2=1h/20h@Top-Ctr []  
Day 17 | Fit:0.8675 Yld:0.8980 Sav:0.8305 Hlth:0.8294 | Surv: 97.8% (44/45) | Cost:\$ 84.75 Sav  
e:\$ 415.25 | Sim:0.2s LLM:0.5s  
| food=10x/8h@Top-Ctr prob=1x/48h@Top-Ctr O2=1h/20h@Top-Ctr []  
Day 18 | Fit:0.8619 Yld:0.8980 Sav:0.8225 Hlth:0.8053 | Surv: 97.8% (44/45) | Cost:\$ 88.75 Sav  
e:\$ 411.25 | Sim:0.2s LLM:0.5s  
| food=10x/8h@Top-Ctr prob=1x/48h@Top-Ctr O2=1h/20h@Top-Ctr []  
Day 19 | Fit:0.8455 Yld:0.8776 Sav:0.8130 Hlth:0.7878 | Surv: 95.6% (43/45) | Cost:\$ 93.50 Sav  
e:\$ 406.50 | Sim:0.2s LLM:0.5s  
| food=10x/8h@Top-Ctr prob=1x/48h@Top-Ctr O2=1h/20h@Top-Ctr []  
Day 20 | Fit:0.8305 Yld:0.8571 Sav:0.8050 Hlth:0.7789 | Surv: 93.3% (42/45) | Cost:\$ 97.50 Sav  
e:\$ 402.50 | Sim:0.2s LLM:0.5s  
| food=10x/8h@Top-Ctr prob=1x/48h@Top-Ctr O2=1h/20h@Top-Ctr []  
Day 21 | Fit:0.7829 Yld:0.7755 Sav:0.7905 Hlth:0.7960 | Surv: 84.4% (38/45) | Cost:\$ 104.75 Sav  
e:\$ 395.25 | Sim:0.2s LLM:0.5s  
| food=12x/8h@Top-Ctr prob=1x/48h@Top-Ctr O2=1h/20h@Top-Ctr []  
Day 22 | Fit:0.7372 Yld:0.6939 Sav:0.7819 Hlth:0.8132 | Surv: 75.6% (34/45) | Cost:\$ 109.05 Sav  
e:\$ 390.95 | Sim:0.2s LLM:0.6s  
| food=20x/6h@Top-Ctr prob=1x/48h@Top-Ctr O2=1h/20h@Top-Ctr []  
Day 23 | Fit:0.7269 Yld:0.6735 Sav:0.7674 Hlth:0.8602 | Surv: 73.3% (33/45) | Cost:\$ 116.30 Sav  
e:\$ 383.70 | Sim:0.2s LLM:0.6s  
| food=20x/6h@Top-Ctr prob=1x/48h@Top-Ctr O2=1h/20h@Top-Ctr []  
Day 24 | Fit:0.7077 Yld:0.6531 Sav:0.7544 Hlth:0.8300 | Surv: 71.1% (32/45) | Cost:\$ 122.80 Sav  
e:\$ 377.20 | Sim:0.2s LLM:0.6s  
| food=20x/6h@Top-Ctr prob=1x/48h@Top-Ctr O2=1h/20h@Top-Ctr []  
Day 25 | Fit:0.7059 Yld:0.6531 Sav:0.7399 Hlth:0.8544 | Surv: 71.1% (32/45) | Cost:\$ 130.05 Sav  
e:\$ 369.95 | Sim:0.2s LLM:0.5s  
| food=20x/6h@Top-Ctr prob=1x/48h@Top-Ctr O2=1h/20h@Top-Ctr []  
Day 26 | Fit:0.6795 Yld:0.6122 Sav:0.7219 Hlth:0.8713 | Surv: 66.7% (30/45) | Cost:\$ 139.05 Sav  
e:\$ 360.95 | Sim:0.2s LLM:0.5s  
| food=20x/6h@Top-Ctr prob=1x/48h@Top-Ctr O2=1h/20h@Top-Ctr []  
Day 27 | Fit:0.6658 Yld:0.5918 Sav:0.7074 Hlth:0.8902 | Surv: 64.4% (29/45) | Cost:\$ 146.30 Sav  
e:\$ 353.70 | Sim:0.2s LLM:0.6s  
| food=20x/6h@Top-Ctr prob=1x/48h@Top-Ctr O2=1h/20h@Top-Ctr []  
Day 28 | Fit:0.6593 Yld:0.5918 Sav:0.6944 Hlth:0.8723 | Surv: 64.4% (29/45) | Cost:\$ 152.80 Sav  
e:\$ 347.20 | Sim:0.3s LLM:0.5s  
| food=20x/6h@Top-Ctr prob=1x/48h@Top-Ctr O2=1h/20h@Top-Ctr []  
Day 29 | Fit:0.6551 Yld:0.5918 Sav:0.6799 Hlth:0.8767 | Surv: 64.4% (29/45) | Cost:\$ 160.05 Sav  
e:\$ 339.95 | Sim:0.3s LLM:0.6s  
| food=20x/6h@Top-Ctr prob=1x/48h@Top-Ctr O2=1h/20h@Top-Ctr []  
Day 30 | Fit:0.6331 Yld:0.5510 Sav:0.6669 Hlth:0.9167 | Surv: 60.0% (27/45) | Cost:\$ 166.55 Sav  
e:\$ 333.45 | Sim:0.3s [END]

Gen 2 FINAL (attempt 1):  
Fit:0.6331 | Yield:0.5510 | SavingRate:0.6669 | Healthiness:0.9167  
Surv:60.0% (27/45) | Cost:\$166.55 | Save:\$333.45

~~~~~  
Generation 3/5 (attempt 1)  
~~~~~

Day 0 | Stocked 45 fish | food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random | LLM:0.6s  
Day 1 | Fit:0.9440 Yld:0.9184 Sav:0.9855 Hlth:0.9477 | Surv:100.0% (45/45) | Cost:\$ 7.25 Sav  
e:\$ 492.75 | Sim:0.1s LLM:0.5s

[illegible]

Day 29 | Fit:0.4214 Yld:0.2449 Sav:0.6408 Hlth:0.6266 | Surv: 26.7% (12/45) | Cost:\$ 179.60 Sav  
e:\$ 320.40 | Sim:0.2s LLM:0.6s  
| food=30x/5h@Random prob=1x/48h@Top-Ctr O2=1h/12h@Top-Ctr []  
Day 30 | Fit:0.3992 Yld:0.2245 Sav:0.6188 Hlth:0.5962 | Surv: 24.4% (11/45) | Cost:\$ 190.60 Sav  
e:\$ 309.40 | Sim:0.2s [END]

Gen 3 FINAL (attempt 1):  
Fit:0.3992 | Yield:0.2245 | SavingRate:0.6188 | Healthiness:0.5962  
Surv:24.4% (11/45) | Cost:\$190.60 | Save:\$309.40

~~~~~  
Generation 4/5 (attempt 1)  
~~~~~

Day 0 | Stocked 45 fish | food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random | LLM:0.6s  
Day 1 | Fit:0.9428 Yld:0.9184 Sav:0.9855 Hlth:0.9369 | Surv:100.0% (45/45) | Cost:\$ 7.25 Sav  
e:\$ 492.75 | Sim:0.1s LLM:0.5s  
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random []  
Day 2 | Fit:0.9403 Yld:0.9184 Sav:0.9775 Hlth:0.9383 | Surv:100.0% (45/45) | Cost:\$ 11.25 Sav  
e:\$ 488.75 | Sim:0.1s LLM:0.5s  
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random []  
Day 3 | Fit:0.9356 Yld:0.9184 Sav:0.9680 Hlth:0.9254 | Surv:100.0% (45/45) | Cost:\$ 16.00 Sav  
e:\$ 484.00 | Sim:0.1s LLM:0.5s  
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random []  
Day 4 | Fit:0.9321 Yld:0.9184 Sav:0.9600 Hlth:0.9180 | Surv:100.0% (45/45) | Cost:\$ 20.00 Sav  
e:\$ 480.00 | Sim:0.1s LLM:0.5s  
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random []  
Day 5 | Fit:0.9304 Yld:0.9184 Sav:0.9505 Hlth:0.9300 | Surv:100.0% (45/45) | Cost:\$ 24.75 Sav  
e:\$ 475.25 | Sim:0.1s LLM:0.5s  
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random []  
Day 6 | Fit:0.9261 Yld:0.9184 Sav:0.9375 Hlth:0.9299 | Surv:100.0% (45/45) | Cost:\$ 31.25 Sav  
e:\$ 468.75 | Sim:0.1s LLM:0.5s  
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random []  
Day 7 | Fit:0.9217 Yld:0.9184 Sav:0.9280 Hlth:0.9198 | Surv:100.0% (45/45) | Cost:\$ 36.00 Sav  
e:\$ 464.00 | Sim:0.1s LLM:0.5s  
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random []  
Day 8 | Fit:0.9196 Yld:0.9184 Sav:0.9200 Hlth:0.9242 | Surv:100.0% (45/45) | Cost:\$ 40.00 Sav  
e:\$ 460.00 | Sim:0.1s LLM:0.5s  
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random []  
Day 9 | Fit:0.9146 Yld:0.9184 Sav:0.9105 Hlth:0.9088 | Surv:100.0% (45/45) | Cost:\$ 44.75 Sav  
e:\$ 455.25 | Sim:0.1s LLM:0.5s  
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random []  
Day 10 | Fit:0.9106 Yld:0.9184 Sav:0.9025 Hlth:0.8976 | Surv:100.0% (45/45) | Cost:\$ 48.75 Sav  
e:\$ 451.25 | Sim:0.1s LLM:0.5s  
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random []  
Day 11 | Fit:0.9061 Yld:0.9184 Sav:0.8880 Hlth:0.8994 | Surv:100.0% (45/45) | Cost:\$ 56.00 Sav  
e:\$ 444.00 | Sim:0.1s LLM:0.5s  
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random []  
Day 12 | Fit:0.8898 Yld:0.8980 Sav:0.8800 Hlth:0.8792 | Surv: 97.8% (44/45) | Cost:\$ 60.00 Sav  
e:\$ 440.00 | Sim:0.1s LLM:0.5s  
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random []  
Day 13 | Fit:0.8853 Yld:0.8980 Sav:0.8705 Hlth:0.8681 | Surv: 97.8% (44/45) | Cost:\$ 64.75 Sav  
e:\$ 435.25 | Sim:0.2s LLM:0.5s  
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random []  
Day 14 | Fit:0.8844 Yld:0.8980 Sav:0.8625 Hlth:0.8823 | Surv: 97.8% (44/45) | Cost:\$ 68.75 Sav  
e:\$ 431.25 | Sim:0.2s LLM:0.5s  
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random []  
Day 15 | Fit:0.8819 Yld:0.8980 Sav:0.8530 Hlth:0.8875 | Surv: 97.8% (44/45) | Cost:\$ 73.50 Sav  
e:\$ 426.50 | Sim:0.2s LLM:0.5s  
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random []  
Day 16 | Fit:0.8794 Yld:0.8980 Sav:0.8400 Hlth:0.9025 | Surv: 97.8% (44/45) | Cost:\$ 80.00 Sav  
e:\$ 420.00 | Sim:0.2s LLM:0.5s  
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random []  
Day 17 | Fit:0.8763 Yld:0.8980 Sav:0.8305 Hlth:0.9033 | Surv: 97.8% (44/45) | Cost:\$ 84.75 Sav  
e:\$ 415.25 | Sim:0.2s LLM:0.5s  
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random []  
Day 18 | Fit:0.8739 Yld:0.8980 Sav:0.8225 Hlth:0.9046 | Surv: 97.8% (44/45) | Cost:\$ 88.75 Sav  
e:\$ 411.25 | Sim:0.2s LLM:0.5s  
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random []  
Day 19 | Fit:0.8699 Yld:0.8980 Sav:0.8130 Hlth:0.8975 | Surv: 97.8% (44/45) | Cost:\$ 93.50 Sav  
e:\$ 406.50 | Sim:0.2s LLM:0.5s  
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random []  
Day 20 | Fit:0.8674 Yld:0.8980 Sav:0.8050 Hlth:0.8992 | Surv: 97.8% (44/45) | Cost:\$ 97.50 Sav  
e:\$ 402.50 | Sim:0.2s LLM:0.5s  
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random []  
Day 21 | Fit:0.8611 Yld:0.8980 Sav:0.7905 Hlth:0.8862 | Surv: 97.8% (44/45) | Cost:\$ 104.75 Sav  
e:\$ 395.25 | Sim:0.2s LLM:0.5s  
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random []  
Day 22 | Fit:0.8581 Yld:0.8980 Sav:0.7825 Hlth:0.8830 | Surv: 97.8% (44/45) | Cost:\$ 108.75 Sav  
e:\$ 391.25 | Sim:0.2s LLM:0.6s  
| food=10x/8h@Top-Ctr prob=1x/48h@Top-Ctr O2=1h/20h@Top-Ctr []  
Day 23 | Fit:0.8391 Yld:0.8776 Sav:0.7730 Hlth:0.8449 | Surv: 95.6% (43/45) | Cost:\$ 113.50 Sav  
e:\$ 386.50 | Sim:0.2s LLM:0.6s  
| food=10x/8h@Top-Ctr prob=1x/48h@Top-Ctr O2=1h/20h@Top-Ctr []

Day 23 | Sim:0.2s LLM:0.6s  
| food=10x/8h@Top-Ctr prob=1x/48h@Top-Ctr O2=1h/20h@Top-Ctr []  
Day 24 | Fit:0.8221 Yld:0.8571 Sav:0.7650 Hlth:0.8188 | Surv: 93.3% (42/45) | Cost:\$ 117.50 Sav  
e:\$ 382.50 | Sim:0.2s LLM:0.5s  
| food=10x/8h@Top-Ctr prob=1x/48h@Top-Ctr O2=1h/20h@Top-Ctr []  
Day 25 | Fit:0.7894 Yld:0.8163 Sav:0.7555 Hlth:0.7589 | Surv: 88.9% (40/45) | Cost:\$ 122.25 Sav  
e:\$ 377.75 | Sim:0.2s LLM:0.6s  
| food=10x/8h@Top-Ctr prob=1x/48h@Top-Ctr O2=1h/20h@Top-Ctr []  
Day 26 | Fit:0.7674 Yld:0.7959 Sav:0.7425 Hlth:0.7051 | Surv: 86.7% (39/45) | Cost:\$ 128.75 Sav  
e:\$ 371.25 | Sim:0.2s LLM:0.6s  
| food=10x/8h@Top-Ctr prob=1x/48h@Top-Ctr O2=1h/20h@Top-Ctr []  
Day 27 | Fit:0.7576 Yld:0.7959 Sav:0.7330 Hlth:0.6494 | Surv: 86.7% (39/45) | Cost:\$ 133.50 Sav  
e:\$ 366.50 | Sim:0.2s LLM:0.5s  
| food=15x/8h@Top-Ctr prob=1x/48h@Top-Ctr O2=1h/20h@Top-Ctr []  
Day 28 | Fit:0.7311 Yld:0.7551 Sav:0.7235 Hlth:0.6422 | Surv: 82.2% (37/45) | Cost:\$ 138.25 Sav  
e:\$ 361.75 | Sim:0.2s LLM:0.6s  
| food=15x/4h@Random prob=1x/48h@Top-Ctr O2=1h/20h@Top-Ctr []  
Day 29 | Fit:0.7501 Yld:0.7551 Sav:0.7080 Hlth:0.8431 | Surv: 82.2% (37/45) | Cost:\$ 146.00 Sav  
e:\$ 354.00 | Sim:0.2s LLM:0.6s  
| food=15x/4h@Random prob=1x/48h@Top-Ctr O2=1h/20h@Top-Ctr []  
Day 30 | Fit:0.7395 Yld:0.7347 Sav:0.6940 Hlth:0.8870 | Surv: 80.0% (36/45) | Cost:\$ 153.00 Sav  
e:\$ 347.00 | Sim:0.2s [END]

Gen 4 FINAL (attempt 1):  
Fit:0.7395 | Yield:0.7347 | SavingRate:0.6940 | Healthiness:0.8870  
Surv:80.0% (36/45) | Cost:\$153.00 | Save:\$347.00

~~~~~  
Generation 5/5 (attempt 1)  
~~~~~

Day 0 | Stocked 45 fish | food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random | LLM:0.5s  
Day 1 | Fit:0.9438 Yld:0.9184 Sav:0.9855 Hlth:0.9460 | Surv:100.0% (45/45) | Cost:\$ 7.25 Sav  
e:\$ 492.75 | Sim:0.1s LLM:0.5s  
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random []  
Day 2 | Fit:0.9399 Yld:0.9184 Sav:0.9775 Hlth:0.9349 | Surv:100.0% (45/45) | Cost:\$ 11.25 Sav  
e:\$ 488.75 | Sim:0.1s LLM:0.5s  
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random []  
Day 3 | Fit:0.9350 Yld:0.9184 Sav:0.9680 Hlth:0.9203 | Surv:100.0% (45/45) | Cost:\$ 16.00 Sav  
e:\$ 484.00 | Sim:0.1s LLM:0.5s  
| food=10x/8h@Random prob=1x/48h@Random O2=1h/20h@Random []  
Day 4 | Fit:0.9322 Yld:0.9184 Sav:0.9600 Hlth:0.9188 | Surv:100.0% (45/45) | Cost:\$ 20.00 Sav  
e:\$ 480.00 | Sim:0.1s LLM:0.5s  
| food=10x/8h@Left-Ctr prob=1x/48h@Top-Ctr O2=1h/20h@Bot-Ctr []  
Day 5 | Fit:0.9167 Yld:0.8980 Sav:0.9505 Hlth:0.9100 | Surv: 97.8% (44/45) | Cost:\$ 24.75 Sav  
e:\$ 475.25 | Sim:0.1s LLM:0.5s  
| food=10x/8h@Left-Ctr prob=1x/48h@Top-Ctr O2=1h/20h@Bot-Ctr []  
Day 6 | Fit:0.9085 Yld:0.8980 Sav:0.9375 Hlth:0.8769 | Surv: 97.8% (44/45) | Cost:\$ 31.25 Sav  
e:\$ 468.75 | Sim:0.1s LLM:0.5s  
| food=10x/8h@Left-Ctr prob=1x/48h@Top-Ctr O2=1h/20h@Bot-Ctr []  
Day 7 | Fit:0.9025 Yld:0.8980 Sav:0.9280 Hlth:0.8530 | Surv: 97.8% (44/45) | Cost:\$ 36.00 Sav  
e:\$ 464.00 | Sim:0.1s LLM:0.5s  
| food=10x/8h@Left-Ctr prob=1x/48h@Top-Ctr O2=1h/20h@Bot-Ctr []  
Day 8 | Fit:0.8995 Yld:0.8980 Sav:0.9200 Hlth:0.8503 | Surv: 97.8% (44/45) | Cost:\$ 40.00 Sav  
e:\$ 460.00 | Sim:0.1s LLM:0.5s  
| food=10x/8h@Left-Ctr prob=1x/48h@Top-Ctr O2=1h/20h@Bot-Ctr []  
Day 9 | Fit:0.8735 Yld:0.8571 Sav:0.9105 Hlth:0.8468 | Surv: 93.3% (42/45) | Cost:\$ 44.75 Sav  
e:\$ 455.25 | Sim:0.1s LLM:0.5s  
| food=10x/8h@Left-Ctr prob=1x/48h@Top-Ctr O2=1h/20h@Bot-Ctr []  
Day 10 | Fit:0.8585 Yld:0.8367 Sav:0.9025 Hlth:0.8373 | Surv: 91.1% (41/45) | Cost:\$ 48.75 Sav  
e:\$ 451.25 | Sim:0.1s LLM:0.5s  
| food=10x/8h@Left-Ctr prob=1x/48h@Top-Ctr O2=1h/20h@Bot-Ctr []  
Day 11 | Fit:0.8412 Yld:0.8163 Sav:0.8880 Hlth:0.8262 | Surv: 88.9% (40/45) | Cost:\$ 56.00 Sav  
e:\$ 444.00 | Sim:0.1s LLM:0.5s  
| food=10x/8h@Left-Ctr prob=1x/48h@Top-Ctr O2=1h/20h@Bot-Ctr []  
Day 12 | Fit:0.8239 Yld:0.7959 Sav:0.8800 Hlth:0.7977 | Surv: 86.7% (39/45) | Cost:\$ 60.00 Sav  
e:\$ 440.00 | Sim:0.1s LLM:0.5s  
| food=10x/8h@Left-Ctr prob=1x/48h@Top-Ctr O2=1h/20h@Bot-Ctr []  
Day 13 | Fit:0.7974 Yld:0.7551 Sav:0.8705 Hlth:0.7901 | Surv: 82.2% (37/45) | Cost:\$ 64.75 Sav  
e:\$ 435.25 | Sim:0.2s LLM:0.6s  
| food=10x/8h@Left-Ctr prob=1x/48h@Top-Ctr O2=1h/20h@Bot-Ctr []  
Day 14 | Fit:0.7708 Yld:0.7143 Sav:0.8625 Hlth:0.7777 | Surv: 77.8% (35/45) | Cost:\$ 68.75 Sav  
e:\$ 431.25 | Sim:0.2s LLM:0.6s  
| food=10x/8h@Left-Ctr prob=1x/48h@Top-Ctr O2=1h/20h@Bot-Ctr []  
Day 15 | Fit:0.7528 Yld:0.6939 Sav:0.8530 Hlth:0.7475 | Surv: 75.6% (34/45) | Cost:\$ 73.50 Sav  
e:\$ 426.50 | Sim:0.1s LLM:0.6s  
| food=12x/8h@Left-Ctr prob=1x/48h@Top-Ctr O2=1h/20h@Bot-Ctr []  
Day 16 | Fit:0.7248 Yld:0.6327 Sav:0.8394 Hlth:0.8319 | Surv: 68.9% (31/45) | Cost:\$ 80.30 Sav  
e:\$ 419.70 | Sim:0.1s LLM:0.6s  
| food=12x/8h@Left-Ctr prob=1x/48h@Top-Ctr O2=1h/20h@Bot-Ctr []  
Day 17 | Fit:0.7011 Yld:0.5918 Sav:0.8293 Hlth:0.8495 | Surv: 64.4% (29/45) | Cost:\$ 85.35 Sav  
e:\$ 414.65 | Sim:0.1s LLM:0.5s  
| food=12x/8h@Left-Ctr prob=1x/48h@Top-Ctr O2=1h/20h@Bot-Ctr []

Day 18 | Fit:0.6976 Yld:0.5918 Sav:0.8207 Hlth:0.8442 | Surv: 64.4% (29/45) | Cost:\$ 89.65 Sav  
e:\$ 410.35 | Sim:0.1s LLM:0.6s  
| food=12x/8h@Left-Ctr prob=1x/48h@Top-Ctr O2=1h/20h@Bot-Ctr []  
Day 19 | Fit:0.6513 Yld:0.5102 Sav:0.8106 Hlth:0.8596 | Surv: 55.6% (25/45) | Cost:\$ 94.70 Sav  
e:\$ 405.30 | Sim:0.1s LLM:0.6s  
| food=12x/8h@Left-Ctr prob=1x/48h@Top-Ctr O2=1h/20h@Bot-Ctr []  
Day 20 | Fit:0.6483 Yld:0.5102 Sav:0.8020 Hlth:0.8586 | Surv: 55.6% (25/45) | Cost:\$ 99.00 Sav  
e:\$ 401.00 | Sim:0.1s LLM:0.6s  
| food=12x/8h@Left-Ctr prob=1x/48h@Top-Ctr O2=1h/20h@Bot-Ctr []  
Day 21 | Fit:0.6117 Yld:0.4490 Sav:0.7869 Hlth:0.8755 | Surv: 48.9% (22/45) | Cost:\$ 106.55 Sav  
e:\$ 393.45 | Sim:0.1s LLM:0.6s  
| food=12x/8h@Left-Ctr prob=1x/48h@Top-Ctr O2=1h/20h@Bot-Ctr []  
Day 22 | Fit:0.5973 Yld:0.4286 Sav:0.7783 Hlth:0.8731 | Surv: 46.7% (21/45) | Cost:\$ 110.85 Sav  
e:\$ 389.15 | Sim:0.2s LLM:0.5s  
| food=12x/8h@Left-Ctr prob=1x/48h@Top-Ctr O2=1h/20h@Bot-Ctr []  
Day 23 | Fit:0.5679 Yld:0.3878 Sav:0.7682 Hlth:0.8428 | Surv: 42.2% (19/45) | Cost:\$ 115.90 Sav  
e:\$ 384.10 | Sim:0.2s LLM:0.6s  
| food=12x/8h@Left-Ctr prob=1x/48h@Top-Ctr O2=1h/20h@Bot-Ctr []  
Day 24 | Fit:0.5641 Yld:0.3878 Sav:0.7596 Hlth:0.8350 | Surv: 42.2% (19/45) | Cost:\$ 120.20 Sav  
e:\$ 379.80 | Sim:0.1s LLM:0.6s  
| food=12x/8h@Left-Ctr prob=1x/48h@Top-Ctr O2=1h/20h@Bot-Ctr []  
Day 25 | Fit:0.5395 Yld:0.3469 Sav:0.7495 Hlth:0.8442 | Surv: 37.8% (17/45) | Cost:\$ 125.25 Sav  
e:\$ 374.75 | Sim:0.1s LLM:0.6s  
| food=12x/8h@Random prob=1x/48h@Top-Ctr O2=1h/20h@Bot-Ctr []  
Day 26 | Fit:0.5127 Yld:0.3061 Sav:0.7359 Hlth:0.8460 | Surv: 33.3% (15/45) | Cost:\$ 132.05 Sav  
e:\$ 367.95 | Sim:0.1s LLM:0.5s  
| food=12x/8h@Random prob=1x/48h@Top-Ctr O2=1h/20h@Bot-Ctr []  
Day 27 | Fit:0.5013 Yld:0.2857 Sav:0.7258 Hlth:0.8719 | Surv: 31.1% (14/45) | Cost:\$ 137.10 Sav  
e:\$ 362.90 | Sim:0.1s LLM:0.5s  
| food=12x/8h@Random prob=1x/48h@Top-Ctr O2=1h/20h@Bot-Ctr []  
Day 28 | Fit:0.5012 Yld:0.2857 Sav:0.7172 Hlth:0.8950 | Surv: 31.1% (14/45) | Cost:\$ 141.40 Sav  
e:\$ 358.60 | Sim:0.1s LLM:0.6s  
| food=12x/8h@Random prob=1x/48h@Top-Ctr O2=1h/20h@Bot-Ctr []  
Day 29 | Fit:0.4978 Yld:0.2857 Sav:0.7071 Hlth:0.8945 | Surv: 31.1% (14/45) | Cost:\$ 146.45 Sav  
e:\$ 353.55 | Sim:0.1s LLM:0.6s  
| food=12x/8h@Random prob=1x/48h@Top-Ctr O2=1h/20h@Bot-Ctr []  
Day 30 | Fit:0.4946 Yld:0.2857 Sav:0.6985 Hlth:0.8913 | Surv: 31.1% (14/45) | Cost:\$ 150.75 Sav  
e:\$ 349.25 | Sim:0.1s [END]

Gen 5 FINAL (attempt 1):  
Fit:0.4946 | Yield:0.2857 | SavingRate:0.6985 | Healthiness:0.8913  
Surv:31.1% (14/45) | Cost:\$150.75 | Save:\$349.25

Saved memory: /content/AquaLLM/logs/memory\_minimal.json  
Saved trace: /content/AquaLLM/logs/trace\_minimal.json (335,100 bytes)  
Saved CSV: /content/AquaLLM/logs/llm\_results.csv (756 rows)  
Saved viz: /content/AquaLLM/logs/simulation\_data.json (720 frames) [best: minimal\_gen4]

=====  
BEST: minimal\_gen4  
Fit:0.7395 | Yield:0.7347 | SavingRate:0.6940 | Healthiness:0.8870  
=====

Total wall time: 4411.1s (73.5 min)

# CELL 14: Download Visualization + Memory Files

In [59]:

```
@title
import os, glob
from google.colab import files

for path, desc in [
 ("/content/AquaLLM/logs/simulation_data.json", "Visualization data"),
 ("/content/AquaLLM/logs/llm_results.csv", "Results CSV"),
]:
 if os.path.exists(path):
 size = os.path.getsize(path)
 print("Downloading {} ({} bytes) - {}".format(
 os.path.basename(path), size, desc))
 files.download(path)
 else:
 print("Not found: {} - run Cell 13 first".format(path))

Download all trace files
for path in sorted(glob.glob("/content/AquaLLM/logs/trace_*.json")):
 size = os.path.getsize(path)
 print("Downloading {} ({} bytes) - Trace".format(os.path.basename(path), size))
```

```
files.download(path)

Download all memory files
for path in sorted(glob.glob("/content/AquaLLM/logs/memory_*.json")):
 size = os.path.getsize(path)
 print("Downloading {} ({} bytes) - Memory".format(os.path.basename(path), size))
 files.download(path)

print("\nTo view visualization:")
print(" 1. Put simulation_data.json in AquaLLM/logs/")
print(" 2. cd AquaLLM && python -m http.server")
print(" 3. Open http://localhost:8000/visuals/index.html")
```

Downloading simulation\_data.json (23,082,217 bytes) - Visualization data

Downloading llm\_results.csv (75,619 bytes) - Results CSV

Downloading trace\_full.json (736,047 bytes) - Trace

Downloading trace\_minimal.json (335,100 bytes) - Trace

Downloading trace\_no\_memory.json (611,116 bytes) - Trace

Downloading trace\_no\_obs.json (899,399 bytes) - Trace

Downloading trace\_no\_planning.json (408,173 bytes) - Trace

Downloading trace\_no\_tools.json (367,370 bytes) - Trace

Downloading memory\_full.json (27,327 bytes) - Memory

Downloading memory\_minimal.json (604 bytes) - Memory

Downloading memory\_no\_memory.json (1,111 bytes) - Memory

Downloading memory\_no\_obs.json (5,857 bytes) - Memory

Downloading memory\_no\_planning.json (29,873 bytes) - Memory

Downloading memory\_no\_tools.json (30,828 bytes) - Memory

To view visualization:

1. Put simulation\_data.json in AquaLLM/logs/
2. cd AquaLLM && python -m http.server
3. Open http://localhost:8000/visuals/index.html

## Cell 15: Ablation Analysis & Plots

In [60]:

```
@title
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib
import numpy as np
import os

matplotlib.rcParams.update({'font.size': 11})

CSV_PATH = "/content/AquaLLM/logs/llm_results.csv"
PLOTS_DIR = "/content/AquaLLM/plots"
os.makedirs(PLOTS_DIR, exist_ok=True)

if not os.path.exists(CSV_PATH):
 print("No results CSV found. Run Cell 13 first.")
```

```

else:
 df = pd.read_csv(CSV_PATH)
 print("Loaded {} rows from {}".format(len(df), CSV_PATH))
 print("Configs found: {}".format(df["config"].unique().tolist()))
 print("Generations: {}".format(sorted(df["generation"].unique()))))

 # — Shared color map —
 configs = sorted(df["config"].unique())
 cmap = plt.cm.tab10
 CONFIG_COLORS = {cfg: cmap(i) for i, cfg in enumerate(configs)}

 # — Per-generation final fitness (last day of each gen) —
 final = df.groupby(["config", "run", "generation"]).last().reset_index()

 # — 1. Bar chart: Final metrics by config (2x2) —
 fig, axes = plt.subplots(2, 2, figsize=(14, 10))
 axes = axes.flatten()

 for idx, metric, label in [
 (0, "fitness", "Fitness"),
 (1, "yield", "Yield"),
 (2, "saving_ratio", "Saving Rate"),
 (3, "healthiness", "Healthiness"),
]:
 ax = axes[idx]
 means = [final[final["config"] == c][metric].astype(float).mean() for c in configs]
 stds = [final[final["config"] == c][metric].astype(float).std() for c in configs]

 colors = [CONFIG_COLORS[c] for c in configs]
 bars = ax.bar(configs, means, yerr=stds, capsize=4, color=colors, edgecolor="black", linewidth=0.5)

 ax.set_ylabel(label)
 ax.set_title("Final {} by Config".format(label))
 ax.set_ylim(0, max(means) * 1.3 if max(means) > 0 else 1)
 for bar, m in zip(bars, means):
 ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.01,
 "{:.3f}".format(m), ha="center", va="bottom", fontsize=9)
 ax.tick_params(axis='x', rotation=30)

 plt.tight_layout()
 plt.savefig(os.path.join(PLOTS_DIR, "ablation_bars.png"), dpi=150, bbox_inches="tight")
 plt.show()
 print("Saved ablation_bars.png")

 # — 2. Line chart: Fitness over days by config —
 fig, ax = plt.subplots(figsize=(14, 6))

 for cfg in configs:
 cfg_df = df[df["config"] == cfg]
 daily_mean = cfg_df.groupby("day")["fitness"].mean().astype(float)
 daily_std = cfg_df.groupby("day")["fitness"].std().astype(float).fillna(0)
 days = daily_mean.index
 ax.plot(days, daily_mean, label=cfg, color=CONFIG_COLORS[cfg], linewidth=2)
 ax.fill_between(days, daily_mean - daily_std, daily_mean + daily_std,
 alpha=0.15, color=CONFIG_COLORS[cfg])

 ax.set_xlabel("Day")
 ax.set_ylabel("Fitness")
 ax.set_title("Fitness Trajectory by Ablation Config (mean ± std)")
 ax.legend(loc="lower right")
 ax.grid(True, alpha=0.3)
 plt.tight_layout()
 plt.savefig(os.path.join(PLOTS_DIR, "ablation_trajectories.png"), dpi=150, bbox_inches="tight")
 plt.show()
 print("Saved ablation_trajectories.png")

 # — 3. Survival rate comparison —
 fig, ax = plt.subplots(figsize=(10, 5))
 for cfg in configs:
 cfg_df = df[df["config"] == cfg]
 daily_surv = cfg_df.groupby("day")["survival_pct"].mean().astype(float)
 ax.plot(daily_surv.index, daily_surv, label=cfg, color=CONFIG_COLORS[cfg], linewidth=2)

 ax.set_xlabel("Day")
 ax.set_ylabel("Survival %")
 ax.set_title("Fish Survival Rate by Config")
 ax.legend(loc="lower left")
 ax.grid(True, alpha=0.3)
 ax.set_ylim(0, 105)
 plt.tight_layout()
 plt.savefig(os.path.join(PLOTS_DIR, "ablation_survival.png"), dpi=150, bbox_inches="tight")

```

```

plt.show()
print("Saved ablation_survival.png")

— 4. Cost efficiency —
fig, ax = plt.subplots(figsize=(10, 5))
for cfg in configs:
 cfg_df = df[df["config"] == cfg]
 daily_cost = cfg_df.groupby("day")["cost"].mean().astype(float)
 ax.plot(daily_cost.index, daily_cost, label=cfg, color=CONFIG_COLORS[cfg], linewidth=2)

ax.axhline(y=500, color="red", linestyle="--", alpha=0.7, label="Budget limit ($500)")
ax.set_xlabel("Day")
ax.set_ylabel("Cumulative Cost ($)")
ax.set_title("Spending Trajectory by Config")
ax.legend(loc="upper left")
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig(os.path.join(PLOTS_DIR, "ablation_cost.png"), dpi=150, bbox_inches="tight")
plt.show()
print("Saved ablation_cost.png")

— 5. Heatmap: Component contribution —
fig, ax = plt.subplots(figsize=(10, 5))
components = ["fitness", "yield", "saving_ratio", "healthiness", "survival_pct"]
comp_labels = ["Fitness", "Yield", "Saving", "Health", "Survival%"]

matrix = []
for cfg in configs:
 row = []
 for comp in components:
 val = final[final["config"] == cfg][comp].astype(float).mean()
 if comp == "survival_pct":
 val /= 100 # Normalize to 0-1
 row.append(val)
 matrix.append(row)

matrix = np.array(matrix)
im = ax.imshow(matrix, cmap="YlOrRd", aspect="auto", vmin=0, vmax=1)
ax.set_xticks(range(len(comp_labels)))
ax.set_xticklabels(comp_labels)
ax.set_yticks(range(len(configs)))
ax.set_yticklabels(configs)
for i in range(len(configs)):
 for j in range(len(comp_labels)):
 ax.text(j, i, "{:.3f}".format(matrix[i, j]),
 ha="center", va="center", fontsize=9,
 color="white" if matrix[i, j] > 0.6 else "black")
plt.colorbar(im, ax=ax, label="Fitness (0-1)")
ax.set_title("Ablation Performance Heatmap")
plt.tight_layout()
plt.savefig(os.path.join(PLOTS_DIR, "ablation_heatmap.png"), dpi=150, bbox_inches="tight")
plt.show()
print("Saved ablation_heatmap.png")

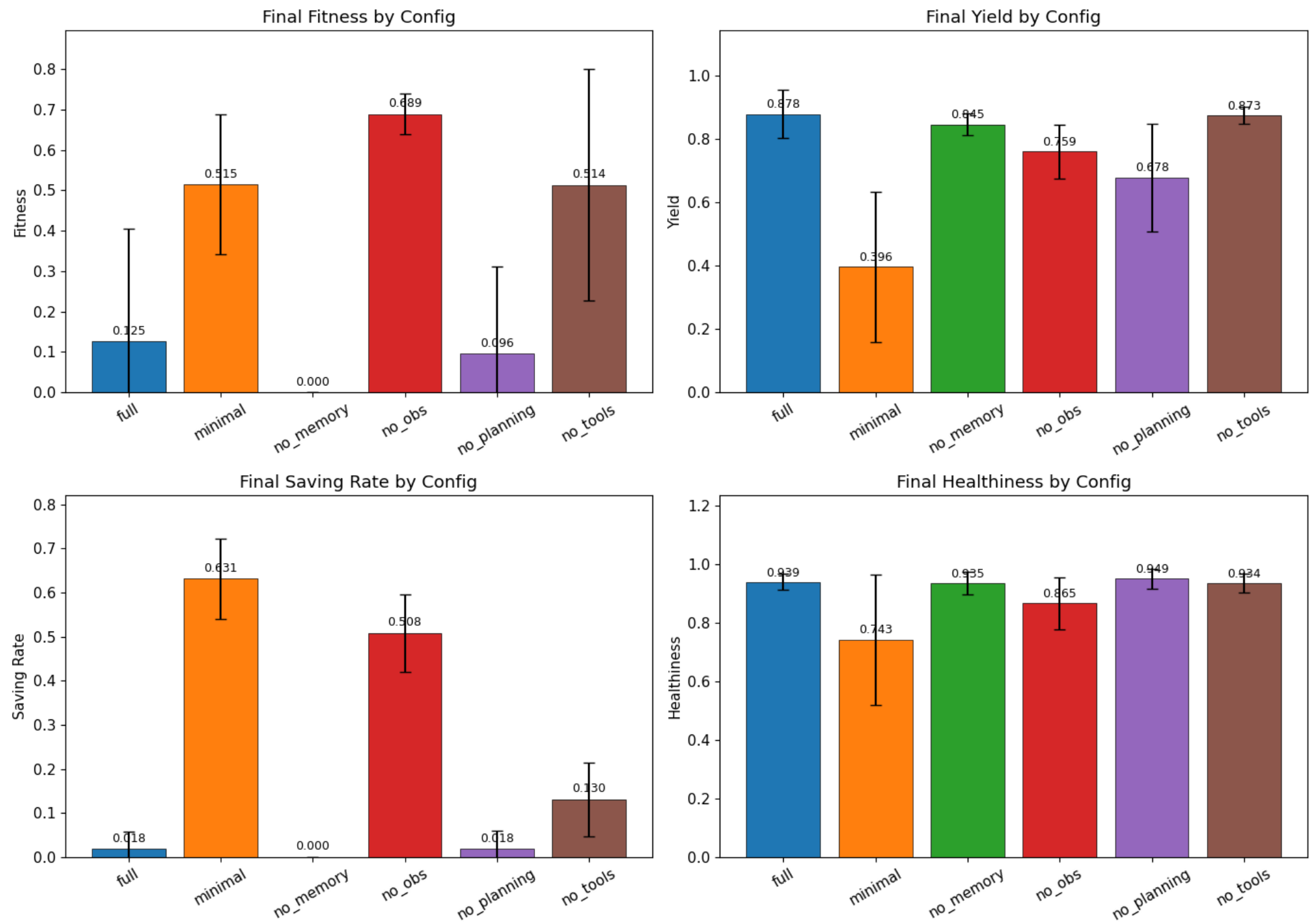
— Download all plots —
from google.colab import files
for fname in [
 "ablation_bars.png",
 "ablation_trajectories.png",
 "ablation_survival.png",
 "ablation_cost.png",
 "ablation_heatmap.png",
]:
 path = os.path.join(PLOTS_DIR, fname)
 if os.path.exists(path):
 files.download(path)
print("\n All plots downloaded.")

— 6. Summary table —
print("\n" + "=" * 80)
print("ABLATION SUMMARY")
print("=" * 80)
summary = final.groupby("config").agg({
 "fitness": ["mean", "std"],
 "yield": "mean",
 "saving_ratio": "mean",
 "healthiness": "mean",
 "survival_pct": "mean",
 "cost": "mean",
}).round(4)
print(summary.to_string())

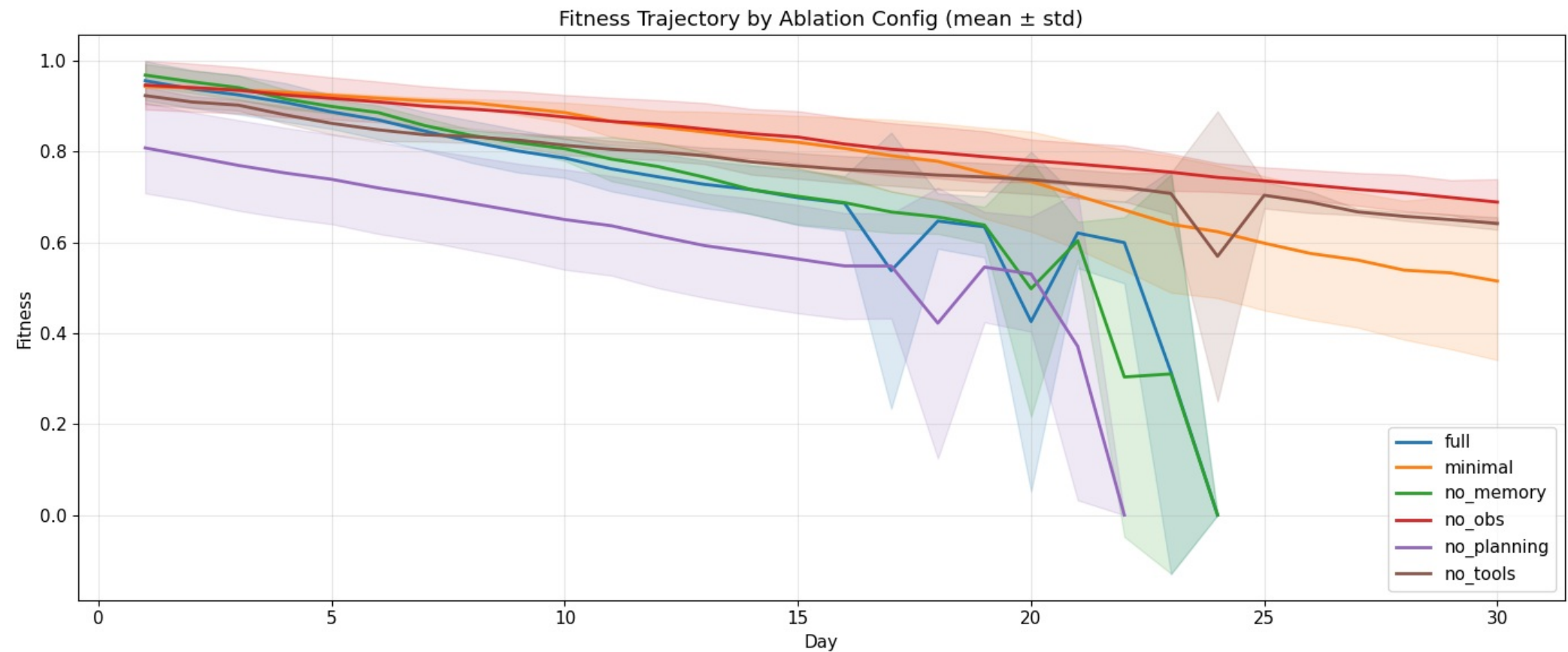
```

```
— Rank configs —
print("\n RANKING (by mean fitness):")
ranking = final.groupby("config")["fitness"].mean().astype(float).sort_values(ascending=False)
for rank, (cfg, fit) in enumerate(ranking.items(), 1):
 print(" {}. {} — {:.4f}".format(rank, cfg, fit))
```

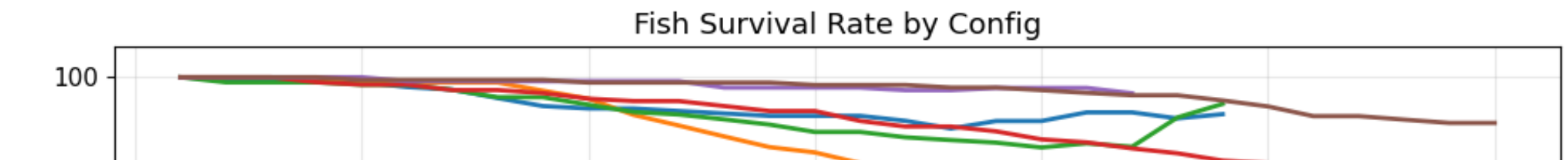
Loaded 756 rows from /content/AquaLLM/logs/llm\_results.csv  
Configs found: ['full', 'no\_planning', 'no\_memory', 'no\_tools', 'no\_obs', 'minimal']  
Generations: [np.int64(1), np.int64(2), np.int64(3), np.int64(4), np.int64(5)]

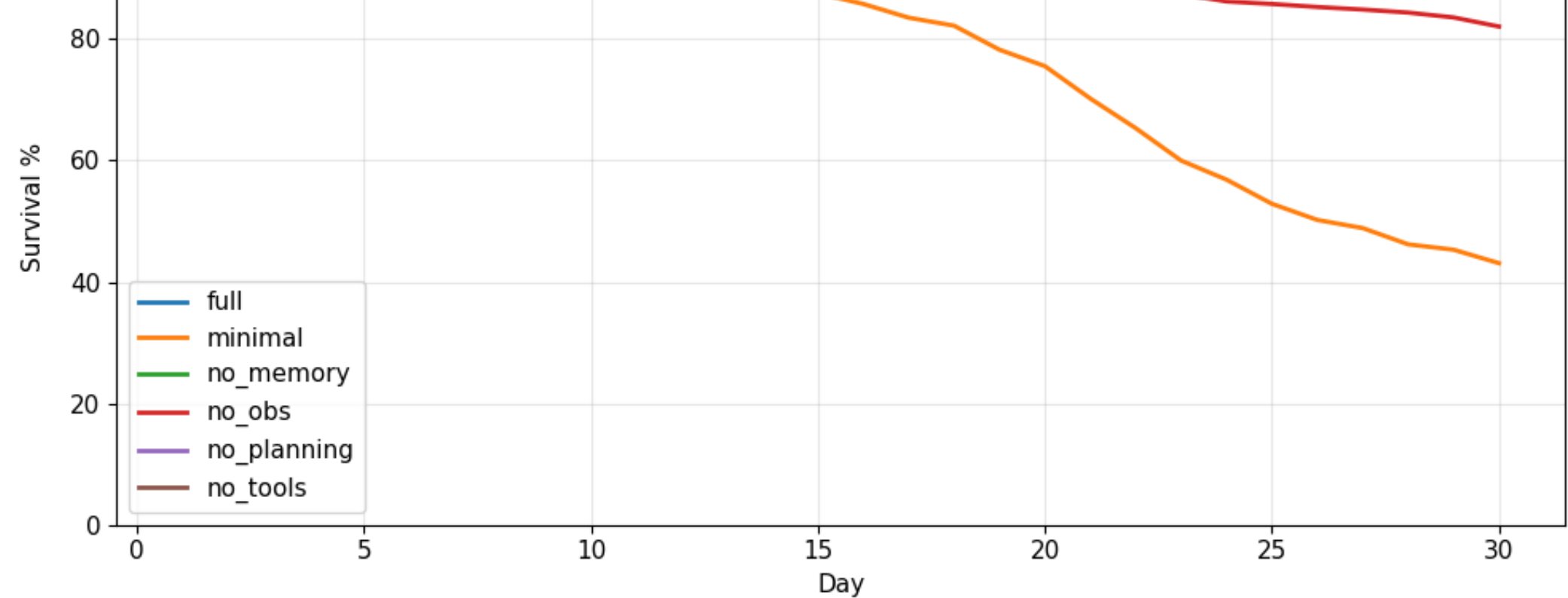


Saved ablation\_bars.png

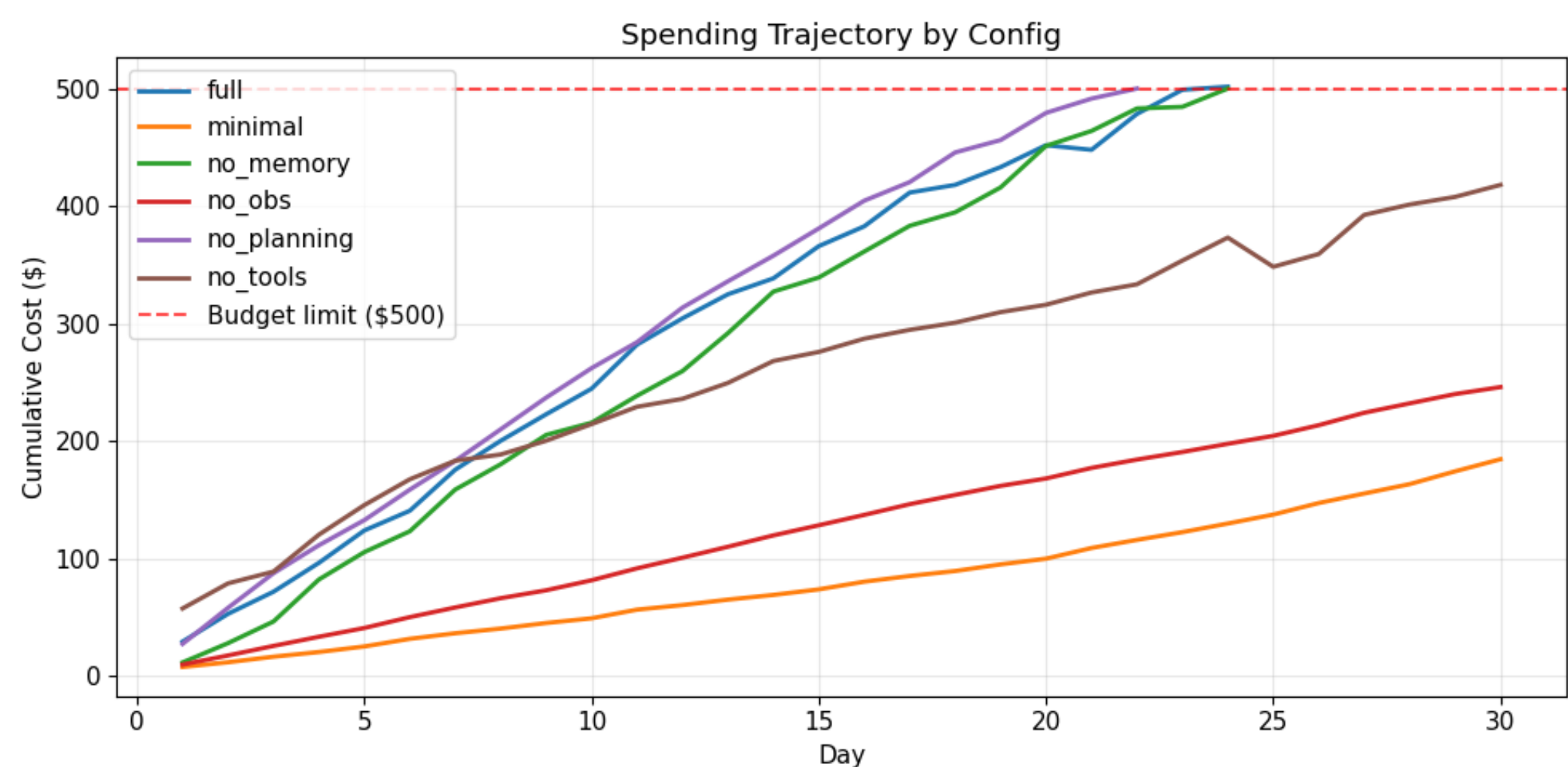


Saved ablation\_trajectories.png

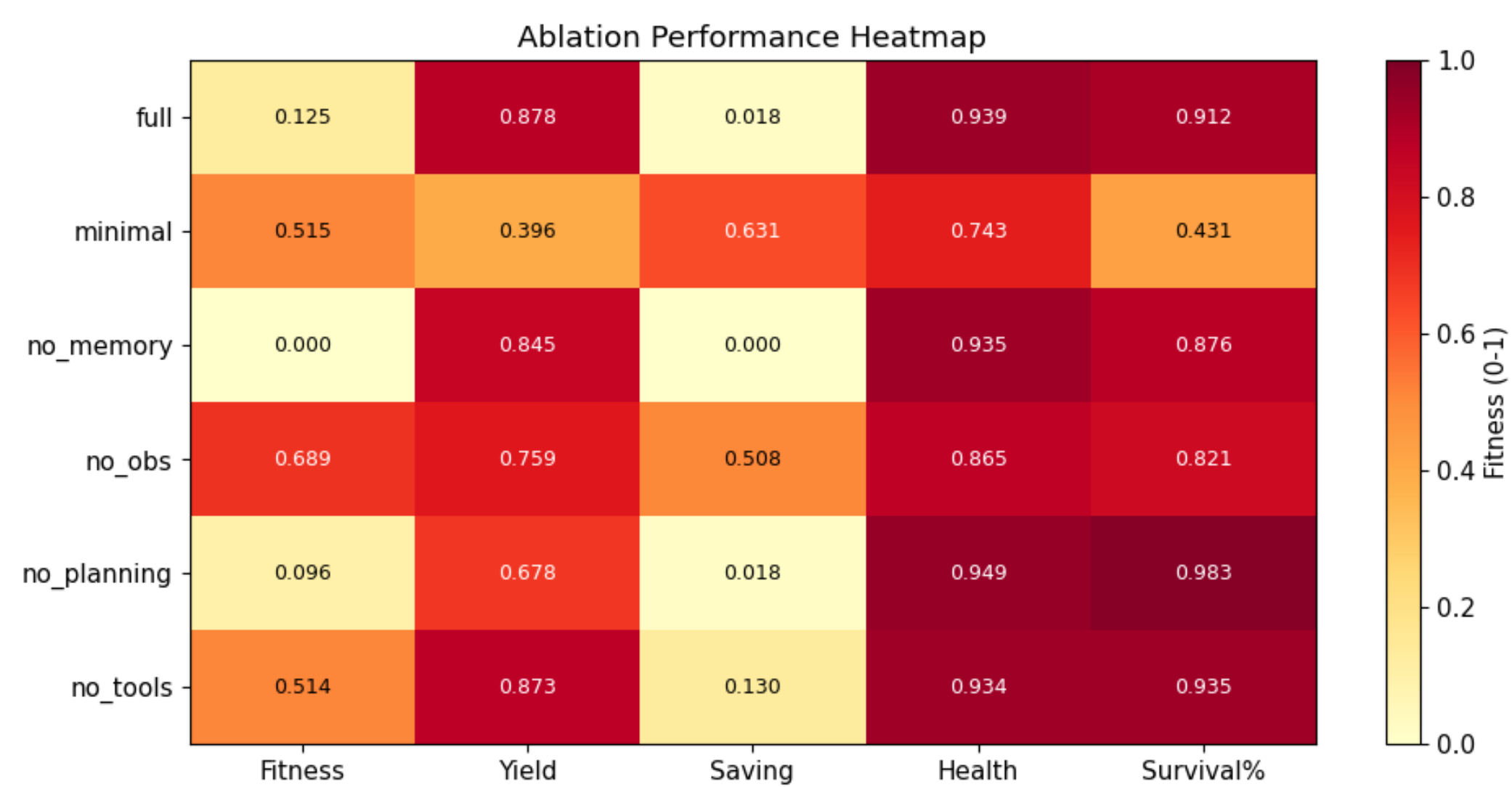




Saved ablation\_survival.png



Saved ablation\_cost.png



Saved ablation\_heatmap.png

☐ All plots downloaded.

=====

ABLATION SUMMARY

=====

| config      | fitness |        | yield  | saving_ratio |        | healthiness | survival_pct |      | cost |
|-------------|---------|--------|--------|--------------|--------|-------------|--------------|------|------|
|             | mean    | std    |        | mean         | mean   |             | mean         | mean |      |
| full        | 0.1254  | 0.2804 | 0.8776 | 0.0180       | 0.9386 | 91.18       | 492.04       |      |      |
| minimal     | 0.5151  | 0.1736 | 0.3959 | 0.6310       | 0.7425 | 43.10       | 184.48       |      |      |
| no_memory   | 0.0000  | 0.0000 | 0.8449 | 0.0000       | 0.9347 | 87.56       | 501.71       |      |      |
| no_obs      | 0.6890  | 0.0503 | 0.7592 | 0.5080       | 0.8653 | 82.08       | 246.01       |      |      |
| no_planning | 0.0959  | 0.2144 | 0.6775 | 0.0182       | 0.9493 | 98.28       | 491.49       |      |      |
| no_tools    | 0.5136  | 0.2874 | 0.8735 | 0.1304       | 0.9344 | 93.52       | 435.20       |      |      |

- RANKING (by mean fitness):
- 1. no\_obs - 0.6890
  - 2. minimal - 0.5151
  - 3. no\_tools - 0.5136
  - 4. full - 0.1254
  - 5. no\_planning - 0.0959
  - 6. no\_memory - 0.0000